Relazione DB Servizio Civile Universale

Università degli Studi Napoli Parthenope
Dipartimento di scienze e tecnologie
Corso di Basi di Dati e Laboratorio

Proponente

- Giovanni De Filippo





Indice

- Introduzione al Progetto
- Descrizione Generale del Sistema
- Progettazione del Database
- Glossario dei Termini
- Diagramma EE/R (Enhanced Entity Relationship)
- Diagramma Relazionale
- Gestione degli Utenti e dei Ruoli
- Operazioni di Base (DML)
- Operazioni Complesse del Sistema
- Schede Descrittive delle Operazioni
- Tavola delle Operazioni
- Tabella dei Volumi
- Vincoli di Integrità
- Vincoli Statici
- Vincoli Dinamici
- Verifica di Normalizzazione (1NF, 2NF, 3NF)
- Possibili Estensioni del Sistema
- Implementazione del Database
- Gestione degli Utenti Oracle e Assegnazione dei Permessi
- Data Definition Language (DDL)
- Definizione delle Tabelle
- Stored Procedures e Programmazione PL/SQL
- Trigger di Controllo e Automazione
- Popolamento del Database con Dati di Esempio
- Query SQL di Interrogazione e Reportistica
- Test e Validazione delle Procedure
- Viste

Illustrazione generale del database

Il database del Servizio Civile Universale è stato progettato per supportare la gestione amministrativa e organizzativa delle attività svolte da un ente del Terzo Settore nell'ambito del Servizio Civile Universale. Il sistema consente di gestire in modo strutturato volontari, operatori, progetti, sedi, formazione, servizio giornaliero, richieste di variazione dell'assegnazione e monitoraggio del percorso del volontario.

La gestione del database parte dall'ente, che organizza uno o più progetti di Servizio Civile Universale. Ogni progetto può svolgersi in una o più sedi, e ogni sede può ospitare più progetti. Un volontario viene assegnato a un determinato progetto presso una determinata sede, per un intervallo temporale definito. Tale collocazione concreta è rappresentata dall'entità Assegnazione, che costituisce il fulcro del database.

Durante il periodo di servizio, il volontario svolge attività giornaliere registrate nel RegistroServizio, che consente di tracciare presenza, permesso ordinario, malattia retribuita, malattia non retribuita e permesso straordinario. Il sistema è costruito per rispettare le regole amministrative del dominio: cinque giorni di servizio a settimana, due giorni di riposo, minimo trentacinque ore settimanali, massimo otto ore giornaliere conteggiabili, gestione del recupero ore e distinzione tra permessi ordinari, malattia e permessi straordinari.

Il database gestisce anche la formazione, obbligatoria per tutti i volontari. Poiché ogni volontario partecipa a più eventi formativi durante l'anno e ogni evento coinvolge più volontari, tra Assegnazione e Formazione esiste una relazione molti-a-molti che verrà poi tradotta nel modello relazionale tramite una tabella associativa.

In caso di problemi particolari, il volontario può presentare una RichiestaVariazione dell'assegnazione. Se la richiesta viene approvata, la vecchia assegnazione non viene eliminata ma chiusa, e ne viene creata una nuova collegata alla precedente, in modo da preservare lo storico.
Il sistema include inoltre l'entità Monitoraggio, che rappresenta la sintesi periodica e valutativa del percorso del volontario, ricavata dai dati amministrativi e integrata con valutazioni qualitative dell'operatore.

Glossario

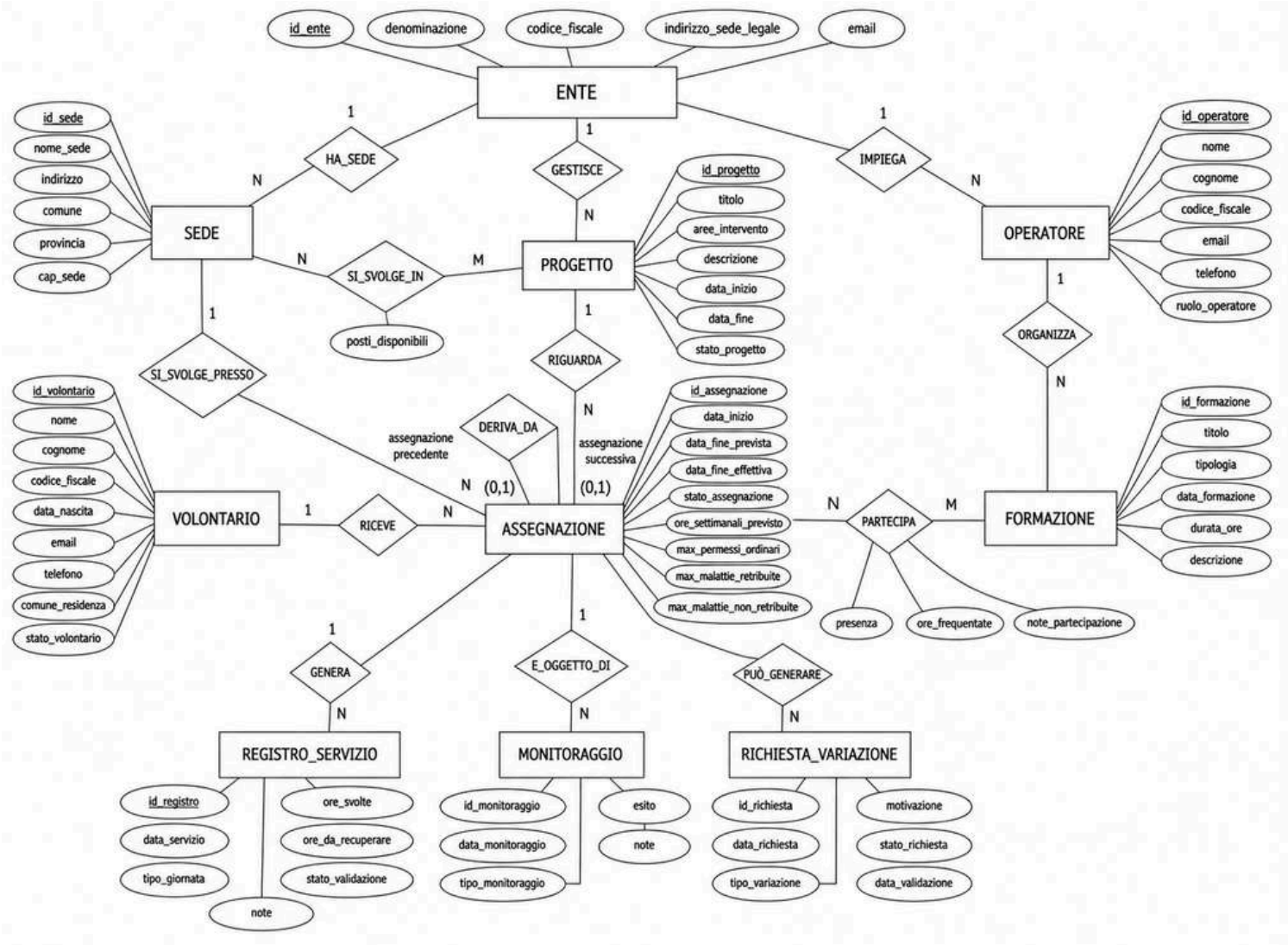
Il glossario chiarisce il gergo tecnico utilizzato nell a descrizione dei requisiti e evidezia eventuali sinonimi.

PAROLA	DESCRIZIONE	SINONIMO
Admin_user	??	??
Volunteer_user	Soggetto che svolge il Servizio Civile	Operatore volontario
Operator_user	Figura interna dell'ente	Tutor, OLP, coordinatore
Progetto	Insieme di obiettivi e attività del Servizio Civile	Programma Operativo
Sede	Luogo fisico in cui il progetto è attuato	Sede di attuazione
Assegnazione	Collocazione concreta del volontario in progetto e sede	Destinazione
Formazione	Evento formativo previsto per i volontari	Modulo formativo

PAROLA	DESCRIZIONE	SINONIMO
Monitoraggio	Valutazione periodica del percorso del volontario	Verifica periodica
RichiestaVariazione	Procedimento di modifica eccezionale dell'assegnazione	Istanza di variazione
Tupla	Riga di una tabella	Record
Chiave primaria	Attributo che identifica univocamente una tupla	Primary key
Chiave esterna	Attributo che referenzia una chiave primaria	Foreign key
RegistroServizio	Registrazione giornaliera dello stato del servizio	Registro presenze

Diagramma EE/R

Il diagramma Entità-Relazione (ER) è uno strumento utilizzato per modellare le informazioni di un sistema in termini di entità e delle relazioni tra di esse. Questo tipo di diagramma è ampiamente utilizzato nella progettazione di database.



- Entità: Rappresentano oggetti del mondo reale con una distinta esistenza. Le entità sono solitamente rappresentate come rettangoli;
- Attributi: Caratteristiche delle entità, rappresentati come ovali collegati ai rettangoli delle entità. Gli attributi possono includere chiavi primarie e chiavi esterne;
- Relazioni: Descrivono come le entità interagiscono tra di loro, rappresentate come rombi collegati alle entità coinvolte.

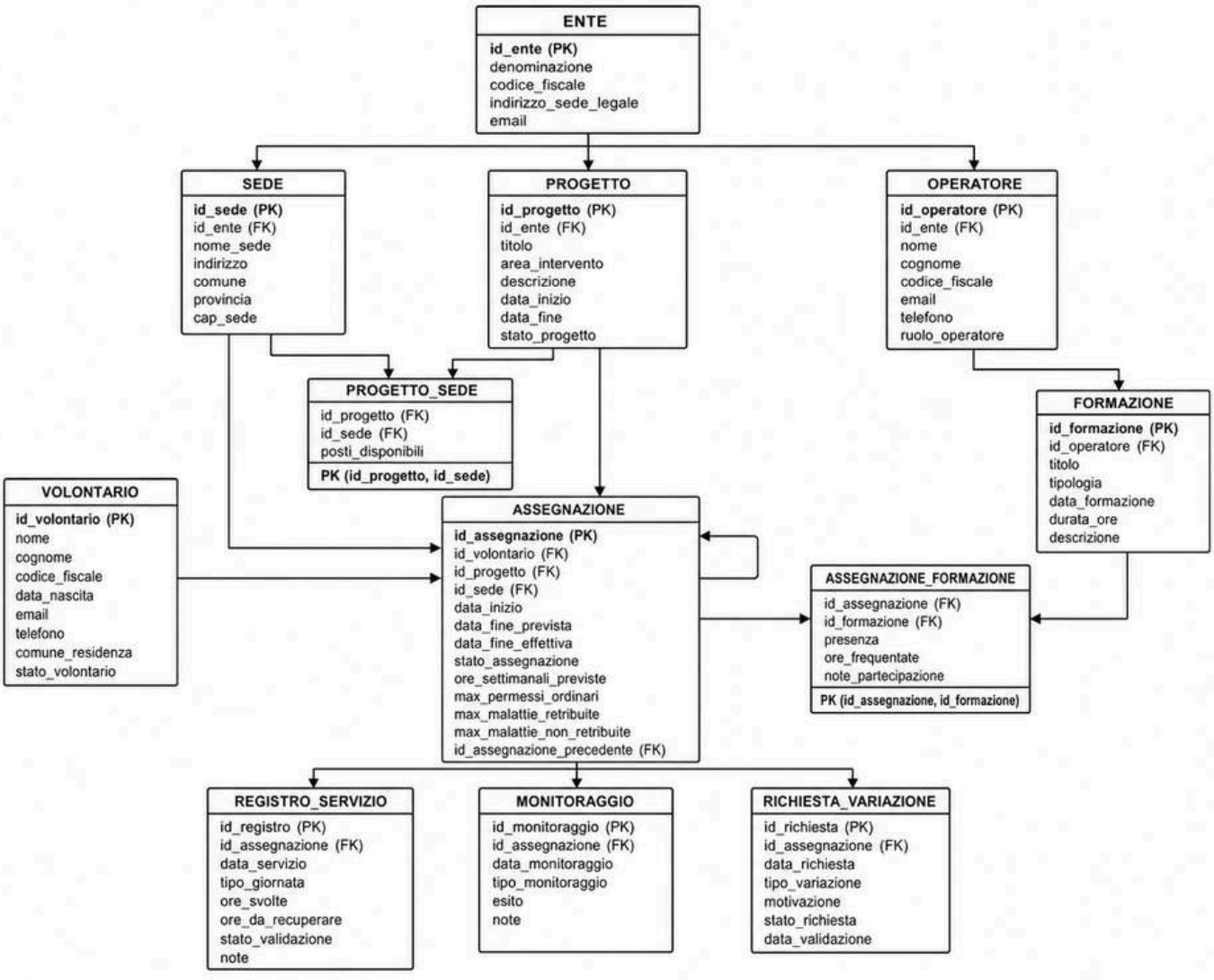
Diagramma Relazionale

Il diagramma relazionale, noto anche come schema relazionale, è una rappresentazione grafica di un database relazionale. Viene utilizzato per illustrare come le tabelle (o relazioni) del database sono strutturate e come queste tabelle sono interconnesse. In un database relazionale, i dati sono organizzati in tabelle, ognuna delle quali contiene righe (o record) e colonne (o campi).

Componenti principali di un Diagramma Relazionale

1. Tabelle (Relazioni): Ogni tabella rappresenta un'entità o un concetto del mondo reale. Le tabelle sono composte da righe e colonne;
    a. Righe (Record o Tuples): Ogni riga rappresenta un'istanza dell'entità descritta dalla tabella;
    b. Colonne (Attributi o Campi): Ogni colonna rappresenta una proprietà dell'entità;
2. Chiavi Primarie (Primary Keys): Una colonna (o un insieme di colonne) che identifica univocamente ogni riga in una tabella. La chiave primaria garantisce l'unicità dei record.
3. Chiavi Esterne (Foreign Keys): Una colonna (o un insieme di colonne) in una tabella che crea un collegamento a una chiave primaria in un'altra tabella. Le chiavi esterne sono utilizzate per definire le relazioni tra le tabelle.
4. Relazioni: I collegamenti tra le tabelle tramite chiavi primarie e chiavi esterne. Le relazioni possono essere di vari tipi:
    a. Uno a Uno (1:1): Ogni riga di una tabella è collegata a una sola riga di un'altra tabella.
    b. Uno a Molti (1: M): Una riga di una tabella può essere collegata a più righe di un'altra tabella.

c. Molti a Molti (M: N): Più righe di una tabella possono essere collegate a più righe di un'altra tabella, spesso implementate con una tabella di giunzione.



Utenti

Per descrivere gli utenti possiamo creare una struttura tabellare che riassume le informazioni degli utenti, il loro tipo (amministratore o normale), il volume delle operazioni (ad esempio, il numero di operazioni o il tipo di dati che possono gestire) e i permessi associati.

UTENTE	TIPO	VOLUME	PERMESSI
Admin_user	Personale interno dell'ente capofila	Tutti i dati e operazioni	SELECT, INSERT, UPDATE, DELETE su tutte le tabelle
Volunteer_user	Volontario	Solo i dati personali	SELECT sui propri dati; INSERT e UPDATE limitato sulle proprie presenze in registro_servizio
Operator_user	Operatore	Dati di servizio e supporto al percorso	SELECT sui dati organizzativi; INSERT e UPDATE su formazione, partecipazione_formazione e monitoraggio

Spiegazione del Volume delle Operazioni

- Amministratore (Admin_user): Può gestire tutti i dati e tutte le operazioni nel database. Questo include la creazione, lettura, aggiornamento e cancellazione di record in tutte le tabelle del sistema, come ENTE, SEDE, OPERATORE, VOLONTARIO, PROGETTO, PROGETTO_SEDE, ASSEGNAZIONE, FORMAZIONE, PARTECIPAZIONE_FORMAZIONE, REGISTRO_SERVIZIO, MONITORAGGIO e RICHIESTA_VARIAZIONE.
- Operatore (Operator_User: Ha accesso in lettura ai dati organizzativi necessari allo svolgimento delle proprie attività, come volontari, assegnazioni, progetti, sedi e richieste di variazione. Può creare, leggere e aggiornare le informazioni relative alla formazione (FORMAZIONE), alla partecipazione alla formazione (PARTECIPAZIONE_FORMAZIONE) e ai monitoraggi (MONITORAGGIO). Può inoltre leggere, inserire e aggiornare le proprie presenze nella tabella REGISTRO_SERVIZIO, limitatamente ai record che lo riguardano personalmente.
- Volontario (Volunteer_User: Ha accesso in lettura ai dati che riguardano il proprio percorso di Servizio Civile. Può consultare la propria assegnazione (ASSEGNAZIONE), la formazione prevista (FORMAZIONE), la propria partecipazione alla formazione (PARTECIPAZIONE_FORMAZIONE), il proprio registro del servizio (REGISTRO_SERVIZIO), i monitoraggi che lo riguardano (MONITORAGGIO) e le eventuali richieste di variazione associate al proprio percorso (RICHIESTA_VARIAZIONE). Può inoltre creare, leggere e aggiornare le proprie presenze nella tabella REGISTRO_SERVIZIO, che saranno successivamente verificate e validate dall’amministratore.

Operazioni di base degli utenti

Le operazioni di base degli utenti in un database relazionale, che possono essere eseguite con semplici comandi DML (Data Manipulation Language), sono generalmente quattro:

1. SELECT: per leggere i dati dalle tabelle;
2. INSERT: per inserire nuovi record nelle tabelle;
3. UPDATE: per modificare i record esistenti nelle tabelle;
4. DELETE: per cancellare i record dalle tabelle;

Operazioni di base per admin_user (Amministratore)

L'amministratore ha tutti i permessi sulle tabelle, quindi può eseguire tutte le operazioni di base su qualsiasi tabella del database.

Operazioni di base per operator_user (Operatore)

L'operatore ha permessi limitati rispetto all'amministratore, quindi può eseguire operazioni di base solo su determinate tabelle e campi. In particolare, può leggere i dati relativi a volontari, assegnazioni, sedi e progetti, e può inserire o aggiornare i dati riguardanti formazione, partecipazione alla formazione, monitoraggi.

Operazioni di base per volunteer_user (Volontario)

Il volontario ha accesso limitato ai dati che riguardano il proprio percorso. Può eseguire operazioni di lettura sui dati personali e sulle informazioni relative alla propria assegnazione, alla formazione, ai monitoraggi e alle richieste di variazione. Può inoltre inserire e aggiornare le proprie presenze nella tabella REGISTRO_SERVIZIO, che saranno successivamente controllate e validate dall'amministratore.

Operazioni Complesse degli utenti

Le operazioni più complesse che richiedono l'implementazione tramite procedure (stored procedures) possono includere attività che coinvolgono più tabelle, logica di business, controlli di integrità, transazioni e altre operazioni complesse che non possono essere gestite con semplici comandi DML. Di seguito viene riportata una lista delle operazioni complesse per gli utenti admin_user, operator_user e volunteer_user.

Operazioni complesse per admin_user (Amministratore)

1. **Creare Progetto:** Creazione di un progetto con le relative sedi di attuazione.
2. **Modificare Progetto:**Modifica di un progetto e delle sue relazioni con le sedi.
3. **Disattivare Progetto:** Disattivazione di un progetto e gestione delle relazioni collegate.
4. **Creare Assegnazione:** Creazione di una assegnazione tra volontario, progetto e sede.

5. **Gestire Richiesta Variazione:** Gestione di una richiesta di variazione dell'assegnazione, con eventuale aggiornamento dello stato dell'assegnazione e mantenimento dello storico.
6. **Validare Presenze:** Controllo, correzione e validazione finale delle presenze inserite nel sistema.

Operazioni complesse per operator_user (Operatore)

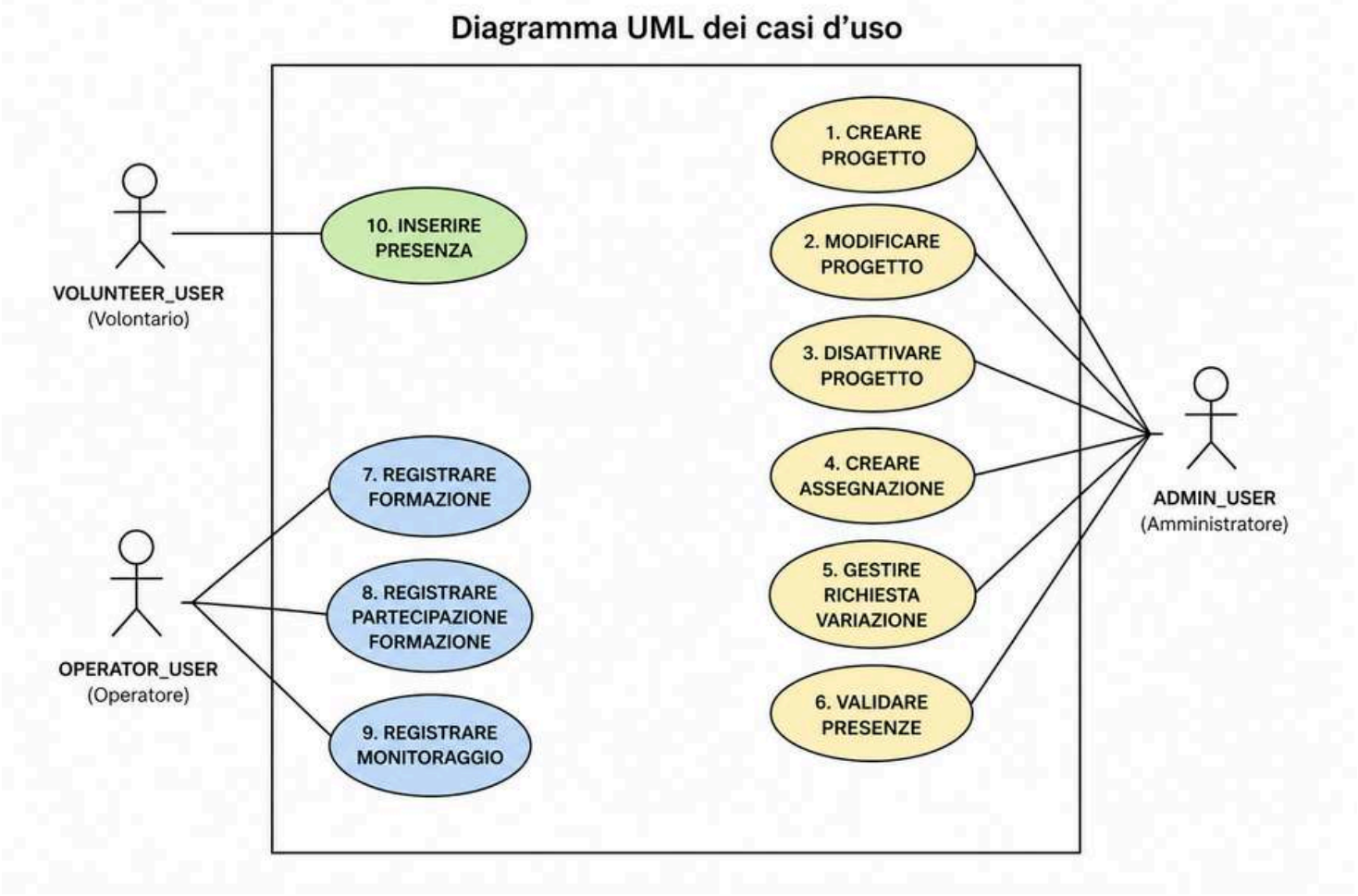
7. **Registrare Formazione:** Registrazione di un evento formativo;
8. **Registrare Partecipazione Formazione:** Registrazione della partecipazione dei volontari alla formazione;
9. **Registrare Monitoraggio:** Registrazione di un monitoraggio periodico relativo al percorso del volontario.

Operazioni complesse per voulunteer_user (Volontario)

10. **Inserire Presenza:**

Registrazione della propria presenza giornaliera nel registro del servizio.

OPERAZIONE	DESCRIZIONE	UTENTE
Creare Progetto	Creazione di un progetto con le relative sedi di attuazione	Admin_user
Modificare Progetto	Modifica di un progetto e delle sue relazioni	Admin_user
Disattivazione Progetto	Disattivazione di un progetto e gestione delle relazioni collegate	Admin_user
Creare Assegnazione	Creazione di una assegnazione tra volontario, progetto e sede	Admin_user
Gestire Richiesta Variazione	Gestione di una richiesta di variazione dell'assegnazione	Admin_user
Validare Presenze	Controllo e validazione finale delle presenze inserite nel sistema	Admin_user
Registrare Formazione	Registrazione di un evento formativo	Operator_user
Registrare Partecipazione Formazione	Registrazione della partecipazione alla formazione	Operator_user
Registrare Monitoraggio	Registrazione di un monitoraggio periodico	Operator_user
Inserire Presenza	Registrazione della propria presenza	Volunteer_user


Diagramma UML dei casi d'uso
VOLUNTEER_USER
(Volontario)
10. INSERIRE PRESENZA
OPERATOR_USER
(Operatore)
7. REGISTRARE FORMAZIONE
8. REGISTRARE PARTECIPAZIONE FORMAZIONE
9. REGISTRARE MONITORAGGIO
1. CREARE PROGETTO
2. MODIFICARE PROGETTO
3. DISATTIVARE PROGETTO
4. CREARE ASSEGNAZIONE
5. GESTIRE RICHIESTA VARIAZIONE
6. VALIDARE PRESENZE
ADMIN_USER
(Amministratore)

Schede descrittore operazione per le operazioni degli utenti

CREARE PROGETTO

- **Scopo:** Creare un nuovo progetto di Servizio Civile Universale e associarlo alle relative sedi di attuazione.
- **Argomenti:**
  - **p_titolo_progetto:** Titolo del progetto (VARCHAR2)
  - **p_descrizione_progetto:** Descrizione del progetto (VARCHAR2)
  - **p_data_inizio:** Data di inizio del progetto (DATE)
  - **p_data_fine:** Data di fine del progetto (DATE)
  - **p_stato_progetto:** Stato del progetto (VARCHAR2)
  - **p_area_intervento:** Area dove si svolge il progetto **(**VARCHAR2)
  - **p_sedi_list:** Lista degli identificativi delle sedi associate (VARCHAR2, separata da virgole)
  - **p_id_ente:** codice identificativo dell'ente, a cui il progetto è associato (NUMBER)
  - **p_posti_disponibili:** numero di posti disponibili da associare a ciascuna sede del progetto (NUMBER)
  - **p_stato_progetto**: stato del progetto (VARCHAR2)
- **Risultato:** Nuovo progetto creato e memorizzato nel sistema, associato all'ente indicato e collegato alle sedi di attuazione specificate.
- **Errori:**
  - Dati mancanti o non validi
  - Ente inesistente
  - Date incoerenti
  - Stato del progetto non valido
  - Una o più sedi inesistenti
  - Lista sedi vuota o malformata
  - Presenza di sedi duplicate nella lista
  - Numero di posti disponibili non valido
  - Errore SQL durante la creazione del progetto

Schede descrittore operazione per le operazioni degli utenti

CREARE PROGETTO

- **Usa:**
  - **PROGETTO:** per creare il nuovo progetto
  - **SEDE:** per verificare l'esistenza delle sedi indicate
  - **PROGETTO_SEDE:** per creare le associazioni tra progetto e sedi
  - **ENTE:** per verificare l'esistenza dell'ente indicato
- **Modifica:**
  - **PROGETTO:** inserisce un nuovo record
  - **PROGETTO_SEDE:** inserisce i record di collegamento con le sedi
- **Prima:** Il progetto non è presente nel sistema e non esistono ancora associazioni tra il nuovo progetto e le sedi di attuazione indicate.
- **Poi:** Il progetto risulta memorizzato nel sistema, associato all'ente indicato e collegato alle sedi di attuazione specificate, con il numero di posti disponibili previsto per ciascuna sede.

MODIFICARE PROGETTO

- **Scopo:** Modificare un progetto di Servizio Civile Universale già esistente e aggiornare le relative sedi di attuazione, insieme ai posti disponibili associati a ciascuna sede.
- **Argomenti:**
  - **p_id_progetto:** Identificativo del progetto (NUMBER);
  - **p_titolo_progetto:** Nuovo titolo del progetto (VARCHAR2);
  - **p_area_intervento:** nuova area di intervento del progetto (VARCHAR2)
  - **p_descrizione_progetto:** Nuova descrizione (VARCHAR2);
  - **p_data_inizio:** Nuova data di inizio (DATE);
  - **p_data_fine:** Nuova data di fine (DATE);
  - **p_stato_progetto:** Nuovo stato del progetto (VARCHAR2);
  - **p_sedi_list:** Nuova lista delle sedi associate (VARCHAR2, separata da virgole);
  - **p_posti_disponibili:** nuovo numero di posti disponibili da associare a ciascuna sede (NUMBER).
- **Risultato:** Progetto aggiornato nel sistema con le nuove informazioni e con le nuove associazioni alle sedi.
- **Errori:**
  - Progetto inesistente
  - Dati mancanti o non validi
  - Date incoerenti
  - Stato del progetto non valido
  - Lista sedi vuota o malformata
  - Presenza di sedi duplicate nella lista
  - Una o più sedi inesistenti
  - Una o più sedi non coerenti con l'ente del progetto
  - Numero di posti disponibili non valido
  - Errore SQL durante l'aggiornamento

MODIFICARE PROGETTO

- **Usa:**
  - **PROGETTO:** per recuperare e aggiornare il progetto;
  - **SEDE:** per verificare l'esistenza delle sedi indicate;
  - **PROGETTO_SEDE:** per aggiornare le associazioni tra progetto e sedi;
  - **ENTE:** indirettamente, tramite l'ente associato al progetto esistente, per verificare la coerenza delle sedi
- **Modifica:**
  - **PROGETTO:** aggiorna il record del progetto
  - **PROGETTO_SEDE:** elimina e reinserisce oppure aggiorna i collegamenti con le sedi
- **Prima:** Il progetto esiste nel sistema con una certa configurazione di dati e con un determinato insieme di sedi associate.
- **Poi:** Il progetto risulta aggiornato con i nuovi dati e con il nuovo insieme di sedi associate, ciascuna con il numero di posti disponibili specificato.

DISATTIVARE PROGETTO

- **Scopo:** Disattivare un progetto esistente e impedirne l'utilizzo per nuove assegnazioni, mantenendo coerente lo storico delle relazioni già presenti nel sistema.
- **Argomenti:**
  - **p_id_progetto:** Identificativo del progetto (NUMBER);.
- **Risultato:** Il progetto viene disattivato e non può più essere utilizzato per nuove assegnazioni.
- **Errori:**
  - Progetto inesistente;
  - Progetto già disattivato;
  - Presenza di assegnazioni incompatibili con la disattivazione;
  - Errore SQL durante l'operazione.
- **Usa:**
  - **PROGETTO:** per recuperare e disattivare il progetto;
  - **PROGETTO_SEDE:** per verificare le associazioni esistenti;
  - **ASSEGNAZIONE:** per controllare eventuali assegnazioni collegate al progetto.
- **Modifica:**
  - **PROGETTO:** aggiorna lo stato del progetto;
- **Prima:** Il progetto risulta attivo nel sistema;
- **Poi:** Il progetto risulta disattivato e non più disponibile per nuove operazioni di assegnazione, senza compromettere la consistenza delle relazioni storiche già presenti. L'operazione complessa di disattivazione progetto è prevista esplicitamente nelle operazioni dell'amministratore.

CREARE ASSEGNAZIONE

- **Scopo:** Creare una nuova assegnazione tra un volontario, un progetto e una sede, per un determinato intervallo temporale, nel rispetto dei vincoli di compatibilità e dei limiti amministrativi previsti dal sistema.
- **Argomenti:**
  - **p_id_volontario:** identificativo del volontario (NUMBER)
  - **p_id_progetto:** identificativo del progetto (NUMBER)
  - **p_id_sede:** identificativo della sede (NUMBER)
  - **p_id_assegnazione_precedente:** identificativo dell'eventuale assegnazione precedente da cui deriva la nuova assegnazione (NUMBER, opzionale)
  - **p_data_inizio:** data di inizio assegnazione (DATE)
  - **p_data_fine_prevista:** data di fine prevista dell'assegnazione (DATE)
  - **p_stato_assegnazione:** stato dell'assegnazione (VARCHAR2)
  - **p_ore_settimanali_previste:** monte ore settimanale previsto (NUMBER)
  - **p_max_permessi_ordinari:** limite massimo di permessi ordinari (NUMBER)
  - **p_max_malattie_retribuite:** limite massimo di malattia retribuita (NUMBER)
  - **p_max_malattie_non_retribuite:** limite massimo di malattia non retribuita (NUMBER)
- **Risultato:** Nuova assegnazione memorizzata nel sistema e associata correttamente al volontario, al progetto e alla sede indicati.
- **Errori:**
  - volontario inesistente
  - progetto inesistente
  - progetto non disponibile per nuove assegnazioni
  - sede inesistente
  - associazione progetto-sede non valida
  - volontario già assegnato in modo incompatibile nello stesso periodo
  - date incoerenti
  - stato dell'assegnazione non valido
  - valori numerici amministrativi non validi
  - assegnazione precedente non valida o incoerente, se valorizzata
  - errore SQL durante la creazione

CREARE ASSEGNAZIONE

- **Usa:**
  - **VOLONTARIO:** per verificare il soggetto assegnato
  - **PROGETTO:** per verificare l'esistenza e la disponibilità del progetto
  - **SEDE:** per verificare l'esistenza della sede
  - **PROGETTO_SEDE:** per verificare che la sede sia associata al progetto scelto
  - **ASSEGNAZIONE:** per controllare conflitti e inserire la nuova assegnazione
- **Modifica:**
  - **ASSEGNAZIONE:** inserisce un nuovo record;
- **Prima:** Il volontario non ha una assegnazione incompatibile con quella richiesta e la coppia progetto-sede è valida.
- **Poi:** Il volontario risulta assegnato al progetto presso la sede indicata, per il periodo specificato, con i relativi parametri amministrativi associati.

GESTIRE RICHIESTA VARIAZIONE

- **Scopo:** Gestire una richiesta di variazione dell'assegnazione di un volontario, preservando lo storico del percorso amministrativo.
- **Argomenti:**
  - **p_id_richiesta_variazione:** identificativo della richiesta (NUMBER)
  - **p_stato_richiesta:** nuovo stato della richiesta (VARCHAR2)
  - **p_motivazione_esito:** motivazione dell'esito (VARCHAR2)
  - **p_nuovo_id_progetto:** nuovo progetto, se previsto (NUMBER)
  - **p_nuovo_id_sede:** nuova sede, se prevista (NUMBER)
  - **p_data_decorrenza:** data di decorrenza della variazione (DATE)
  - **p_data_validazione:** data di validazione della richiesta (DATE)
- **Risultato:** La richiesta viene valutata; se approvata, la vecchia assegnazione viene chiusa e ne viene creata una nuova collegata alla precedente, mantenendo lo storico del percorso.
- **Errori:**
  - richiesta inesistente
  - richiesta già valutata
  - stato richiesta non valido
  - nuovo progetto o nuova sede non validi
  - associazione progetto-sede non valida
  - data di decorrenza incoerente
  - errore SQL durante la gestione della richiesta

GESTIRE RICHIESTA VARIAZIONE

- **Usa:**
  - **RICHIESTA_VARIAZIONE:** per recuperare e aggiornare la richiesta
  - **ASSEGNAZIONE:** per recuperare l'assegnazione corrente, chiuderla ed eventualmente crearne una nuova
  - **PROGETTO:** per verificare l'esistenza del nuovo progetto, se previsto
  - **SEDE:** per verificare l'esistenza della nuova sede, se prevista
  - **PROGETTO_SEDE:** per verificare la validità della nuova coppia progetto-sede
- **Modifica:**
  - **RICHIESTA_VARIAZIONE:** aggiorna stato della richiesta, motivazione dell'esito e data di validazione
  - **ASSEGNAZIONE:** chiude l'assegnazione corrente e, se necessario, inserisce la nuova assegnazione collegata alla precedente
- **Prima:** Esiste una richiesta pendente associata a una assegnazione attiva.
- **Poi:** La richiesta risulta gestita e il sistema conserva correttamente lo storico delle assegnazioni. In caso di approvazione, la vecchia assegnazione viene chiusa e la nuova eredita i parametri amministrativi della precedente, salvo variazioni esplicite non previste in questa operazione.

VALIDARE PRESENZE

- **Scopo:** Controllare, correggere e validare le presenze inserite nel sistema, verificandone la coerenza amministrativa rispetto a una specifica assegnazione e alla relativa giornata di servizio.
- **Argomenti:**
  - **p_id_assegnazione:** identificativo dell'assegnazione a cui la presenza si riferisce (NUMBER)
  - **p_data_servizio:** data del servizio da validare (DATE)
  - **p_esito_validazione:** esito della validazione (VARCHAR2)
  - **p_note_validazione:** eventuali note di validazione (VARCHAR2)
  - **p_nuovo_tipo_giornata:** nuova tipologia della giornata, se si intende correggere il record (VARCHAR2, opzionale)
  - **p_nuove_ore_svolte:** nuovo numero di ore svolte, se si intende correggere il record (NUMBER, opzionale)
  - **p_nuove_ore_da_recuperare:** nuovo numero di ore da recuperare, se si intende correggere il record (NUMBER, opzionale)
- **Risultato:** La presenza relativa all'assegnazione e alla data indicate viene validata, corretta oppure segnalata come non valida, in base all'esito del controllo amministrativo.
- **Errori:**
  - assegnazione inesistente
  - assegnazione non valida nella data indicata
  - presenza non trovata
  - esito di validazione non valido
  - valori di correzione non coerenti
  - errore SQL durante la validazione

VALIDARE PRESENZE

- **Usa:**
  - **ASSEGNAZIONE:** per verificare che l'assegnazione esista e sia coerente con la data considerata
- **REGISTRO_SERVIZIO:** per recuperare, correggere e validare la registrazione della presenza
- **Modifica:**
  - **REGISTRO_SERVIZIO:** aggiorna lo stato di validazione, eventuali note o correzioni.
- **Prima:** La presenza risulta inserita ma non ancora validata.
- **Poi:** La presenza risulta validata, corretta oppure rifiutata secondo l'esito del controllo amministrativo, mantenendo la coerenza con l'assegnazione e con le regole del registro del servizio.

REGISTRARE FORMAZIONE

- **Scopo:** Registrare un nuovo evento formativo nel sistema, associandolo all'operatore responsabile e memorizzandone le principali informazioni descrittive, temporali e organizzative.
- **Argomenti:**
  - **p_titolo_formazione:** titolo della formazione (VARCHAR2)
  - **p_tipologia_formazione:** tipologia della formazione (VARCHAR2)
  - **p_descrizione_formazione:** descrizione della formazione (VARCHAR2)
  - **p_data_formazione:** data della formazione (DATE)
  - **p_durata_ore:** durata della formazione in ore (NUMBER)
  - **p_id_operatore:** identificativo dell'operatore responsabile (NUMBER)
- **Risultato:** Nuovo evento formativo memorizzato nel sistema e correttamente associato all'operatore responsabile.
- **Errori:**
  - dati obbligatori mancanti
  - operatore inesistente
  - tipologia della formazione non valida
  - data della formazione non valida
  - durata della formazione non valida
  - errore SQL durante l'inserimento
- **Usa:**
  - **FORMAZIONE:** per creare il nuovo evento formativo
  - **OPERATORE:** per verificare l'esistenza dell'operatore responsabile
- **Modifica:**
  - **FORMAZIONE:** inserisce un nuovo record
- **Prima:** L'evento formativo non è ancora presente nel sistema.
- **Poi:** L'evento formativo risulta registrato nel sistema con titolo, tipologia, descrizione, data, durata e operatore responsabile correttamente associati.

REGISTRARE PARTECIPAZIONE FORMAZIONE

- **Scopo:** Registrare la partecipazione di un volontario a un evento formativo nel contesto della propria assegnazione.
- **Argomenti:**
  - **p_id_assegnazione:** identificativo dell'assegnazione (NUMBER**)**
  - **p_id_formazione:** identificativo della formazione (NUMBER)
  - **p_presenza:** stato di presenza del volontario (VARCHAR2)
  - **p_ore_frequentate:** numero di ore frequentate (NUMBER)
  - **p_note:** eventuali note di partecipazione (VARCHAR2)
- **Risultato:** La partecipazione del volontario alla formazione viene registrata nel sistema.
- **Errori:**
  - assegnazione inesistente
  - formazione inesistente
  - partecipazione già registrata
  - dati non validi
  - presenza e ore frequentate non coerenti
  - errore SQL durante l'operazione
- **Usa:**
  - **ASSEGNAZIONE:** per identificare il volontario nel contesto del progetto
  - **FORMAZIONE:** per identificare l'evento formativo
  - **ASSEGNAZIONE_FORMAZIONE:** per registrare la partecipazione
- **Modifica:**
  - **ASSEGNAZIONE_FORMAZIONE:** inserisce un nuovo record
- **Prima:** La partecipazione del volontario alla formazione non è ancora registrata.
- **Poi:** Il volontario risulta associato alla formazione con il relativo stato di presenza, il numero di ore frequentate e le eventuali note di partecipazione.

REGISTRARE MONITORAGGIO

- **Scopo:** Registrare un monitoraggio periodico relativo al percorso del volontario, associandolo a una specifica assegnazione e sintetizzando l'andamento del percorso sulla base dei dati amministrativi disponibili e delle osservazioni qualitative.
- **Argomenti:**
  - **p_id_assegnazione:** identificativo dell'assegnazione (NUMBER)
  - **p_data_monitoraggio:** data del monitoraggio (DATE)
  - **p_tipo_monitoraggio:** tipologia del monitoraggio (VARCHAR2)
  - **p_esito_monitoraggio:** esito sintetico del monitoraggio (VARCHAR2)
  - **p_note:** note aggiuntive (VARCHAR2)
- **Risultato**: Nuovo monitoraggio memorizzato nel sistema in riferimento al percorso del volontario.
- **Errori:** assegnazione inesistente, assegnazione non coerente con il periodo del monitoraggio, tipo di monitoraggio non valido, esito del monitoraggio non valido, dati mancanti o non validi, errore SQL durante l'operazione
- **Usa:**
  - **ASSEGNAZIONE:** per identificare il percorso del volontario e verificarne la coerenza temporale
  - **REGISTRO_SERVIZIO:** per consultare i dati di servizio già registrati, se necessario in fase istruttoria
  - **ASSEGNAZIONE_FORMAZIONE:** per consultare i dati di partecipazione alla formazione, se necessario in fase istruttoria
  - **MONITORAGGIO:** per registrare il nuovo monitoraggio
- **Modifica:**
  - **MONITORAGGIO:** inserisce un nuovo record
- **Prima:** Il monitoraggio non è ancora presente nel sistema per il periodo considerato.
- **Poi:** Il monitoraggio risulta associato al percorso del volontario e coerente con i dati amministrativi disponibili.

INSERIRE PRESENZA

Scopo: Consentire al volontario di registrare la propria presenza giornaliera nel sistema.

Argomenti:

- **p_id_volontario:** Identificativo del volontario (NUMBER);
- **p_data_servizio:** Data della giornata di servizio (DATE);
- **p_tipo_giornata:** Tipo di giornata (VARCHAR2);
- **p_ore_svolte:** Ore svolte (NUMBER);
- **p_note:** Eventuali note (VARCHAR2).
- **Risultato:** La presenza giornaliera del volontario viene registrata nel sistema.

Errori:

- Volontario inesistente;
- Assegnazione non attiva nella data indicata;
- Presenza già registrata per la stessa data;
- Dati mancanti o non validi;
- Errore SQL durante l'inserimento.

Usa:

- **VOLONTARIO:** per identificare il soggetto che inserisce la presenza;
- **ASSEGNAZIONE:** per verificare l'esistenza di una assegnazione attiva nella data considerata;
- **REGISTRO_SERVIZIO:** per registrare la presenza.

Modifica:

- **REGISTRO_SERVIZIO:** inserisce un nuovo record di presenza.

Prima: Non esiste alcuna registrazione di presenza per il volontario nella data indicata.

Poi: La presenza risulta registrata nel sistema e disponibile per la successiva validazione amministrativa.

Tavola delle operazioni

La tavola delle operazioni rappresenta il numero di volte in cui l'operazione è stata lanciata, non il numero di volte in cui ha successo.

OPERAZIONE	TIPO	VOLUME	PERIODO
Creare Progetto	B	1	Mese
Modificare Progetto	B	3	Mese
Disattivazione Progetto	B	1	Mese
Creare Assegnazione	B	10	Mese
Gestire Richiesta Variazione	B	2	Mese
Validare Presenze	B	700	Settimana
Registrare Formazione	B	4	Mese
Registrare Partecipazione Formazione	B	30	Mese
Registrare Monitoraggio	B	20	Mese

Inserire Presenza	B	700	Settimana

Tabella dei Volumi

La tabella dei volumi riporta una stima delle tuple (righe) presenti nelle tabelle. Il tipo indica la tipologia della tabella e può dividersi in E (entità), ED (entità debole) e A (associazione). L'incremento rappresenta la quantità ipotizzata del flusso della tabella in un periodo determinato.

TABELLA	TIPO	VOLUME	INCREMENTO	PERIODO
ENTE	E	1	0	Annuale
SEDE	E	10	1	Annuale
OPERATORE	E	15	2	Annuale
VOLONTARIO	E	140	30	Annuale
PROGETTO	E	10	3	Annuale
ASSEGNAZIONE	E	160	30	Annuale
FORMAZIONE	E	40	5	Mensile
MONITORAGGIO	E	300	20	Mensile

Progettazione

TABELLA	TIPO	VOLUME	INCREMENTO	PERIODO
RICHIESTA_VARIAZIONE	E	20	2	Annuale
REGISTRO_SERVIZIO	E	22000	700	Annuale
PROGETTO_SEDE	A	18	2	Annuale
ASSEGNAZIONE_FORMAZIONE	A	900	30	Annuale

Vincoli di integrità

I vincoli di integrità in un database sono regole o condizioni che assicurano la correttezza, l'affidabilità e la coerenza dei dati all'interno del database stesso. Teoricamente, i vincoli di integrità servono a garantire che i dati siano accurati, validi e completi. Ne esistono diversi tipi principali: **Vincoli di integrità di entità:** Assicurano che ogni riga in una tabella abbia un valore univoco per la sua chiave primaria, garantendo che non ci siano duplicati e che ogni entità sia identificabile in modo univoco. **Vincoli di integrità referenziale:** Mantengono la coerenza tra le tabelle, stabilendo relazioni tra le chiavi esterne e primarie. Ad esempio, se una tabella A ha una chiave esterna che fa riferimento a una tabella B, il valore della chiave esterna deve esistere come valore valido nella chiave primaria della tabella B. **Vincoli di integrità di dominio:** Definiscono i limiti e le regole valide per i valori che possono essere inseriti in una colonna specifica di una tabella. Ad esempio, una colonna che rappresenta l'età di una persona potrebbe avere un vincolo di integrità di dominio che limita i valori validi tra 0 e 150. **Vincoli di integrità di attributo:** Assicurano che i valori inseriti in una colonna rispettino determinate regole, come il tipo di dati (es. numeri interi, stringhe), la lunghezza massima, il formato (es. indirizzi email validi) o altre proprietà specifiche.

Vincoli di Integrità Statici e Dinamici

I vincoli di integrità possono essere classificati in due categorie principali: **statici e dinamici.** Questa distinzione si riferisce al momento in cui vengono valutati e applicati i vincoli rispetto alle operazioni di manipolazione dei dati nel database. I vincoli di integrità statici sono regole che vengono definiti durante la progettazione del database e vengono applicati ogni volta che i dati vengono inseriti, aggiornati o eliminati nel database. I vincoli di integrità dinamici sono regole che dipendono dallo stato attuale dei dati nel database e vengono verificati solo quando viene eseguita un'operazione di manipolazione dei dati (come un'operazione di inserimento, aggiornamento o eliminazione).

VINCOLI STATICI

- **ENTE:**
  - **UNIQUE(codice_fiscale_ente):** Assicura l'unicità del codice fiscale dell'ente;
  - **chk_email_ente:** Controlla che l'indirizzo email dell'ente sia espresso in un formato valido.
- **SEDE:**
  - **chk_cap_sede**: Assicura che cap_sede sia valorizzato correttamente secondo il formato previsto dal sistema;
  - **chk_nome_sede:** Assicura che nome_sede sia valorizzato;
  - **chk_indirizzo_sede:** Assicura che indirizzo sia valorizzato;
  - **chk_comune_sede:** Assicura che comune sia valorizzato;
  - **chk_provincia_sede:** Assicura che provincia sia valorizzata.
- **OPERATORE:**
  - **UNIQUE(cf_operatore):** Assicura l'unicità del codice fiscale dell'operatore;
  - **chk_email_operatore:** Controlla che l'indirizzo email dell'operatore sia espresso in un formato valido;
  - **chk_ruolo_operatore:** Limita il valore di ruolo_operatore ai soli ruoli ammessi dal sistema.
- **VOLONTARIO:**
  - **UNIQUE(cf_volontario):** Assicura l'unicità del codice fiscale del volontario;
  - **chk_email_volontario:** Controlla che l'indirizzo email del volontario sia espresso in un formato valido;
  - **chk_data_nascita:** Assicura che la data di nascita del volontario sia antecedente alla data corrente.
- **PROGETTO:**
  - **chk_periodo_progetto:** Assicura che data_inizio sia antecedente a data_fine;
  - **chk_stato_progetto:** Limita il valore di stato_progetto ai soli stati ammessi dal sistema.

VINCOLI STATICI

- **ASSEGNAZIONE:**
  - **chk_periodo_assegnazione:** Assicura che data_inizio sia antecedente o uguale a data_fine, se presente;
  - **chk_stato_assegnazione**: Limita il valore di stato_assegnazione ai soli stati ammessi dal sistema;
  - **chk_ore_settimanali_previste:** Assicura che ore_settimanali_previste sia maggiore di zero;
  - **chk_max_permessi_ordinari:** Assicura che max_permessi_ordinari sia maggiore o uguale a zero;
  - **chk_max_malattia_retribuita:** Assicura che max_malattia_retribuita sia maggiore o uguale a zero;
  - **chk_max_malattia_non_retribuita:** Assicura che max_malattia_non_retribuita sia maggiore o uguale a zero.
- **FORMAZIONE:**
  - **chk_durata_ore_formazione:** Assicura che durata_ore sia maggiore di zero;
  - **chk_data_formazione:** Assicura che la data della formazione sia valorizzata correttamente.

VINCOLI STATICI

- **ASSEGNAZIONE_FORMAZIONE:**
  - **chk_presenza_formazione:** Limita il valore di presenza ai soli valori ammessi dal sistema;
  - **chk_ore_frequentate:** Assicura che ore_frequentate sia maggiore o uguale a zero;
  - **chk_coerenza_presenza_ore:** Assicura che, se presenza indica assenza, ore_frequentate sia pari a zero, mentre se presenza indica presenza, ore_frequentate sia maggiore di zero;
  - **UNIQUE(id_assegnazione, id_formazione):** Assicura che una stessa assegnazione non possa essere registrata più di una volta per la medesima formazione.
- **REGISTRO_SERVIZIO:**
  - **chk_data_servizio:** Assicura che la data del servizio sia valorizzata correttamente;
  - **chk_tipo_giornata:** Limita il valore di tipo_giornata ai soli valori ammessi dal sistema;
  - **chk_ore_svolte:** Assicura che ore_svolte sia maggiore o uguale a zero;
  - **chk_ore_da_recuperare:** Assicura che ore_da_recuperare sia maggiore o uguale a zero;
  - **chk_stato_validazione:** Limita il valore di stato_validazione ai soli stati ammessi dal sistema;
  - **chk_coerenza_tipo_giornata_ore:** Assicura che, se tipo_giornata corrisponde a una giornata di servizio ordinario, ore_svolte sia maggiore di zero, mentre se tipo_giornata corrisponde a permesso ordinario, malattia retribuita, malattia non retribuita o permesso straordinario, ore_svolte sia pari a zero;
  - **UNIQUE(id_assegnazione, data_servizio):** Assicura che per una stessa assegnazione non esistano duplicati nella stessa data.

VINCOLI STATICI

- **MONITORAGGIO:**
  - **chk_data_monitoraggio:** Assicura che la data del monitoraggio sia valorizzata correttamente;
  - **chk_esito_monitoraggio:** Limita il valore di esito_monitoraggio ai soli valori ammessi dal sistema, se previsto.
- **RICHIESTA_VARIAZIONE:**
  - **chk_data_richiesta:** Assicura che la data della richiesta sia valorizzata correttamente;
  - **chk_stato_richiesta:** Limita il valore di stato_richiesta ai soli stati ammessi dal sistema;

VINCOLI DINAMICI

- **Unicità dell'assegnazione attiva**
  - **Descrizione:** Controlla che un volontario non abbia più di una assegnazione attiva nello stesso intervallo temporale;
  - **Scopo:** Garantire che ogni volontario sia associato a un solo progetto e a una sola sede per ciascun periodo di servizio;
  - **Vincolo Dinamico:** Durante l'inserimento o l'aggiornamento di una assegnazione, il sistema verifica che non esistano altre assegnazioni attive e temporalmente sovrapposte per lo stesso volontario.
- **Validità della coppia progetto-sede**
  - **Descrizione:** Controlla che la sede indicata in una assegnazione sia effettivamente associata al progetto scelto;
  - **Scopo:** Evitare assegnazioni incoerenti tra progetto e sede;
  - **Vincolo Dinamico:** Durante l'inserimento o l'aggiornamento di una assegnazione, il sistema verifica l'esistenza della corrispondente associazione nella tabella PROGETTO_SEDE.
- **Gestione dello storico nelle variazioni**
  - **Descrizione:** Controlla che, in caso di approvazione di una richiesta di variazione, la vecchia assegnazione venga chiusa e non eliminata;
  - **Scopo:** Preservare lo storico amministrativo del percorso del volontario;
  - **Vincolo Dinamico:** Durante la gestione di una richiesta di variazione approvata, il sistema chiude l'assegnazione precedente e crea una nuova assegnazione collegata alla precedente.

VINCOLI DINAMICI

- **Validità dell'inserimento della presenza**
  - **Descrizione:** Controlla che una presenza possa essere registrata solo se esiste una assegnazione attiva e valida nella data considerata;
  - **Scopo:** Impedire la registrazione di giornate di servizio al di fuori del periodo di assegnazione;
  - **Vincolo Dinamico:** Durante l'inserimento di una presenza, il sistema verifica l'esistenza di una assegnazione attiva nella data indicata.
- **Limite dei permessi ordinari**
  - **Descrizione:** Controlla che il numero totale di permessi ordinari fruiti non superi il limite previsto per l'assegnazione;
  - **Scopo:** Garantire il rispetto del monte permessi ordinari concesso al volontario;
  - **Vincolo Dinamico:** Durante l'inserimento o l'aggiornamento di una giornata classificata come permesso ordinario, il sistema calcola il totale dei permessi già registrati e verifica che non venga superato il valore massimo consentito altrimenti l'assegnazione del volontario decadrà.

VINCOLI DINAMICI

- **Limite delle malattie retribuite**
  - **Descrizione:** Controlla che il numero totale di giorni di malattia retribuita non superi il limite previsto per l'assegnazione;
  - **Scopo:** Garantire il rispetto del monte giorni di malattia retribuita;
  - **Vincolo Dinamico:** Durante l'inserimento o l'aggiornamento di una giornata classificata come malattia retribuita, il sistema verifica il numero totale di giorni già registrati per quella tipologia, altrimenti l'assegnazione del volontario decadrà.
- **Limite delle malattie non retribuite**
  - **Descrizione:** Controlla che il numero totale di giorni di malattia non retribuita non superi il limite previsto per l'assegnazione;
  - **Scopo:** Garantire il rispetto del monte giorni di malattia non retribuita;
  - **Vincolo Dinamico:** Durante l'inserimento o l'aggiornamento di una giornata classificata come malattia non retribuita, il sistema verifica il numero totale di giorni già registrati per quella tipologia, altrimenti l'assegnazione del volontario decadrà.

VINCOLI DINAMICI

- **Validità del permesso straordinario**
  - **Descrizione:** Controlla che il permesso straordinario possa essere registrato solo in presenza di una causale ammessa e di una giustificazione valida;
  - **Scopo:** Evitare l'uso improprio del permesso straordinario;
  - **Vincolo Dinamico:** Durante l'inserimento o l'aggiornamento di una giornata classificata come permesso straordinario, il sistema verifica la presenza della motivazione prevista e dell'eventuale documentazione richiesta.
- **Limite massimo di ore giornaliere conteggiabili**
  - **Descrizione:** Controlla che le ore di servizio conteggiate in una giornata non superino il limite massimo di 8 ore;
  - **Scopo:** Garantire il rispetto della regola amministrativa sul monte ore giornaliero;
  - **Vincolo Dinamico:** Durante l'inserimento o l'aggiornamento di una presenza, il sistema verifica che le ore conteggiate non superino 8; eventuali ore eccedenti non vengono considerate ai fini del monte ore di servizio.

VINCOLI DINAMICI

- **Monitoraggio su assegnazioni esistenti**
  - **Descrizione:** Controlla che un monitoraggio possa essere registrato solo in riferimento a una assegnazione esistente e coerente con il periodo del percorso monitorato.
  - **Scopo:** Garantire che i monitoraggi siano associati a percorsi reali del volontario.
  - **Vincolo Dinamico:** Durante l'inserimento di un monitoraggio, il sistema verifica l'esistenza dell'assegnazione e la coerenza del monitoraggio con il percorso amministrativo registrato.
- **Coerenza tra tipo di giornata e ore registrate**
  - **Descrizione:** Controlla che il numero di ore registrate sia coerente con la tipologia della giornata di servizio;
  - **Scopo:** Evitare registrazioni incompatibili tra stato della giornata e quantità di ore svolte;
  - **Vincolo Dinamico:** Durante l'inserimento o l'aggiornamento di un record in REGISTRO_SERVIZIO, il sistema verifica che una giornata classificata come servizio presenti un numero di ore maggiore di zero e che una giornata classificata come permesso ordinario, malattia retribuita, malattia non retribuita o permesso straordinario presenti un numero di ore pari a zero.

Verifica di normalità

Per verificare la normalizzazione delle tabelle e correggere eventuali anomalie nelle dipendenze funzionali, analizziamo le tabelle alla luce delle regole delle forme normali, in particolare della Prima (1NF), Seconda (2NF), e Terza Forma Normale (3NF).

Vincoli di Integrità Statici e Dinamici

La 1NF richiede che ogni colonna di una tabella contenga solo valori atomici e ogni riga deve essere unica.

Tutte le tabelle rispettano la 1NF, poiché:

• Ogni colonna contiene valori atomici;

• Ogni tabella ha una chiave primaria definita.

Vincoli di Integrità Statici e Dinamici

La 2NF richiede che la tabella sia già in 1NF e che tutti gli attributi non chiave dipendano interamente dalla chiave primaria.

Tabella ASSEGNAZIONE — È in 2NF perché tutti gli attributi non chiave, come id_volontario, id_progetto, id_sede, id_assegnazione_precedente, data_inizio, data_fine_prevista, data_fine_effettiva, stato_assegnazione, ore_settimanali_previste, max_permessi_ordinari, max_malattie_retribuite e max_malattie_non_retribuite, dipendono interamente dalla chiave primaria id_assegnazione.

Tabella PROGETTO_SEDE — È in 2NF perché l'associazione tra progetto e sede è identificata dalla chiave composta (id_progetto, id_sede) e l'eventuale attributo descrittivo posti_disponibili dipende interamente dall'intera chiave composta, senza dipendenze parziali.

Tabella PARTECIPAZIONE_FORMAZIONE — È in 2NF perché gli attributi presenza e ore_frequentatedipendono interamente dalla chiave composta (id_assegnazione, id_formazione).

Tabella REGISTRO_SERVIZIO — È in 2NF perché gli attributi descrittivi della giornata di servizio dipendono dalla chiave primaria id_registro, mentre la coppia (id_assegnazione, data_servizio) è resa univoca da un vincolo di unicità.

Progettazione

1 ASSEGNAZIONE

```
CREATE TABLE ASSEGNAZIONE (
   id_assegnazione NUMBER(10,0) PRIMARY KEY,
   id_volontario NUMBER(10,0) NOT NULL,
   id_progetto NUMBER(10,0) NOT NULL,
   id_sede NUMBER(10,0) NOT NULL,
   data_inizio DATE NOT NULL,
   data_fine_prevista DATE,
   data_fine_effettiva DATE,
   stato_assegnazione VARCHAR2(20) NOT NULL,
   ore_settimanali_previste NUMBER(2,0) NOT NULL,
   max_permessi_ordinari NUMBER(2,0) DEFAULT 15 NOT NULL,
   max_malattie_retribuite NUMBER(2,0) DEFAULT 10 NOT NULL,
   max_malattie_non_retribuite NUMBER(2,0) DEFAULT 10 NOT NULL,

   CONSTRAINT fk_ass_vol FOREIGN KEY (id_volontario)
      REFERENCES VOLONTARIO(id_volontario),

   CONSTRAINT fk_ass_pro FOREIGN KEY (id_progetto)
      REFERENCES PROGETTO(id_progetto),

   CONSTRAINT fk_ass_sede FOREIGN KEY (id_sede)
      REFERENCES SEDE(id_sede),

   CONSTRAINT chk_periodo_prev_ass CHECK (
      data_fine_prevista IS NULL OR data_inizio <= data_fine_prevista
   ),
   CONSTRAINT chk_periodo_eff_ass CHECK (
      data_fine_effettiva IS NULL OR data_inizio <= data_fine_effettiva
   ),
   CONSTRAINT chk_ore_ass CHECK (
      ore_settimanali_previste > 0
   ),
   CONSTRAINT chk_max_perm CHECK (
      max_permessi_ordinari >= 0
   ),
   CONSTRAINT chk_max_mal_ret CHECK (
      max_malattie_retribuite >= 0
   ),
   CONSTRAINT chk_max_mal_non_ret CHECK (
      max_malattie_non_retribuite >= 0
   ),
   CONSTRAINT chk_stato_ass CHECK (
      stato_assegnazione IN ('ATTIVA', 'CONCLUSA', 'SOSPESA', 'DECADUTA')
   )
);
```

2 PROGETTO_SEDE

```
CREATE TABLE PROGETTO_SEDE (
   id_progetto NUMBER(10,0) NOT NULL,
   id_sede NUMBER(10,0) NOT NULL,
   posti_disponibili NUMBER(3,0) NOT NULL,

   PRIMARY KEY (id_progetto, id_sede),

   FOREIGN KEY (id_progetto)
      REFERENCES PROGETTO(id_progetto),

   FOREIGN KEY (id_sede)
      REFERENCES SEDE(id_sede),

   CONSTRAINT chk_posti_disponibili CHECK (
      posti_disponibili >= 0
   )
);
```

3 ASSEGNAZIONE_FORMAZIONE

```
CREATE TABLE ASSEGNAZIONE_FORMAZIONE (
   id_assegnazione NUMBER(10,0),
   id_formazione NUMBER(10,0),
   presenza VARCHAR2(15) NOT NULL,
   ore_frequentate NUMBER(4,1) DEFAULT 0 NOT NULL,
   note_partecipazione VARCHAR2(255),

   PRIMARY KEY (id_assegnazione, id_formazione),

   FOREIGN KEY (id_assegnazione)
      REFERENCES ASSEGNAZIONE(id_assegnazione),
   FOREIGN KEY (id_formazione)
      REFERENCES FORMAZIONE(id_formazione),

   CONSTRAINT chk_ore_freq CHECK (
      ore_frequentate >= 0
   ),
   CONSTRAINT chk_presenza_form CHECK (
      presenza IN ('PRESENTE', 'ASSENTE')
   ),
   CONSTRAINT chk_presenza_ore CHECK (
      (presenza = 'ASSENTE' AND ore_frequentate = 0)
      OR
      (presenza = 'PRESENTE' AND ore_frequentate > 0)
   )
);
```

4 REGISTRO_SERVIZIO

```
CREATE TABLE REGISTRO_SERVIZIO (
   id_registro NUMBER(10,0) PRIMARY KEY,
   id_assegnazione NUMBER(10,0) NOT NULL,
   data_servizio DATE NOT NULL,
   tipo_giornata VARCHAR2(30) NOT NULL,
   ore_svolte NUMBER(4,1) DEFAULT 0 NOT NULL,
   ore_da_recuperare NUMBER(4,1) DEFAULT 0 NOT NULL,
   stato_validazione VARCHAR2(15) NOT NULL,
   note VARCHAR2(255),

   CONSTRAINT unq_reg UNIQUE (id_assegnazione, data_servizio),
   CONSTRAINT chk_ore_svolte CHECK (
     ore_svolte >= 0
    ),
   CONSTRAINT chk_ore_max CHECK (
     ore_svolte <= 8)
   ),
   CONSTRAINT chk_ore_rec CHECK (
     ore_da_recuperare >= 0
   ),
   CONSTRAINT chk_tipo_giornata CHECK (
     tipo_giornata IN (
        'SERVIZIO',
        'MALATTIA_RETRIBUITA',
        'MALATTIA_NON_RETRIBUITA',
        'PERMESSO_ORDINARIO',
        'PERMESSO_STRAORDINARIO'
     )
   ),
   CONSTRAINT chk_stato_valid CHECK (
     stato_validazione IN ('DA_VALIDARE', 'VALIDATA', 'RIFIUTATA')
   ),
   CONSTRAINT chk_tipo_ore CHECK (
     (tipo_giornata = 'SERVIZIO' AND ore_svolte > 0)
     OR
     (tipo_giornata IN (
        'MALATTIA_RETRIBUITA',
        'MALATTIA_NON_RETRIBUITA',
        'PERMESSO_ORDINARIO',
        'PERMESSO_STRAORDINARIO'
   ),) AND ore_svolte = 0)
   FOREIGN KEY (id_assegnazione)
);    REFERENCES ASSEGNAZIONE(id_assegnazione)
```

Terza Forma Normale (3NF)

La Terza Forma Normale (3NF) richiede che una tabella sia già in 2NF e che nessun attributo non chiave dipenda transitivamente dalla chiave primaria. Nel presente progetto, le principali tabelle analizzate rispettano la 3NF, poiché tutti gli attributi non chiave dipendono direttamente dalla rispettiva chiave primaria e non risultano dipendenze transitive tra attributi non chiave.

Tabella ASSEGNAZIONE:

È in 3NF perché gli attributi non chiave, tra cui id_assegnazione_precedente, data_inizio, data_fine_prevista, data_fine_effettiva, stato_assegnazione, ore_settimanali_previste, max_permessi_ordinari, max_malattie_retribuite e max_malattie_non_retribuite, dipendono direttamente dalla chiave primaria id_assegnazione e non da altri attributi non chiave.

Tabella PROGETTO_SEDE:

Questa tabella è in 3NF perché rappresenta l'associazione tra progetto e sede e l'attributo posti_disponibili dipende direttamente dalla chiave composta (id_progetto, id_sede), senza generare dipendenze transitive.

Tabella ASSEGNAZIONE_FORMAZIONE:

Questa tabella è in 3NF perché gli attributi presenza, ore_frequentate e note_partecipazione dipendono direttamente dalla chiave composta (id_assegnazione, id_formazione) e non risultano dipendenze tra attributi non chiave.

Tabella REGISTRO_SERVIZIO:

Questa tabella è in 3NF perché gli attributi data_servizio, tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione e note dipendono direttamente dalla chiave primaria id_registro e non da altri attributi non chiave. Inoltre, il vincolo di unicità sulla coppia (id_assegnazione, data_servizio) impedisce duplicazioni logiche della registrazione giornaliera senza introdurre dipendenze transitive.

Possibili estensioni

Le possibili estensioni sono modifiche o aggiunte alla struttura del database per supportare nuove esigenze organizzative, nuovi controlli amministrativi o ulteriori funzionalità del sistema. Esse possono essere:

- Introdurre una gestione dettagliata dei documenti giustificativi, associando in modo strutturato certificati medici, attestazioni universitarie o altra documentazione ai record di REGISTRO_SERVIZIO o alle RICHIESTA_VARIAZIONE;
- Aggiungere una gestione più avanzata delle tipologie di formazione, distinguendo tra formazione generale, formazione specifica e moduli integrativi, con eventuali soglie minime di frequenza obbligatoria;
- Prevedere una sezione dedicata alla valutazione finale del volontario, collegata ai monitoraggi periodici e utile per sintetizzare l'intero percorso svolto;
- Introdurre una gestione più dettagliata delle cause di decadenza, registrando in modo esplicito il motivo amministrativo che ha determinato la chiusura anticipata dell'assegnazione;
- Aggiungere funzionalità di reportistica e statistiche, ad esempio sul totale delle ore svolte, sul numero di permessi fruiti, sulla frequenza alla formazione o sull'andamento dei monitoraggi;
- Prevedere una struttura per la gestione delle comunicazioni interne, in modo da tracciare notifiche o messaggi inviati a volontari e operatori relativamente a presenze, variazioni o obblighi formativi.

Gestione degli utenti e assegnazione dei permessi

Nel sistema sono previsti tre profili di utenza:

- **admin_user:** amministratore del sistema;
- **operator_user:** operatore interno dell'ente;
- **volunteer_user:** volontario del Servizio Civile Universale.

È opportuno distinguere tra:

- **profili logici del sistema,** cioè i ruoli funzionali del progetto;
- **utenti del DBMS,** cioè gli account Oracle a cui vengono assegnati i privilegi di accesso.

I privilegi di accesso vengono assegnati a livello di database tramite utenti e comandi GRANT, in modo coerente con i ruoli previsti dal sistema.

```
-- Creazione degli utenti del sistema
CREATE USER admin_user IDENTIFIED BY securePassword1;
CREATE USER operator_user IDENTIFIED BY securePassword2;
CREATE USER volunteer_user IDENTIFIED BY securePassword3;

-- Abilitazione dell'accesso al database
GRANT CREATE SESSION TO admin_user;
GRANT CREATE SESSION TO operator_user;
GRANT CREATE SESSION TO volunteer_user;

-- Concessione dei permessi per admin_user
GRANT SELECT, INSERT, UPDATE, DELETE ON ENTE TO admin_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON SEDE TO admin_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON OPERATORE TO admin_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON PROGETTO TO admin_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON PROGETTO_SEDE TO admin_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON VOLONTARIO TO admin_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ASSEGNAZIONE TO admin_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON FORMAZIONE TO admin_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ASSEGNAZIONE_FORMAZIONE TO admin_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON MONITORAGGIO TO admin_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON REGISTRO_SERVIZIO TO admin_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON RICHIESTA_VARIAZIONE TO admin_user;

-- Concessione dei permessi per operator_user
GRANT SELECT ON VOLONTARIO TO operator_user;
GRANT SELECT ON PROGETTO TO operator_user;
GRANT SELECT ON SEDE TO operator_user;
GRANT SELECT ON ASSEGNAZIONE TO operator_user;
GRANT SELECT, INSERT, UPDATE ON FORMAZIONE TO operator_user;
GRANT SELECT, INSERT, UPDATE ON ASSEGNAZIONE_FORMAZIONE TO operator_user;
GRANT SELECT, INSERT, UPDATE ON MONITORAGGIO TO operator_user;
GRANT SELECT ON REGISTRO_SERVIZIO TO operator_user;
GRANT SELECT ON RICHIESTA_VARIAZIONE TO operator_user;

-- Concessione dei permessi per volunteer_user
GRANT SELECT ON ASSEGNAZIONE TO volunteer_user;
GRANT SELECT ON FORMAZIONE TO volunteer_user;
GRANT SELECT ON ASSEGNAZIONE_FORMAZIONE TO volunteer_user;
GRANT SELECT ON MONITORAGGIO TO volunteer_user;
GRANT SELECT ON RICHIESTA_VARIAZIONE TO volunteer_user;
GRANT SELECT, INSERT, UPDATE ON REGISTRO_SERVIZIO TO volunteer_user;
```

Nota: eventuali limitazioni ai soli dati di competenza del singolo utente sono demandate alla logica applicativa del sistema.

Implementazione

Creazione degli utenti

Gli utenti del sistema vengono creati mediante le seguenti istruzioni:

CREATE USER admin_user **IDENTIFIED BY** securePassword1;

CREATE USER operator_user **IDENTIFIED BY** securePassword2;

CREATE USER volunteer_user **IDENTIFIED BY** securePassword3;

Abilitazione all'accesso al database

Per consentire agli utenti di aprire una sessione di lavoro nel database, viene assegnato il privilegio CREATE SESSION:

GRANT CREATE SESSION TO admin_user;

GRANT CREATE SESSION TO operator_user;

GRANT CREATE SESSION TO volunteer_user;

Permessi per admin_user

L'utente admin_user è il profilo con privilegi completi sul sistema. Per questo motivo riceve i permessi di lettura, inserimento, aggiornamento e cancellazione sulle principali tabelle del progetto:

GRANT SELECT, INSERT, UPDATE, DELETE ON PROGETTO TO admin_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON ENTE TO admin_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON OPERATORE TO admin_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON SEDE TO admin_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON PROGETTO_SEDE TO admin_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON VOLONTARIO TO admin_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON ASSEGNAZIONE TO admin_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON FORMAZIONE TO admin_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON PARTECIPAZIONE_FORMAZIONE TO admin_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON MONITORAGGIO TO admin_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON REGISTRO_SERVIZIO TO admin_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON RICHIESTA_VARIAZIONE TO admin_user**;**

Implementazione

Permessi per operator_user

L'utente operator_user dispone di privilegi di consultazione sul registro del servizio e di privilegi di inserimento e aggiornamento sulle tabelle che riguardano la formazione, l'associazione tra assegnazione e formazione e il monitoraggio.

GRANT SELECT ON VOLONTARIO TO operator_user;
GRANT SELECT ON PROGETTO TO operator_user;
GRANT SELECT ON SEDE TO operator_user;
GRANT SELECT ON ASSEGNAZIONE TO operator_user;
GRANT SELECT, INSERT, UPDATE ON FORMAZIONE TO operator_user;
GRANT SELECT, INSERT, UPDATE ON PARTECIPAZIONE_FORMAZIONE TO operator_user;
GRANT SELECT, INSERT, UPDATE ON MONITORAGGIO TO operator_user;
GRANT SELECT ON REGISTRO_SERVIZIO TO operator_user;
GRANT SELECT ON RICHIESTA_VARIAZIONE TO operator_user;

Permessi per operator_user

L'utente volunteer_user dispone di privilegi di lettura sulle tabelle che descrivono il proprio percorso nel sistema e di privilegi di inserimento e aggiornamento sulla tabella REGISTRO_SERVIZIO, attraverso la quale vengono registrate le giornate di servizio.
GRANT SELECT ON ASSEGNAZIONE TO volunteer_user;
GRANT SELECT ON FORMAZIONE TO volunteer_user;
GRANT SELECT ON PARTECIPAZIONE_FORMAZIONE TO volunteer_user;
GRANT SELECT ON MONITORAGGIO TO volunteer_user;
GRANT SELECT ON RICHIESTA_VARIAZIONE TO volunteer_user;
GRANT SELECT, INSERT, UPDATE ON REGISTRO_SERVIZIO TO volunteer_user;

Implementazione

Spiegazione delle scelte effettuate

L’utente admin_user corrisponde al profilo amministrativo con capacità complete di gestione del sistema. Per questo motivo può intervenire su tutte le principali tabelle del database, incluse quelle relative all’ente, alle sedi, agli operatori, ai progetti, alle associazioni progetto-sede, ai volontari, alle assegnazioni, alla formazione, all’associazione tra assegnazione e formazione, ai monitoraggi, al registro del servizio e alle richieste di variazione.

L’utente operator_user svolge funzioni operative e organizzative. Di conseguenza, dispone di privilegi di consultazione sui dati necessari allo svolgimento delle attività quotidiane, come volontari, progetti, sedi, assegnazioni, registro del servizio e richieste di variazione. Dispone inoltre di privilegi di inserimento e aggiornamento sulle tabelle che riguardano la formazione, l’associazione tra assegnazione e formazione e il monitoraggio.

L’utente volunteer_user rappresenta invece il volontario. Per questo motivo possiede privilegi più limitati: può consultare le informazioni relative al proprio percorso, tra cui assegnazione, formazione, associazione tra assegnazione e formazione, monitoraggi e richieste di variazione. Può inoltre intervenire sul registro del servizio, inserendo o aggiornando le proprie giornate.

Le istruzioni riportate consentono di definire i tre profili di utenza del sistema — amministratore, operatore e volontario — e di assegnare a ciascuno i privilegi coerenti con il proprio ruolo e con le funzioni effettivamente svolte nel progetto.

Implementazione

Nel sistema, la richiesta di variazione non viene registrata direttamente dal volontario nel database. Il volontario manifesta informalmente la volontà di modificare la propria assegnazione, ad esempio rispetto a progetto o sede, all'operatore locale di progetto. La registrazione formale della richiesta nel database e la successiva gestione della stessa vengono però effettuate dall'amministratore. Per questo motivo i privilegi completi sulla tabella RICHIESTA_VARIAZIONE sono attribuiti a admin_user, mentre operator_user e volunteer_user dispongono soltanto di privilegi di lettura. Eventuali limitazioni ai soli dati di competenza del singolo utente sono demandate alla logica applicativa del sistema.

Data Definition Language (DDL)

Il Data Definition Language (DDL) è un insieme di comandi SQL utilizzati per definire e gestire la struttura e lo schema di un database. I comandi DDL vengono utilizzati per creare, modificare e eliminare oggetti del database come tabelle, indici, e viste. I comandi principali del DDL includono:

CREATE: Utilizzato per creare nuovi oggetti del database come tabelle, indici, viste, e schemi;

ALTER: Utilizzato per modificare la struttura di oggetti esistenti nel database;

DROP: Utilizzato per eliminare oggetti del database.

Tabella Ente

La tabella **ENTE** rappresenta l'ente che gestisce il sistema di Servizio Civile Universale. Questa tabella consente di identificare univocamente ogni ente tramite un codice numerico interno e di memorizzare le principali informazioni istituzionali, come denominazione, codice fiscale, indirizzo della sede legale ed email.

La tabella costituisce il riferimento organizzativo dell'intero database, poiché da essa dipendono le sedi, i progetti e gli operatori.

Sono previsti vincoli di integrità per garantire l'unicità del codice fiscale e la correttezza formale dell'indirizzo email.

```
CREATE TABLE ENTE (
    id_ente NUMBER(10,0) PRIMARY KEY,
    denominazione VARCHAR2(100) NOT NULL,
    codice_fiscale VARCHAR2(16) NOT NULL UNIQUE,
    indirizzo_sede_legale VARCHAR2(150) NOT NULL,
    email VARCHAR2(100) NOT NULL,

    CONSTRAINT chk_email_ente
        CHECK (REGEXP_LIKE(email, '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'))
);
```

Implementazione

Data Definition Language (DDL)

Il Data Definition Language (DDL) è un insieme di comandi SQL utilizzati per definire e gestire la struttura e lo schema di un database. I comandi DDL vengono utilizzati per creare, modificare e eliminare oggetti del database come tabelle, indici, e viste. I comandi principali del DDL includono:

CREATE: Utilizzato per creare nuovi oggetti del database come tabelle, indici, viste, e schemi;

ALTER: Utilizzato per modificare la struttura di oggetti esistenti nel database;

DROP: Utilizzato per eliminare oggetti del database.

Tabella Ente

La tabella **ENTE** rappresenta l'ente che gestisce il sistema di Servizio Civile Universale. Questa tabella consente di identificare univocamente ogni ente tramite un codice numerico interno e di memorizzare le principali informazioni istituzionali, come denominazione, codice fiscale, indirizzo della sede legale ed email.

La tabella costituisce il riferimento organizzativo dell'intero database, poiché da essa dipendono le sedi, i progetti e gli operatori.

Sono previsti vincoli di integrità per garantire l'unicità del codice fiscale e la correttezza formale dell'indirizzo email.

```
CREATE TABLE ENTE (
    id_ente NUMBER(10,0) PRIMARY KEY,
    denominazione VARCHAR2(100) NOT NULL,
    codice_fiscale VARCHAR2(16) NOT NULL UNIQUE,
    indirizzo_sede_legale VARCHAR2(150) NOT NULL,
    email VARCHAR2(100) NOT NULL,

    CONSTRAINT chk_email_ente
        CHECK (REGEXP_LIKE(email, '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'))
);
```

Implementazione

Tabella Sede

La tabella **SEDE** rappresenta una sede operativa dell'ente presso cui possono svolgersi i progetti di Servizio Civile Universale. Questa tabella consente di identificare univocamente ogni sede tramite un codice numerico interno e di memorizzare le principali informazioni localizzative, come nome della sede, indirizzo, comune, provincia e CAP.

Ogni sede è associata a un solo ente, mediante una chiave esterna verso la tabella **ENTE**.

Sono previsti vincoli di integrità per garantire la valorizzazione corretta degli attributi principali e la validità formale del CAP.

```
CREATE TABLE SEDE (
    id_sede NUMBER(10,0) PRIMARY KEY,
    nome_sede VARCHAR2(100) NOT NULL,
    indirizzo VARCHAR2(150) NOT NULL,
    comune VARCHAR2(100) NOT NULL,
    provincia VARCHAR2(2) NOT NULL,
    cap_sede VARCHAR2(5) NOT NULL,
    id_ente NUMBER(10,0) NOT NULL,

    CONSTRAINT fk_sede_ente FOREIGN KEY (id_ente)
        REFERENCES ENTE(id_ente),

    CONSTRAINT chk_cap_sede CHECK (REGEXP_LIKE(cap_sede, '^[0-9]{5}$'))
);
```

Implementazione

Tabella Operatore

La tabella OPERATORE rappresenta il personale interno dell'ente che svolge funzioni organizzative e di supporto nel sistema di Servizio Civile Universale. Questa tabella consente di identificare univocamente ogni operatore tramite un codice numerico interno e di memorizzare le principali informazioni anagrafiche e di contatto, come nome, cognome, codice fiscale, email, telefono e ruolo.

Ogni operatore è associato a un solo ente, mediante una chiave esterna verso la tabella ENTE.

Sono previsti vincoli di integrità per garantire l'unicità del codice fiscale, la correttezza formale dell'indirizzo email e la validità del ruolo assegnato.

```
CREATE TABLE OPERATORE (
    id_operatore NUMBER(10,0) PRIMARY KEY,
    nome VARCHAR2(100) NOT NULL,
    cognome VARCHAR2(100) NOT NULL,
    codice_fiscale VARCHAR2(16) NOT NULL,
    email VARCHAR2(100) NOT NULL,
    telefono VARCHAR2(15) NOT NULL,
    ruolo_operatore VARCHAR2(50) NOT NULL,
    id_ente NUMBER(10,0) NOT NULL,
    CONSTRAINT unq_operatore_cf UNIQUE (codice_fiscale),
    CONSTRAINT unq_operatore_email UNIQUE (email),
    CONSTRAINT fk_operatore_ente FOREIGN KEY (id_ente)
        REFERENCES ENTE(id_ente),
    CONSTRAINT chk_cf_operatore CHECK (REGEXP_LIKE(codice_fiscale, '^[A-Z0-9]{16}$')),
    CONSTRAINT chk_email_operatore CHECK (REGEXP_LIKE(email,
        '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'|+\.[A-Za-z}}"))
);
```

Implementazione

Tabella Progetto

La tabella **PROGETTO** rappresenta un progetto di Servizio Civile Universale gestito da un ente. Questa tabella consente di identificare univocamente ogni progetto tramite un codice numerico interno e di memorizzare le principali informazioni descrittive e temporali, come titolo, area di intervento, descrizione, data di inizio, data di fine e stato del progetto.

Ogni progetto è associato a un solo ente, mediante una chiave esterna verso la tabella **ENTE**.

Sono previsti vincoli di integrità per garantire la coerenza del periodo temporale e la validità dello stato del progetto.

```
CREATE TABLE PROGETTO (
      id_progetto NUMBER(10,0) PRIMARY KEY,
      id_ente NUMBER(10,0) NOT NULL,
      titolo VARCHAR2(100) NOT NULL,
      area_intervento VARCHAR2(100) NOT NULL,
      descrizione VARCHAR2(255) NOT NULL,
      data_inizio DATE NOT NULL,
      data_fine DATE,
      stato_progetto VARCHAR2(20) NOT NULL,

      CONSTRAINT fk_progetto_ente FOREIGN KEY (id_ente)
            REFERENCES ENTE(id_ente),

      CONSTRAINT chk_periodo_progetto CHECK (data_fine IS NULL OR data_inizio <= data_fine),

      CONSTRAINT chk_stato_progetto CHECK (stato_progetto IN ('ATTIVO', 'CONCLUSO', 'SOSPESO'))
);
```

Implementazione

Tabella Progetto_Sede

La tabella **PROGETTO_SEDE** rappresenta l'associazione tra un progetto e una sede operativa. Essa deriva dalla trasformazione della relazione molti-a-molti tra **PROGETTO** e **SEDE** e consente di stabilire in quali sedi può svolgersi ciascun progetto.

La tabella memorizza inoltre il numero di posti disponibili per ogni specifica coppia progetto-sede, informazione necessaria per la gestione amministrativa delle assegnazioni.

La chiave primaria è composta dagli identificativi del progetto e della sede, mentre sono previsti vincoli di integrità referenziale verso le rispettive tabelle padre e un vincolo di validità sul numero dei posti disponibili.

```
CREATE TABLE PROGETTO_SEDE (

    id_progetto NUMBER(10,0) NOT NULL,
    id_sede NUMBER(10,0) NOT NULL,
    posti_disponibili NUMBER(3,0) NOT NULL,

    CONSTRAINT pk_progetto_sede
        PRIMARY KEY (id_progetto, id_sede),

    CONSTRAINT fk_progetto_sede_progetto
        FOREIGN KEY (id_progetto)
        REFERENCES PROGETTO(id_progetto),

    CONSTRAINT fk_progetto_sede_sede
        FOREIGN KEY (id_sede)
        REFERENCES SEDE(id_sede),

    CONSTRAINT chk_posti_disponibili
        CHECK (posti_disponibili >= 0)
);
```

Implementazione

Tabella Volontario

La tabella **VOLONTARIO** rappresenta il soggetto che svolge il percorso di Servizio Civile Universale all'interno del sistema. Questa tabella consente di identificare univocamente ogni volontario tramite un codice numerico interno e di memorizzare le principali informazioni anagrafiche e di contatto, come nome, cognome, codice fiscale, data di nascita, email, telefono, comune di residenza e stato del volontario.

La tabella costituisce il riferimento personale del percorso amministrativo, poiché a partire da essa vengono poi definite le assegnazioni, il registro del servizio, il monitoraggio e le eventuali richieste di variazione.

Sono previsti vincoli di integrità per garantire l'unicità del codice fiscale, la correttezza formale dell'indirizzo email, la validità della data di nascita e la coerenza dello stato del volontario.

```
CREATE TABLE VOLONTARIO (
    id_volontario NUMBER(10,0) PRIMARY KEY,
    nome VARCHAR2(50) NOT NULL,
    cognome VARCHAR2(50) NOT NULL,
    codice_fiscale VARCHAR2(16) NOT NULL UNIQUE,
    data_nascita DATE NOT NULL,
    email VARCHAR2(100) NOT NULL,
    telefono VARCHAR2(15),
    comune_residenza VARCHAR2(100) NOT NULL,
    stato_volontario VARCHAR2(30) NOT NULL,

    CONSTRAINT chk_email_volontario
        CHECK (REGEXP_LIKE(email, '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')),

    CONSTRAINT chk_data_nascita_volontario
        CHECK (data_nascita < CURRENT_DATE),

    CONSTRAINT chk_stato_volontario
        CHECK (stato_volontario IN ('ATTIVO', 'CONCLUSO', 'RINUNCIATARIO', 'DECADUTO'))
);
```

Implementazione

Tabella Assegnazione

La tabella **ASSEGNAZIONE** rappresenta il fulcro del percorso amministrativo del volontario nel sistema di Servizio Civile Universale. Essa consente di associare un volontario a un determinato progetto e a una specifica sede per un intervallo temporale definito.

La tabella memorizza inoltre lo stato dell'assegnazione, il monte ore settimanale previsto e i massimali relativi a permessi ordinari, malattie retribuite e malattie non retribuite.

Sono previsti vincoli di integrità per garantire la coerenza temporale dell'assegnazione, la validità dello stato e la non negatività dei valori numerici di controllo. Restano demandati alla logica dinamica del sistema il controllo di unicità dell'assegnazione attiva e la validità della coppia progetto-sede. La gestione dello storico in caso di variazione è invece supportata anche strutturalmente tramite l'attributo id_assegnazione_precedente, che collega la nuova assegnazione a quella da cui deriva.

Implementazione

Tabella Assegnazione

```
CREATE TABLE ASSEGNAZIONE (
      id_assegnazione NUMBER(10,0) PRIMARY KEY,
      id_volontario NUMBER(10,0) NOT NULL,
      id_progetto NUMBER(10,0) NOT NULL,
      id_sede NUMBER(10,0) NOT NULL,
      id_assegnazione_precedente NUMBER(10,0),
      data_inizio DATE NOT NULL,
      data_fine_prevista DATE,
      data_fine_effettiva DATE,
      stato_assegnazione VARCHAR2(20) NOT NULL,
      ore_settimanali_previste NUMBER(2,0) NOT NULL,
      max_permessi_ordinari NUMBER(2,0) DEFAULT 15 NOT NULL,
      max_malattie_retribuite NUMBER(2,0) DEFAULT 10 NOT NULL,
      max_malattie_non_retribuite NUMBER(2,0) DEFAULT 10 NOT NULL,

      CONSTRAINT fk_assegnazione_volontario
           FOREIGN KEY (id_volontario)
           REFERENCES VOLONTARIO(id_volontario),

      CONSTRAINT fk_assegnazione_progetto
           FOREIGN KEY (id_progetto)
           REFERENCES PROGETTO(id_progetto),

      CONSTRAINT fk_assegnazione_sede
           FOREIGN KEY (id_sede)
           REFERENCES SEDE(id_sede),

      CONSTRAINT fk_assegnazione_precedente
           FOREIGN KEY (id_assegnazione_precedente)
           REFERENCES ASSEGNAZIONE(id_assegnazione),

      CONSTRAINT uq_assegnazione_precedente
           UNIQUE (id_assegnazione_precedente),

      CONSTRAINT chk_assegnazione_precedente_diversa
           CHECK (
               id_assegnazione_precedente IS NULL
               OR id_assegnazione_precedente <> id_assegnazione
           ),

      CONSTRAINT chk_periodo_assegnazione_prev
           CHECK (data_fine_prevista IS NULL OR data_inizio <= data_fine_prevista),

      CONSTRAINT chk_periodo_assegnazione_eff
           CHECK (data_fine_effettiva IS NULL OR data_inizio <= data_fine_effettiva),

      CONSTRAINT chk_stato_assegnazione
           CHECK (stato_assegnazione IN ('ATTIVA', 'CONCLUSA', 'DECADUTA')),

      CONSTRAINT chk_ore_settimanali_previste
           CHECK (ore_settimanali_previste > 0),
      CONSTRAINT chk_max_permessi_ordinari
           CHECK (max_permessi_ordinari >= 0),
      CONSTRAINT chk_max_malattie_retribuite
           CHECK (max_malattie_retribuite >= 0),
      CONSTRAINT chk_max_malattie_non_retribuite
           CHECK (max_malattie_non_retribuite >= 0)
);
```

Implementazione

Tabella Formazione

La tabella **FORMAZIONE** rappresenta un evento formativo previsto all'interno del percorso di Servizio Civile Universale. Questa tabella consente di identificare univocamente ogni attività formativa tramite un codice numerico interno e di memorizzare le principali informazioni descrittive e organizzative, come titolo, tipologia, data della formazione, durata in ore e descrizione.

Ogni formazione è associata a un operatore, mediante una chiave esterna verso la tabella **OPERATORE**, che rappresenta la figura organizzativa responsabile dell'attività.

Sono previsti vincoli di integrità per garantire la correttezza della data, il fatto che la durata della formazione sia strettamente positiva e la validità della tipologia formativa.

```
CREATE TABLE FORMAZIONE (
    id_formazione NUMBER(10,0) PRIMARY KEY,
    id_operatore NUMBER(10,0) NOT NULL,
    titolo VARCHAR2(100) NOT NULL,
    tipologia VARCHAR2(30) NOT NULL,
    data_formazione DATE NOT NULL,
    durata_ore NUMBER(4,1) NOT NULL,
    descrizione VARCHAR2(255) NOT NULL,

    CONSTRAINT fk_formazione_operatore
        FOREIGN KEY (id_operatore)
        REFERENCES OPERATORE(id_operatore),

    CONSTRAINT chk_durata_ore_formazione
        CHECK (durata_ore > 0),

    CONSTRAINT chk_tipologia_formazione
        CHECK (tipologia IN ('GENERALE', 'SPECIFICA', 'INTEGRATIVA'))
);
```

Implementazione

Tabella Assegnazione_Formazione

La tabella **ASSEGNAZIONE_FORMAZIONE** rappresenta l'associazione tra una specifica assegnazione e un evento formativo. Essa deriva dalla trasformazione della relazione molti-a-molti tra **ASSEGNAZIONE** e **FORMAZIONE** e consente di registrare la partecipazione del volontario, nel contesto della propria assegnazione, a una determinata attività formativa.
La tabella memorizza inoltre informazioni descrittive sulla partecipazione, come lo stato di presenza, il numero di ore frequentate e le eventuali note di partecipazione.
La chiave primaria è composta dagli identificativi dell'assegnazione e della formazione. Sono previsti vincoli di integrità per garantire la validità del valore di presenza, la non negatività delle ore frequentate e la coerenza tra stato di presenza e numero di ore registrate.

```
CREATE TABLE ASSEGNAZIONE_FORMAZIONE (
    id_assegnazione NUMBER(10,0) NOT NULL,
    id_formazione NUMBER(10,0) NOT NULL,
    presenza VARCHAR2(15) NOT NULL,
    ore_frequentate NUMBER(4,1) DEFAULT 0 NOT NULL,
    note_partecipazione VARCHAR2(255),

    CONSTRAINT pk_assegnazione_formazione
        PRIMARY KEY (id_assegnazione, id_formazione),

    CONSTRAINT fk_assegnazione_formazione_assegnazione
        FOREIGN KEY (id_assegnazione)
        REFERENCES ASSEGNAZIONE(id_assegnazione),

    CONSTRAINT fk_assegnazione_formazione_formazione
        FOREIGN KEY (id_formazione)
        REFERENCES FORMAZIONE(id_formazione),

    CONSTRAINT chk_presenza_formazione
        CHECK (presenza IN ('PRESENTE', 'ASSENTE', 'PARZIALE')),

    CONSTRAINT chk_ore_frequentate
        CHECK (ore_frequentate >= 0),

    CONSTRAINT chk_coerenza_presenza_ore
        CHECK (
            (presenza = 'ASSENTE' AND ore_frequentate = 0)
            OR
            (presenza IN ('PRESENTE', 'PARZIALE') AND ore_frequentate > 0)
        )
);
```

Implementazione

Tabella Registro_Servizio

La tabella **REGISTRO_SERVIZIO** rappresenta il registro giornaliero delle attività svolte dal volontario nell'ambito di una specifica assegnazione. Essa consente di memorizzare, per ciascuna giornata, la data del servizio, la tipologia della giornata, il numero di ore svolte, le eventuali ore da recuperare, lo stato di validazione e note descrittive.

Ogni registrazione è associata a una sola assegnazione, mediante una chiave esterna verso la tabella **ASSEGNAZIONE**.

Sono previsti vincoli di integrità per garantire la correttezza dei valori registrati, l'assenza di duplicazioni per la stessa data e la coerenza tra tipo di giornata e ore svolte. Restano demandati alla logica dinamica del sistema i controlli cumulativi sui limiti di permessi, malattie, ore settimanali e decadenza amministrativa.

```
CREATE TABLE REGISTRO_SERVIZIO (
	id_registro NUMBER(10,0) PRIMARY KEY,
	id_assegnazione NUMBER(10,0) NOT NULL,
	data_servizio DATE NOT NULL,
	tipo_giornata VARCHAR2(30) NOT NULL,
	ore_svolte NUMBER(4,1) DEFAULT 0 NOT NULL,
	ore_da_recuperare NUMBER(4,1) DEFAULT 0 NOT NULL,
	stato_validazione VARCHAR2(15) NOT NULL,
	note VARCHAR2(255),
	CONSTRAINT fk_registro_servizio_assegnazione
		FOREIGN KEY (id_assegnazione)
		REFERENCES ASSEGNAZIONE(id_assegnazione),
	CONSTRAINT unq_registro_servizio
		UNIQUE (id_assegnazione, data_servizio),
	CONSTRAINT chk_tipo_giornata
		CHECK (tipo_giornata IN (
			'SERVIZIO',
			'MALATTIA_RETRIBUITA',
			'MALATTIA_NON_RETRIBUITA',
			'PERMESSO_ORDINARIO',
			'PERMESSO_STRAORDINARIO'
		)),
	CONSTRAINT chk_ore_svolte
		CHECK (ore_svolte >= 0),
	CONSTRAINT chk_ore_da_recuperare
		CHECK (ore_da_recuperare >= 0),
	CONSTRAINT chk_stato_validazione
		CHECK (stato_validazione IN ('DA_VALIDARE', 'VALIDATA', 'RIFIUTATA')),
	CONSTRAINT chk_coerenza_tipo_giornata_ore
		CHECK (
			(tipo_giornata = 'SERVIZIO' AND ore_svolte > 0)
			OR
			(tipo_giornata IN (
				'MALATTIA_RETRIBUITA',
				'MALATTIA_NON_RETRIBUITA',
				'PERMESSO_ORDINARIO',
				'PERMESSO_STRAORDINARIO'
			) AND ore_svolte = 0)
		),
	CONSTRAINT chk_ore_massime_giornaliere
		CHECK (ore_svolte <= 8)
);
```

Implementazione

Tabella Monitoraggio

La tabella **MONITORAGGIO** rappresenta la registrazione periodica delle osservazioni e delle valutazioni relative al percorso del volontario nell'ambito di una specifica assegnazione. Essa consente di memorizzare la data del monitoraggio, la tipologia del monitoraggio, l'esito e eventuali note descrittive.

Ogni monitoraggio è associato a una sola assegnazione, mediante una chiave esterna verso la tabella **ASSEGNAZIONE**.

Sono previsti vincoli di integrità per garantire la corretta valorizzazione della data, la validità dell'esito e la coerenza della tipologia di monitoraggio.

```
CREATE TABLE MONITORAGGIO (
      id_monitoraggio NUMBER(10,0) PRIMARY KEY,
      id_assegnazione NUMBER(10,0) NOT NULL,
      data_monitoraggio DATE NOT NULL,
      tipo_monitoraggio VARCHAR2(20) NOT NULL,
      esito VARCHAR2(20),
      note VARCHAR2(255),

      CONSTRAINT fk_monitoraggio_assegnazione FOREIGN KEY (id_assegnazione)
            REFERENCES ASSEGNAZIONE(id_assegnazione),

      CONSTRAINT chk_tipo_monitoraggio CHECK (tipo_monitoraggio IN ('INIZIALE', 'INTERMEDIO', 'FINALE')),

      CONSTRAINT chk_esito_monitoraggio CHECK (esito IS NULL OR esito IN ('POSITIVO', 'CRITICO', 'NEGATIVO'))
);
```

Implementazione

Tabella Richiesta_Variazione

La tabella **RICHIESTA_VARIAZIONE** rappresenta la richiesta formale di modifica relativa a una specifica assegnazione del volontario. Essa consente di memorizzare la data della richiesta, la tipologia della variazione, la motivazione, lo stato della richiesta e la data di validazione.

Ogni richiesta di variazione è associata a una sola assegnazione, mediante una chiave esterna verso la tabella **ASSEGNAZIONE**.

Sono previsti vincoli di integrità per garantire la corretta valorizzazione della data, la validità dello stato della richiesta e la coerenza della tipologia di variazione. Restano invece demandati alla logica dinamica del sistema i controlli legati all'approvazione della variazione, alla chiusura dell'assegnazione precedente e alla creazione della nuova assegnazione.

```
CREATE TABLE RICHIESTA_VARIAZIONE (

    id_richiesta NUMBER(10,0) PRIMARY KEY,
    id_assegnazione NUMBER(10,0) NOT NULL,
    data_richiesta DATE NOT NULL,
    tipo_variazione VARCHAR2(30) NOT NULL,
    motivazione VARCHAR2(255) NOT NULL,
    stato_richiesta VARCHAR2(20) NOT NULL,
    data_validazione DATE,

    CONSTRAINT fk_richiesta_variazione_assegnazione
        FOREIGN KEY (id_assegnazione)
        REFERENCES ASSEGNAZIONE(id_assegnazione),

    CONSTRAINT chk_tipo_variazione
        CHECK (tipo_variazione IN ('CAMBIO_SEDE', 'CAMBIO_PROGETTO', 'CAMBIO_ENTRAMBI')),

    CONSTRAINT chk_stato_richiesta
        CHECK (stato_richiesta IN ('IN_ATTESA', 'APPROVATA', 'RIFIUTATA'))

);
```

Data Manipulation Language (DDL)

Il Data Manipulation Language (DML) è una parte del linguaggio SQL (Structured Query Language) che consente agli utenti di interagire e manipolare i dati all'interno di un database. Il DML include comandi che permettono di inserire, aggiornare, eliminare e recuperare dati da tabelle di database. Le operazioni principali del DML
sono:

1. INSERT: Utilizzato per inserire nuovi record in una tabella.
2. UPDATE: Utilizzato per aggiornare i dati esistenti in una tabella.
3. DELETE: Utilizzato per eliminare uno o più record da una tabella.
4. SELECT: Utilizzato per recuperare dati da una o più tabelle.

Il DML è fondamentale per la gestione quotidiana dei dati in qualsiasi sistema di database. Consente agli utenti di interagire con i dati in modo dinamico, aggiornando e recuperando informazioni in base alle necessità aziendali. A seguito tutti gli INSERT relativi alle varie tabelle.

Inserimenti Ente

```
INSERT INTO ENTE (id_ente, denominazione, codice_fiscale, indirizzo_sede_legale, email)
VALUES (1, 'Associazione Orizzonti Solidali', '1234567890123456',
          'Via Roma 12, Napoli', 'info@orizzontisolidali.it');

INSERT INTO ENTE (id_ente, denominazione, codice_fiscale, indirizzo_sede_legale, email)
VALUES (2, 'Fondazione Cittadinanza Attiva', '2345678901234567',
          'Corso Garibaldi 45, Salerno', 'segreteria@cittadinanzaattiva.org');

INSERT INTO ENTE (id_ente, denominazione, codice_fiscale, indirizzo_sede_legale, email)
VALUES (3, 'Cooperativa Nuove Energie Sociali', '3456789012345678',
          'Piazza Municipio 8, Caserta', 'contatti@nuoveenergie.it');
```

Implementazione

Inserimenti Sede

```
INSERT INTO SEDE (id_sede, id_ente, nome_sede, indirizzo, comune, provincia, cap_sede)
VALUES (1, 1, 'Sede Centrale Napoli', 'Via Toledo 120', 'Napoli', 'NA', '80134');

INSERT INTO SEDE (id_sede, id_ente, nome_sede, indirizzo, comune, provincia, cap_sede)
VALUES (2, 1, 'Sede Vomero', 'Via Scarlatti 88', 'Napoli', 'NA', '80129');

INSERT INTO SEDE (id_sede, id_ente, nome_sede, indirizzo, comune, provincia, cap_sede)
VALUES (3, 1, 'Sede Casoria', 'Via Principe di Piemonte 45', 'Casoria', 'NA', '80026');

INSERT INTO SEDE (id_sede, id_ente, nome_sede, indirizzo, comune, provincia, cap_sede)
VALUES (4, 2, 'Sede Salerno Centro', 'Corso Vittorio Emanuele 75', 'Salerno', 'SA', '84122');

INSERT INTO SEDE (id_sede, id_ente, nome_sede, indirizzo, comune, provincia, cap_sede)
VALUES (5, 2, 'Sede Cava de Tirreni', 'Via XXV Luglio 30', 'Cava de Tirreni', 'SA', '84013');

INSERT INTO SEDE (id_sede, id_ente, nome_sede, indirizzo, comune, provincia, cap_sede)
VALUES (6, 2, 'Sede Battipaglia', 'Via Mazzini 14', 'Battipaglia', 'SA', '84091');

INSERT INTO SEDE (id_sede, id_ente, nome_sede, indirizzo, comune, provincia, cap_sede)
VALUES (7, 3, 'Sede Caserta Centro', 'Via Giuseppe Mazzini 52', 'Caserta', 'CE', '81100');

INSERT INTO SEDE (id_sede, id_ente, nome_sede, indirizzo, comune, provincia, cap_sede)
VALUES (8, 3, 'Sede Aversa', 'Via Roma 101', 'Aversa', 'CE', '81031');

INSERT INTO SEDE (id_sede, id_ente, nome_sede, indirizzo, comune, provincia, cap_sede)
VALUES (9, 3, 'Sede Santa Maria Capua Vetere', 'Corso Aldo Moro 19',
            'Santa Maria Capua Vetere', 'CE', '81055');
```

Implementazione

Inserimenti Progetto

```
-- P1
INSERT INTO PROGETTO (id_progetto, id_ente, titolo, area_intervento, descrizione, data_inizio, data_fine, stato_progetto)
  VALUES (1, 1,  'Giovani e Inclusione Sociale', 'ASSISTENZA',
                 'Progetto dedicato al supporto di giovani in condizioni di fragilita sociale.',
                 DATE '2025-01-10', NULL, 'ATTIVO');
-- P2
INSERT INTO PROGETTO (id_progetto, id_ente, titolo, area_intervento, descrizione, data_inizio, data_fine, stato_progetto)
  VALUES (2, 1,  'Educazione Digitale per Tutti', 'EDUCAZIONE',
                 'Percorso di alfabetizzazione digitale rivolto a minori, anziani e cittadini vulnerabili.',
                 DATE '2024-03-01', DATE '2025-02-28', 'CONCLUSO');
-- P3
INSERT INTO PROGETTO (id_progetto, id_ente, titolo, area_intervento, descrizione, data_inizio, data_fine, stato_progetto)
  VALUES (3, 1,  'Territorio e Partecipazione', 'CITTADINANZA_ATTIVA',
                 'Progetto orientato alla promozione della partecipazione civica e del volontariato locale.',
                 DATE '2025-04-01', NULL, 'ATTIVO');
-- P4
INSERT INTO PROGETTO (id_progetto, id_ente, titolo, area_intervento, descrizione, data_inizio, data_fine, stato_progetto)
  VALUES (4, 2,  'Supporto alle Fragilita Urbane', 'ASSISTENZA',
                 'Interventi di supporto sociale per persone e famiglie in contesti urbani complessi.',
                 DATE '2025-02-15', NULL, 'ATTIVO');
-- P5
INSERT INTO PROGETTO (id_progetto, id_ente, titolo, area_intervento, descrizione, data_inizio, data_fine, stato_progetto)
  VALUES (5, 2,  'Memoria e Comunita', 'PATRIMONIO_CULTURALE',
                 'Attivita di valorizzazione della memoria storica locale e del patrimonio culturale cittadino.',
                 DATE '2024-01-20', DATE '2024-12-20', 'CONCLUSO');
-- P6
INSERT INTO PROGETTO (id_progetto, id_ente, titolo, area_intervento, descrizione, data_inizio, data_fine, stato_progetto)
  VALUES (6, 2,  'Scuola Aperta e Innovazione', 'EDUCAZIONE',
                 'Azioni di supporto educativo extrascolastico e innovazione nei contesti formativi.',
                 DATE '2025-03-10', NULL, 'ATTIVO');
-- P7
INSERT INTO PROGETTO (id_progetto, id_ente, titolo, area_intervento, descrizione, data_inizio, data_fine, stato_progetto)
  VALUES (7, 3,  'Reti di Solidarieta Territoriale', 'ASSISTENZA',
                 'Costruzione di reti territoriali di sostegno tra enti, cittadini e volontari.',
                 DATE '2025-01-05', NULL, 'ATTIVO');
-- P8
INSERT INTO PROGETTO (id_progetto, id_ente, titolo, area_intervento, descrizione, data_inizio, data_fine, stato_progetto)
  VALUES (8, 3,  'Laboratori per la Cittadinanza', 'CITTADINANZA_ATTIVA',
                 'Laboratori territoriali per la promozione della cittadinanza consapevole e partecipata.',
                 DATE '2025-02-01', NULL, 'SOSPESO');
```

Implementazione

Inserimenti Operatore

```
-- 01
INSERT INTO OPERATORE (id_operatore, id_ente, nome, cognome, codice_fiscale, email, telefono, ruolo_operatore)
VALUES (1, 1, 'Luca', 'Esposito', 'LCSPSP850101H501', 'luca.esposito@orizzontisolidali.it', '3331000001', 'OLP');

-- 02
INSERT INTO OPERATORE (id_operatore, id_ente, nome, cognome, codice_fiscale, email, telefono, ruolo_operatore)
VALUES (2, 1, 'Maria', 'Ferraro', 'MRAFRR870202H501', 'maria.ferraro@orizzontisolidali.it', '3331000002', 'OLP');

-- 03
INSERT INTO OPERATORE (id_operatore, id_ente, nome, cognome, codice_fiscale, email, telefono, ruolo_operatore)
VALUES (3, 1, 'Giovanni', 'Romano', 'GVNRMN820303H501', 'giovanni.romano@orizzontisolidali.it', '3331000003', 'COORDINATORE');

-- 04
INSERT INTO OPERATORE (id_operatore, id_ente, nome, cognome, codice_fiscale, email, telefono, ruolo_operatore)
VALUES (4, 1, 'Anna', 'Greco', 'NNAGRC890404H501', 'anna.greco@orizzontisolidali.it', '3331000004', 'AMMINISTRATIVO');

-- 05
INSERT INTO OPERATORE (id_operatore, id_ente, nome, cognome, codice_fiscale, email, telefono, ruolo_operatore)
VALUES (5, 2, 'Francesco', 'Marino', 'FRNMRN840505H703', 'francesco.marino@cittadinanzaattiva.org', '3332000001', 'OLP');

-- 06
INSERT INTO OPERATORE (id_operatore, id_ente, nome, cognome, codice_fiscale, email, telefono, ruolo_operatore)
VALUES (6, 2, 'Sara', 'Rinaldi', 'SRARNL880606H703', 'sara.rinaldi@cittadinanzaattiva.org', '3332000002', 'COORDINATORE');

-- 07
INSERT INTO OPERATORE (id_operatore, id_ente, nome, cognome, codice_fiscale, email, telefono, ruolo_operatore)
VALUES (7, 2, 'Paolo', 'De Luca', 'PLADLC860707H703', 'paolo.deluca@cittadinanzaattiva.org', '3332000003', 'AMMINISTRATIVO');

-- 08
INSERT INTO OPERATORE (id_operatore, id_ente, nome, cognome, codice_fiscale, email, telefono, ruolo_operatore)
VALUES (8, 3, 'Elena', 'Coppola', 'LNECPL900808B963', 'elena.coppola@nuoveenergie.it', '3333000001', 'OLP');

-- 09
INSERT INTO OPERATORE (id_operatore, id_ente, nome, cognome, codice_fiscale, email, telefono, ruolo_operatore)
VALUES (9, 3, 'Davide', 'Mancini', 'DVDMCN830909B963', 'davide.mancini@nuoveenergie.it', '3333000002', 'COORDINATORE');

-- 010
INSERT INTO OPERATORE (id_operatore, id_ente, nome, cognome, codice_fiscale, email, telefono, ruolo_operatore)
VALUES (10, 3, 'Chiara', 'Russo', 'CHRRSS910101B963', 'chiara.russo@nuoveenergie.it', '3333000003', 'AMMINISTRATIVO');
```

Implementazione

Inserimenti Volontario

```
-- V1
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (1, 'Alessandro', 'Vitale', 'LSNVTL000101H501', DATE '2000-01-01', 'alessandro.vitale@email.it', '3471000001', 'Napoli', 'ATTIVO');
-- V2
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (2, 'Martina', 'Lombardi', 'MRTLMB010202H501', DATE '2001-02-02', 'martina.lombardi@email.it', '3471000002', 'Napoli', 'ATTIVO');
-- V3
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (3, 'Simone', 'Giordano', 'SMNGRD990303H703', DATE '1999-03-03', 'simone.giordano@email.it', '3471000003', 'Salerno', 'ATTIVO');
-- V4
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (4, 'Giulia', 'Moretti', 'GLIMRT020404B963', DATE '2002-04-04', 'giulia.moretti@email.it', '3471000004', 'Caserta', 'ATTIVO');
-- V5
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (5, 'Antonio', 'Ricci', 'NTNRCC980505H501', DATE '1998-05-05', 'antonio.ricci@email.it', '3471000005', 'Casoria', 'ATTIVO');
-- V6
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (6, 'Federica', 'Testa', 'FDRTST000606H703', DATE '2000-06-06', 'federica.testa@email.it', '3471000006', 'Battipaglia', 'ATTIVO');
-- V7
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (7, 'Davide', 'Serra', 'DVDSRR970707B963', DATE '1997-07-07', 'davide.serra@email.it', '3471000007', 'Aversa', 'ATTIVO');
-- V8
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (8, 'Chiara', 'Palmieri', 'CHRPLM010808H501', DATE '2001-08-08', 'chiara.palmieri@email.it', '3471000008', 'Napoli', 'ATTIVO');
-- V9
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (9, 'Francesca', 'Leone', 'FRNLNE990909H703', DATE '1999-09-09', 'francesca.leone@email.it', '3471000009', 'Cava de Tirreni', 'ATTIVO');
-- V10
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (10, 'Marco', 'D Amico', 'MRCDMC000110B963', DATE '2000-01-10', 'marco.damico@email.it', '3471000010', 'Santa Maria Capua Vetere', 'ATTIVO');
-- V11
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (11, 'Valeria', 'Conti', 'VLRCNT980211H501', DATE '1998-02-11', 'valeria.conti@email.it', '3471000011', 'Napoli', 'ATTIVO');
-- V12
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (12, 'Gabriele', 'Fiore', 'GBRFR0990312H703', DATE '1999-03-12', 'gabriele.fiore@email.it', '3471000012', 'Salerno', 'CONCLUSO');
-- V13
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (13, 'Elisa', 'Ruggiero', 'LSRGGR010413B963', DATE '2001-04-13', 'elisa.ruggiero@email.it', '3471000013', 'Caserta', 'ATTIVO');
-- V14
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (14, 'Matteo', 'Parisi', 'MTTPRS000514H501', DATE '2000-05-14', 'matteo.parisi@email.it', '3471000014', 'Napoli', 'RINUNCIATARIO');
-- V15
INSERT INTO VOLONTARIO (id_volontario, nome, cognome, codice_fiscale, data_nascita, email, telefono, comune_residenza, stato_volontario)
  VALUES (15, 'Serena', 'De Rosa', 'SRNDRS990615H703', DATE '1999-06-15', 'serena.derosa@email.it', '3471000015', 'Salerno', 'DECADUTO');
```

Implementazione

Inserimenti Progetto_Sede

```
-- P1 in S1
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (1, 1, 6);

-- P1 in S2
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (1, 2, 4);

-- P2 in S1
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (2, 1, 0);

-- P2 in S3
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (2, 3, 3);

-- P3 in S2
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (3, 2, 5);

-- P3 in S3
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (3, 3, 2);

-- P4 in S4
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (4, 4, 7);

-- P4 in S5
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (4, 5, 3);

-- P5 in S4
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (5, 4, 2);
-- P6 in S5
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (6, 5, 6);
-- P6 in S6
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (6, 6, 4);
-- P7 in S7
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (7, 7, 5);
-- P7 in S8
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (7, 8, 3);
-- P8 in S9
INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
  VALUES (8, 9, 4);
```

Implementazione

Inserimenti Formazione

```
-- F1
INSERT INTO FORMAZIONE (id_formazione, id_operatore, titolo, tipologia, data_formazione, durata_ore, descrizione)
  VALUES (1, 1, 'Introduzione al Servizio Civile', 'GENERALE',
          DATE '2025-01-20', 4.0,
          'Modulo introdutivo sui principi, i valori e l organizzazione del serviziocivile.');
-- F2
INSERT INTO FORMAZIONE (id_formazione, id_operatore, titolo, tipologia, data_formazione, durata_ore, descrizione)
  VALUES (2, 2, 'Comunicazione con l Utenza Fragile', 'SPECIFICA',
          DATE '2025-02-05', 6.0,
          'Percorso formativo dedicato alla comunicazione efficace con utenti in condizioni di fragilita.');
-- F3
INSERT INTO FORMAZIONE (id_formazione, id_operatore, titolo, tipologia, data_formazione, durata_ore, descrizione)
  VALUES (3, 3, 'Cittadinanza Attiva e Partecipazione', 'INTEGRATIVA',
          DATE '2025-03-12', 3.5,
          'Laboratorio sul ruolo del volontario nella promozione della partecipazione civica.');
-- F4
INSERT INTO FORMAZIONE (id_formazione, id_operatore, titolo, tipologia, data_formazione, durata_ore, descrizione)
  VALUES (4, 5, 'Tutela dei Minori e Contesti Educativi', 'SPECIFICA',
          DATE '2025-02-18', 5.0,
          'Formazione operativa sui contesti educativi e sulla tutela dei minori.');
-- F5
INSERT INTO FORMAZIONE (id_formazione, id_operatore, titolo, tipologia, data_formazione, durata_ore, descrizione)
  VALUES (5, 6, 'Organizzazione del Progetto e Lavoro in Rete', 'GENERALE',
          DATE '2025-03-03', 4.5,
          'Modulo sulla gestione organizzativa del progetto e sul coordinamento tra soggetti coinvolti.');
-- F6
INSERT INTO FORMAZIONE (id_formazione, id_operatore, titolo, tipologia, data_formazione, durata_ore, descrizione)
  VALUES (6, 7, 'Strumenti Digitali per l Inclusione', 'INTEGRATIVA',
          DATE '2025-04-10', 3.0,
          'Formazione sull utilizzo di strumenti digitali a supporto dell inclusione sociale.');
-- F7
INSERT INTO FORMAZIONE (id_formazione, id_operatore, titolo, tipologia, data_formazione, durata_ore, descrizione)
  VALUES (7, 8, 'Animazione Territoriale e Comunita', 'SPECIFICA',
          DATE '2025-02-25', 5.5,
          'Percorso dedicato alle tecniche di animazione territoriale e coinvolgimento della comunita.');
-- F8
INSERT INTO FORMAZIONE (id_formazione, id_operatore, titolo, tipologia, data_formazione, durata_ore, descrizione)
  VALUES (8, 9, 'Monitoraggio del Percorso del Volontario', 'GENERALE',
          DATE '2025-03-20', 4.0,
          'Formazione sulle attivita di osservazione, valutazione e monitoraggio del percorso individuale.');
-- F9
INSERT INTO FORMAZIONE (id_formazione, id_operatore, titolo, tipologia, data_formazione, durata_ore, descrizione)
  VALUES (9, 10, 'Patrimonio Culturale e Valorizzazione Locale', 'SPECIFICA',
          DATE '2025-04-08', 6.5,
          'Modulo sulla valorizzazione del patrimonio culturale e delle risorse del territorio.');
-- F10
INSERT INTO FORMAZIONE (id_formazione, id_operatore, titolo, tipologia, data_formazione, durata_ore, descrizione)
  VALUES (10, 4, 'Gestione Documentale e Procedure Interne', 'INTEGRATIVA',
          DATE '2025-05-06', 2.5,
          'Formazione interna sulle procedure amministrative e sulla gestione documentale del progetto.');
```

Implementazione

Inserimenti Assegnazione

```
-- A1
INSERT INTO ASSEGNAZIONE (
       id_assegnazione, id_volontario, id_progetto, id_sede,
       data_inizio, data_fine_prevista, data_fine_effettiva,
       stato_assegnazione, ore_settimanali_previste,
       max_permessi_ordinari, max_malattie_retribuite, max_malattie_non_retribuite
)
VALUES (
       1, 1, 1, 1,
       DATE '2025-01-15', DATE '2025-12-31', NULL,
       'ATTIVA', 35,
       15, 10, 10
);

-- A2
INSERT INTO ASSEGNAZIONE (
       id_assegnazione, id_volontario, id_progetto, id_sede,
       data_inizio, data_fine_prevista, data_fine_effettiva,
       stato_assegnazione, ore_settimanali_previste,
       max_permessi_ordinari, max_malattie_retribuite, max_malattie_non_retribuite
)
VALUES (
       2, 2, 1, 2,
       DATE '2025-02-01', DATE '2025-12-31', NULL,
       'ATTIVA', 35,
       15, 10, 10
);

-- A3
INSERT INTO ASSEGNAZIONE (
       id_assegnazione, id_volontario, id_progetto, id_sede,
       data_inizio, data_fine_prevista, data_fine_effettiva,
       stato_assegnazione, ore_settimanali_previste,
       max_permessi_ordinari, max_malattie_retribuite, max_malattie_non_retribuite
)
VALUES (
       3, 3, 4, 4,
       DATE '2025-02-20', DATE '2025-04-30', DATE '2025-04-30',
       'CONCLUSA', 35,
       15, 10, 10
);
-- A4
INSERT INTO ASSEGNAZIONE (
       id_assegnazione, id_volontario, id_progetto, id_sede,
       data_inizio, data_fine_prevista, data_fine_effettiva,
       stato_assegnazione, ore_settimanali_previste,
       max_permessi_ordinari, max_malattie_retribuite, max_malattie_non_retribuite
)
VALUES (   4,  3,  5,
         DATE '2025-05-05', DATE '2025-12-31', NULL,
         'ATTIVA', 35,
         15, 10, 10
);
-- A5
INSERT INTO ASSEGNAZIONE (
       id_assegnazione, id_volontario, id_progetto, id_sede,
)      data_inizio, data_fine_prevista, data_fine_effettiva,
VALUES (   5,  4,  7,  7,
         DATE '2025-01-20', DATE '2025-12-31', NULL,
         'ATTIVA', 35,
         15, 10, 10
```

Implementazione

Inserimenti Assegnazione

```
-- A6
INSERT INTO ASSEGNAZIONE (
       id_assegnazione, id_volontario, id_progetto, id_sede,
       data_inizio, data_fine_prevista, data_fine_effettiva,
       stato_assegnazione, ore_settimanali_previste,
       max_permessi_ordinari, max_malattie_retribbuite, max_malattie_non_retribuite
)
VALUES (
       6, 5, 3, 2,
       DATE '2025-04-05', DATE '2025-12-31', NULL,
       'ATTIVA', 35,
       15, 10, 10
);

-- A7
INSERT INTO ASSEGNAZIONE (
       id_assegnazione, id_volontario, id_progetto, id_sede,
       data_inizio, data_fine_prevista, data_fine_effettiva,
       stato_assegnazione, ore_settimanali_previste,
       max_permessi_ordinari, max_malattie_retribbuite, max_malattie_non_retribuite
)
VALUES (
       7, 6, 6, 6,
       DATE '2025-03-15', DATE '2025-12-31', NULL,
       'ATTIVA', 35,
       15, 10, 10
);

-- A8
INSERT INTO ASSEGNAZIONE (
       id_assegnazione, id_volontario, id_progetto, id_sede,
       data_inizio, data_fine_prevista, data_fine_effettiva,
       stato_assegnazione, ore_settimanali_previste,
       max_permessi_ordinari, max_malattie_retribbuite, max_malattie_non_retribuite
)
VALUES (
       8, 7, 7, 8,
       DATE '2025-01-25', DATE '2025-12-31', NULL,
       'ATTIVA', 35,
       15, 10, 10
);

-- A9
INSERT INTO ASSEGNAZIONE (
       id_assegnazione, id_volontario, id_progetto, id_sede,
       data_inizio, data_fine_prevista, data_fine_effettiva,
       stato_assegnazione, ore_settimanali_previste,
       max_permessi_ordinari, max_malattie_retribbuite, max_malattie_non_retribuite
)
VALUES (
       9, 8, 1, 1,
       DATE '2025-02-10', DATE '2025-12-31', NULL,
       'ATTIVA', 35,
       15, 10, 10
);

-- A10
INSERT INTO ASSEGNAZIONE (
       10, 9, 4, 5,
       DATE '2025-03-01', DATE '2025-12-31', NULL,
       'ATTIVA', 35,
       15, 10, 10
);
```

Implementazione

Inserimenti Assegnazione

```
-- A15
INSERT INTO ASSEGNAZIONE (
      id_assegnazione, id_volontario, id_progetto, id_sede,
      data_inizio, data_fine_prevista, data_fine_effettiva,
      stato_assegnazione, ore_settimanali_previste,
      max_permessi_ordinari, max_malattie_retribuite, max_malattie_non_retribuite
)
VALUES (
      15, 14, 3, 3,
      DATE '2025-04-10', DATE '2025-12-31', DATE '2025-05-10',
      'DECADUTA', 35,
      15, 10, 10
);

-- A16
INSERT INTO ASSEGNAZIONE (
      id_assegnazione, id_volontario, id_progetto, id_sede,
      data_inizio, data_fine_prevista, data_fine_effettiva,
      stato_assegnazione, ore_settimanali_previste,
      max_permessi_ordinari, max_malattie_retribuite, max_malattie_non_retribuite
)
VALUES (
      16, 15, 4, 4,
      DATE '2025-02-20', DATE '2025-12-31', DATE '2025-04-01',
      'DECADUTA', 35,
      15, 10, 10
);

-- A17
INSERT INTO ASSEGNAZIONE (
      id_assegnazione, id_volontario, id_progetto, id_sede,
      data_inizio, data_fine_prevista, data_fine_effettiva,
      stato_assegnazione, ore_settimanali_previste,
      max_permessi_ordinari, max_malattie_retribuite, max_malattie_non_retribuite
)
VALUES (
      17, 11, 1, 2,
      DATE '2025-03-10', DATE '2025-12-31', NULL,
      'ATTIVA', 35,
      15, 10, 10
);
-- A18
INSERT INTO ASSEGNAZIONE (
      id_assegnazione, id_volontario, id_progetto, id_sede,
      data_inizio, data_fine_prevista, data_fine_effettiva,
      stato_assegnazione, ore_settimanali_previste,
      max_permessi_ordinari, max_malattie_retribuite, max_malattie_non_retribuite
)
VALUES (
      18, 13, 7, 7,
      DATE '2025-03-01', DATE '2025-12-31', NULL,
      'ATTIVA', 35,
      15, 10, 10
);
```

Implementazione

Inserimenti Assegnazione_Formazione

```
-- A1
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (1, 1, 'PRESENTE', 4.0, 'Partecipazione completa al modulo introduttivo.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (1, 2, 'PARSENTE', 6.0, 'Partecipazione regolare alla formazione specifica.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (1, 3, 'PARZIALE', 2.0, 'Uscita anticipata per esigenze personali.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (1, 10, 'PRESENTE', 2.5, 'Presenza completa al modulo integrativo.');

-- A2
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (2, 2, 'PRESENTE', 6.0, 'Frequenza completa.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (2, 3, 'ASSENTE', 0, 'Assenza comunicata.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (2, 10, 'PRESENTE', 2.5, 'Presenza completa al modulo integrativo.');

-- A3
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (3, 5, 'PRESENTE', 4.5, 'Partecipazione completa.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (3, 6, 'PARZIALE', 2.0, 'Ha seguito solo una parte del modulo.');

-- A4
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (4, 10, 'PRESENTE', 2.5, 'Partecipazione completa dopo l avvio della nuova assegnazione.');

-- A5
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (5, 7, 'PRESENTE', 5.5, 'Ottima partecipazione.');
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (5, 8, 'PRESENTE', 4.0, 'Presenza regolare per l intero incontro.');
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (5, 9, 'PARZIALE', 3.0, 'Partecipazione parziale.');
```

Implementazione

Inserimenti Assegnazione_Formazione

```
-- A6
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (6, 6, 'PRESENTE', 3.0, 'Partecipazione completa al modulo integrativo.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (6, 10, 'PRESENTE', 2.5, 'Presenza completa.');

-- A7
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (7, 6, 'PRESENTE', 3.0, 'Buona partecipazione al modulo integrativo.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (7, 10, 'ASSENTE', 0, 'Assenza giustificata.');

-- A8
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (8, 7, 'ASSENTE', 0, 'Assenza registrata e giustificata.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (8, 8, 'PRESENTE', 4.0, 'Partecipazione completa.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (8, 9, 'PRESENTE', 6.5, 'Presenza piena al modulo specialistico.');

-- A9
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (9, 2, 'ASSENTE', 0, 'Assente alla formazione specifica.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (9, 3, 'PRESENTE', 3.5, 'Partecipazione attiva.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (9, 10, 'PARZIALE', 1.5, 'Partecipazione ridotta.');

-- A10
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (10, 5, 'PRESENTE', 4.5, 'Frequenza completa.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (10, 6, 'ASSENTE', 0, 'Assenza per indisponibilita.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (10, 10, 'PRESENTE', 2.5, 'Partecipazione al modulo integrativo.');
```

Implementazione

Inserimenti Assegnazione_Formazione

```
-- A11
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (11, 6, 'PRESENTE', 3.0, 'Partecipazione regolare.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (11, 9, 'PRESENTE', 6.5, 'Partecipazione completa al modulo specialistico.');

-- A12
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (12, 1, 'PRESENTE', 4.0, 'Presenza completa.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (12, 2, 'PARZIALE', 3.0, 'Ha seguito solo parte della formazione.');

-- A14
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (14, 1, 'ASSENTE', 0, 'Assente al primo modulo.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (14, 2, 'PRESENTE', 6.0, 'Frequenza completa.');

-- A15
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
  VALUES (15, 10, 'PRESENTE', 2.5, 'Partecipazione completa prima della chiusura anticipata.');
```

Implementazione

Inserimenti Assegnazione_Formazione

```
-- A16
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (16, 5, 'PARZIALE', 2.0, 'Partecipazione parziale prima della decadenza.');

-- A17
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (17, 3, 'PRESENTE', 3.5, 'Partecipazione piena al laboratorio.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (17, 10, 'PRESENTE', 2.5, 'Presenza completa.');

-- A18
INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (18, 7, 'PRESENTE', 5.5, 'Partecipazione completa.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (18, 8, 'PRESENTE', 4.0, 'Partecipazione completa.');

INSERT INTO ASSEGNAZIONE_FORMAZIONE (id_assegnazione, id_formazione, presenza, ore_frequentate, note_partecipazione)
   VALUES (18, 9, 'PARZIALE', 3.5, 'Partecipazione parziale con uscita anticipata.');
```

Implementazione

Inserimenti Registro_Servizio

```
-- A1
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (1, 1, DATE '2025-01-21', 'SERVIZIO', 7.0, 0, 'VALIDATA',
  'Prima giornata operativa regolarmente svolta.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (2, 1, DATE '2025-01-22', 'SERVIZIO', 5.0,  2.0, 'VALIDATA',
  'Uscita anticipata con recupero previsto.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (3, 1, DATE '2025-01-23', 'PERMESSO_ORDINARIO', 0,  0, 'VALIDATA',
  'Permesso ordinario autorizzato.');

-- A2
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (4, 2, DATE '2025-02-06', 'SERVIZIO', 8.0, 0, 'VALIDATA',
  'Giornata piena di servizio.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (5, 2, DATE '2025-02-07', 'MALATTIA_RETRIBUITA', 0, 0, 'VALIDATA',
  'Assenza per malattia con certificazione.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (6, 2, DATE '2025-02-10', 'SERVIZIO', 6.0, 1.0, 'DA_VALIDARE',
  'Servizio svolto con recupero residuo.');

-- A3
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (7, 3, DATE '2025-03-05', 'SERVIZIO', 7.0, 0, 'VALIDATA',
  'Attivita ordinaria svolta regolarmente.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (8, 3, DATE '2025-03-06', 'PERMESSO_STRAORDINARIO', 0, 0, 'VALIDATA',
  'Permesso straordinario con motivazione approvata.');

-- A4
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (9, 4, DATE '2025-05-06', 'SERVIZIO', 8.0, 0, 'VALIDATA',
  'Avvio regolare della nuova assegnazione.');
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (10, 4, DATE '2025-05-07', 'SERVIZIO', 4.0,  3.0, 'DA_VALIDARE',
  'Presenza ridotta con ore da recuperare.');

-- A5
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (11, 5, DATE '2025-02-26', 'SERVIZIO', 8.0, 0, 'VALIDATA',
  'Servizio completato regolarmente.');
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (12, 5, DATE '2025-03-03', 'MALATTIA_NON_RETRIBUITA', 0, 0, 'VALIDATA',
  'Giornata di malattia non retribuita.');
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio, tipo_giornata,
  ore_svolte, ore_da_recuperare, stato_validazione, note)
VALUES (13, 5, DATE '2025-03-04', 'SERVIZIO', 6.0, 0, 'RIFIUTATA',
  'Registrazione respinta per errore nella descrizione.');
```

Implementazione

Inserimenti Registro_Servizio

```
-- A6
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
  tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
  VALUES (14, 6, DATE '2025-04-11', 'SERVIZIO', 7.0, 0, 'VALIDATA',
    'Giornata svolta correttamente.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
  tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
  VALUES (15, 6, DATE '2025-04-12', 'PERMESSO_ORDINARIO', 0, 0, 'VALIDATA',
    'Permesso ordinario richiesto e concesso.');

-- A7
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
  tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
  VALUES (16, 7, DATE '2025-03-18', 'SERVIZIO', 8.0, 0, 'VALIDATA',
    'Giornata piena.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
  tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
  VALUES (17, 7, DATE '2025-03-19', 'SERVIZIO', 5.0, 2.0, 'DA_VALIDARE',
    'Riduzione oraria con recupero previsto.');

-- A8
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
  tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
  VALUES (18, 8, DATE '2025-03-21', 'SERVIZIO', 7.5, 0, 'VALIDATA',
    'Servizio ordinario.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
  tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
  VALUES (19, 8, DATE '2025-03-22', 'PERMESSO_STRAORDINARIO', 0, 0, 'VALIDATA',
    'Permesso straordinario autorizzato.');

-- A9
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
  tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
  VALUES (20, 9, DATE '2025-03-13', 'SERVIZIO', 8.0, 0, 'VALIDATA',
    'Presenza completa.');
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
  tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
  VALUES (21, 9, DATE '2025-03-14', 'MALATTIA_RETRIBUITA', 0, 0, 'VALIDATA',
    'Malattia retribuita registrata.');

-- A10
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
  tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
  VALUES (22, 10, DATE '2025-03-11', 'SERVIZIO', 6.0, 1.0, 'VALIDATA',
    'Servizio svolto con lieve recupero residuo.');
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
  tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
  VALUES (23, 10, DATE '2025-03-12', 'PERMESSO_ORDINARIO', 0, 0, 'VALIDATA',
    'Permesso ordinario.');
```

Implementazione

Inserimenti Registro_Servizio

```
-- A10
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
    tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
 VALUES (22, 10, DATE '2025-03-11', 'SERVIZIO', 6.0, 1.0, 'VALIDATA',
    'Servizio svolto con lieve recupero residuo.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
    tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
 VALUES (23, 10, DATE '2025-03-12', 'PERMESSO_ORDINARIO', 0, 0, 'VALIDATA',
    'Permesso ordinario.');

-- A11
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
    tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
 VALUES (24, 11, DATE '2025-04-09', 'SERVIZIO', 8.0, 0, 'VALIDATA',
    'Servizio regolare.');

-- A12
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
    tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
 VALUES (25, 12, DATE '2025-02-20', 'SERVIZIO', 7.0, 0, 'VALIDATA',
    'Ultime giornate del progetto concluse regolarmente.');

-- A13
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
    tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
 VALUES (33, 13, DATE '2024-11-18', 'SERVIZIO', 7.0, 0, 'VALIDATA',
    'Giornata ordinaria di servizio.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
    tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
 VALUES (34, 13, DATE '2024-11-19', 'PERMESSO_ORDINARIO', 0, 0, 'VALIDATA',
    'Permesso ordinario autorizzato.');

-- A14
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
    tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
 VALUES (26, 14, DATE '2025-02-10', 'SERVIZIO', 6.0, 0, 'VALIDATA',
    'Servizio svolto prima della conclusione.');

-- A15
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione, data_servizio,
    tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note)
 VALUES (27, 15, DATE '2025-05-05', 'SERVIZIO', 4.0, 3.0, 'VALIDATA',
    'Riduzione oraria nel periodo finale.');
```

Implementazione

Inserimenti Registro_Servizio

```
-- A16
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione,
data_servizio, tipo_giornata, ore_svolte, ore_da_recuperare,
stato_validazione, note)
VALUES (28, 16, DATE '2025-03-25', 'MALATTIA_NON_RETRIBUITA',
0, 0, 'VALIDATA', 'Giornata di malattia non retribuita.');

-- A17
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione,
data_servizio, tipo_giornata, ore_svolte, ore_da_recuperare,
stato_validazione, note)
VALUES (29, 17, DATE '2025-05-07', 'SERVIZIO', 8.0, 0,
'VALIDATA', 'Giornata ordinaria.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione,
data_servizio, tipo_giornata, ore_svolte, ore_da_recuperare,
stato_validazione, note)
VALUES (30, 17, DATE '2025-05-08', 'SERVIZIO', 5.0, 2.0,
'DA_VALIDARE', 'Recupero da pianificare entro la settimana.');

-- A18
INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione,
data_servizio, tipo_giornata, ore_svolte, ore_da_recuperare,
stato_validazione, note)
VALUES (31, 18, DATE '2025-04-15', 'SERVIZIO', 7.0, 0,
'VALIDATA', 'Giornata regolare di servizio.');

INSERT INTO REGISTRO_SERVIZIO (id_registro, id_assegnazione,
data_servizio, tipo_giornata, ore_svolte, ore_da_recuperare,
stato_validazione, note)
VALUES (32, 18, DATE '2025-04-16', 'PERMESSO_STRAORDINARIO',
0, 0, 'VALIDATA', 'Permesso straordinario con motivazione
documentata.');
```

Implementazione

Inserimenti Monitoraggio

```
-- M1
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (1, 1, DATE '2025-02-15', 'INIZIALE', 'POSITIVO', 'Inserimento iniziale positivo e buona
                integrazione nel contesto operativo.');

-- M2
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (2, 1, DATE '2025-06-20', 'INTERMEDIO', 'POSITIVO', 'Andamento regolare del percorso e
                partecipazione costante.');

-- M3
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (3, 2, DATE '2025-03-10', 'INIZIALE', 'POSITIVO', 'Avvio positivo del percorso con buona
                risposta alle attivita proposte.');

-- M4
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (4, 3, DATE '2025-04-25', 'FINALE', 'POSITIVO', 'Percorso concluso con risultati soddisfacenti.');

-- M5
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (5, 4, DATE '2025-06-10', 'INIZIALE', 'POSITIVO', 'Nuova assegnazione avviata regolarmente
                dopo la precedente conclusione.');
```

Implementazione

Inserimenti Monitoraggio

```
-- M6
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (6, 5, DATE '2025-03-30', 'INTERMEDIO', 'POSITIVO', 'Partecipazione costante e buon rapporto
                con il gruppo di lavoro.');

-- M7
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (7, 6, DATE '2025-05-05', 'INIZIALE', 'POSITIVO', 'Monitoraggio iniziale positivo e obiettivi
                ben compresi.');

-- M8
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (8, 7, DATE '2025-04-18', 'INTERMEDIO', 'CRITICO', 'Emerse difficolta nella continuita
                della presenza e nella gestione delle attivita.');

-- M9
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (9, 8, DATE '2025-04-22', 'INTERMEDIO', 'POSITIVO', 'Andamento complessivamente buono
                con partecipazione sufficiente.');
-- M10
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (10, 9, DATE '2025-04-10', 'INTERMEDIO', 'POSITIVO', 'Percorso regolare con buona partecipazione
                 alle attivita.');
```

Implementazione

Inserimenti Monitoraggio

```
-- M11
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (11, 10, DATE '2025-05-15', 'INTERMEDIO', 'POSITIVO', 'Percorso stabile e buona capacita
                    di adattamento al contesto.');

-- M12
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (12, 11, DATE '2025-05-25', 'INTERMEDIO', 'POSITIVO', 'Andamento complessivamente
                    stabile e buona integrazione.');

-- M13
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (13, 12, DATE '2025-02-25', 'FINALE', 'POSITIVO', 'Conclusione regolare del percorso
                    con obiettivi raggiunti.');

-- M14
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (14, 13, DATE '2024-12-15', 'FINALE', 'POSITIVO', 'Monitoraggio finale positivo e
                    chiusura ordinata del progetto.');

-- M15
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (15, 14, DATE '2025-02-14', 'FINALE', 'POSITIVO', 'Conclusione ordinata del percorso
                    con esito soddisfacente.');
```

Implementazione

Inserimenti Monitoraggio

```
-- M16
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (16, 15, DATE '2025-05-08', 'FINALE', 'NEGATIVO', 'Decadenza preceduta da ripetute
                    criticita organizzative e di presenza.');

-- M17
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (17, 16, DATE '2025-03-30', 'FINALE', 'NEGATIVO', 'Esito negativo associato alla
                    chiusura anticipata del percorso.');

-- M18
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (18, 17, DATE '2025-06-05', 'INIZIALE', 'POSITIVO', 'Nuova assegnazione avviata con
                    buon inserimento nel gruppo.');

-- M19
INSERT INTO MONITORAGGIO (id_monitoraggio, id_assegnazione, data_monitoraggio, tipo_monitoraggio, esito, note)
VALUES (19, 18, DATE '2025-05-20', 'INTERMEDIO', 'POSITIVO', 'Percorso attuale positivo dopo
                    la riassegnazione.');
```

Implementazione

Inserimenti Richiesta_Variazione

```
-- R1
INSERT INTO RICHIESTA_VARIAZIONE (
    id_richiesta, id_assegnazione, data_richiesta,
    tipo_variazione, motivazione, stato_richiesta, data_validazione
)
VALUES (
    1, 3, DATE '2025-04-28',
    'CAMBIO_ENTRAMBI',
    'Richiesta di variazione per migliore compatibilita tra sede operativa e attivita progettuali.',
    'APPROVATA', DATE '2025-05-02'
);

-- R2
INSERT INTO RICHIESTA_VARIAZIONE (
    id_richiesta, id_assegnazione, data_richiesta,
    tipo_variazione, motivazione, stato_richiesta, data_validazione
)
VALUES (
    2, 12, DATE '2025-02-27',
    'CAMBIO_ENTRAMBI',
    'Richiesta di riassegnazione a seguito di esigenze organizzative e disponibilita su altro progetto.',
    'APPROVATA', DATE '2025-03-05'
);

-- R3
INSERT INTO RICHIESTA_VARIAZIONE (
    id_richiesta, id_assegnazione, data_richiesta,
    tipo_variazione, motivazione, stato_richiesta, data_validazione
)
VALUES (
    3, 14, DATE '2025-02-16',
    'CAMBIO_ENTRAMBI',
    'Richiesta di riallocazione del volontario su nuova sede e nuovo progetto per continuita del percorso.',
    'APPROVATA', DATE '2025-02-25'
);

-- R4
INSERT INTO RICHIESTA_VARIAZIONE (
    id_richiesta, id_assegnazione, data_richiesta,
    tipo_variazione, motivazione, stato_richiesta, data_validazione
)
VALUES (
    4, 1, DATE '2025-06-25',
    'CAMBIO_SEDE',
    'Richiesta di trasferimento per maggiore vicinanza territoriale alla residenza.',
    'RIFIUTATA', DATE '2025-06-30'
);

-- R5
INSERT INTO RICHIESTA_VARIAZIONE (
    id_richiesta, id_assegnazione, data_richiesta,
    tipo_variazione, motivazione, stato_richiesta, data_validazione
)
VALUES (
    5, 2, DATE '2025-07-05',
    'CAMBIO_PROGETTO',
    'Richiesta di spostamento verso attivita maggiormente coerenti con il profilo formativo del volontario.',
    'IN_ATTESA', NULL
);

-- R6
INSERT INTO RICHIESTA_VARIAZIONE (
    id_richiesta, id_assegnazione, data_richiesta,
    tipo_variazione, motivazione, stato_richiesta, data_validazione
)
VALUES (
    6, 5, DATE '2025-05-12',
    'CAMBIO_SEDE',
    'Richiesta presentata per difficolta logistiche negli spostamenti verso la sede assegnata.',
    'RIFIUTATA', DATE '2025-05-18'
);
```

Trigger

I trigger sono un tipo di meccanismo nel database management system (DBMS) che permette l'esecuzione automatica di una serie di istruzioni SQL in risposta a determinati eventi che avvengono su una tabella o una vista. Questi eventi possono includere operazioni come INSERT, UPDATE e DELETE. Un trigger è definito con una specifica condizione e azione. Quando la condizione si verifica (ad esempio, un record viene inserito in una tabella), l'azione specificata viene automaticamente eseguita. I trigger vengono utilizzati per mantenere l'integrità dei dati, applicare regole di business e molte altre funzioni.

Componenti di un Trigger

Evento: L'azione che scatena l'esecuzione del trigger (ad esempio, INSERT, UPDATE, DELETE).

Condizione: Un'espressione logica che determina se l'azione deve essere eseguita. Non tutti i trigger necessitano di una condizione.

Azione: L'insieme di istruzioni SQL che vengono eseguite quando si verifica l'evento e, se presente, la condizione è soddisfatta.

Implementazione

Trigger Check_Assegnazione_Attiva

Il trigger **check_assegnazione_attiva** è progettato per garantire che uno stesso volontario non possa avere più di una assegnazione attiva nello stesso intervallo temporale. Funziona come segue:

-Esecuzione: Il trigger si attiva dopo ogni inserimento o aggiornamento di un record nella tabella **ASSEGNAZIONE**.

-Recupero dei dati della nuova assegnazione: Il trigger memorizza temporaneamente i dati della nuova assegnazione o dell'assegnazione modificata, in particolare volontario, data di inizio, data di fine prevista, data di fine effettiva e stato dell'assegnazione.

-Controllo della sovrapposizione temporale: Il trigger verifica se, per lo stesso volontario, esiste già un'altra assegnazione con stato **ATTIVA** il cui intervallo temporale si sovrappone a quello della nuova assegnazione.

-**Generazione dell'errore:** Se viene trovata un'altra assegnazione attiva e temporalmente sovrapposta per lo stesso volontario, viene generato un errore tramite **RAISE_APPLICATION_ERROR.** Questo impedisce l'inserimento o l'aggiornamento dell'assegnazione non valida.

Implementazione

Trigger Check_Assegnazione_Attiva

```
CREATE OR REPLACE TRIGGER check_assegnazione_attiva
FOR INSERT OR UPDATE ON ASSEGNAZIONE
COMPOUND TRIGGER

    TYPE t_assegnazione IS RECORD (
        id_assegnazione ASSEGNAZIONE.id_assegnazione%TYPE,
        id_volontario ASSEGNAZIONE.id_volontario%TYPE,
        data_inizio ASSEGNAZIONE.data_inizio%TYPE,
        data_fine_prevista ASSEGNAZIONE.data_fine_prevista%TYPE,
        data_fine_effettiva ASSEGNAZIONE.data_fine_effettiva%TYPE,
        stato_assegnazione ASSEGNAZIONE.stato_assegnazione%TYPE
    );

    TYPE t_lista_assegnazioni IS TABLE OF t_assegnazione INDEX BY PLS_INTEGER;

    v_assegnazioni t_lista_assegnazioni;
    v_indice PLS_INTEGER := 0;

AFTER EACH ROW IS
BEGIN
    v_indice := v_indice + 1;

    v_assegnazioni(v_indice).id_assegnazione := :NEW.id_assegnazione;
    v_assegnazioni(v_indice).id_volontario := :NEW.id_volontario;
    v_assegnazioni(v_indice).data_inizio := :NEW.data_inizio;
    v_assegnazioni(v_indice).data_fine_prevista := :NEW.data_fine_prevista;
    v_assegnazioni(v_indice).data_fine_effettiva := :NEW.data_fine_effettiva;
    v_assegnazioni(v_indice).stato_assegnazione := :NEW.stato_assegnazione;
END AFTER EACH ROW;

AFTER STATEMENT IS
    v_conteggio NUMBER;
BEGIN
    FOR i IN 1 .. v_indice LOOP

        IF v_assegnazioni(i).stato_assegnazione = 'ATTIVA' THEN

            SELECT COUNT(*)
            INTO v_conteggio
            FROM ASSEGNAZIONE a
            WHERE a.id_volontario = v_assegnazioni(i).id_volontario
              AND a.id_assegnazione <> v_assegnazioni(i).id_assegnazione
              AND a.stato_assegnazione = 'ATTIVA'
              AND v_assegnazioni(i).data_inizio <=
                 NVL(a.data_fine_effettiva, NVL(a.data_fine_prevista, DATE '9999-12-31'))
              AND NVL(v_assegnazioni(i).data_fine_effettiva,
                  NVL(v_assegnazioni(i).data_fine_prevista, DATE '9999-12-31'))
                  >= a.data_inizio;

            IF v_conteggio > 0 THEN
                RAISE_APPLICATION_ERROR(
                    -20001,
                    'Il volontario ha gia una assegnazione attiva nello stesso
                     intervallo temporale.'
                );
            END IF;
        END IF;
    END LOOP;
END AFTER STATEMENT;

END;
/
```

Implementazione

Trigger check_progetto_sede_assegnazione

Il trigger **check_progetto_sede_assegnazione** è progettato per garantire che la sede indicata in una assegnazione sia effettivamente associata al progetto scelto. Questo controllo è coerente con il vincolo dinamico “Validità della coppia progetto-sede”, che richiede di verificare l’esistenza della corrispondente associazione nella tabella **PROGETTO_SEDE** durante l’inserimento o l’aggiornamento di una assegnazione.
Funziona come segue:

- **Esecuzione:** Il trigger si attiva prima di ogni inserimento o aggiornamento dei campi id_progetto e id_sede nella tabella ASSEGNAZIONE;
- **Recupero della coppia progetto-sede:** Il trigger prende dal nuovo record di ASSEGNAZIONE i valori id_progetto e id_sede;
- **Controllo della tabella associativa:** Il trigger verifica se nella tabella PROGETTO_SEDE esiste una riga con lo stesso id_progetto e lo stesso id_sede. Questa tabella rappresenta proprio l’associazione tra progetto e sede operativa e stabilisce in quali sedi può svolgersi ciascun progetto;
- **Generazione dell’errore:** Se la coppia progetto-sede non esiste in PROGETTO_SEDE, viene generato un errore tramite RAISE_APPLICATION_ERROR. Questo impedisce l’inserimento o l’aggiornamento di assegnazioni incoerenti.

Implementazione

Trigger check_progetto_sede_assegnazione

```
CREATE OR REPLACE TRIGGER check_progetto_sede_assegnazione
BEFORE INSERT OR UPDATE OF id_progetto, id_sede
ON ASSEGNAZIONE
FOR EACH ROW

DECLARE
    v_count NUMBER;

BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM PROGETTO_SEDE
    WHERE id_progetto = :NEW.id_progetto
      AND id_sede = :NEW.id_sede;


    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(
            -20002,
            'La sede indicata non e associata al progetto scelto.'
        );
    END IF;

END;
/
```

Implementazione

Trigger check_storico_variazione

Il trigger **check_storico_variazione** è progettato per garantire che, quando una richiesta di variazione viene approvata, la vecchia assegnazione non venga eliminata, ma risulti chiusa e collegata a una nuova assegnazione successiva. Questo controllo è coerente con il vincolo dinamico “Gestione dello storico nelle variazioni”, secondo cui, in caso di richiesta approvata, il sistema chiude l’assegnazione precedente e crea una nuova assegnazione collegata alla precedente. È coerente anche con la descrizione generale del progetto, dove è specificato che la vecchia assegnazione non viene eliminata ma chiusa, e che la nuova viene collegata alla precedente per preservare lo storico. Funziona come segue:

- **Esecuzione:** Il trigger si attiva prima dell’aggiornamento dello stato di una richiesta nella tabella RICHIESTA_VARIAZIONE.
- **Controllo dello stato della richiesta:** Il trigger entra in azione solo quando il nuovo stato della richiesta diventa APPROVATA.
- **Recupero dell’assegnazione precedente:** Il trigger prende l’id_assegnazione presente nella richiesta di variazione. Questa rappresenta l’assegnazione originaria da modificare.
- **Controllo della chiusura dell’assegnazione precedente:** Il trigger verifica che l’assegnazione precedente sia stata chiusa, cioè che non sia più ATTIVA e che abbia una data_fine_effettiva valorizzata.
- **Controllo della nuova assegnazione collegata:** Il trigger verifica che esista una nuova riga nella tabella ASSEGNAZIONEcon id_assegnazione_precedente uguale all’assegnazione originaria. In questo modo si garantisce che la nuova assegnazione sia collegata alla precedente.
- **Generazione dell’errore:** Se l’assegnazione precedente non è chiusa oppure non esiste una nuova assegnazione collegata, viene generato un errore tramite RAISE_APPLICATION_ERROR. Questo impedisce di approvare una richiesta di variazione senza preservare correttamente lo storico.

Implementazione

Trigger check_storico_variazione

```
CREATE OR REPLACE TRIGGER check_storico_variazione
BEFORE UPDATE OF stato_richiesta
ON RICHIESTA_VARIAZIONE
FOR EACH ROW

DECLARE
    v_count NUMBER;

BEGIN
    IF :NEW.stato_richiesta = 'APPROVATA'
        AND :OLD.stato_richiesta <> 'APPROVATA' THEN

        SELECT COUNT(*)
        INTO v_count
        FROM ASSEGNAZIONE precedente
        JOIN ASSEGNAZIONE successiva
          ON successiva.id_assegnazione_precedente = precedente.id_assegnazione
        WHERE precedente.id_assegnazione = :NEW.id_assegnazione
          AND precedente.stato_assegnazione = 'CONCLUSA'
          AND precedente.data_fine_effettiva IS NOT NULL
          AND successiva.id_volontario = precedente.id_volontario
          AND successiva.data_inizio > precedente.data_fine_effettiva;

        IF v_count = 0 THEN
            RAISE_APPLICATION_ERROR(
                -20003,
                'La richiesta non puo essere approvata: la vecchia assegnazione
                deve essere conclusa e deve esistere una nuova assegnazione
                collegata correttamente.'
            );
        END IF;

    END IF;
END;
/
```

Implementazione

Trigger check_presenza_assegnazione_attiva

Il trigger **check_presenza_assegnazione_attiva** è progettato per garantire che una presenza possa essere registrata solo se esiste una assegnazione attiva e valida nella data considerata. Questo controllo è coerente con il vincolo dinamico “Validità dell’inserimento della presenza”, secondo cui, durante l’inserimento di una presenza, il sistema deve verificare l’esistenza di una assegnazione attiva nella data indicata.

È coerente anche con la struttura del progetto, perché **REGISTRO_SERVIZIO** rappresenta le giornate di servizio svolte nell’ambito di una specifica assegnazione, mentre **ASSEGNAZIONE** definisce l’intervallo temporale del percorso del volontario.

Funziona come segue:

- **Esecuzione:** Il trigger si attiva prima di ogni inserimento o aggiornamento dei campi id_assegnazione o data_servizio nella tabella REGISTRO_SERVIZIO.
- **Recupero dei dati della presenza:** Il trigger prende dal nuovo record di REGISTRO_SERVIZIO l’identificativo dell’assegnazione e la data del servizio che si sta cercando di registrare.
- **Controllo dell’assegnazione attiva:** Il trigger verifica che esista una riga nella tabella ASSEGNAZIONE con:
  - Lo stesso id_assegnazione;
  - Stato ATTIVA;
  - Una data_inizio antecedente o uguale alla data del servizio;
  - Una data di fine compatibile con la data del servizio, per la fine dell’intervallo, il trigger considera prima data_fine_effettiva, poi data_fine_prevista; se entrambe sono nulle, usa una data futura convenzionale.
- **Generazione dell’errore:** Se non esiste alcuna assegnazione attiva e valida nella data indicata, viene generato un errore tramite RAISE_APPLICATION_ERROR. Questo impedisce la registrazione di giornate di servizio fuori dal periodo ammesso.

Implementazione

Trigger check_presenza_assegnazione_attiva

```
CREATE OR REPLACE TRIGGER check_presenza_assegnazione_attiva
BEFORE INSERT OR UPDATE OF id_assegnazione, data_servizio
ON REGISTRO_SERVIZIO
FOR EACH ROW

DECLARE
   v_count NUMBER;

BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM ASSEGNAZIONE
    WHERE id_assegnazione = :NEW.id_assegnazione
      AND stato_assegnazione = 'ATTIVA'
      AND :NEW.data_servizio >= data_inizio
      AND :NEW.data_servizio <= NVL(data_fine_effettiva, NVL(data_fine_prevista,
          DATE '9999-12-31'));

    IF v_count = 0 THEN
       RAISE_APPLICATION_ERROR(
           -20004,
           'La presenza puo essere registrata solo per una assegnazione attiva e
            valida nella data indicata.'
       );
    END IF;

END;
/
```

Implementazione

Trigger check_limite_permessi_ordinari*

Il trigger **check_limite_permessi_ordinari** è progettato per garantire che il numero totale di permessi ordinari fruiti da un volontario non superi il limite previsto per la relativa assegnazione.

Questo controllo è coerente con il vincolo dinamico “Limite dei permessi ordinari”, secondo cui, durante l’inserimento o l’aggiornamento di una giornata classificata come permesso ordinario, il sistema deve calcolare il totale dei permessi già registrati e verificare che non venga superato il valore massimo consentito.

È coerente anche con la struttura del progetto, perché la tabella ASSEGNAZIONE contiene il campo max_permessi_ordinari, cioè il limite massimo associato a quella specifica assegnazione, mentre REGISTRO_SERVIZIO registra le giornate classificate come PERMESSO_ORDINARIO.

Funziona come segue:

- **Esecuzione:** Il trigger si attiva dopo ogni inserimento o aggiornamento nella tabella REGISTRO_SERVIZIO;
- **Recupero dell’assegnazione coinvolta:** Il trigger memorizza l’id_assegnazione della riga inserita o modificata;
- **Calcolo dei permessi ordinari già registrati:** Per l’assegnazione interessata, il trigger conta quante giornate risultano già registrate con tipo_giornata = 'PERMESSO_ORDINARIO';
- **Recupero del limite massimo consentito:** Il trigger legge dalla tabella ASSEGNAZIONE il valore di max_permessi_ordinari.
- **Generazione dell’errore:** Se il totale dei permessi ordinari supera il limite previsto per l’assegnazione, viene generato un errore tramite RAISE_APPLICATION_ERROR. Questo impedisce l’inserimento o l’aggiornamento non valido.

Implementazione

Trigger check_limite_permessi_ordinari*

```
CREATE OR REPLACE TRIGGER check_limite_permessi_ordinari
FOR INSERT OR UPDATE ON REGISTRO_SERVIZIO
COMPOUND TRIGGER

    TYPE t_id_assegnazione IS TABLE OF REGISTRO_SERVIZIO.id_assegnazione%TYPE INDEX BY PLS_INTEGER;

    v_ids_assegnazione t_id_assegnazione;
    v_indice PLS_INTEGER := 0;

AFTER EACH ROW IS
BEGIN
    v_indice := v_indice + 1;
    v_ids_assegnazione(v_indice) := :NEW.id_assegnazione;
END AFTER EACH ROW;

AFTER STATEMENT IS
    v_totale_permessi NUMBER;
    v_max_permessi NUMBER;
BEGIN
    FOR i IN 1 .. v_indice LOOP

        SELECT COUNT(*)
        INTO v_totale_permessi
        FROM REGISTRO_SERVIZIO
        WHERE id_assegnazione = v_ids_assegnazione(i)
          AND tipo_giornata = 'PERMESSO_ORDINARIO';

        SELECT max_permessi_ordinari
        INTO v_max_permessi
        FROM ASSEGNAZIONE
        WHERE id_assegnazione = v_ids_assegnazione(i);

        IF v_totale_permessi > v_max_permessi THEN
            RAISE_APPLICATION_ERROR(
                -20006,
                'Il numero massimo di permessi ordinari consentiti per questa assegnazione e stato superato.'
            );
        END IF;

    END LOOP;
END AFTER STATEMENT;

END;
/
```

Implementazione

Trigger check_limite_malattia_retribuita*

Il trigger **check_limite_malattia_retribuita** è progettato per garantire che il numero totale di giornate classificate come malattia retribuita non superi il limite massimo previsto per la specifica assegnazione.
Questo controllo è coerente con il vincolo dinamico “Limite della malattia retribuita”, secondo cui, durante l’inserimento o l’aggiornamento di una giornata classificata come malattia retribuita, il sistema deve verificare il numero totale di giorni già registrati per quella tipologia.
È coerente anche con il modello del progetto, perché:

- **REGISTRO_SERVIZIO** registra le giornate del volontario, distinguendo servizio, permessi e malattie;
- **ASSEGNAZIONE** contiene il limite massimo di malattia retribuita previsto per quella specifica assegnazione;
- il controllo riguarda il percorso amministrativo del volontario durante una determinata assegnazione.

Funziona come segue:

- **Esecuzione:** Il trigger si attiva dopo ogni inserimento o aggiornamento nella tabella REGISTRO_SERVIZIO.
- **Recupero dell’assegnazione coinvolta:** Il trigger memorizza l’id_assegnazione della riga inserita o modificata.
- **Calcolo delle giornate di malattia retribuita:** Per l’assegnazione interessata, il trigger conta quante giornate risultano registrate con tipo_giornata = 'MALATTIA_RETRIBUITA'.
- **Recupero del limite massimo consentito: I**l trigger legge dalla tabella ASSEGNAZIONE il valore di max_malattie_retribuite.
- **Generazione dell’errore:** Se il totale delle giornate di malattia retribuita supera il limite previsto per l’assegnazione, viene generato un errore tramite RAISE_APPLICATION_ERROR. Questo impedisce l’inserimento o l’aggiornamento non valido.

Implementazione

Trigger check_limite_malattia_retribuita*

```
CREATE OR REPLACE TRIGGER check_limite_malattia_retribuita
FOR INSERT OR UPDATE ON REGISTRO_SERVIZIO
COMPOUND TRIGGER

    TYPE t_id_assegnazione IS TABLE OF REGISTRO_SERVIZIO.id_assegnazione%TYPE INDEX BY PLS_INTEGER;

    v_ids_assegnazione t_id_assegnazione;
    v_indice PLS_INTEGER := 0;

AFTER EACH ROW IS
BEGIN
    v_indice := v_indice + 1;
    v_ids_assegnazione(v_indice) := :NEW.id_assegnazione;
END AFTER EACH ROW;

AFTER STATEMENT IS
    v_totale_malattie NUMBER;
    v_max_malattie NUMBER;
BEGIN
    FOR i IN 1 .. v_indice LOOP

        SELECT COUNT(*)
        INTO v_totale_malattie
        FROM REGISTRO_SERVIZIO
        WHERE id_assegnazione = v_ids_assegnazione(i)
          AND tipo_giornata = 'MALATTIA_RETRIBUITA';

        SELECT max_malattie_retribuite
        INTO v_max_malattie
        FROM ASSEGNAZIONE
        WHERE id_assegnazione = v_ids_assegnazione(i);

        IF v_totale_malattie > v_max_malattie THEN
            RAISE_APPLICATION_ERROR(
                -20007,
                'Il numero massimo di giorni di malattia retribuita consentiti per questa assegnazione e stato superato.'
            );
        END IF;

    END LOOP;
END AFTER STATEMENT;

END;
/
```

Implementazione

Trigger check_limite_malattia_non_retribuita*

Il trigger **check_limite_malattia_non_retribuita** è progettato per garantire che il numero totale di giornate classificate come malattia non retribuita non superi il limite massimo previsto per la specifica assegnazione.
Questo controllo è coerente con il vincolo dinamico “Limite della malattia non retribuita”, secondo cui, durante l’inserimento o l’aggiornamento di una giornata classificata come malattia non retribuita, il sistema deve verificare il numero totale di giorni già registrati per quella tipologia.
È coerente anche con il modello del progetto, perché:

- **REGISTRO_SERVIZIO** registra le giornate del volontario e distingue le varie tipologie di giornata;
- **ASSEGNAZIONE** contiene il limite massimo di malattia non retribuita previsto per quella specifica assegnazione;
- il controllo riguarda il percorso amministrativo del volontario durante una determinata assegnazione.

Funziona come segue:

- **Esecuzione:** Il trigger si attiva dopo ogni inserimento o aggiornamento nella tabella REGISTRO_SERVIZIO.
- **Recupero dell’assegnazione coinvolta:** Il trigger memorizza l’id_assegnazione della riga inserita o modificata.
- **Calcolo delle giornate di malattia non retribuita:** Per l’assegnazione interessata, il trigger conta quante giornate risultano registrate con tipo_giornata = 'MALATTIA_NON_RETRIBUITA'.
- **Recupero del limite massimo consentito:** Il trigger legge dalla tabella ASSEGNAZIONE il valore di max_malattie_non_retribuite.
- **Generazione dell’errore:** Se il totale delle giornate di malattia non retribuita supera il limite previsto per l’assegnazione, viene generato un errore tramite RAISE_APPLICATION_ERROR. Questo impedisce l’inserimento o l’aggiornamento non valido.

Implementazione

Trigger check_limite_malattia_non_retribuita*

```
CREATE OR REPLACE TRIGGER check_limite_malattia_non_retribuita
FOR INSERT OR UPDATE ON REGISTRO_SERVIZIO
COMPOUND TRIGGER

    TYPE t_id_assegnazione IS TABLE OF REGISTRO_SERVIZIO.id_assegnazione%TYPE INDEX BY PLS_INTEGER;

    v_ids_assegnazione t_id_assegnazione;
    v_indice PLS_INTEGER := 0;

AFTER EACH ROW IS
BEGIN
    v_indice := v_indice + 1;
    v_ids_assegnazione(v_indice) := :NEW.id_assegnazione;
END AFTER EACH ROW;

AFTER STATEMENT IS
    v_totale_malattie NUMBER;
    v_max_malattie NUMBER;
BEGIN
    FOR i IN 1 .. v_indice LOOP

        SELECT COUNT(*)
        INTO v_totale_malattie
        FROM REGISTRO_SERVIZIO
        WHERE id_assegnazione = v_ids_assegnazione(i)
          AND tipo_giornata = 'MALATTIA_NON_RETRIBUITA';

        SELECT max_malattie_non_retribuite
        INTO v_max_malattie
        FROM ASSEGNAZIONE
        WHERE id_assegnazione = v_ids_assegnazione(i);

        IF v_totale_malattie > v_max_malattie THEN
            RAISE_APPLICATION_ERROR(
                -20008,
                'Il numero massimo di giorni di malattia non retribuita consentiti per questa assegnazione e stato superato.'
            );
        END IF;

    END LOOP;
END AFTER STATEMENT;

END;
/
```

Implementazione

check_permesso_straordinario_valido

Il trigger **check_permesso_straordinario_valido** è progettato per controllare che una giornata classificata come permesso straordinario venga registrata solo se è presente una motivazione nella colonna note.
Questo controllo è coerente con il vincolo dinamico “Validità del permesso straordinario”, perché il progetto richiede di evitare l’uso improprio del permesso straordinario verificando la presenza di una motivazione o giustificazione valida.

- **Esecuzione:** Il trigger si attiva prima di ogni inserimento o aggiornamento dei campi tipo_giornata e note nella tabella REGISTRO_SERVIZIO;
- **Controllo della tipologia:** Il trigger verifica se la giornata che si sta inserendo o modificando è classificata come PERMESSO_STRAORDINARIO;
- **Controllo della motivazione:** Se la giornata è un permesso straordinario, il trigger controlla che il campo note sia valorizzato;
- **Generazione dell’errore**: Se il campo note è nullo o vuoto, viene generato un errore tramite RAISE_APPLICATION_ERROR.

Implementazione

check_permesso_straordinario_valido

```
CREATE OR REPLACE TRIGGER check_permesso_straordinario_valido
BEFORE INSERT OR UPDATE OF tipo_giornata, note
ON REGISTRO_SERVIZIO
FOR EACH ROW
BEGIN
    IF :NEW.tipo_giornata = 'PERMESSO_STRAORDINARIO'
        AND (:NEW.note IS NULL OR TRIM(:NEW.note) IS NULL) THEN

        RAISE_APPLICATION_ERROR(
            -20009,
            'Il permesso straordinario richiede una motivazione valida nelle note.'
        );

    END IF;

END;
/
```

Implementazione

check_permesso_straordinario_valido

Il trigger **check_permesso_straordinario_valido** è progettato per controllare che una giornata classificata come permesso straordinario venga registrata solo se è presente una motivazione nella colonna note.
Questo controllo è coerente con il vincolo dinamico “Validità del permesso straordinario”, perché il progetto richiede di evitare l’uso improprio del permesso straordinario verificando la presenza di una motivazione o giustificazione valida.

- **Esecuzione:** Il trigger si attiva prima di ogni inserimento o aggiornamento dei campi tipo_giornata e note nella tabella REGISTRO_SERVIZIO;
- **Controllo della tipologia:** Il trigger verifica se la giornata che si sta inserendo o modificando è classificata come PERMESSO_STRAORDINARIO;
- **Controllo della motivazione:** Se la giornata è un permesso straordinario, il trigger controlla che il campo note sia valorizzato;
- **Generazione dell’errore**: Se il campo note è nullo o vuoto, viene generato un errore tramite RAISE_APPLICATION_ERROR.

Implementazione

check_limite_ore_giornaliere

Il trigger **check_limite_ore_giornaliere** è progettato per garantire che le ore di servizio conteggiate in una giornata non superino il limite massimo di 8 ore.

Questo controllo è coerente con il vincolo dinamico “Limite massimo di ore giornaliere conteggiabili”, secondo cui, durante l’inserimento o l’aggiornamento di una presenza, il sistema verifica che le ore conteggiate non superino 8; eventuali ore eccedenti non vengono considerate ai fini del monte ore di servizio.

È coerente anche con il modello del progetto, perché REGISTRO_SERVIZIO è la tabella in cui vengono registrate le giornate del volontario, comprese le ore svolte e le eventuali ore da recuperare.

- **Esecuzione:** Il trigger si attiva prima di ogni inserimento o aggiornamento del campo ore_svolte nella tabella REGISTRO_SERVIZIO.
- **Controllo delle ore giornaliere:** Il trigger verifica se il valore inserito in ore_svolte è maggiore di 8.
- **Correzione automatica del valore:** Se le ore inserite superano 8, il trigger non blocca l’operazione, ma imposta automaticamente ore_svolte a 8.
- **Motivo della scelta:** Il vincolo dinamico non dice che l’inserimento deve essere rifiutato, ma che le ore eccedenti non devono essere considerate ai fini del monte ore di servizio. Per questo motivo è più coerente correggere il valore invece di generare un errore.

Implementazione

check_limite_ore_giornaliere

```
CREATE OR REPLACE TRIGGER check_limite_ore_giornaliere
BEFORE INSERT OR UPDATE OF ore_svolte
ON REGISTRO_SERVIZIO
FOR EACH ROW
BEGIN
    IF :NEW.ore_svolte > 8 THEN
        :NEW.ore_svolte := 8;
    END IF;

END;
/
```

Implementazione

check_monitoraggio_assegnazione_valida

Il trigger **check_monitoraggio_assegnazione_valida** è progettato per garantire che un monitoraggio possa essere registrato solo se riferito a una assegnazione esistente e temporalmente coerente con il percorso amministrativo del volontario.

Questo controllo è coerente con il vincolo dinamico “Monitoraggio su assegnazioni esistenti”, secondo cui, durante l’inserimento di un monitoraggio, il sistema deve verificare l’esistenza dell’assegnazione e la coerenza del monitoraggio con il percorso registrato.

È coerente anche con il modello del progetto, perché:

- **MONITORAGGIO** è collegata ad ASSEGNAZIONE;
- la descrizione implementativa della tabella MONITORAGGIO dice che ogni monitoraggio è associato a una sola assegnazione;
- **ASSEGNAZIONE** contiene l’intervallo temporale del percorso, quindi è corretto verificare che data_monitoraggiocada all’interno di quel periodo.
- **Esecuzione:** Il trigger si attiva prima di ogni inserimento o aggiornamento dei campi id_assegnazione o data_monitoraggio nella tabella MONITORAGGIO.
- **Recupero dei dati del monitoraggio:** Il trigger prende dal nuovo record l’identificativo dell’assegnazione e la data del monitoraggio.
- **Controllo dell’esistenza e della coerenza temporale:** Il trigger verifica che esista una riga nella tabella ASSEGNAZIONEcon:
  - Lo stesso id_assegnazione;
  - Data_inizio antecedente o uguale alla data_monitoraggio;
  - Una data di fine compatibile con la data_monitoraggio, per la fine dell’intervallo, il trigger considera prima data_fine_effettiva, poi data_fine_prevista; se entrambe sono nulle, usa una data futura convenzionale.
- **Generazione dell’errore:** Se non esiste alcuna assegnazione compatibile con la data del monitoraggio, viene generato un errore tramite RAISE_APPLICATION_ERROR.

Implementazione

check_monitoraggio_assegnazione_valida

```
CREATE OR REPLACE TRIGGER check_monitoraggio_assegnazione_valida
BEFORE INSERT OR UPDATE OF id_assegnazione, data_monitoraggio
ON MONITORAGGIO
FOR EACH ROW
DECLARE
   v_count NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM ASSEGNAZIONE
    WHERE id_assegnazione = :NEW.id_assegnazione
      AND :NEW.data_monitoraggio >= data_inizio
      AND :NEW.data_monitoraggio <= NVL(data_fine_effettiva, NVL(data_fine_prevista,
            DATE '9999-12-31'));

    IF v_count = 0 THEN
       RAISE_APPLICATION_ERROR(
          -20016,
          'Il monitoraggio deve riferirsi a una assegnazione esistente e coerente
           con il periodo del percorso.'
       );
    END IF;
END;
/
```

Implementazione

check_coerenza_tipo_giornata_ore

Il trigger **check_coerenza_tipo_giornata_ore** è progettato per garantire che il numero di ore registrate in REGISTRO_SERVIZIO sia coerente con la tipologia della giornata.

Questo controllo è coerente con il vincolo dinamico “Coerenza tra tipo di giornata e ore registrate”, secondo cui:

- Una giornata classificata come SERVIZIO deve avere un numero di ore maggiore di zero;
- Una giornata classificata come PERMESSO_ORDINARIO, MALATTIA_RETRIBUITA, MALATTIA_NON_RETRIBUITA o PERMESSO_STRAORDINARIO deve avere un numero di ore pari a zero.

È coerente anche con il modello del progetto, perché REGISTRO_SERVIZIO è la tabella in cui vengono memorizzati:tipo_giornata, ore_svolte, ore_da_recuperare, stato_validazione, note

- **Esecuzione:** Il trigger si attiva prima di ogni inserimento o aggiornamento dei campi tipo_giornata e ore_svolte nella tabella REGISTRO_SERVIZIO.
- **Controllo della giornata di servizio**: Se tipo_giornata = 'SERVIZIO', il trigger verifica che ore_svolte sia strettamente maggiore di zero.
- **Controllo delle giornate non lavorative:** Se tipo_giornata è uno tra:
  - PERMESSO_ORDINARIO
  - MALATTIA_RETRIBUITA
  - MALATTIA_NON_RETRIBUITA
  - PERMESSO_STRAORDINARIO

allora il trigger verifica che ore_svolte sia uguale a zero.

- **Generazione dell’errore:** Se il valore di ore_svolte non è coerente con la tipologia della giornata, viene generato un errore tramite RAISE_APPLICATION_ERROR.

Implementazione

check_coerenza_tipo_giornata_ore

```
CREATE OR REPLACE TRIGGER check_coerenza_tipo_giornata_ore
BEFORE INSERT OR UPDATE OF tipo_giornata, ore_svolte
ON REGISTRO_SERVIZIO
FOR EACH ROW
BEGIN
   IF :NEW.tipo_giornata = 'SERVIZIO' AND :NEW.ore_svolte <= 0 THEN
     RAISE_APPLICATION_ERROR(
       -20017,
       'Una giornata di servizio deve avere un numero di ore maggiore di zero.'
     );

   ELSIF :NEW.tipo_giornata IN (
        'PERMESSO_ORDINARIO',
        'MALATTIA_RETRIBUITA',
        'MALATTIA_NON_RETRIBUITA',
        'PERMESSO_STRAORDINARIO'
   ) AND :NEW.ore_svolte <> 0 THEN
     RAISE_APPLICATION_ERROR(
       -20018,
       'Permessi e malattie devono avere ore_svolte pari a zero.'
     );
  END IF;
END;
/
```

Procedure

Le procedure in SQL, conosciute anche come stored procedures, sono blocchi di codice SQL memorizzati nel database che possono essere eseguiti su richiesta. Sono un elemento fondamentale per la gestione e la manipolazione dei dati nei database relazionali, offrendo una serie di vantaggi in termini di performance, sicurezza e manutenzione del codice.

Caratteristiche delle Stored Procedure

1. **Modularità:** Le procedure permettono di scrivere codice SQL una volta sola e di riutilizzarlo in più punti dell'applicazione. Questo riduce la duplicazione del codice e facilita la manutenzione;
2. **Performance:** Poiché le stored procedures sono compilate e ottimizzate dal database management system (DBMS) una volta sola, l'esecuzione è generalmente più veloce rispetto a eseguire molteplici query individuali;
3. **Sicurezza:** Le procedure possono essere utilizzate per controllare l'accesso ai dati. Gli utenti possono essere autorizzati a eseguire una procedura senza avere accesso diretto alle tabelle sottostanti.;
4. **Gestione degli errori:** Le stored procedures possono includere meccanismi di gestione degli errori, migliorando l'affidabilità dell'applicazione.

Struttura di una Stored Procedure

Una stored procedure in SQL si compone tipicamente dei seguenti elementi:

1. **Nome della Procedura:** Un identificatore unico per la procedura;
2. **Parametri:** Opzionali, possono essere utilizzati per passare valori alla procedura e per restituire valori;
3. **Corpo della Procedura:** Un blocco di codice SQL che include dichiarazioni, istruzioni condizionali, cicli e operazioni sui dati.

Implementazione

Procedura crea_progetto

La procedura **crea_progetto** è progettata per creare un nuovo progetto di Servizio Civile Universale, associarlo all'ente indicato e collegarlo alle sedi di attuazione specificate.

I passaggi che vengono eseguiti sono:

1. **Controllo dei dati obbligatori**
    a. Viene verificato che i dati principali del progetto siano valorizzati correttamente. In particolare vengono controllati:
        i. **ente**
        ii. **titolo**
        iii. **area di intervento**
        iv. **descrizione**
        v. **data di inizio**
        vi. **stato del progetto**
        vii. **lista delle sedi**
        viii. **numero di posti disponibili**
2. **Controllo esistenza ente**
    a. Viene verificato che l'ente indicato da p_id_ente esista nella tabella **ENTE.**
    b. Se l'ente non esiste, la procedura termina con messaggio di errore.
3. **Controllo coerenza delle date**
    a. Viene verificato che, se valorizzata, la data di fine non sia antecedente alla data di inizio.
4. **Controllo dello stato del progetto**
    a. Viene verificato che lo stato del progetto sia tra quelli ammessi dal sistema.

Implementazione

Procedura crea_progetto

5. **Analisi della lista delle sedi**
    - La stringa p_sedi_list, contenente gli identificativi delle sedi separati da virgole, viene scomposta e analizzata.
    - **Per ciascuna sede:**
        - si verifica che l'identificativo sia numerico;
        - si verifica che la sede esista;
        - si verifica che la sede appartenga allo stesso ente del progetto;
        - si controlla che non sia duplicata nella lista.
6. **Creazione del progetto**
    - Viene generato un nuovo identificativo per il progetto.
    - Il nuovo record viene inserito nella tabella **PROGETTO.**
7. **Creazione delle associazioni con le sedi**
    - Per ogni sede valida della lista viene inserita una riga nella tabella **PROGETTO_SEDE,** associando il nuovo progetto alla sede e indicando il numero di posti disponibili.
8. **Gestione della transazione**
    - Se tutte le operazioni vanno a buon fine, la procedura esegue **COMMIT.**
    - In caso di errore SQL, viene eseguito **ROLLBACK** e viene mostrato il messaggio di errore.

Implementazione

Procedura crea_progetto

FreeSQL | >_ Worksheet | SQL Tutorials | 23ai

[SQL Worksheet]*

```
CREATE OR REPLACE PROCEDURE crea_progetto (
    p_id_ente               IN NUMBER,
    p_titolo_progetto       IN VARCHAR2,
    p_area_intervento       IN VARCHAR2,
    p_descrizione_progetto  IN VARCHAR2,
    p_data_inizio           IN DATE,
    p_data_fine             IN DATE,
    p_stato_progetto        IN VARCHAR2,
    p_sedi_list             IN VARCHAR2,
    p_posti_disponibili     IN NUMBER
)
AS
    v_count             NUMBER;
    v_id_progetto       NUMBER;
    v_sede_str          VARCHAR2(100);
    v_id_sede           NUMBER;
    v_pos               NUMBER := 1;
    v_sedi_trovate      NUMBER := 0;
    v_sedi_viste        VARCHAR2(4000) := ',';
BEGIN
    IF p_id_ente IS NULL
       OR p_titolo_progetto IS NULL
       OR p_area_intervento IS NULL
       OR p_descrizione_progetto IS NULL
       OR p_data_inizio IS NULL
       OR p_stato_progetto IS NULL
       OR p_sedi_list IS NULL
       OR p_posti_disponibili IS NULL THEN
        DBMS_OUTPUT.PUT_LINE('Errore: dati obbligatori mancanti.');
        RETURN;
    END IF;

    IF p_data_fine IS NOT NULL AND p_data_inizio > p_data_fine THEN
        DBMS_OUTPUT.PUT_LINE('Errore: la data di inizio non puo esser
        successiva alla data di fine.');
        RETURN;
    END IF;

    IF p_posti_disponibili < 0 THEN
        DBMS_OUTPUT.PUT_LINE('Errore: il numero di posti disponibili
        non e valido.');
        RETURN;
    END IF;

 IF UPPER(TRIM(p_stato_progetto)) NOT IN ('ATTIVO', 'CONCLUSO', 'SOSPESO')
     THEN
      DBMS_OUTPUT.PUT_LINE('Errore: stato del progetto non valido.');
      RETURN;
 END IF;
```

Implementazione

Procedura crea_progetto

FreeSQL >_ Worksheet SQL Tutorials 23ai

[SQL Worksheet]*

```
48    SELECT COUNT(*)
49    INTO v_count
50    FROM ENTE
51    WHERE id_ente = p_id_ente;
52
53    IF v_count = 0 THEN
54       DBMS_OUTPUT.PUT_LINE('Errore: l''ente indicato non esiste.');
55       RETURN;
56    END IF;
57
58    LOOP
59       v_sede_str := TRIM(REGEXP_SUBSTR(p_sedi_list, '[^,]+', 1, v_pos));
60       EXIT WHEN v_sede_str IS NULL;
61
62       IF NOT REGEXP_LIKE(v_sede_str, '^[0-9]+$') THEN
63          DBMS_OUTPUT.PUT_LINE('Errore: la lista delle sedi contiene un
64          ' valore non valido: ' || v_sede_str);
65          RETURN;
66       END IF;
67
68       v_id_sede := TO_NUMBER(v_sede_str);
69
70       IF INSTR(v_sedi_viste, ',' || v_id_sede || ',') > 0 THEN
71          DBMS_OUTPUT.PUT_LINE('Errore: la sede ' || v_id_sede ||
72          ' e presente piu di una volta nella lista.');
73          RETURN;
74       END IF;
75
76       v_sedi_viste := v_sedi_viste || v_id_sede || ',';
77
78       SELECT COUNT(*)
79       INTO v_count
80       FROM SEDE
81       WHERE id_sede = v_id_sede;
82
83       IF v_count = 0 THEN
84          DBMS_OUTPUT.PUT_LINE('Errore: la sede ' || v_id_sede ||
85          ' non e valido.');
86          RETURN;
87       END IF;
88
89       SELECT COUNT(*)
90       INTO v_count
91       FROM SEDE
92       WHERE id_sede = v_id_sede
93         AND id_ente = p_id_ente;
94
95       IF v_count = 0 THEN
```

Implementazione

Procedura crea_progetto

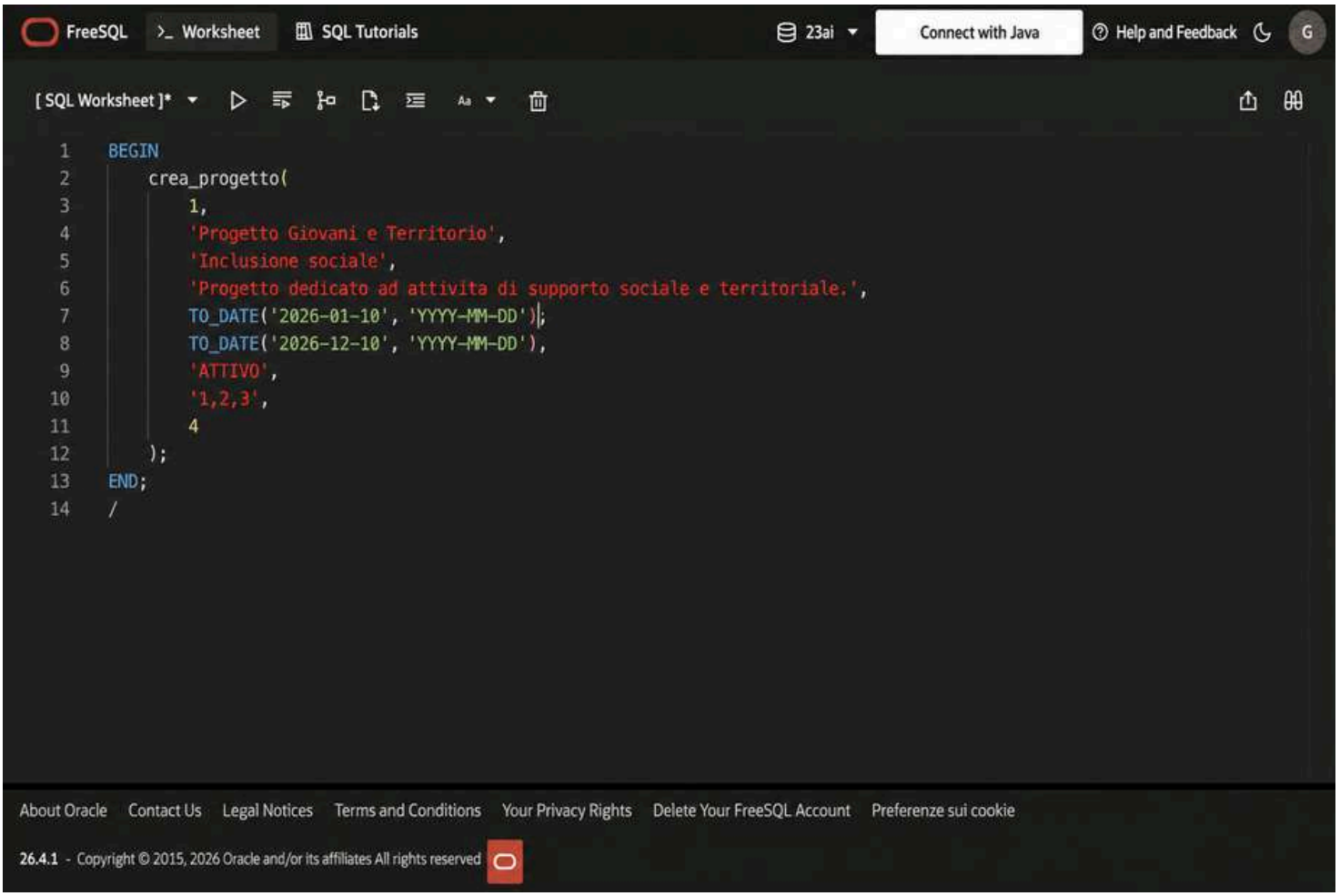
FreeSQL >_ Worksheet SQL Tutorials 23ai

[SQL Worksheet]*

```
95      DBMS_OUTPUT.PUT_LINE('Errore: la sede ' || v_id_sede ||
96        ' non appartiene all''ente indicato.');
97        RETURN;
98    END IF;
99
100   v_sedi_trovate := v_sedi_trovate + 1;
101   v_pos := v_pos + 1;
102 END LOOP;
103
104 IF v_sedi_trovate = 0 THEN
105     DBMS_OUTPUT.PUT_LINE('Errore: la lista delle sedi e vuota
106     o malformata.');
107     RETURN;
108 END IF;
109
110 SELECT NVL(MAX(id_progetto), 0) + 1
111 INTO v_id_progetto
112 FROM PROGETTO;
113
114 INSERT INTO PROGETTO (
115     id_progetto, id_ente, titolo, area_intervento, descrizione,
116     data_inizio, data_fine, stato_progetto
117     )
118 VALUES (
119     v_id_progetto, p_id_ente, p_titolo_progetto, p_area_intervento,
120     p_descrizione_progetto, p_data_inizio, p_data_fine,
121     UPPER(TRIM(p_stato_progetto))
122 );
123
124 v_pos := 1;
125
126 LOOP
127     v_sede_str := TRIM(REGEXP_SUBSTR(p_sedi_list, '[^,]+', 1, v_pos));
128     EXIT WHEN v_sede_str IS NULL;
129
130     v_id_sede := TO_NUMBER(v_sede_str);
131
132     INSERT INTO PROGETTO_SEDE (id_progetto, id_sede, posti_disponibili)
133     VALUES (v_id_progetto, v_id_sede, p_posti_disponibili);
135
136     v_pos := v_pos + 1;
137 END LOOP;
138
139 COMMIT;
140 DBMS_OUTPUT.PUT_LINE('Progetto creato con successo. ID progetto: '
    || v_id_progetto);
141
```

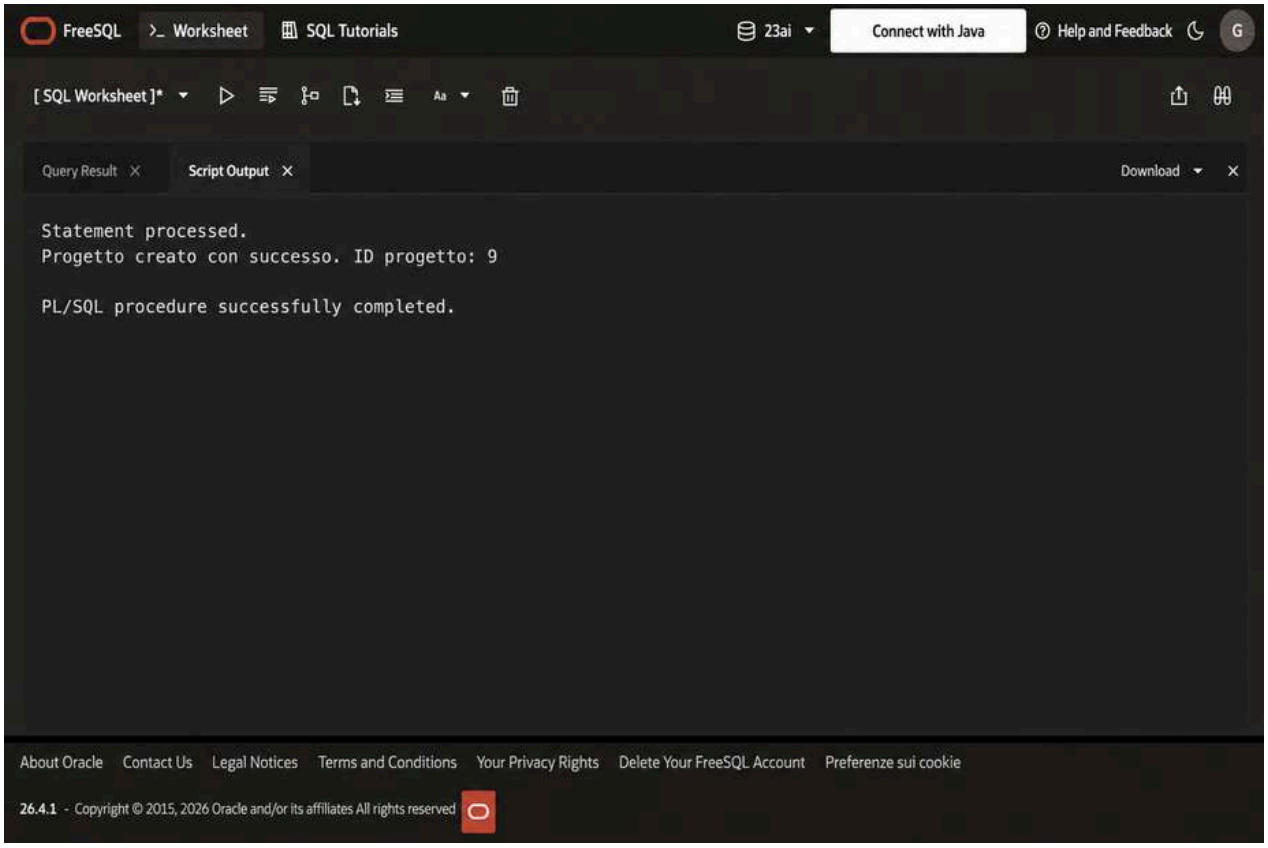
Implementazione

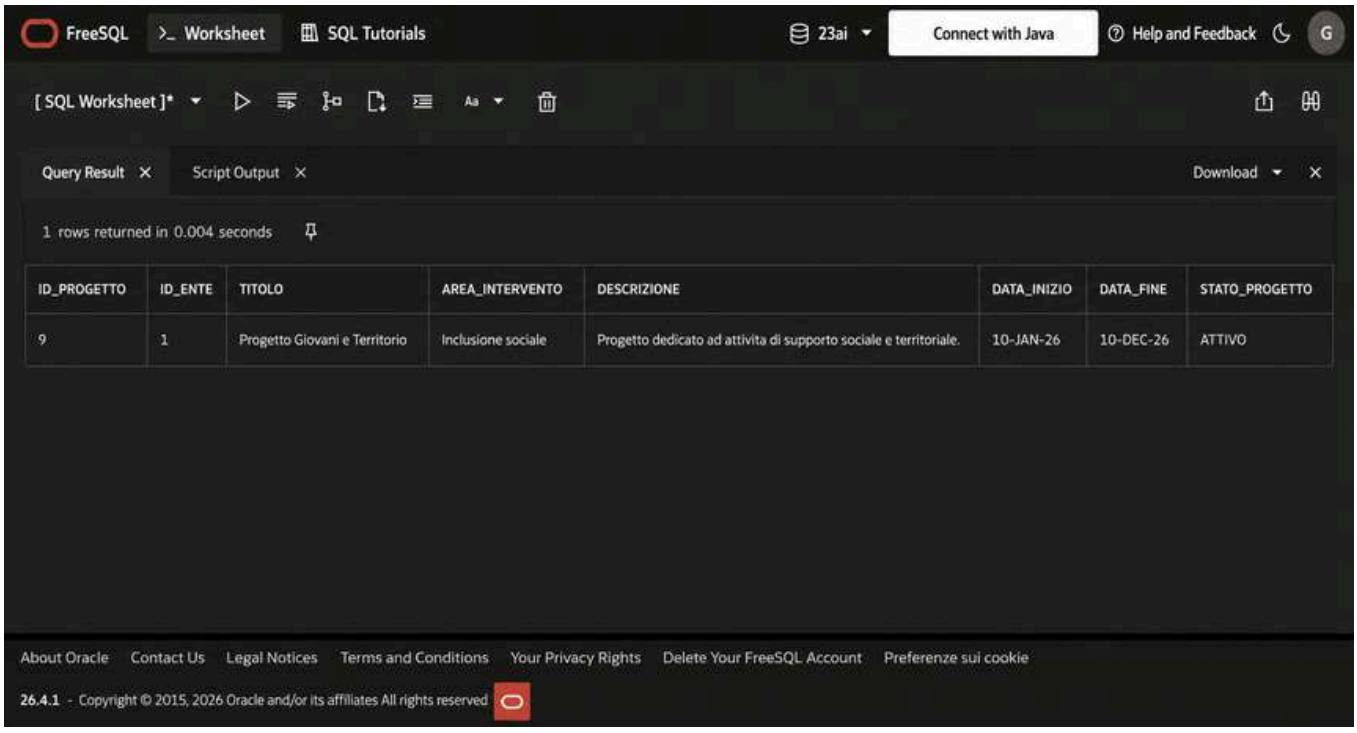
Procedura crea_progetto (Caso corretto)



Implementazione

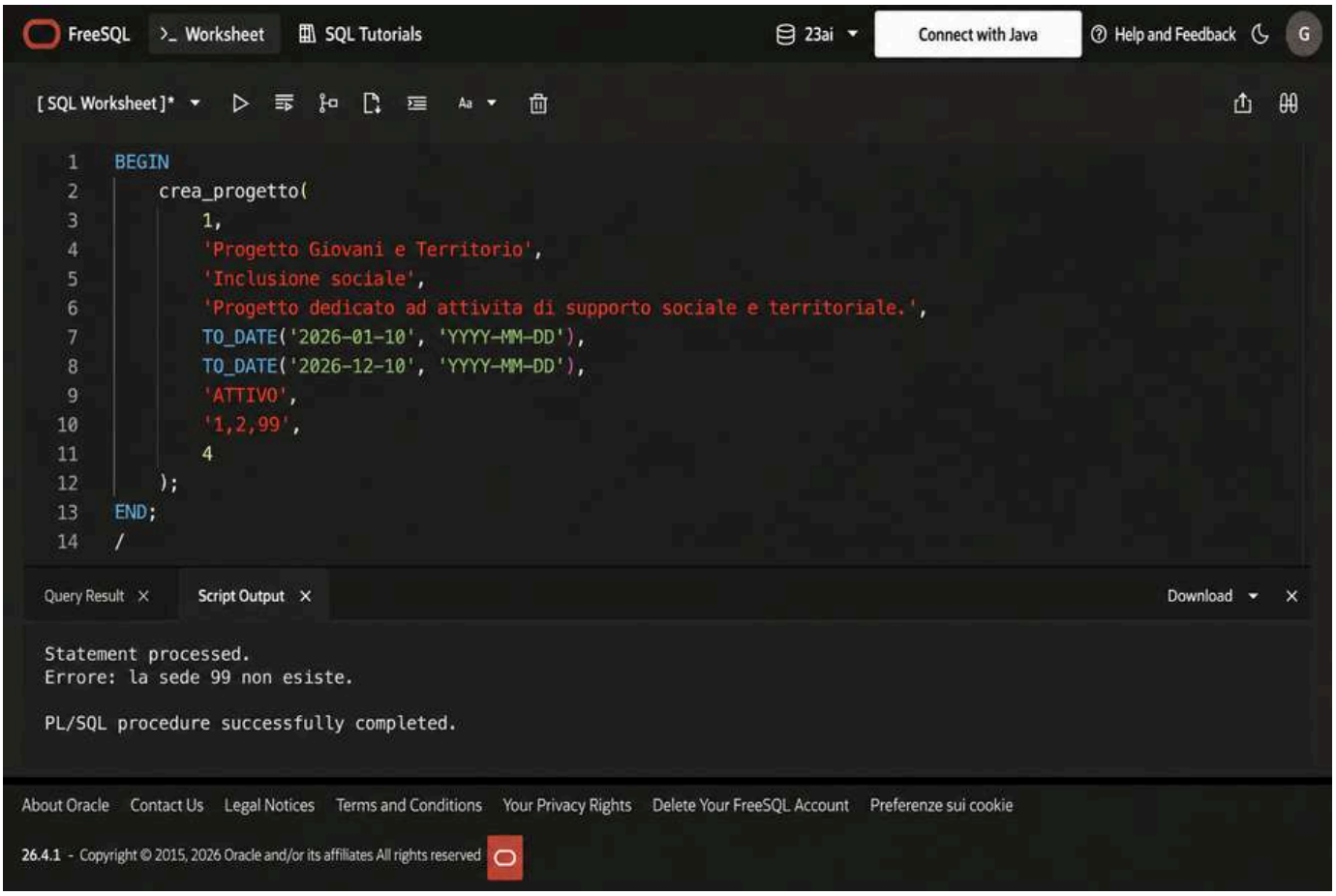
Procedura crea_progetto (Caso corretto)





Implementazione

Procedura crea_progetto (Caso errato)



Implementazione

Procedura modifica_progetto

La procedura **modifica_progetto** è progettata per modificare un progetto di Servizio Civile Universale già esistente e aggiornare le sue associazioni con le sedi di attuazione.

I passaggi eseguiti sono i seguenti:

1. Controllo dei dati obbligatori:
    a. La procedura verifica che siano presenti:
    b. identificativo del progetto
    c. titolo
    d. area di intervento
    e. descrizione
    f. data di inizio
    g. stato del progetto
    h. lista delle sedi
    i. numero di posti disponibili
2. **Controllo esistenza del progetto:** Viene verificato che il progetto esista nella tabella PROGETTO. Ogni progetto è associato a un solo ente.
3. **Recupero dell'ente del progetto:** La procedura recupera id_ente del progetto esistente, perché ogni sede appartiene a un solo ente e le nuove sedi devono essere coerenti con l'ente del progetto.
4. **Controllo coerenza delle date e dello stato:** Viene verificato che data_fine non sia precedente a data_inizio e stato_progetto sia tra i valori ammessi dal sistema. I vincoli statici del progetto prevedono coerenza del periodo temporale e validità dello stato del progetto.

Implementazione

Procedura modifica_progetto

5. **Analisi e validazione della nuova lista sedi:**
    - Per ciascuna sede nella stringa p_sedi_list:
    - l'identificativo deve essere numerico
    - la sede deve esistere
    - la sede deve appartenere allo stesso ente del progetto
    - la sede non deve comparire più di una volta nella lista
6. **Controllo sulle sedi da rimuovere:**

Prima di eliminare le vecchie associazioni in PROGETTO_SEDE, la procedura verifica se una sede che verrebbe rimossa è già usata in ASSEGNAZIONE per quel progetto. Se sì, blocca la modifica. Questo rende la procedura coerente con il vincolo dinamico sulla validità della coppia progetto-sede e protegge la consistenza dei dati già presenti.

7. **Aggiornamento del progetto:** Viene aggiornata la riga di PROGETTO.
8. **Aggiornamento delle associazioni con le sedi:**

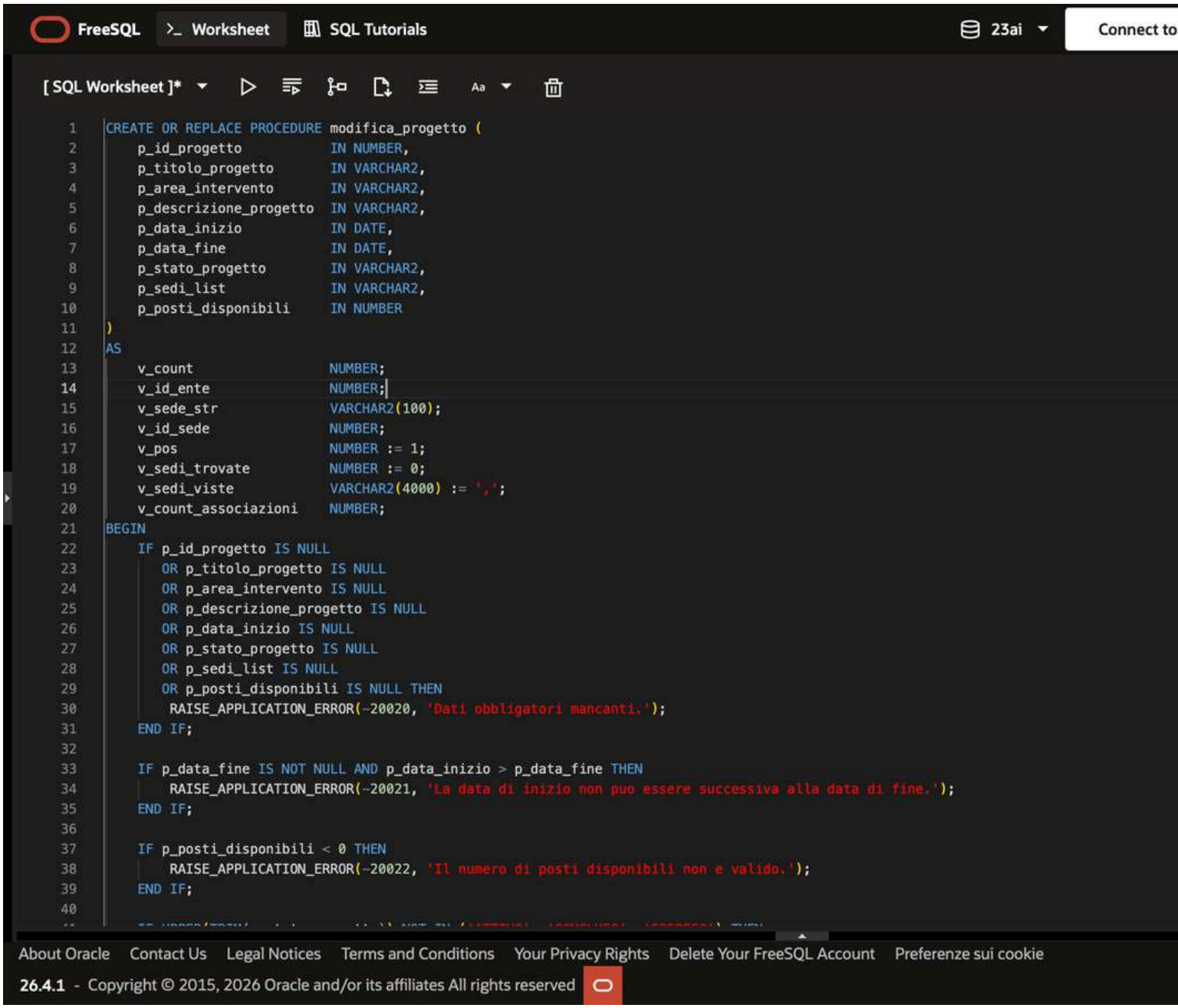
Vengono eliminate le vecchie righe di PROGETTO_SEDE e inserite le nuove, con il numero di posti disponibili richiesto. PROGETTO_SEDE memorizza infatti anche posti_disponibili.

9. **Gestione transazionale:**

Se tutto va bene, viene eseguito COMMIT e viene mostrato un messaggio di successo. In caso di errore, viene eseguito ROLLBACK e l'errore viene rilanciato.

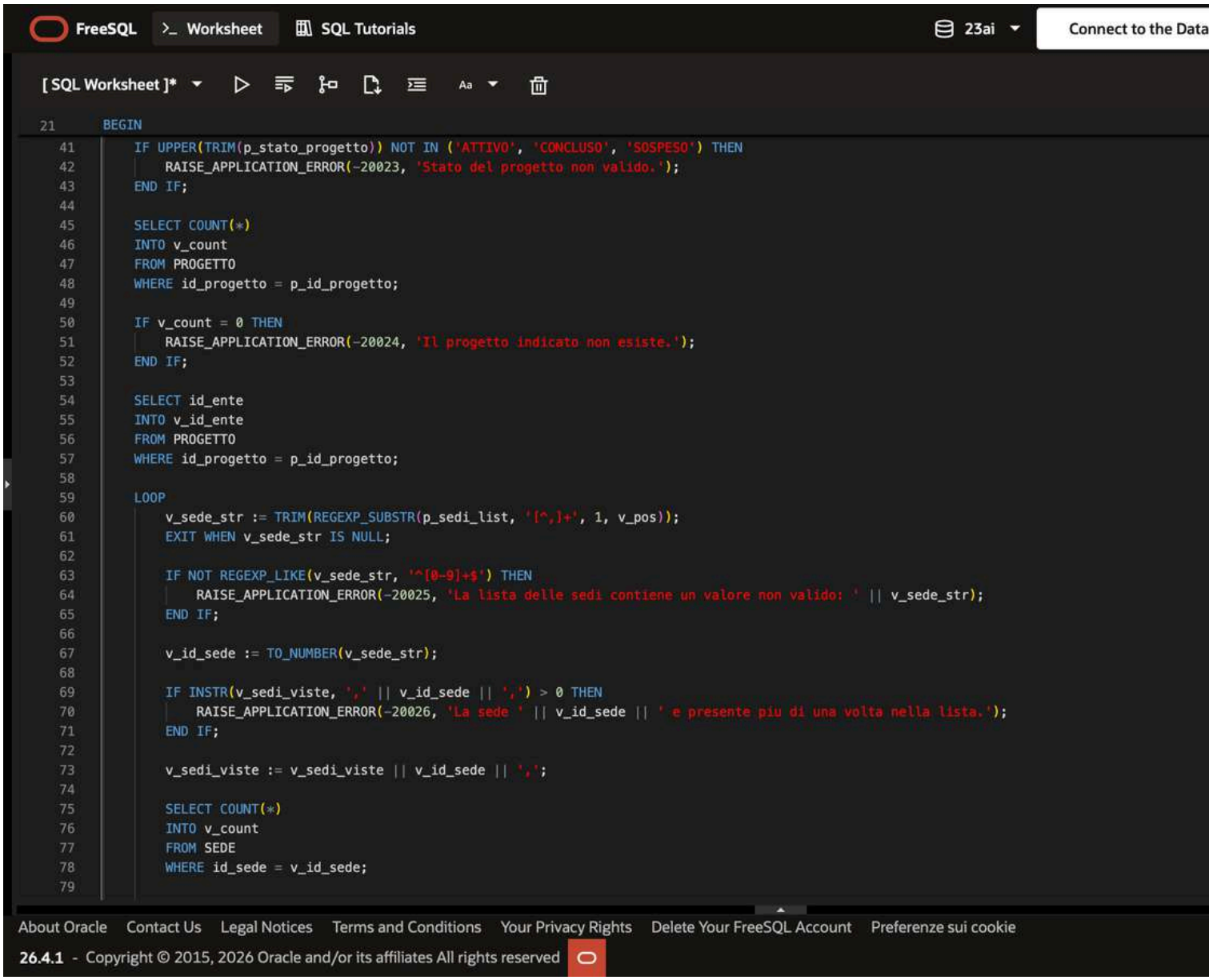
Implementazione

Procedura modifica_progetto



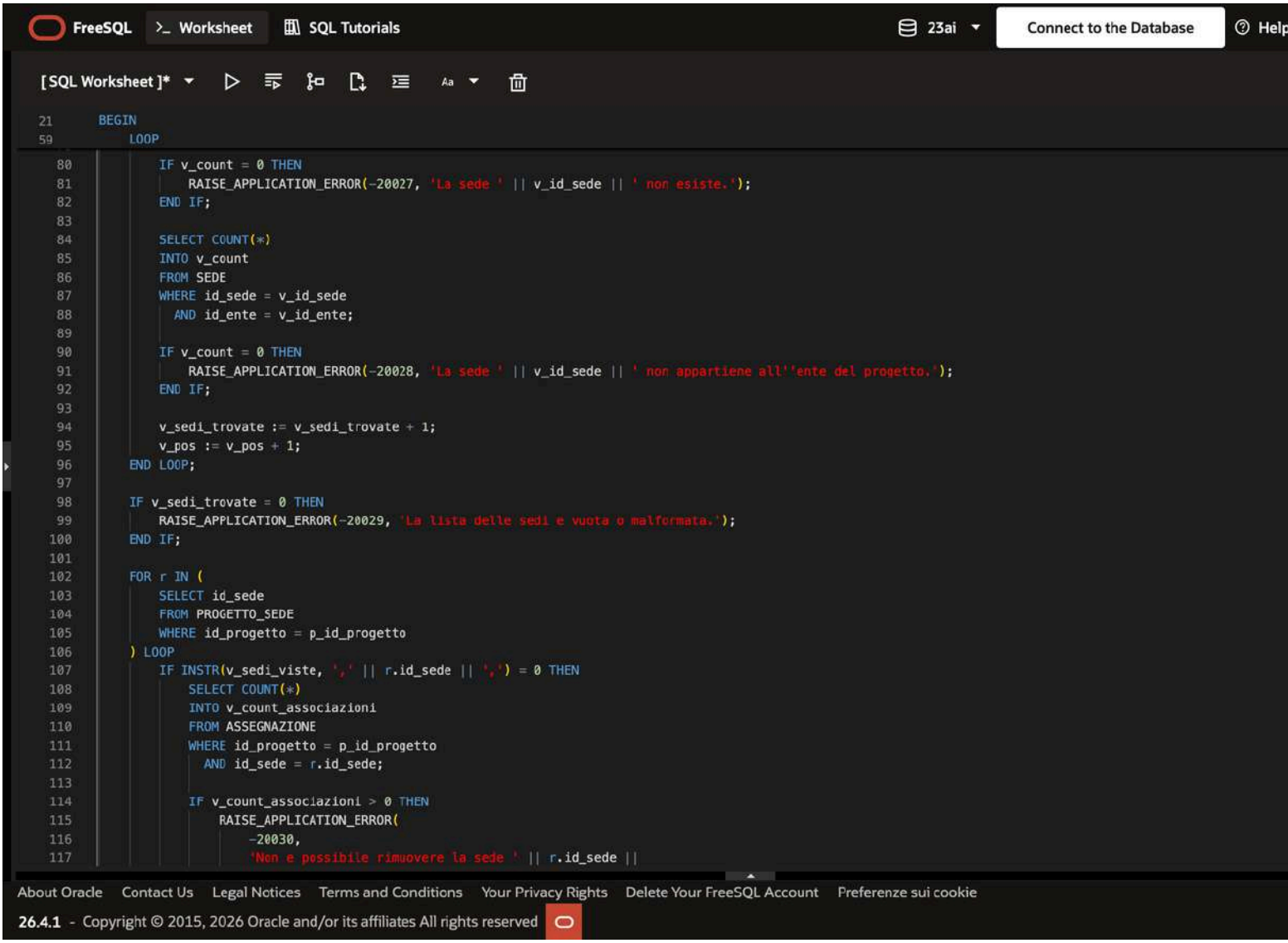
Implementazione

Procedura modifica_progetto



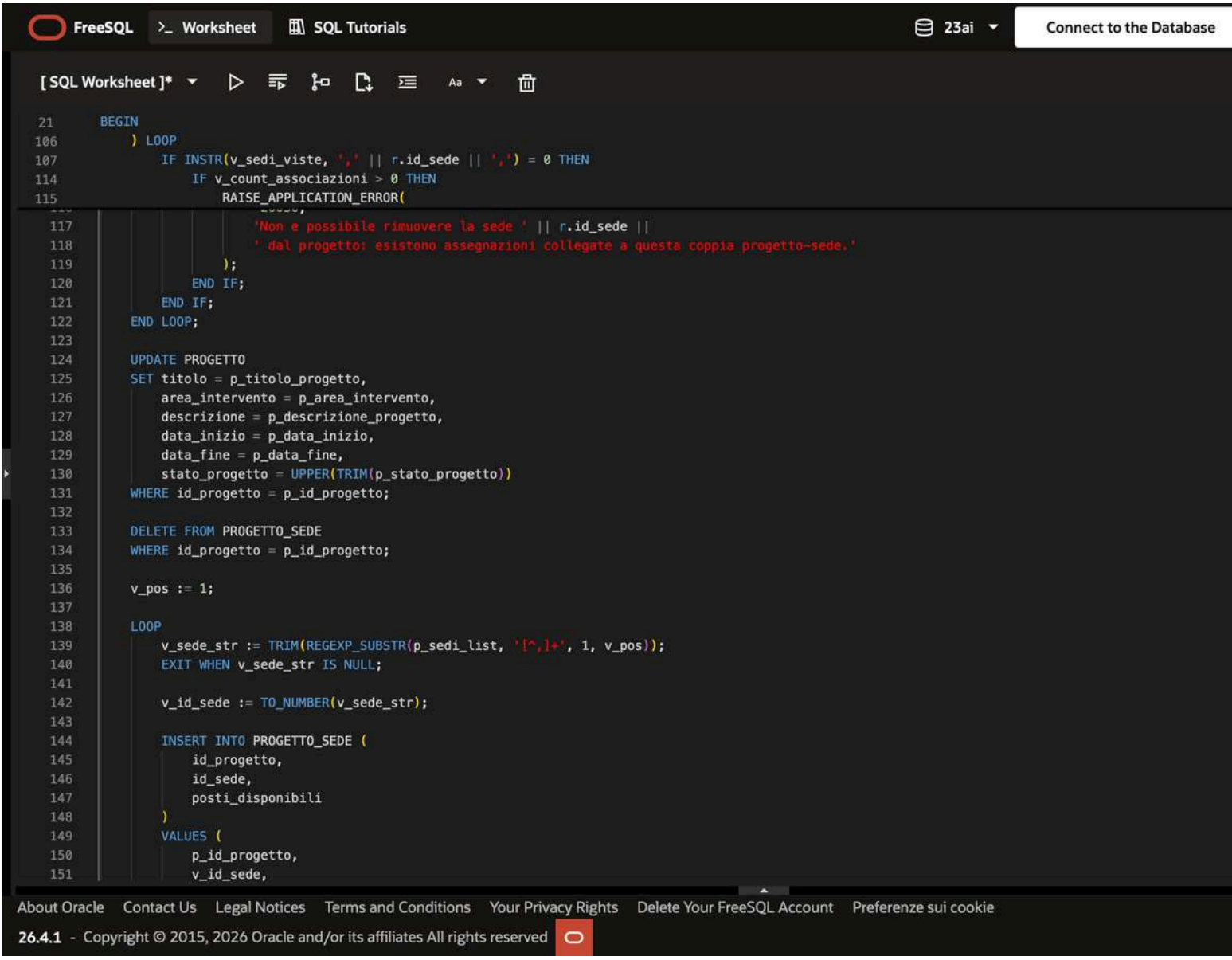
Implementazione

Procedura modifica_progetto



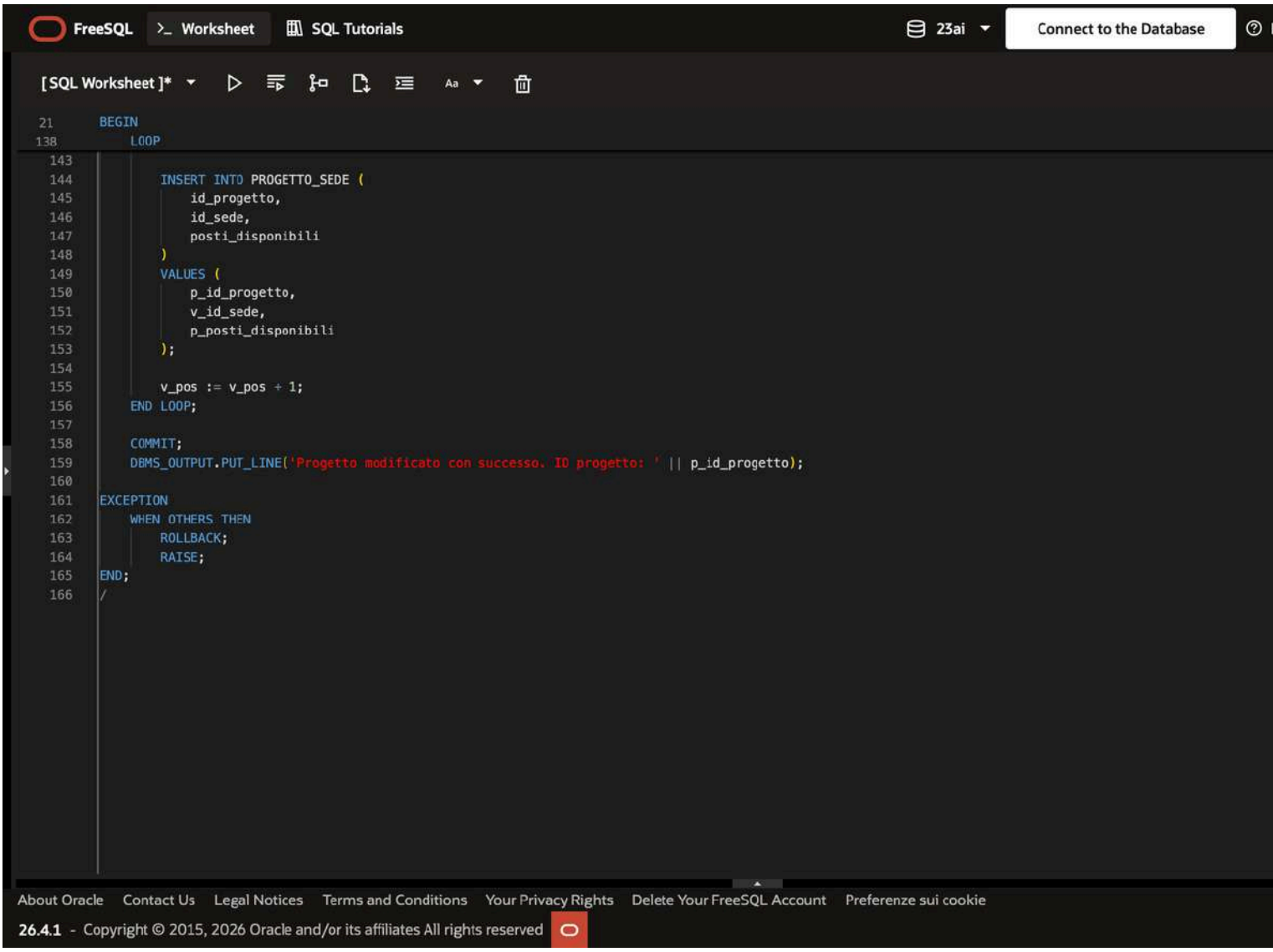
Implementazione

Procedura modifica_progetto



Implementazione

Procedura modifica_progetto



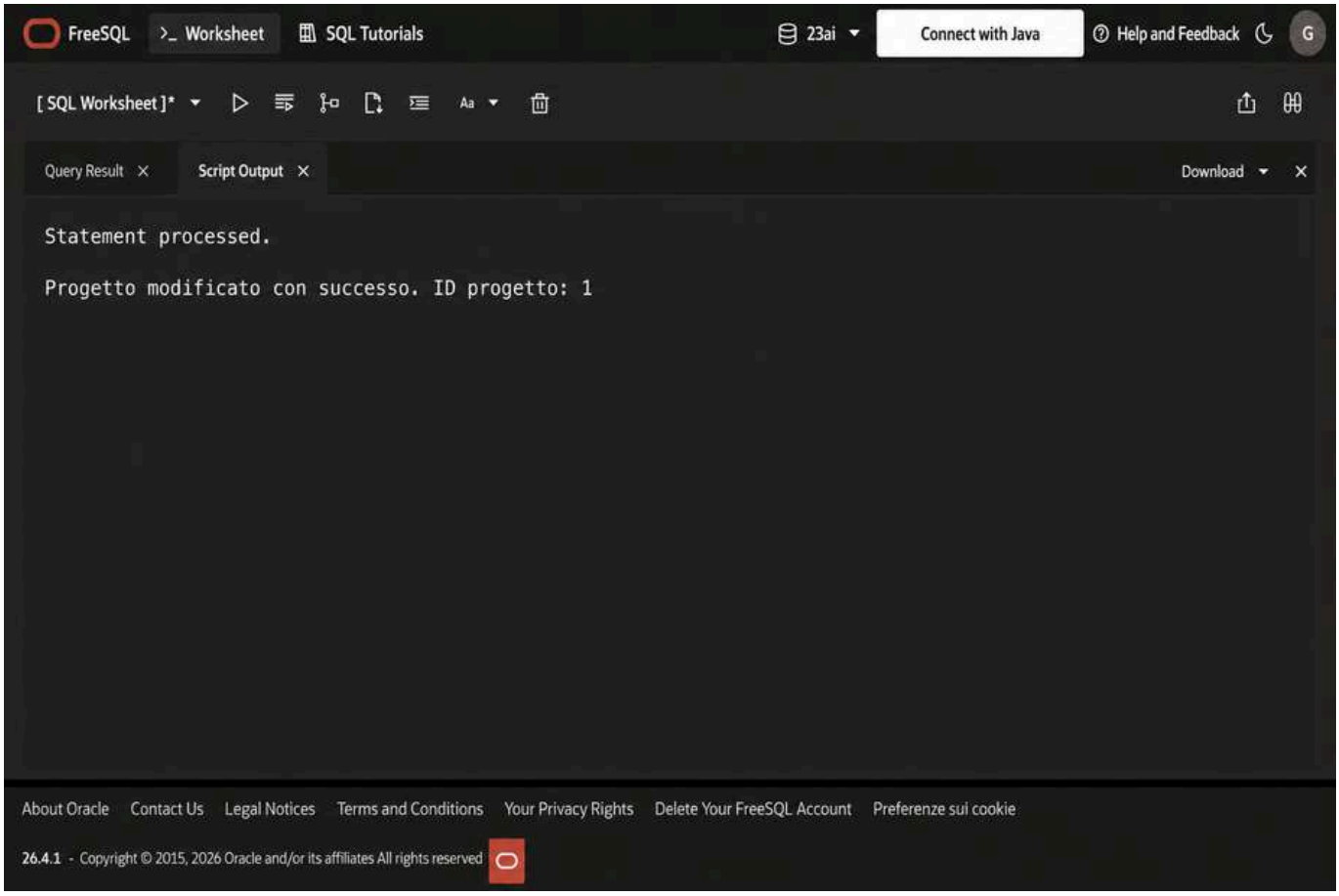
Implementazione

Procedura modifica_progetto (caso corretto)

FreeSQL | Worksheet | SQL Tutorials

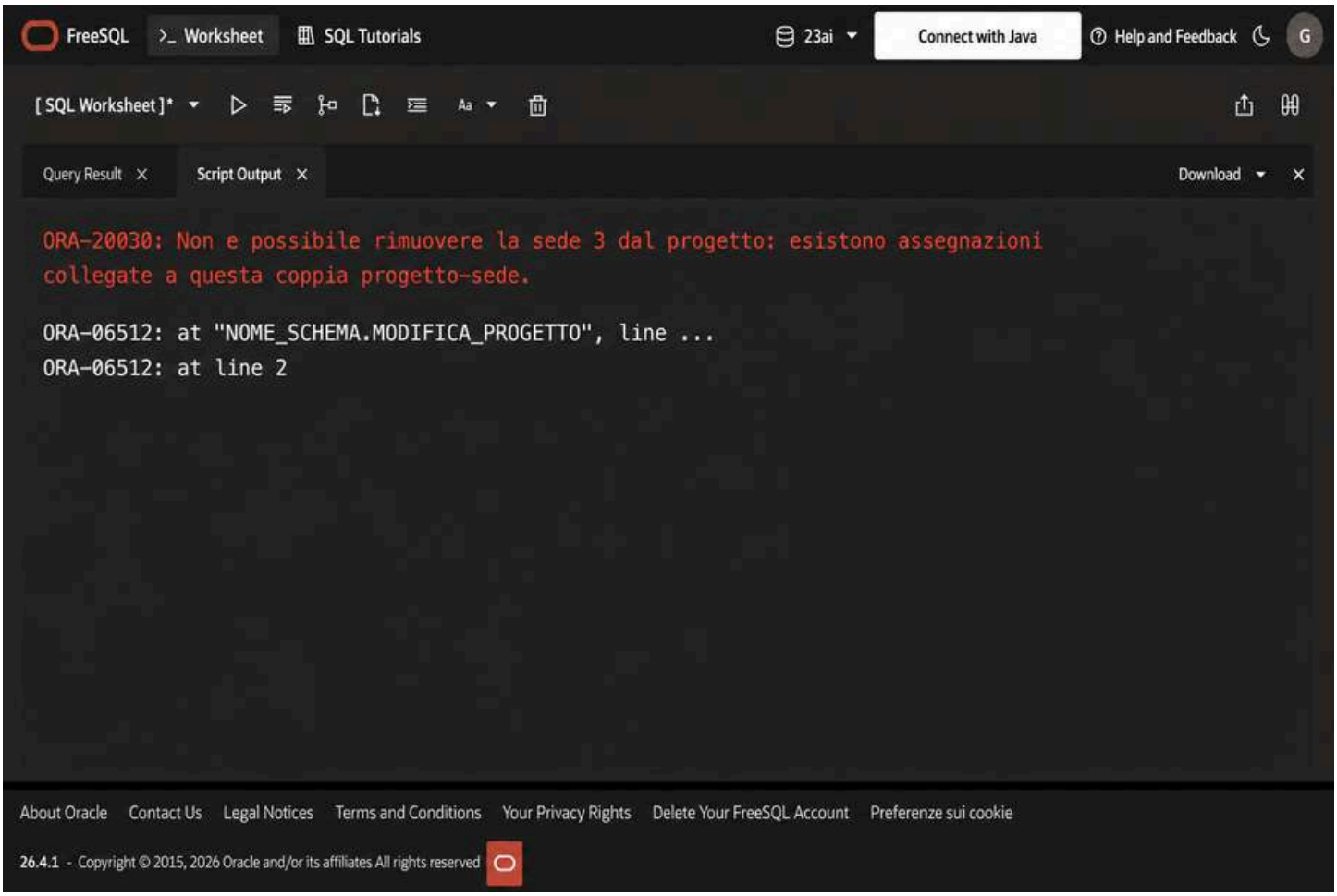
[SQL Worksheet]*

```
BEGIN
    modifica_progetto(
        1,
        'Progetto Giovani e Territorio – Aggiornato',
        'Inclusione sociale',
        'Versione aggiornata del progetto con nuove sedi di attuazione.',
        TO_DATE('2026-01-15','YYYY-MM-DD'),
        TO_DATE('2026-12-20','YYYY-MM-DD'),
        'ATTIVO',
        '1,2',
        5
    );
END;
/
```



Implementazione

Procedura modifica_progetto (caso errato)



Implementazione

Procedura disattiva_progetto

La procedura **disattiva_progetto** è progettata per disattivare un progetto di Servizio Civile Universale già esistente, impedendone l'utilizzo per nuove operazioni di assegnazione e mantenendo coerente il sistema rispetto alle relazioni già presenti nel database.
L'operazione non elimina il progetto dal sistema, ma ne modifica lo stato logico, in modo che il progetto resti memorizzato per finalità storiche, documentali e amministrative, senza però poter essere riutilizzato per nuovi inserimenti operativi.
I passaggi eseguiti dalla procedura sono i seguenti:

1. **Controllo del parametro in ingresso:** La procedura verifica che l'identificativo del progetto sia stato fornito correttamente. Se p_id_progetto è nullo, l'operazione viene interrotta immediatamente.
2. **Verifica dell'esistenza del progetto:** La procedura controlla che il progetto indicato esista realmente nella tabella **PROGETTO**. Se il progetto non è presente nel database, la disattivazione non può essere eseguita.
3. **Controllo dello stato attuale del progetto:** Dopo aver verificato l'esistenza del progetto, la procedura recupera il valore corrente di stato_progetto. Se il progetto risulta già disattivato, la procedura blocca l'operazione per evitare aggiornamenti inutili o incoerenti.

Procedura disattiva_progetto

4. **Controllo delle assegnazioni attive collegate**: La procedura verifica se esistono record nella tabella **ASSEGNAZIONE** associati al progetto con stato **ATTIVA**. Questo controllo è fondamentale, perché un progetto che ha ancora assegnazioni attive non può essere disattivato senza compromettere la coerenza amministrativa del percorso dei volontari.

In altri termini:

- se il progetto è ancora coinvolto in assegnazioni attive, la procedura non procede;
- se invece non esistono assegnazioni attive collegate, la disattivazione può essere eseguita.

5. **Gestione delle relazioni collegate:** La procedura non elimina le righe della tabella **PROGETTO_SEDE** e non modifica i dati storici già presenti nelle altre tabelle. Questa scelta è voluta, perché la disattivazione del progetto deve impedire nuovi utilizzi futuri del progetto stesso, ma non deve cancellare la storia amministrativa delle sedi collegate né alterare i dati già registrati nel sistema.

6. **Aggiornamento dello stato del progetto:** Se tutti i controlli precedenti hanno esito positivo, la procedura aggiorna la tabella PROGETTO, modificando il valore di stato_progetto in uno stato di non operatività, rappresentato in questa implementazione da SOSPESO. In questo modo il progetto:

- rimane presente nel database;
- non viene eliminato;
- non è più considerato disponibile per nuove assegnazioni.

7. **Conferma della transazione:** Una volta completato correttamente l'aggiornamento, la procedura esegue il **COMMIT,** rendendo definitiva la disattivazione.

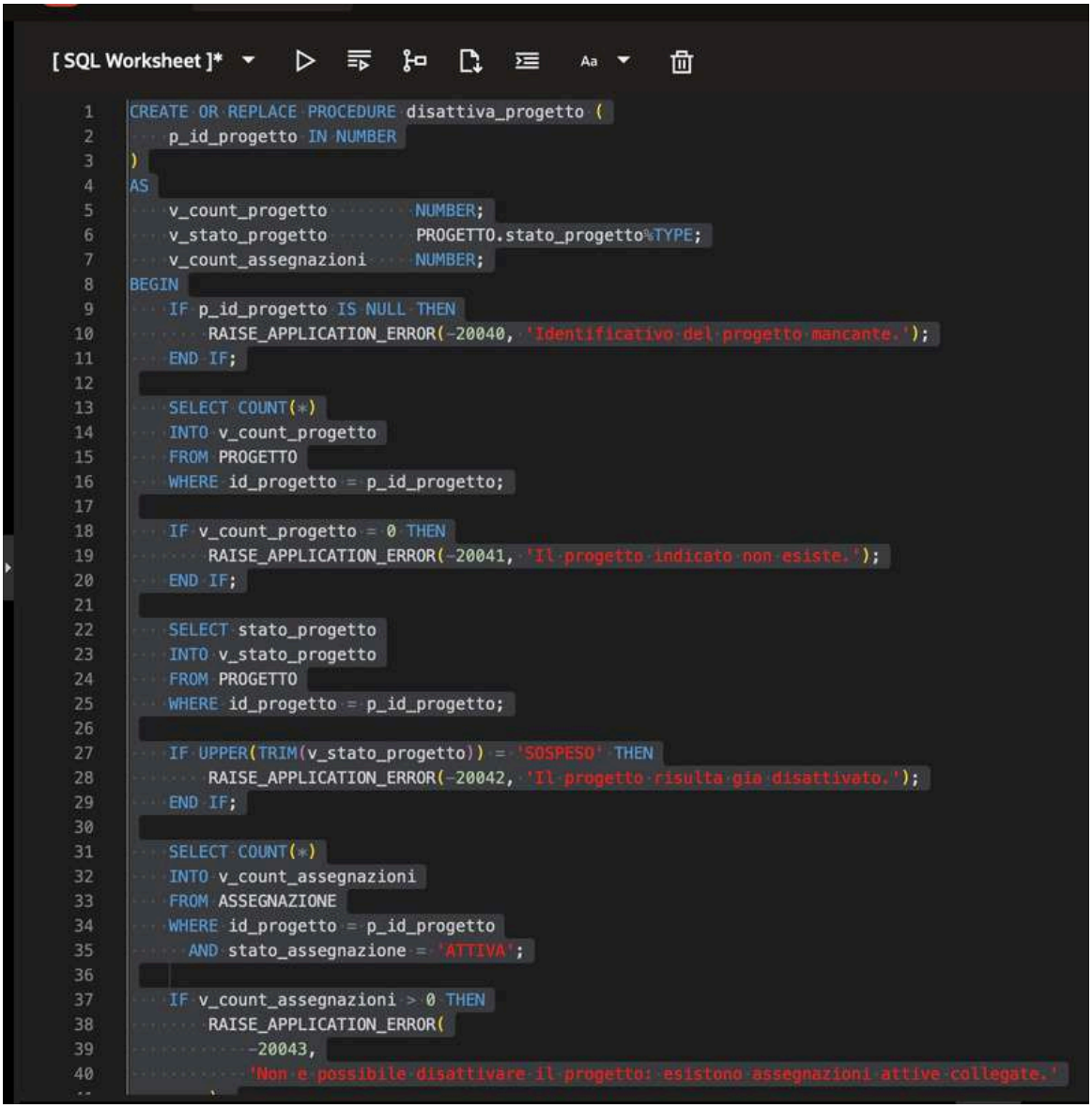
Implementazione

Procedura disattiva_progetto

8. **Messaggio di conferma:** Al termine dell'operazione, la procedura stampa un messaggio tramite **DBMS_OUTPUT.PUT_LINE,** indicando che la disattivazione del progetto è avvenuta con successo.
9. **Gestione esplicita degli errori:** Se durante l'esecuzione si verifica un errore, la procedura:

- esegue **ROLLBACK;**
- annulla tutte le modifiche eventualmente avviate;
- rilancia l'eccezione tramite **RAISE.**

Questo comportamento garantisce la coerenza transazionale dell'operazione e rispetta il requisito progettuale di gestione esplicita delle eccezioni.



Implementazione

Procedura disattiva_progetto

FreeSQL | Worksheet | SQL Tutorials

[SQL Worksheet]*

```
8     BEGIN
37        IF v_count_assegnazioni > 0 THEN
38            RAISE_APPLICATION_ERROR(
39                -20043,
40                'Non è possibile disattivare il progetto: esistono assegnazioni attive collegate.'
41            );
42        END IF;
43
44        UPDATE PROGETTO
45        SET stato_progetto = 'SOSPESO'
46        WHERE id_progetto = p_id_progetto;
47
48        COMMIT;
49        DBMS_OUTPUT.PUT_LINE('Progetto disattivato con successo. ID progetto: ' || p_id_progetto);
50
51    EXCEPTION
52        WHEN OTHERS THEN
53            ROLLBACK;
54            RAISE;
55    END;
56    /
```

Procedura disattiva_progetto (caso corretto)

FreeSQL | Worksheet | SQL Tutorials

[SQL Worksheet]*

```
1   BEGIN
2       disattiva_progetto(
3           8,
4           'Disattivazione per conclusione anticipata del progetto.'
5       );
6   END;
7   /
```

FreeSQL | Worksheet | SQL Tutorials | 23ai | Connect with Java | Help and Feedback

[SQL Worksheet]*

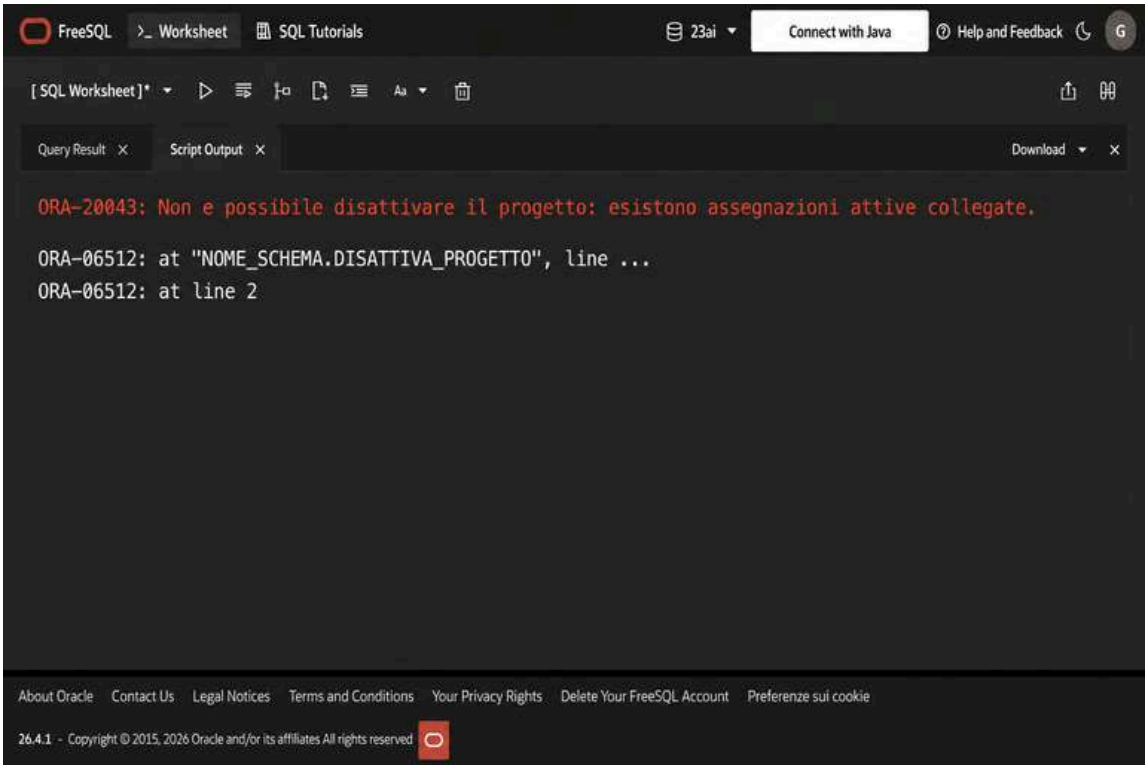
Query Result | Script Output | Download

```
Statement processed.

Progetto disattivato con successo. ID progetto: 8.
Sedi collegate mantenute: N.
```

Implementazione

Procedura disattiva_progetto (caso errato)



Implementazione

Procedura crea_assegnazione

La procedura **crea_assegnazione** è progettata per creare una nuova assegnazione tra un volontario, un progetto e una sede, per un determinato intervallo temporale, nel rispetto dei vincoli di coerenza amministrativa previsti dal sistema.

L'assegnazione rappresenta il fulcro del percorso del volontario nel progetto, poiché collega il soggetto assegnato, il progetto di riferimento, la sede operativa e i parametri amministrativi del servizio, come monte ore settimanale e limiti di permessi e malattie.

I passaggi eseguiti dalla procedura sono i seguenti:

1. **Controllo dei dati obbligatori**
   a. La procedura verifica che siano presenti:
   b. identificativo del volontario
   c. identificativo del progetto
   d. identificativo della sede
   e. data di inizio
   f. data di fine prevista
   g. stato dell'assegnazione
   h. ore settimanali previste
   i. limite massimo di permessi ordinari
   j. limite massimo di malattie retribuite
   k. limite massimo di malattie non retribuite
2. **Controllo di coerenza delle date:** Viene verificato che la data di fine prevista non sia antecedente alla data di inizio.
3. **Controllo dei valori amministrativi:**

La procedura verifica che:

- **Ore_settimanali_previste** sia maggiore di zero;
- I massimali di permessi e malattie siano maggiori o uguali a zero.

4. **Controllo esistenza del volontario**

La procedura verifica che il volontario indicato esista nella tabella **VOLONTARIO.**

Implementazione

Procedura crea_assegnazione

5. **Controllo esistenza e disponibilità del progetto:** La procedura verifica che il progetto indicato esista nella tabella **PROGETTO** e che non sia in stato di non operatività. In questa implementazione si considera non disponibile un progetto con stato **SOSPESO,** coerentemente con la procedura di disattivazione.
6. **Controllo esistenza della sede:** La procedura verifica che la sede indicata esista nella tabella **SEDE.**
7. **Controllo della validità della coppia progetto-sede:** La procedura verifica che la coppia id_progetto / id_sede esista nella tabella **PROGETTO_SEDE,** in coerenza con il vincolo dinamico del progetto.
8. **Controllo di incompatibilità con altre assegnazioni del volontario:** La procedura verifica che il volontario non abbia già un'altra assegnazione attiva e temporalmente sovrapposta nello stesso periodo. Questo recepisce il vincolo dinamico di unicità dell'assegnazione attiva.
9. **Controllo dell'eventuale assegnazione precedente:**

Se viene fornito p_id_assegnazione_precedente, la procedura verifica che:

- l'assegnazione precedente esista;
- appartenga allo stesso volontario;
- sia già conclusa;
- abbia una data_fine_effettiva valorizzata;
- tale data non sia successiva alla nuova data_inizio.

Questo rende la procedura coerente con la gestione dello storico nelle variazioni approvate.

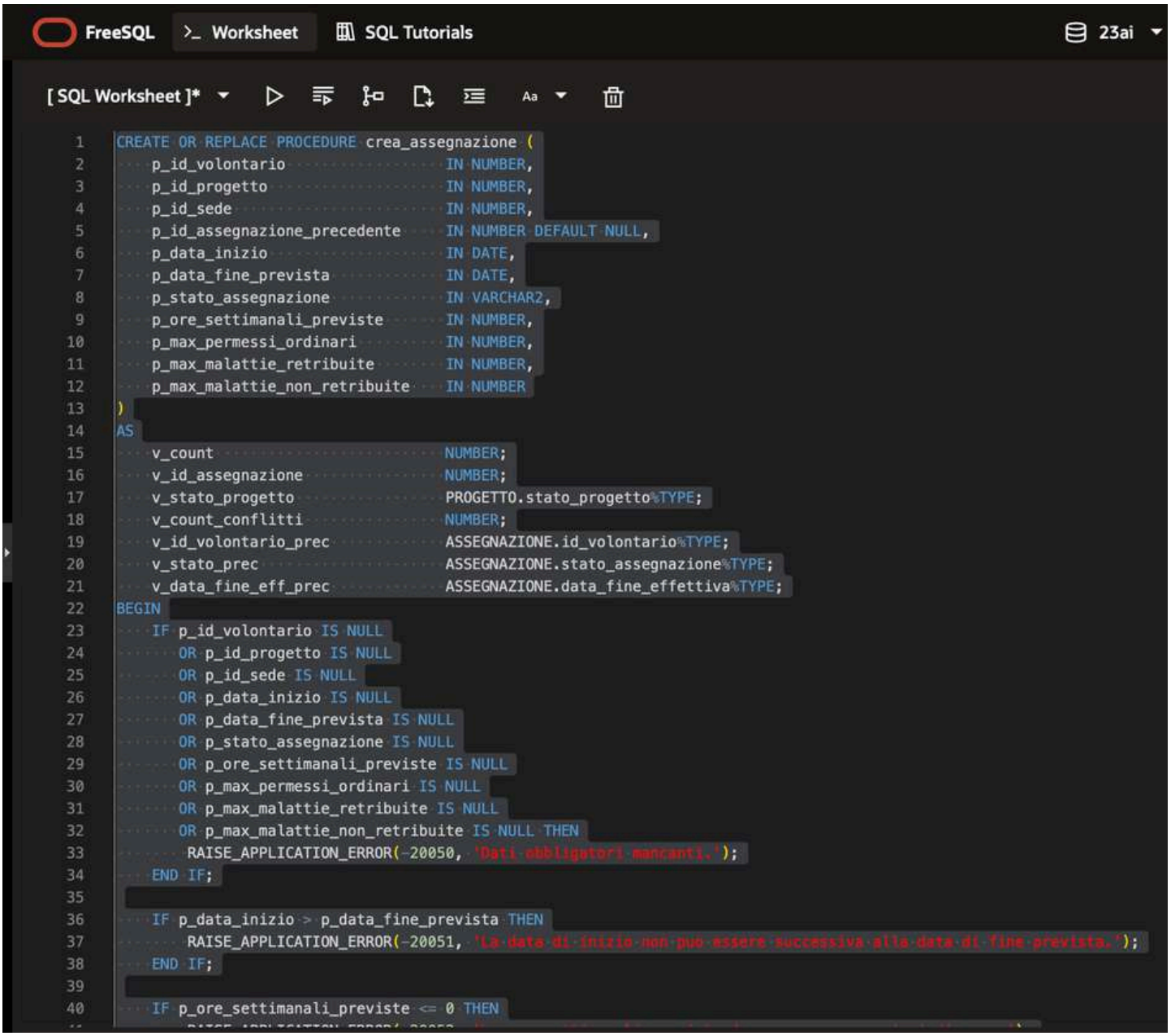
10. **Creazione della nuova assegnazione:** Se tutti i controlli hanno esito positivo:

- viene generato un nuovo identificativo
- viene inserita la nuova riga nella tabella ASSEGNAZIONE
- data_fine_effettiva viene inizialmente impostata a NULL

11. **Gestione della transazione:** Se l'operazione va a buon fine, viene eseguito **COMMIT** e viene stampato un messaggio di conferma.
In caso di errore, viene eseguito **ROLLBACK** e l'eccezione viene rilanciata.

Implementazione

Procedura crea_assegnazione



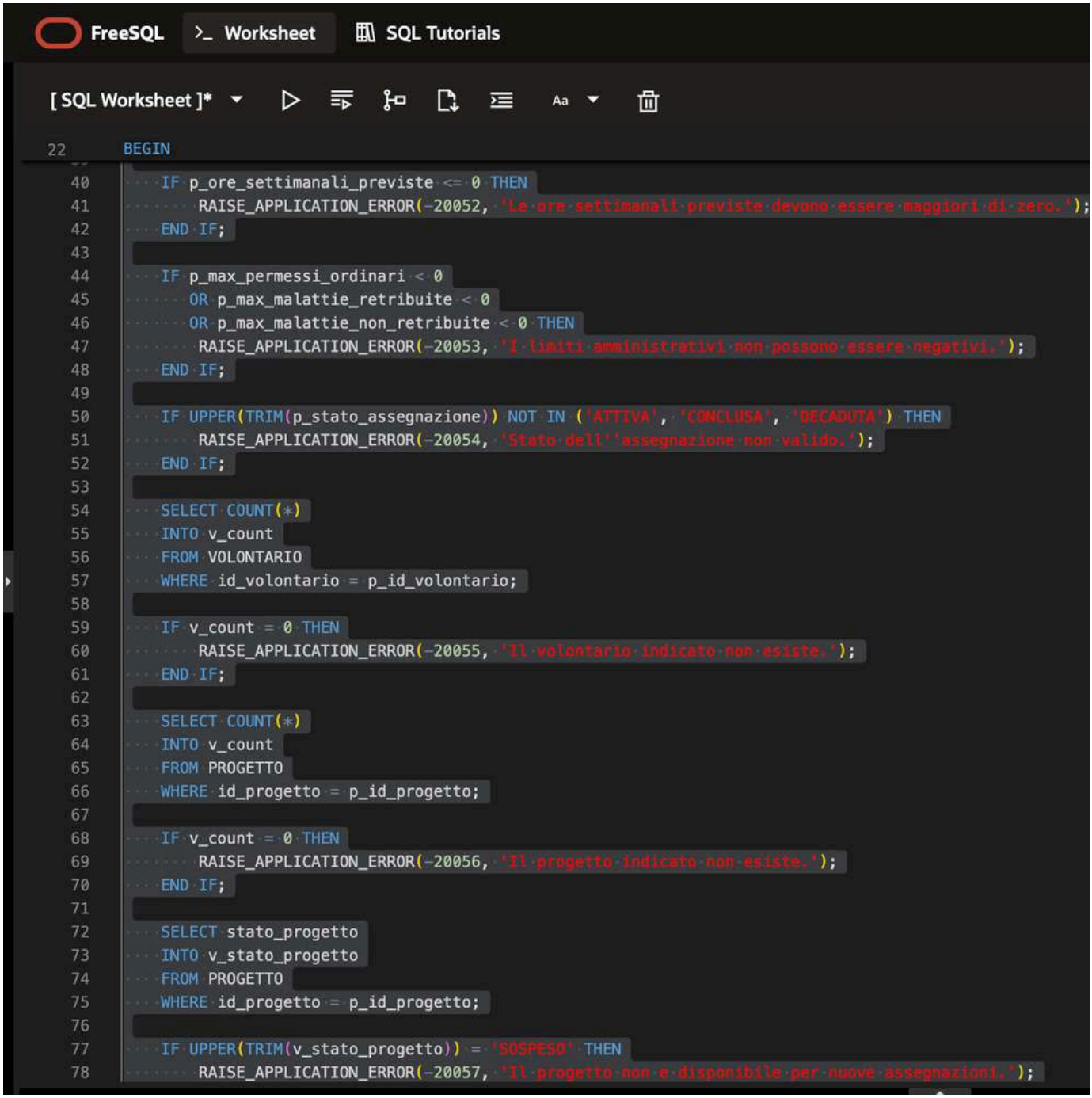

```
CREATE OR REPLACE PROCEDURE crea_assegnazione (
    p_id_volontario                 IN NUMBER,
    p_id_progetto                   IN NUMBER,
    p_id_sede                       IN NUMBER,
    p_id_assegnazione_precedente    IN NUMBER DEFAULT NULL,
    p_data_inizio                   IN DATE,
    p_data_fine_prevista            IN DATE,
    p_stato_assegnazione            IN VARCHAR2,
    p_ore_settimanali_previste      IN NUMBER,
    p_max_permessi_ordinari         IN NUMBER,
    p_max_malattie_retribuite       IN NUMBER,
    p_max_malattie_non_retribuite   IN NUMBER
)
AS
    v_count                         NUMBER;
    v_id_assegnazione               NUMBER;
    v_stato_progetto                PROGETTO.stato_progetto%TYPE;
    v_count_conflitti               NUMBER;
    v_id_volontario_prec            ASSEGNAZIONE.id_volontario%TYPE;
    v_stato_prec                    ASSEGNAZIONE.stato_assegnazione%TYPE;
    v_data_fine_eff_prec            ASSEGNAZIONE.data_fine_effettiva%TYPE;
BEGIN
    IF p_id_volontario IS NULL
       OR p_id_progetto IS NULL
       OR p_id_sede IS NULL
       OR p_data_inizio IS NULL
       OR p_data_fine_prevista IS NULL
       OR p_stato_assegnazione IS NULL
       OR p_ore_settimanali_previste IS NULL
       OR p_max_permessi_ordinari IS NULL
       OR p_max_malattie_retribuite IS NULL
       OR p_max_malattie_non_retribuite IS NULL THEN
        RAISE_APPLICATION_ERROR(-20050, 'Dati obbligatori mancanti.');
    END IF;

    IF p_data_inizio > p_data_fine_prevista THEN
        RAISE_APPLICATION_ERROR(-20051, 'La data di inizio non può essere successiva alla data di fine prevista.');
    END IF;

    IF p_ore_settimanali_previste <= 0 THEN
```

Implementazione

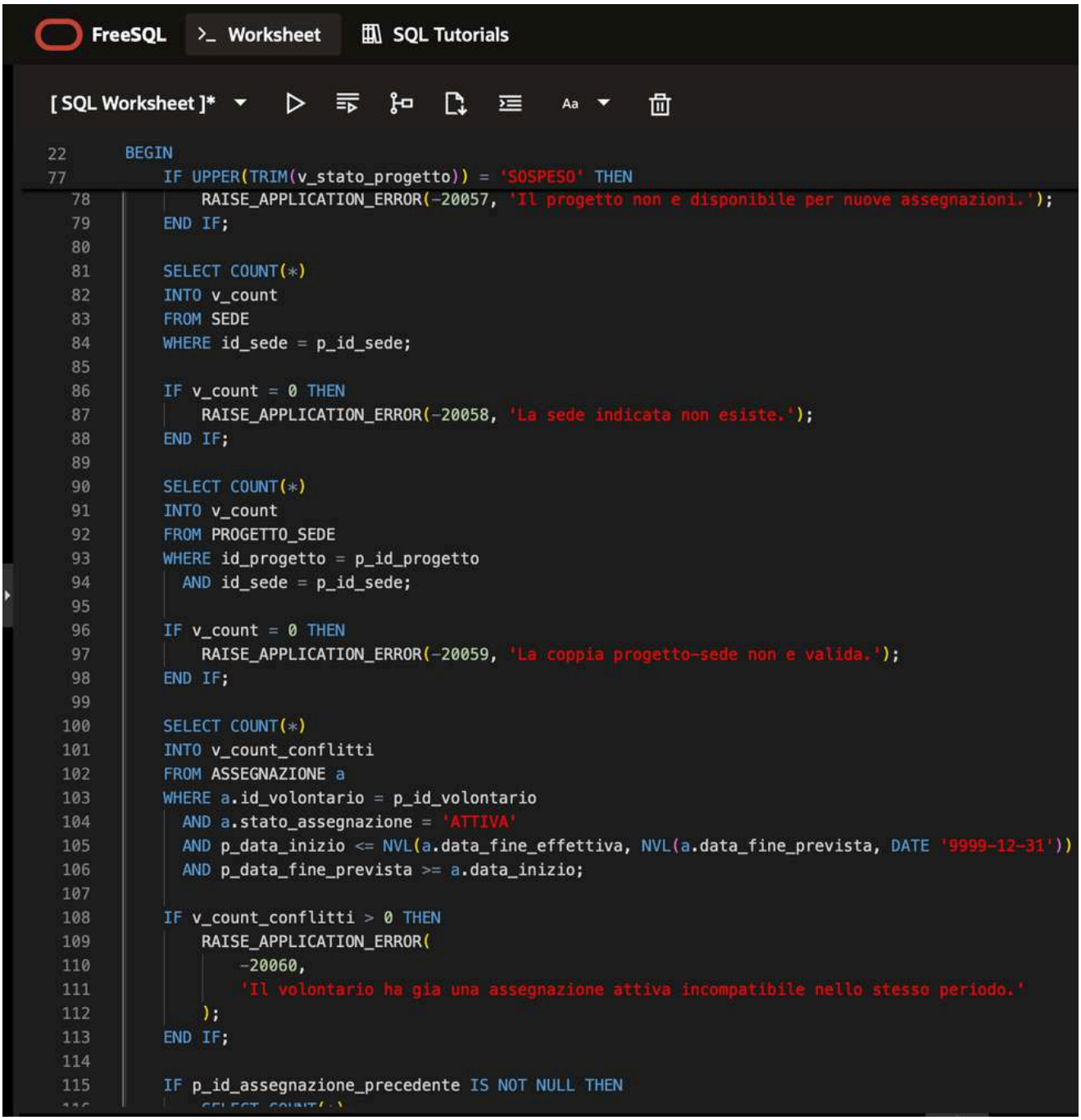
Procedura crea_assegnazione




```
22  BEGIN
40      IF p_ore_settimanali_previste <= 0 THEN
41          RAISE_APPLICATION_ERROR(-20052, 'Le ore settimanali previste devono essere maggiori di zero.');
42      END IF;
43
44      IF p_max_permessi_ordinari < 0
45          OR p_max_malattie_retribuite < 0
46          OR p_max_malattie_non_retribuite < 0 THEN
47          RAISE_APPLICATION_ERROR(-20053, 'I limiti amministrativi non possono essere negativi.');
48      END IF;
49
50      IF UPPER(TRIM(p_stato_assegnazione)) NOT IN ('ATTIVA', 'CONCLUSA', 'DECADUTA') THEN
51          RAISE_APPLICATION_ERROR(-20054, 'Stato dell''assegnazione non valido.');
52      END IF;
53
54      SELECT COUNT(*)
55      INTO v_count
56      FROM VOLONTARIO
57      WHERE id_volontario = p_id_volontario;
58
59      IF v_count = 0 THEN
60          RAISE_APPLICATION_ERROR(-20055, 'Il volontario indicato non esiste.');
61      END IF;
62
63      SELECT COUNT(*)
64      INTO v_count
65      FROM PROGETTO
66      WHERE id_progetto = p_id_progetto;
67
68      IF v_count = 0 THEN
69          RAISE_APPLICATION_ERROR(-20056, 'Il progetto indicato non esiste.');
70      END IF;
71
72      SELECT stato_progetto
73      INTO v_stato_progetto
74      FROM PROGETTO
75      WHERE id_progetto = p_id_progetto;
76
77      IF UPPER(TRIM(v_stato_progetto)) = 'SOSPESO' THEN
78          RAISE_APPLICATION_ERROR(-20057, 'Il progetto non è disponibile per nuove assegnazioni.');
```

Implementazione

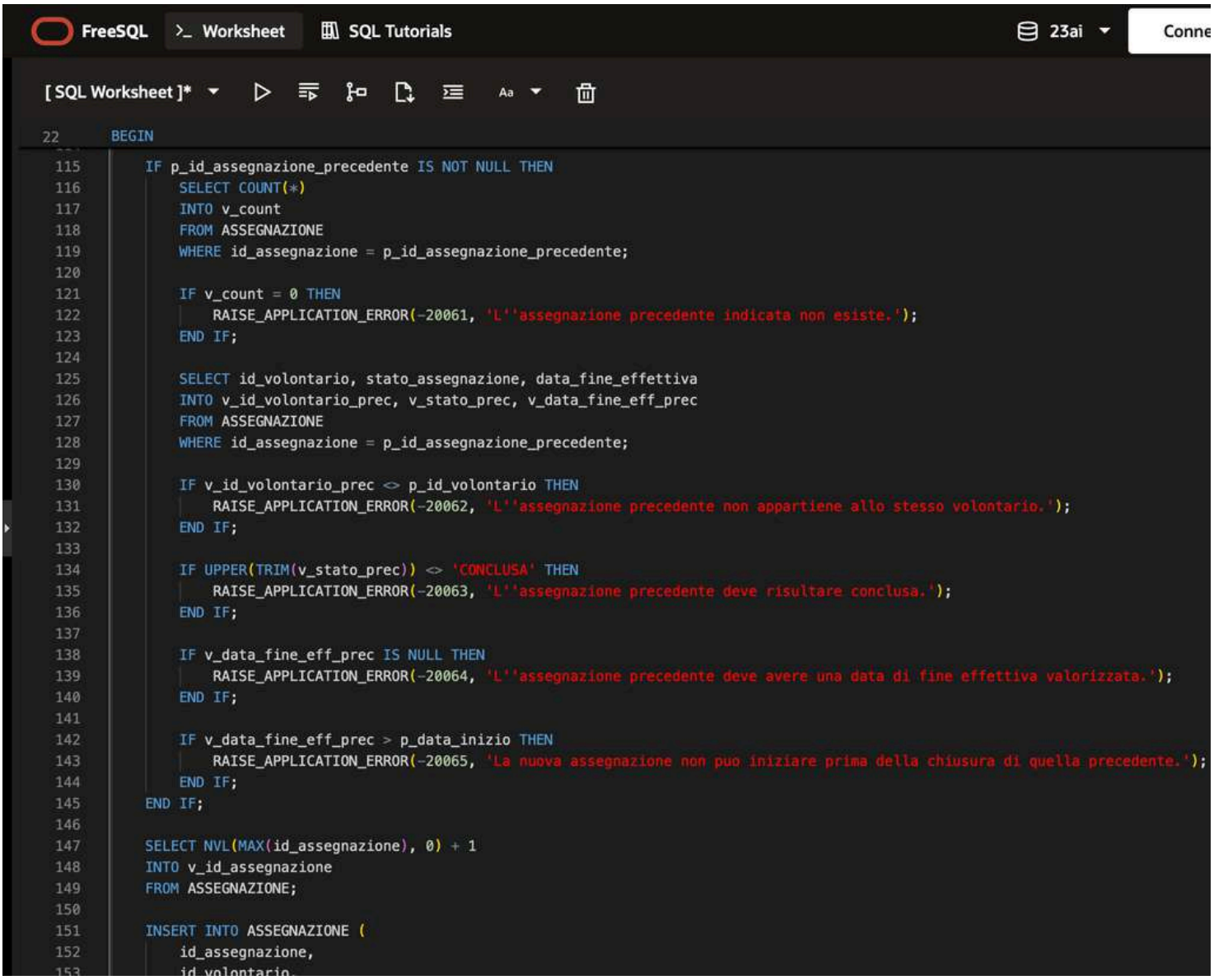
Procedura crea_assegnazione




```
22      BEGIN
77          IF UPPER(TRIM(v_stato_progetto)) = 'SOSPESO' THEN
78              RAISE_APPLICATION_ERROR(-20057, 'Il progetto non e disponibile per nuove assegnazioni.');
79          END IF;
80
81          SELECT COUNT(*)
82          INTO v_count
83          FROM SEDE
84          WHERE id_sede = p_id_sede;
85
86          IF v_count = 0 THEN
87              RAISE_APPLICATION_ERROR(-20058, 'La sede indicata non esiste.');
88          END IF;
89
90          SELECT COUNT(*)
91          INTO v_count
92          FROM PROGETTO_SEDE
93          WHERE id_progetto = p_id_progetto
94            AND id_sede = p_id_sede;
95
96          IF v_count = 0 THEN
97              RAISE_APPLICATION_ERROR(-20059, 'La coppia progetto-sede non e valida.');
98          END IF;
99
100         SELECT COUNT(*)
101         INTO v_count_conflitti
102         FROM ASSEGNAZIONE a
103         WHERE a.id_volontario = p_id_volontario
104           AND a.stato_assegnazione = 'ATTIVA'
105           AND p_data_inizio <= NVL(a.data_fine_effettiva, NVL(a.data_fine_prevista, DATE '9999-12-31'))
106           AND p_data_fine_prevista >= a.data_inizio;
107
108         IF v_count_conflitti > 0 THEN
109             RAISE_APPLICATION_ERROR(
110                 -20060,
111                 'Il volontario ha gia una assegnazione attiva incompatibile nello stesso periodo.'
112             );
113         END IF;
114
115         IF p_id_assegnazione_precedente IS NOT NULL THEN
```

Implementazione

Procedura crea_assegnazione



Implementazione

Procedura crea_assegnazione

FreeSQL | Worksheet | SQL Tutorials

[SQL Worksheet]*

```
22      BEGIN
151         INSERT INTO ASSEGNAZIONE (
152             id_assegnazione,
153             id_volontario,
154             id_progetto,
155             id_sede,
156             id_assegnazione_precedente,
157             data_inizio,
158             data_fine_prevista,
159             data_fine_effettiva,
160             stato_assegnazione,
161             ore_settimanali_previste,
162             max_permessi_ordinari,
163             max_malattie_retribuite,
164             max_malattie_non_retribuite
165         )
166         VALUES (
167             v_id_assegnazione,
168             p_id_volontario,
169             p_id_progetto,
170             p_id_sede,
171             p_id_assegnazione_precedente,
172             p_data_inizio,
173             p_data_fine_prevista,
174             NULL,
175             UPPER(TRIM(p_stato_assegnazione)),
176             p_ore_settimanali_previste,
177             p_max_permessi_ordinari,
178             p_max_malattie_retribuite,
179             p_max_malattie_non_retribuite
180         );
181
182         COMMIT;
183         DBMS_OUTPUT.PUT_LINE('Assegnazione creata con successo. ID assegnazione: ' || v_id_assegnazione);
184
185     EXCEPTION
186         WHEN OTHERS THEN
187             ROLLBACK;
188             RAISE;
189     END;
```

Implementazione

Procedura crea_assegnazione (caso corretto)

FreeSQL >_ Worksheet SQL Tutorials

[SQL Worksheet]*

```
1   BEGIN
2       crea_assegnazione(
3           15,
4           4,
5           7,
6           NULL,
7           TO_DATE('2025-06-01','YYYY-MM-DD'),
8           TO_DATE('2025-12-31','YYYY-MM-DD'),
9           'ATTIVA',
10          35,
11          15,
12          10,
13          10
14      );
15  END;
16  /
```

FreeSQL >_ Worksheet SQL Tutorials 23ai Connect with Java Help and Feedback G

[SQL Worksheet]*

Query Result × Script Output × Download ×

```
Statement processed.

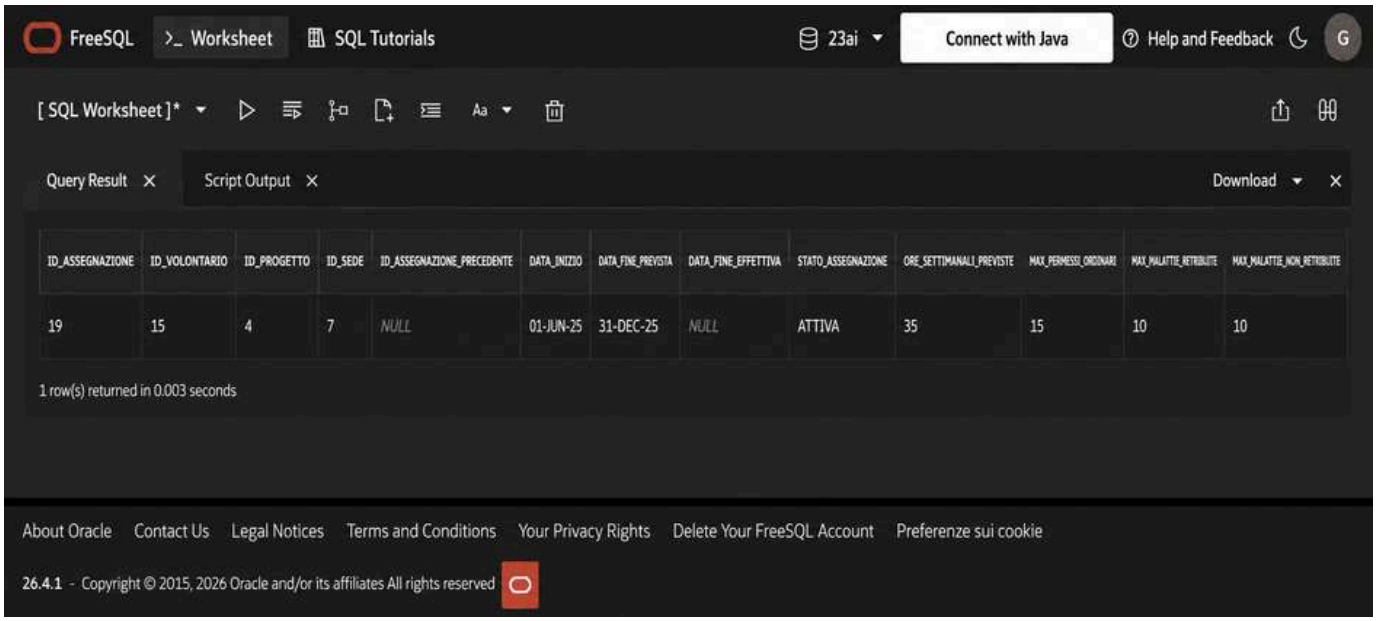
Assegnazione creata con successo. ID assegnazione: 19
```

About Oracle Contact Us Legal Notices Terms and Conditions Your Privacy Rights Delete Your FreeSQL Account Preferenze sui cookie

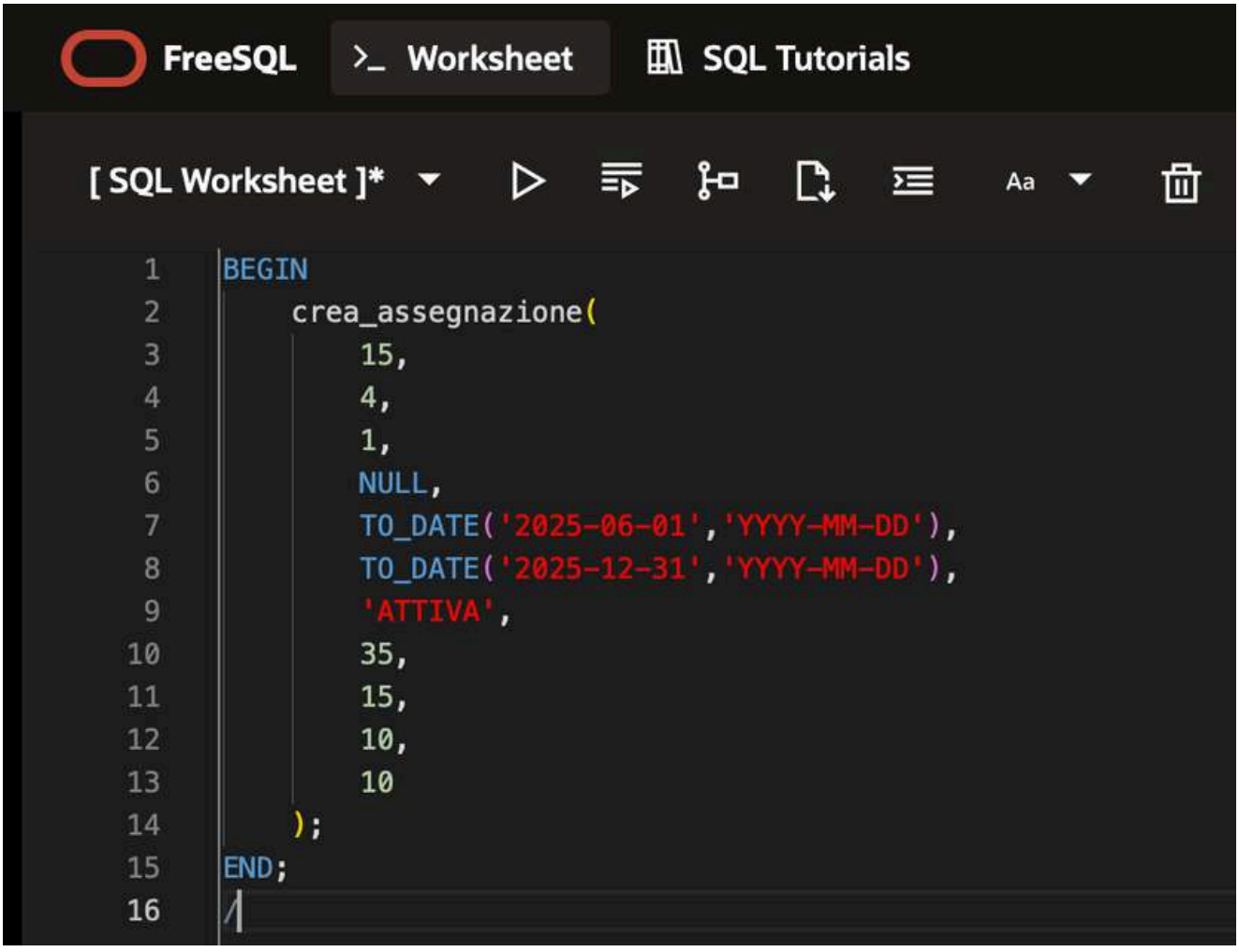
26.4.1 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

Implementazione

Procedura crea_assegnazione (caso corretto)

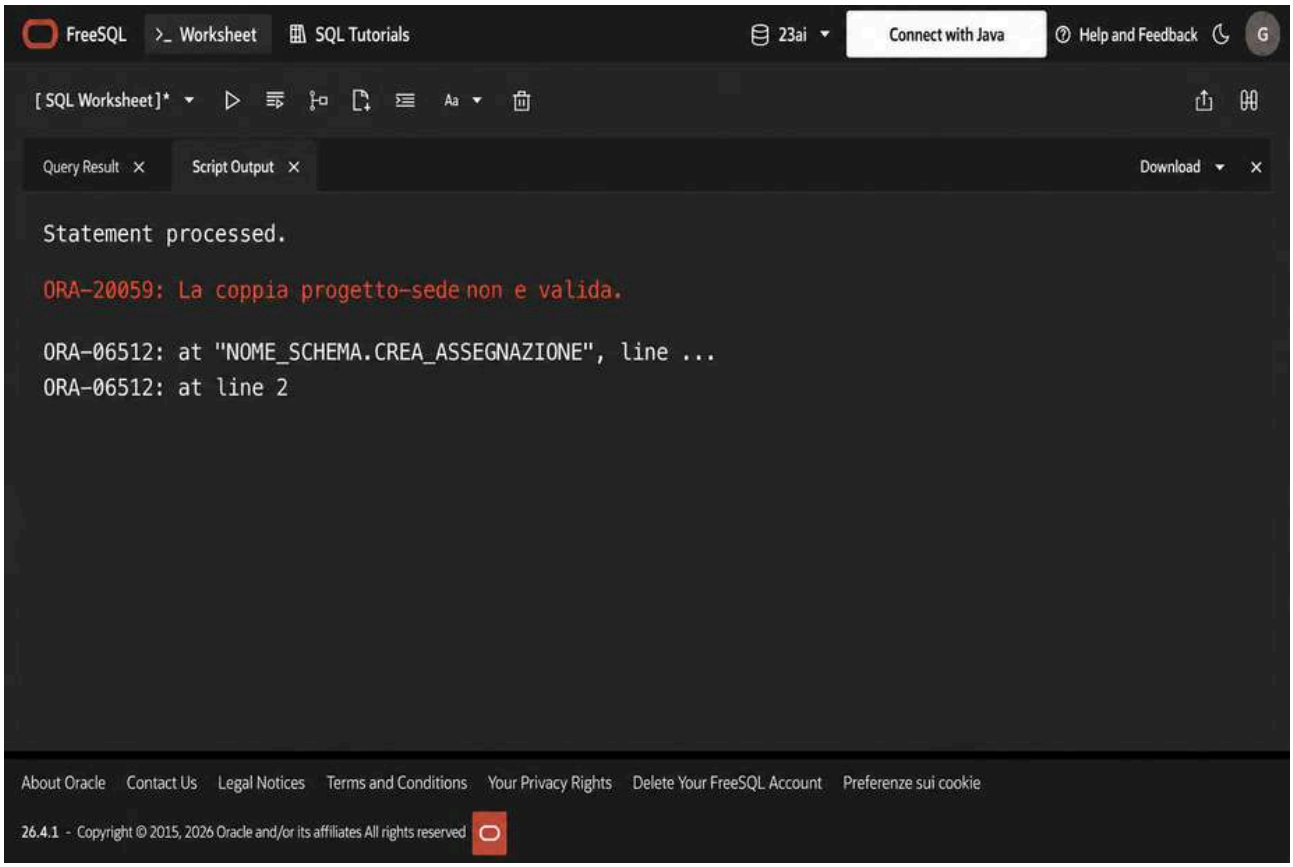


Procedura crea_assegnazione (caso errato)



Implementazione

Procedura crea_assegnazione (caso errato)



Implementazione

Procedura gestisci_richiesta_variazione

La procedura **gestisci_richiesta_variazione** è progettata per gestire formalmente una richiesta di variazione dell'assegnazione di un volontario. Se la richiesta viene approvata, la procedura chiude l'assegnazione corrente e ne crea una nuova collegata alla precedente, preservando lo storico amministrativo del percorso. Questa logica è coerente con l'operazione complessa prevista per l'amministratore e con il vincolo dinamico di gestione dello storico.

I passaggi eseguiti sono i seguenti:

1. **Controllo dei parametri obbligatori:**

La procedura verifica che siano presenti:

- identificativo della richiesta
- stato finale della richiesta
- data di validazione

Se la richiesta viene approvata, richiede anche:

- nuovo progetto;
- nuova sede;
- data di decorrenza.

2. **Controllo esistenza e stato della richiesta:**

La procedura controlla che la richiesta esista e che sia ancora **PENDENTE.**

3. **Recupero dell'assegnazione collegata:**

La procedura recupera l'assegnazione attuale associata alla richiesta, insieme ai parametri amministrativi che dovranno essere ereditati dalla nuova assegnazione.

4. **Controlli in caso di approvazione:**

Se la richiesta è approvata, la procedura verifica:

- esistenza del nuovo progetto
- esistenza della nuova sede
- validità della nuova coppia progetto-sede
- stato attivo dell'assegnazione corrente
- coerenza temporale della data di decorrenza

Implementazione

Procedura gestisci_richiesta_variazione

5. **Aggiornamento della richiesta:**

La procedura aggiorna stato_richiesta e data_validazione.

6. **Chiusura dell'assegnazione corrente:**

Se la richiesta è approvata, la vecchia assegnazione viene chiusa:

- stato_assegnazione = 'CONCLUSA'
- data_fine_effettiva = p_data_decorrenza - 1

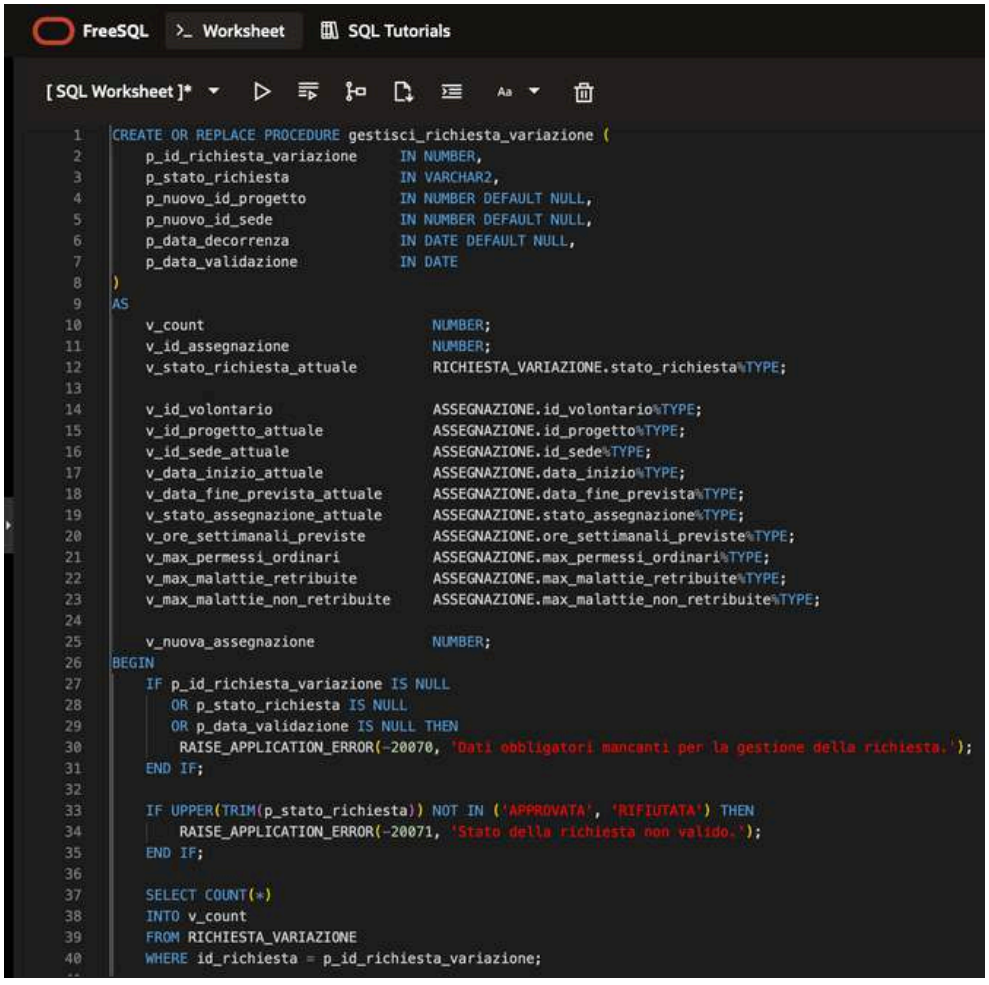
7. **Creazione della nuova assegnazione:**

La procedura crea una nuova assegnazione:

- collegata alla precedente tramite id_assegnazione_precedente
- con nuovo progetto e nuova sede
- con la stessa dotazione amministrativa della precedente
- con data_inizio = p_data_decorrenza

8. **Gestione transazionale:**

Se tutto va bene, viene eseguito COMMIT; in caso di errore, ROLLBACK e rilancio dell'eccezione.



Implementazione

Procedura gestisci_richiesta_variazione

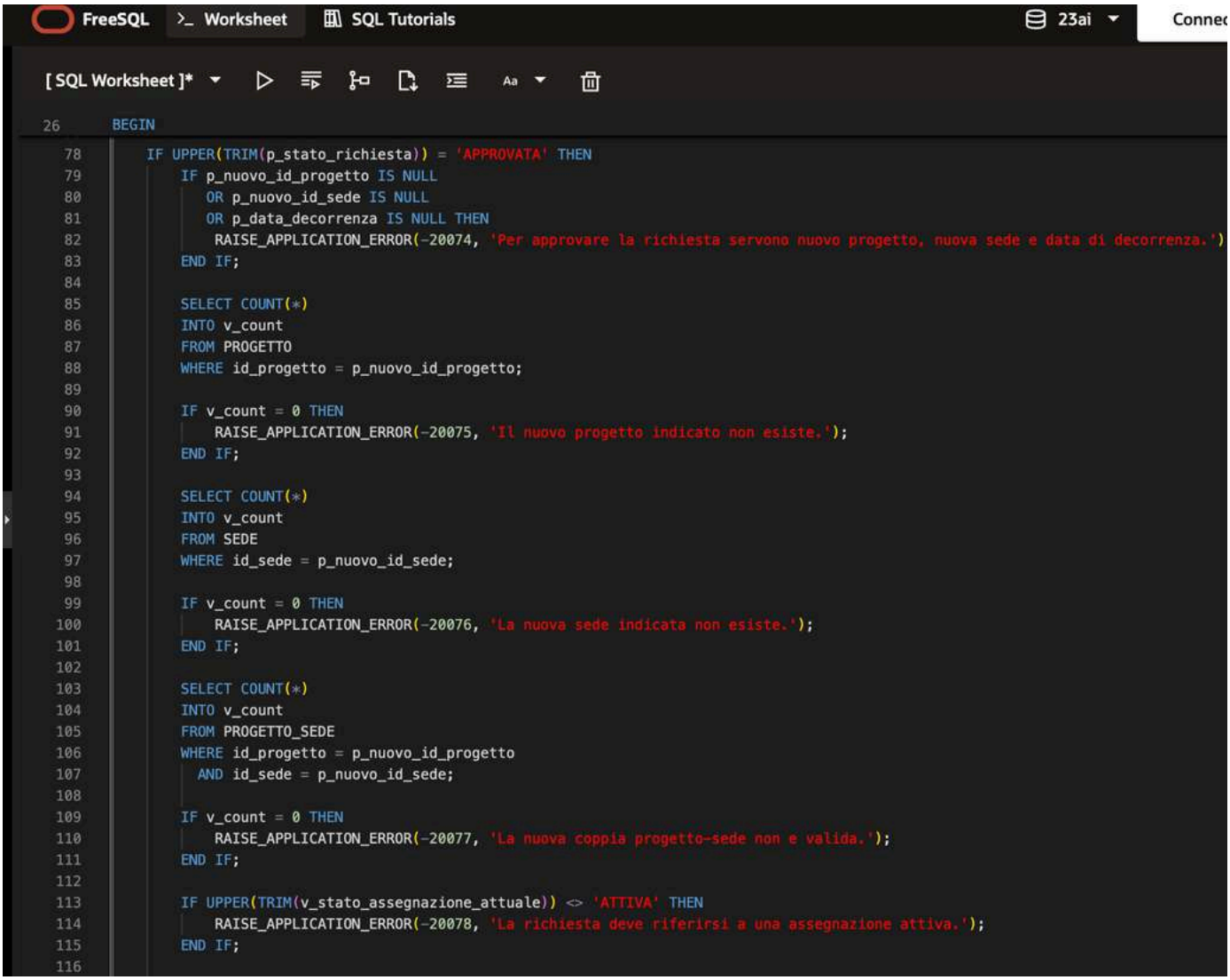
FreeSQL >_ Worksheet SQL Tutorials

[SQL Worksheet]*

```
26      BEGIN
40          WHERE id_richiesta = p_id_richiesta_variazione;
41
42          IF v_count = 0 THEN
43              RAISE_APPLICATION_ERROR(-20072, 'La richiesta indicata non esiste.');
44          END IF;
45
46          SELECT stato_richiesta, id_assegnazione
47          INTO v_stato_richiesta_attuale, v_id_assegnazione
48          FROM RICHIESTA_VARIAZIONE
49          WHERE id_richiesta = p_id_richiesta_variazione;
50
51          IF UPPER(TRIM(v_stato_richiesta_attuale)) <> 'PENDENTE' THEN
52              RAISE_APPLICATION_ERROR(-20073, 'La richiesta risulta gia valutata.');
53          END IF;
54
55          SELECT id_volontario,
56                 id_progetto,
57                 id_sede,
58                 data_inizio,
59                 data_fine_prevista,
60                 stato_assegnazione,
61                 ore_settimanali_previste,
62                 max_permessi_ordinari,
63                 max_malattie_retribuite,
64                 max_malattie_non_retribuite
65          INTO v_id_volontario,
66               v_id_progetto_attuale,
67               v_id_sede_attuale,
68               v_data_inizio_attuale,
69               v_data_fine_prevista_attuale,
70               v_stato_assegnazione_attuale,
71               v_ore_settimanali_previste,
72               v_max_permessi_ordinari,
73               v_max_malattie_retribuite,
74               v_max_malattie_non_retribuite
75          FROM ASSEGNAZIONE
76          WHERE id_assegnazione = v_id_assegnazione;
77
78          IF UPPER(TRIM(p_stato_richiesta)) = 'APPROVATA' THEN
```

Implementazione

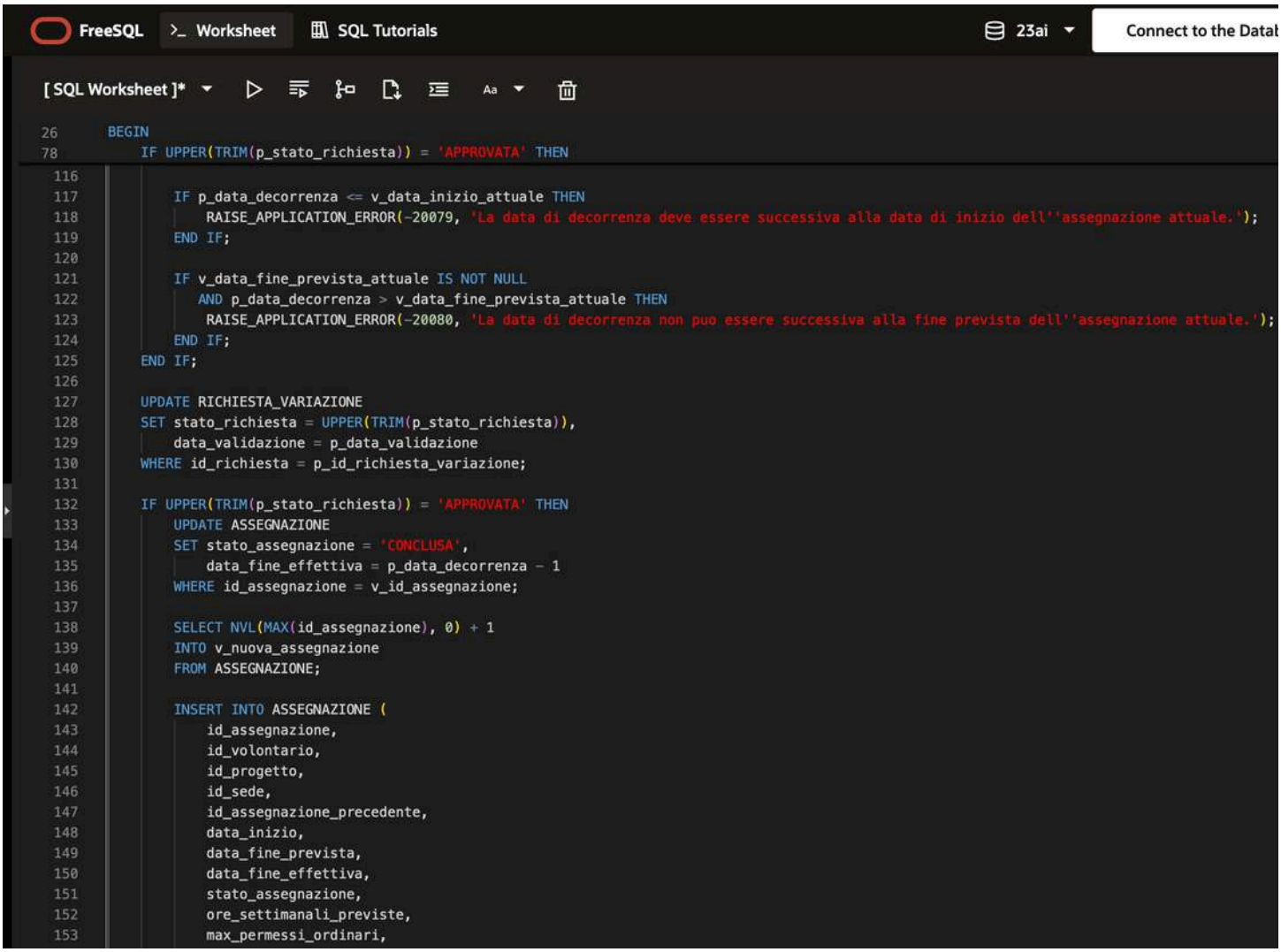
Procedura gestisci_richiesta_variazione




```
26      BEGIN
78          IF UPPER(TRIM(p_stato_richiesta)) = 'APPROVATA' THEN
79              IF p_nuovo_id_progetto IS NULL
80                 OR p_nuovo_id_sede IS NULL
81                 OR p_data_decorrenza IS NULL THEN
82                  RAISE_APPLICATION_ERROR(-20074, 'Per approvare la richiesta servono nuovo progetto, nuova sede e data di decorrenza.');
83              END IF;
84
85              SELECT COUNT(*)
86              INTO v_count
87              FROM PROGETTO
88              WHERE id_progetto = p_nuovo_id_progetto;
89
90              IF v_count = 0 THEN
91                  RAISE_APPLICATION_ERROR(-20075, 'Il nuovo progetto indicato non esiste.');
92              END IF;
93
94              SELECT COUNT(*)
95              INTO v_count
96              FROM SEDE
97              WHERE id_sede = p_nuovo_id_sede;
98
99              IF v_count = 0 THEN
100                 RAISE_APPLICATION_ERROR(-20076, 'La nuova sede indicata non esiste.');
101             END IF;
102
103             SELECT COUNT(*)
104             INTO v_count
105             FROM PROGETTO_SEDE
106             WHERE id_progetto = p_nuovo_id_progetto
107               AND id_sede = p_nuovo_id_sede;
108
109             IF v_count = 0 THEN
110                 RAISE_APPLICATION_ERROR(-20077, 'La nuova coppia progetto-sede non e valida.');
111             END IF;
112
113             IF UPPER(TRIM(v_stato_assegnazione_attuale)) <> 'ATTIVA' THEN
114                 RAISE_APPLICATION_ERROR(-20078, 'La richiesta deve riferirsi a una assegnazione attiva.');
115             END IF;
116
```

Implementazione

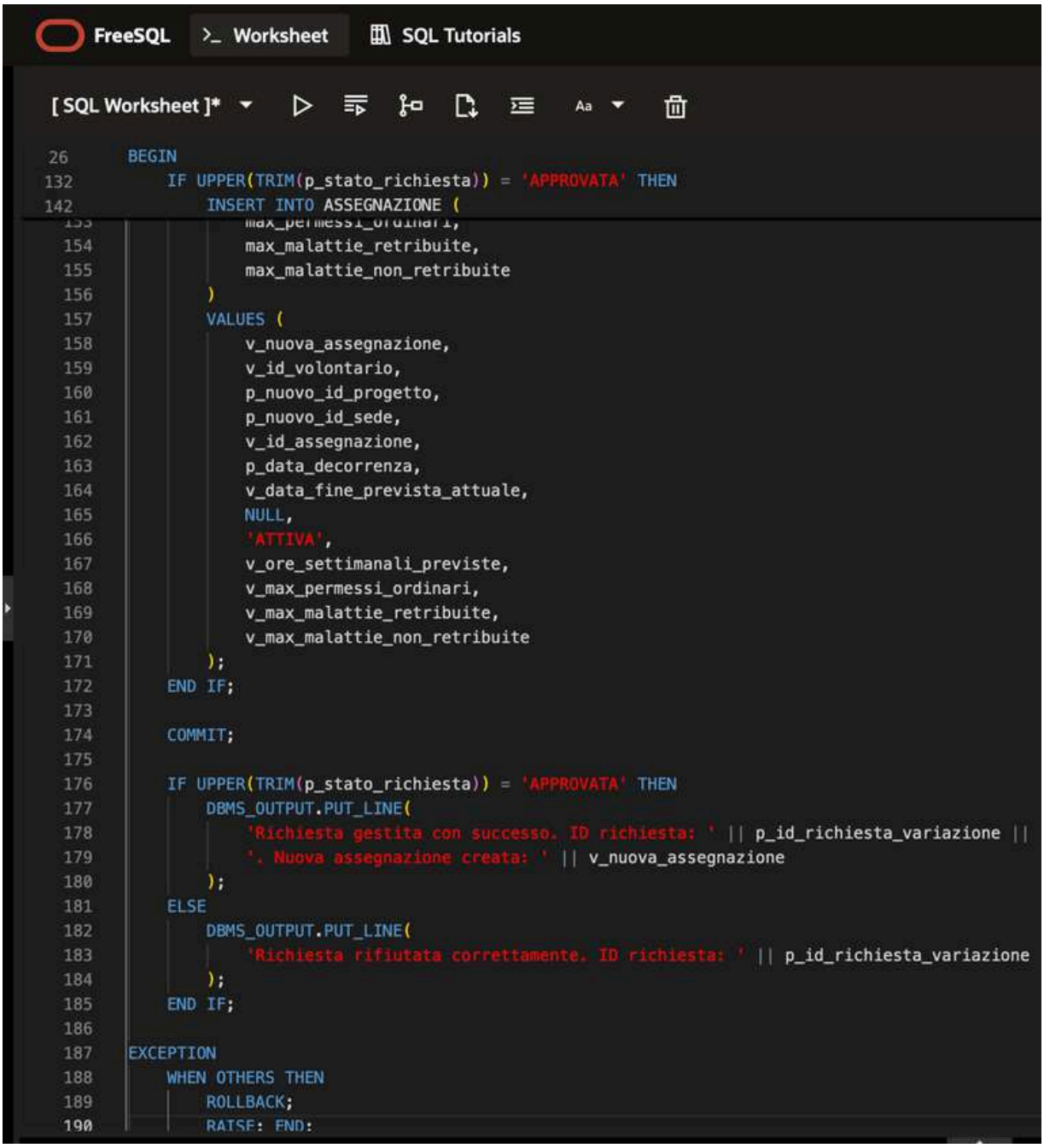
Procedura gestisci_richiesta_variazione




```
26      BEGIN
78          IF UPPER(TRIM(p_stato_richiesta)) = 'APPROVATA' THEN
116
117             IF p_data_decorrenza <= v_data_inizio_attuale THEN
118                 RAISE_APPLICATION_ERROR(-20079, 'La data di decorrenza deve essere successiva alla data di inizio dell''assegnazione attuale.');
119             END IF;
120
121             IF v_data_fine_prevista_attuale IS NOT NULL
122                AND p_data_decorrenza > v_data_fine_prevista_attuale THEN
123                 RAISE_APPLICATION_ERROR(-20080, 'La data di decorrenza non puo essere successiva alla fine prevista dell''assegnazione attuale.');
124             END IF;
125         END IF;
126
127         UPDATE RICHIESTA_VARIAZIONE
128         SET stato_richiesta = UPPER(TRIM(p_stato_richiesta)),
129             data_validazione = p_data_validazione
130         WHERE id_richiesta = p_id_richiesta_variazione;
131
132         IF UPPER(TRIM(p_stato_richiesta)) = 'APPROVATA' THEN
133             UPDATE ASSEGNAZIONE
134             SET stato_assegnazione = 'CONCLUSA',
135                 data_fine_effettiva = p_data_decorrenza - 1
136             WHERE id_assegnazione = v_id_assegnazione;
137
138             SELECT NVL(MAX(id_assegnazione), 0) + 1
139             INTO v_nuova_assegnazione
140             FROM ASSEGNAZIONE;
141
142             INSERT INTO ASSEGNAZIONE (
143                 id_assegnazione,
144                 id_volontario,
145                 id_progetto,
146                 id_sede,
147                 id_assegnazione_precedente,
148                 data_inizio,
149                 data_fine_prevista,
150                 data_fine_effettiva,
151                 stato_assegnazione,
152                 ore_settimanali_previste,
153                 max_permessi_ordinari,
```

Implementazione

Procedura gestisci_richiesta_variazione




```
26      BEGIN
132         IF UPPER(TRIM(p_stato_richiesta)) = 'APPROVATA' THEN
142             INSERT INTO ASSEGNAZIONE (
                    max_permessi_ordinari,
154                 max_malattie_retribuite,
155                 max_malattie_non_retribuite
156             )
157             VALUES (
158                 v_nuova_assegnazione,
159                 v_id_volontario,
160                 p_nuovo_id_progetto,
161                 p_nuovo_id_sede,
162                 v_id_assegnazione,
163                 p_data_decorrenza,
164                 v_data_fine_prevista_attuale,
165                 NULL,
166                 'ATTIVA',
167                 v_ore_settimanali_previste,
168                 v_max_permessi_ordinari,
169                 v_max_malattie_retribuite,
170                 v_max_malattie_non_retribuite
171             );
172         END IF;
173
174         COMMIT;
175
176         IF UPPER(TRIM(p_stato_richiesta)) = 'APPROVATA' THEN
177             DBMS_OUTPUT.PUT_LINE(
178                 'Richiesta gestita con successo. ID richiesta: ' || p_id_richiesta_variazione ||
179                 '. Nuova assegnazione creata: ' || v_nuova_assegnazione
180             );
181         ELSE
182             DBMS_OUTPUT.PUT_LINE(
183                 'Richiesta rifiutata correttamente. ID richiesta: ' || p_id_richiesta_variazione
184             );
185         END IF;
186
187     EXCEPTION
188         WHEN OTHERS THEN
189             ROLLBACK;
190             RAISE; END;
```

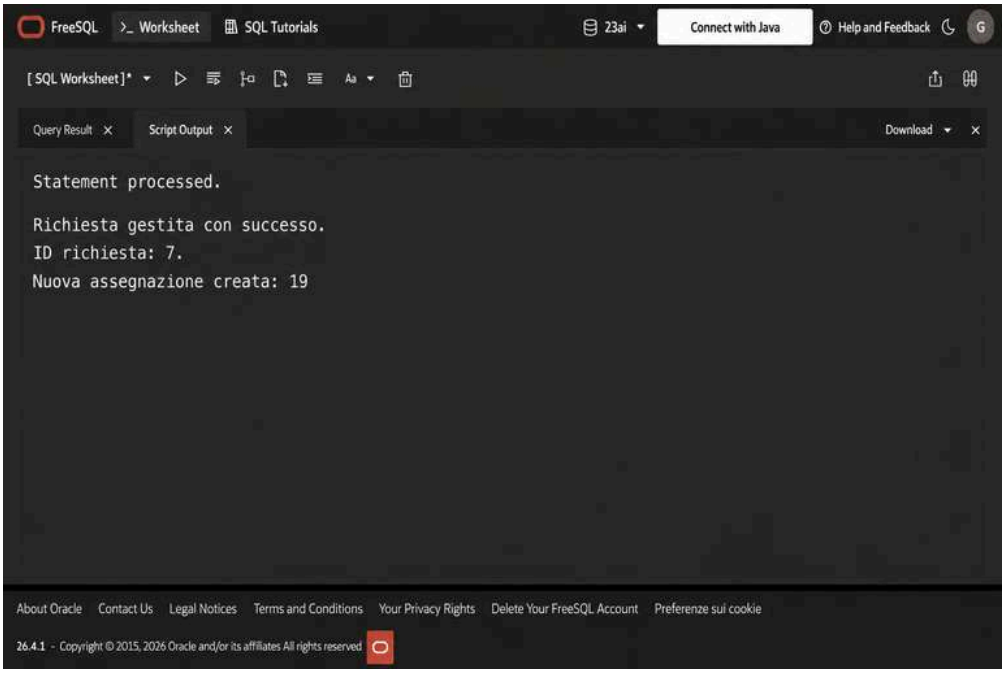
Implementazione

Procedura gestisci_richiesta_variazione (caso corretto)

FreeSQL >_ Worksheet SQL Tutorials

[SQL Worksheet]*

```
1  BEGIN
2      gestisci_richiesta_variazione(
3          7,
4          'APPROVATA',
5          4,
6          7,
7          DATE '2025-06-01',
8          DATE '2025-05-25'
9      );
10 END;
11 /
```



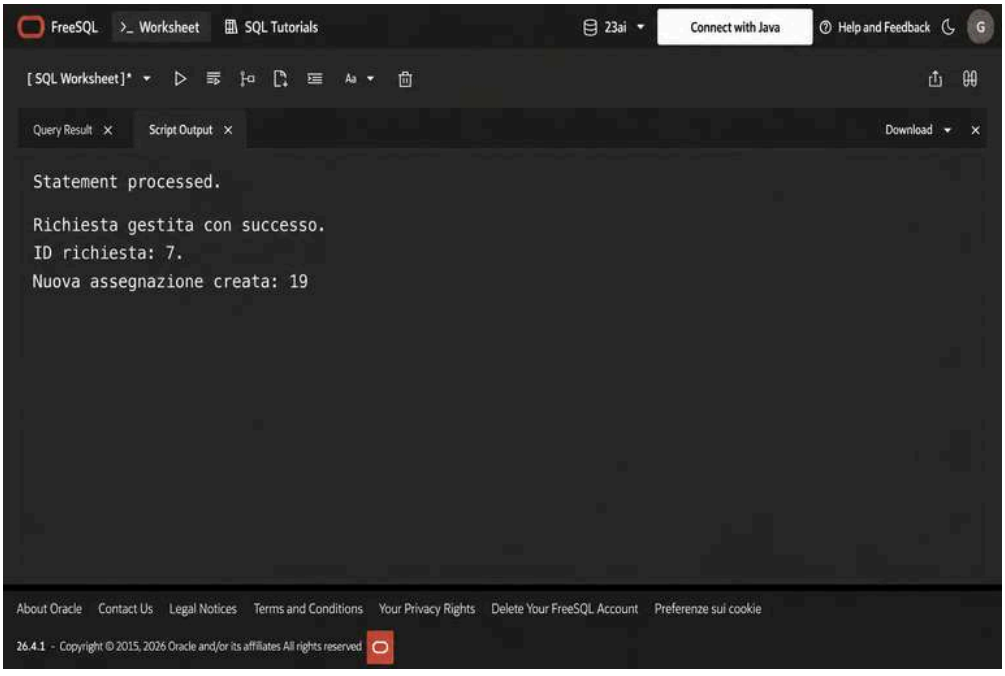
Implementazione

Procedura gestisci_richiesta_variazione (caso corretto)

FreeSQL >_ Worksheet SQL Tutorials

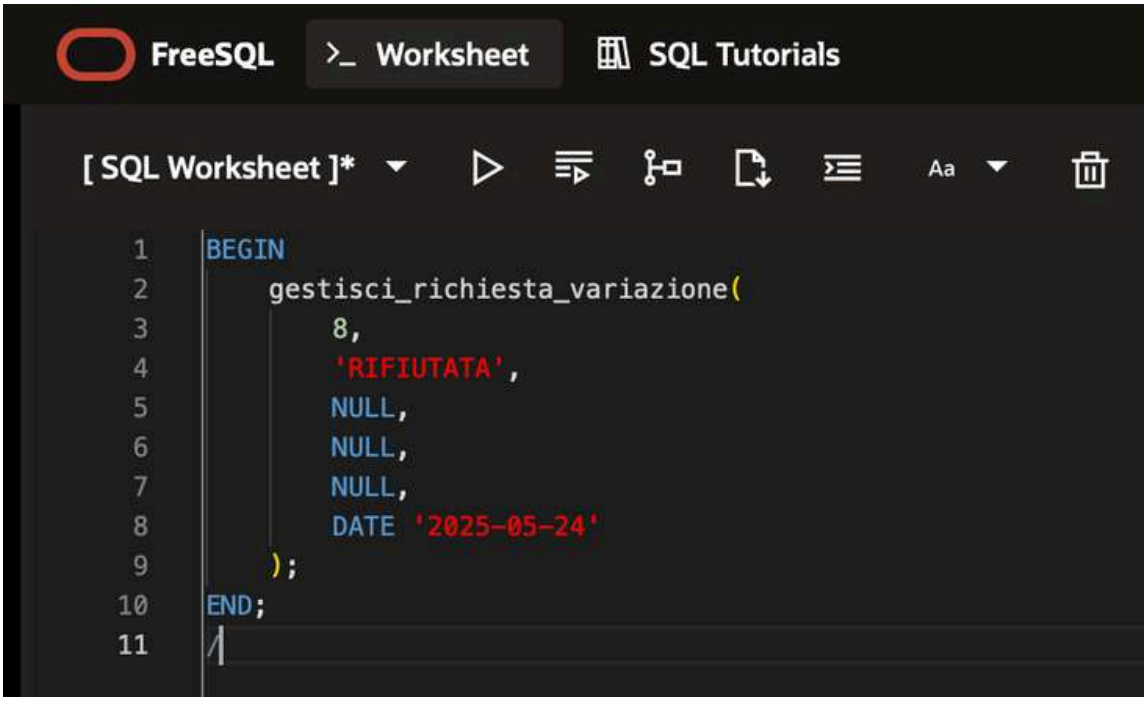
[SQL Worksheet]*

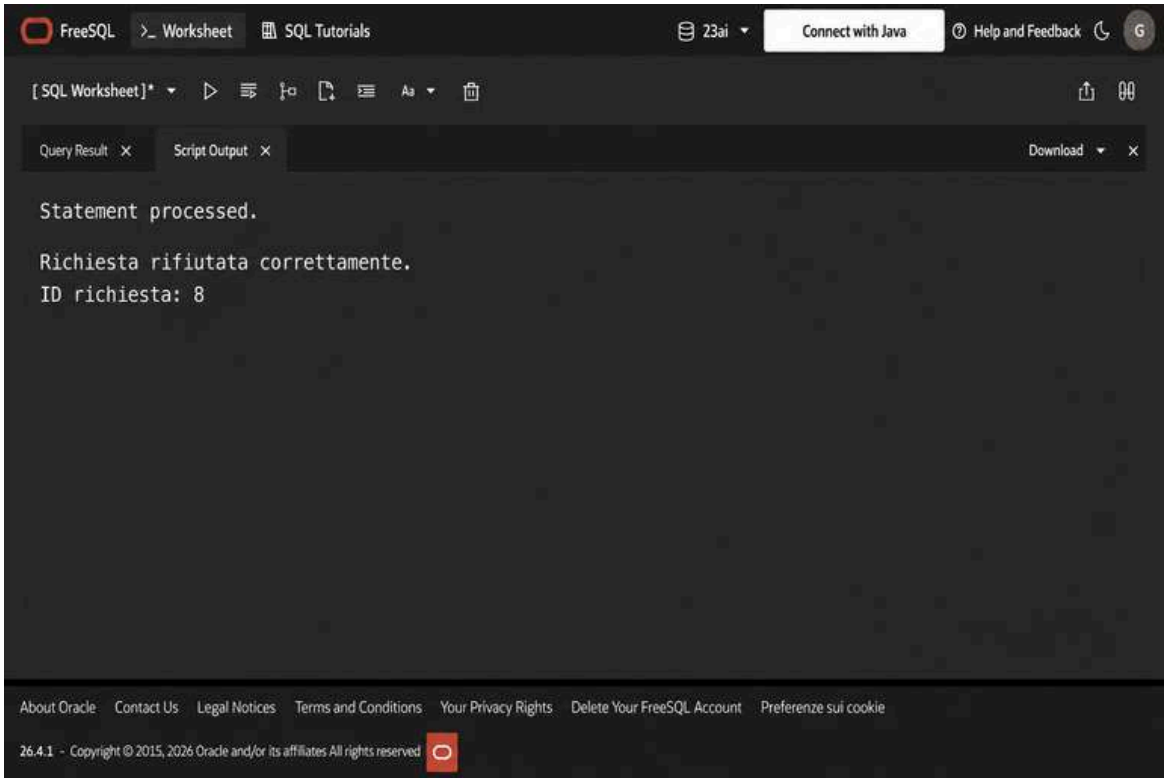
```
1  BEGIN
2      gestisci_richiesta_variazione(
3          7,
4          'APPROVATA',
5          4,
6          7,
7          DATE '2025-06-01',
8          DATE '2025-05-25'
9      );
10 END;
11 /
```



Implementazione

Procedura gestisci_richiesta_variazione (caso errato)





Implementazione

Procedura valida_presenza

La procedura **valida_presenza** è progettata per controllare, correggere e validare in via definitiva una registrazione del registro di servizio già inserita nel sistema. L'operazione viene svolta dall'amministratore nella fase finale di verifica delle presenze e consente di:

- verificare che la presenza appartenga a una assegnazione valida nella data considerata;
- applicare eventuali correzioni ai dati del record;
- aggiornare lo stato di validazione finale della presenza.

I passaggi eseguiti sono i seguenti:

1. **Controllo dei parametri obbligatori**

La procedura verifica che siano presenti:

- identificativo dell'assegnazione
- data del servizio
- esito finale della validazione

2. **Controllo validità dell'esito finale**

L'esito deve essere uno tra:

- VALIDATA
- RIFIUTATA

3. **Controllo esistenza dell'assegnazione**

La procedura verifica che l'assegnazione esista nel sistema.

4. **Controllo coerenza temporale dell'assegnazione**

La data del servizio deve rientrare nel periodo di validità dell'assegnazione.

5. **Controllo esistenza della presenza**

La procedura verifica che esista una riga in REGISTRO_SERVIZIO per la coppia:

- id_assegnazione
- data_servizio

Implementazione

Procedura valida_presenza

6. **Controllo delle eventuali correzioni**

Se vengono passati valori correttivi, questi devono essere coerenti con le regole del registro del servizio.

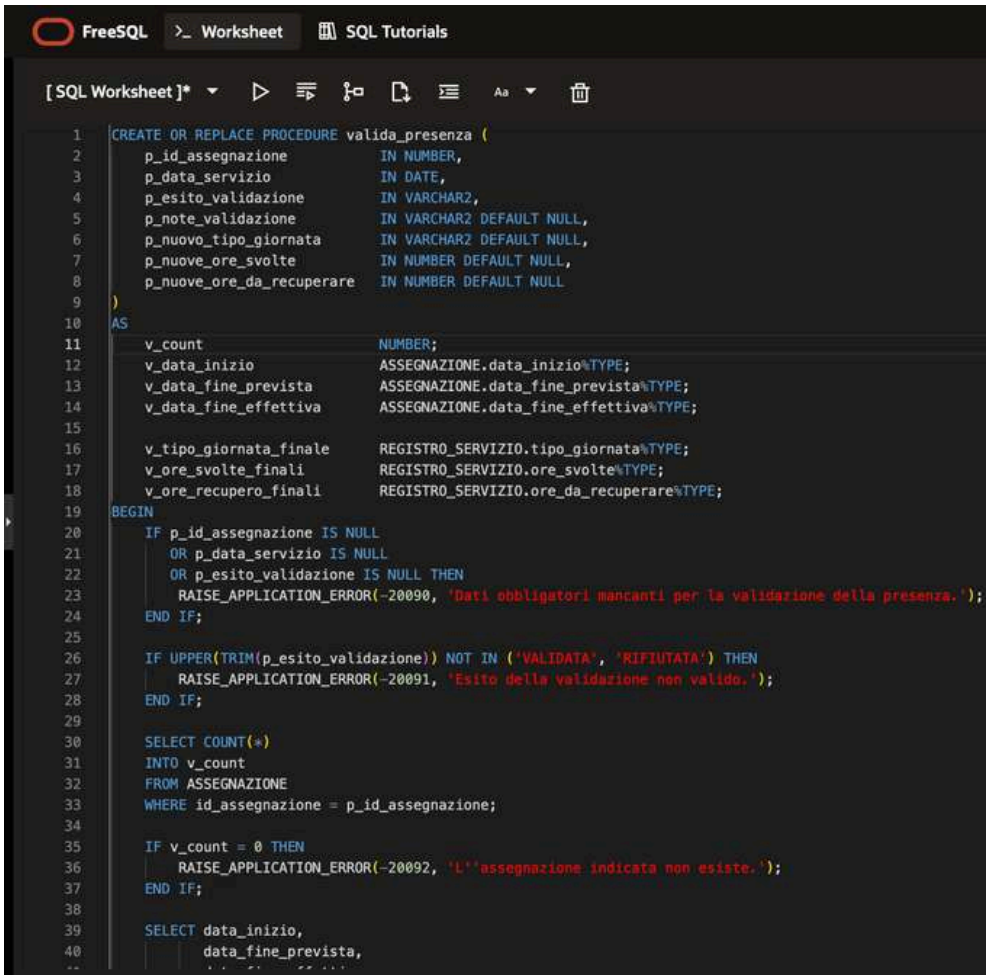
7. **Aggiornamento della presenza**

La procedura aggiorna:

- eventuali campi corretti
- stato finale di validazione
- note amministrative

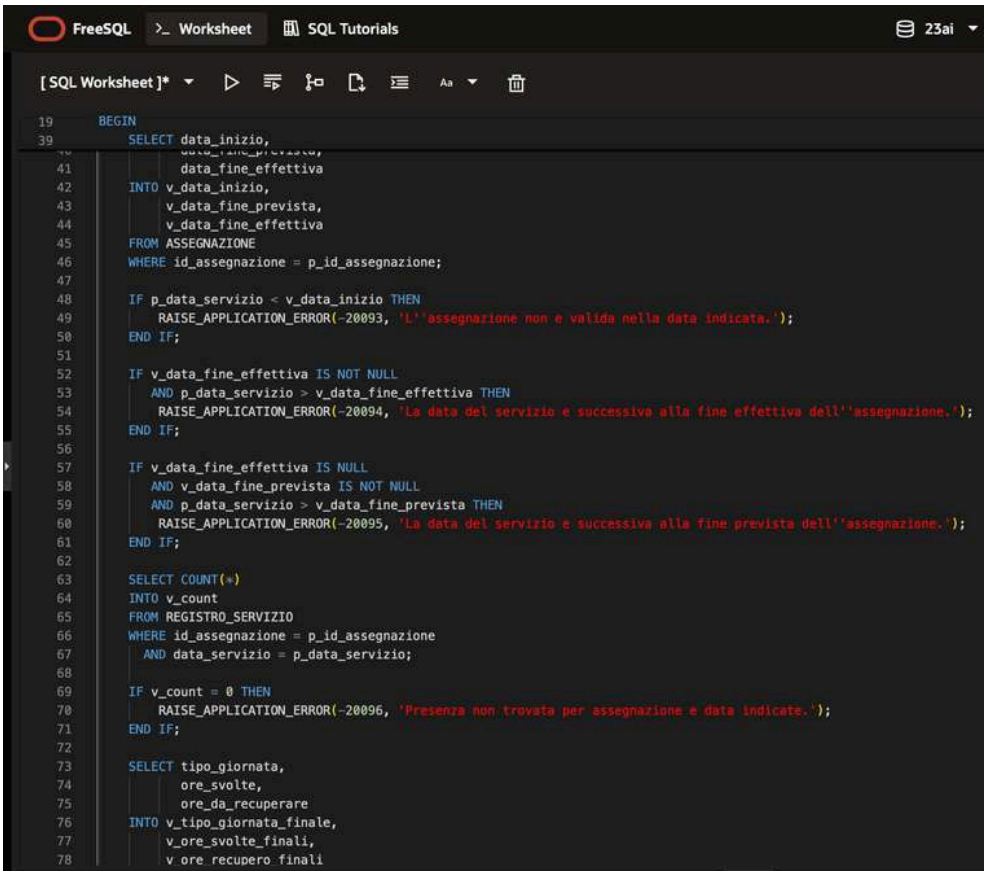
8. **Gestione transazionale**

Se tutto va bene, viene eseguito COMMIT; in caso di errore, ROLLBACK e rilancio dell'eccezione.

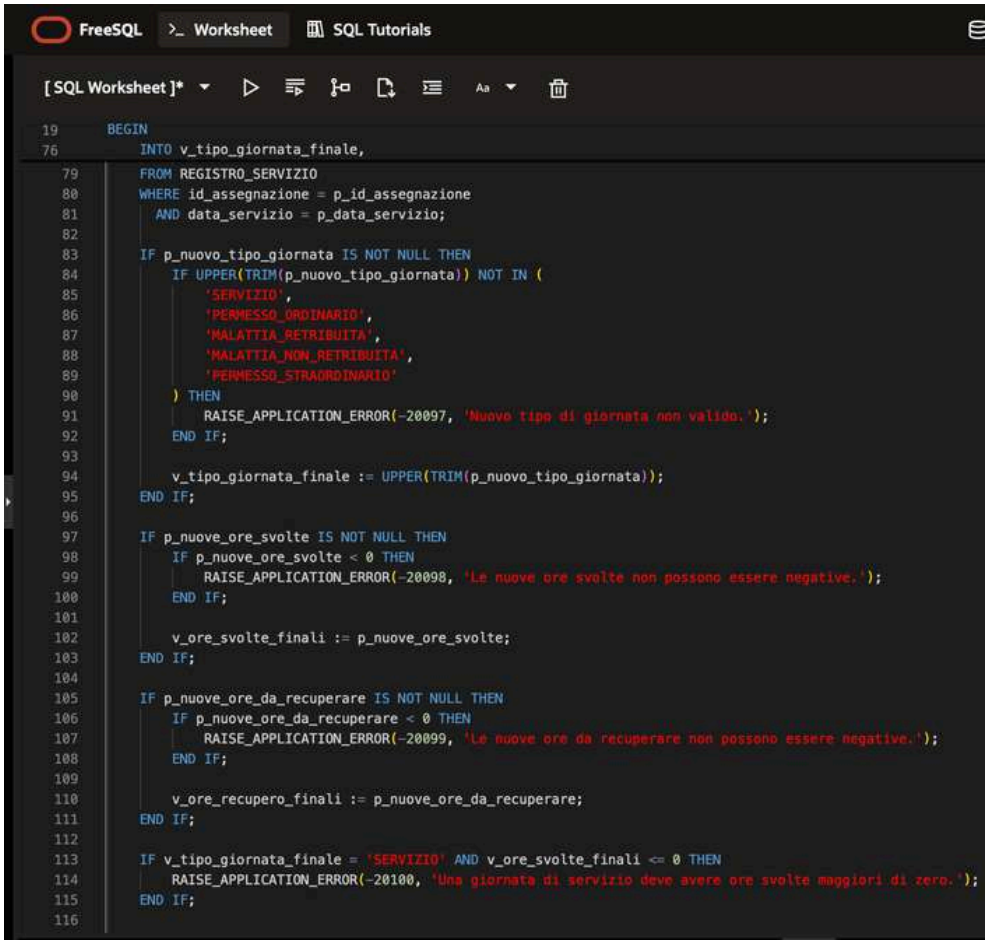


Implementazione

Procedura valida_presenza



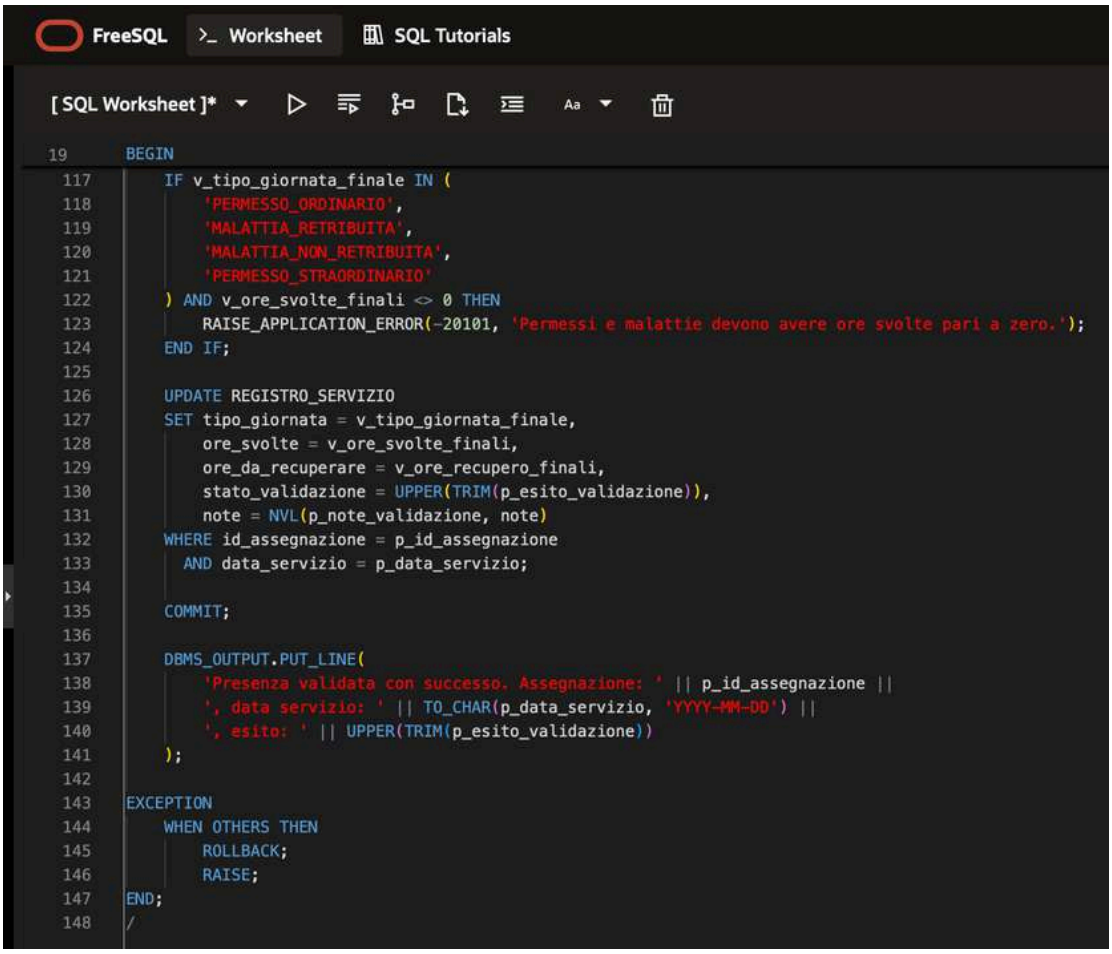
```
19      BEGIN
39          SELECT data_inizio,
41                 data_fine_effettiva
42          INTO v_data_inizio,
43               v_data_fine_prevista,
44               v_data_fine_effettiva
45          FROM ASSEGNAZIONE
46          WHERE id_assegnazione = p_id_assegnazione;
47
48          IF p_data_servizio < v_data_inizio THEN
49              RAISE_APPLICATION_ERROR(-20093, 'L''assegnazione non è valida nella data indicata.');
50          END IF;
51
52          IF v_data_fine_effettiva IS NOT NULL
53             AND p_data_servizio > v_data_fine_effettiva THEN
54              RAISE_APPLICATION_ERROR(-20094, 'La data del servizio è successiva alla fine effettiva dell''assegnazione.');
55          END IF;
56
57          IF v_data_fine_effettiva IS NULL
58             AND v_data_fine_prevista IS NOT NULL
59             AND p_data_servizio > v_data_fine_prevista THEN
60              RAISE_APPLICATION_ERROR(-20095, 'La data del servizio è successiva alla fine prevista dell''assegnazione.');
61          END IF;
62
63          SELECT COUNT(*)
64          INTO v_count
65          FROM REGISTRO_SERVIZIO
66          WHERE id_assegnazione = p_id_assegnazione
67            AND data_servizio = p_data_servizio;
68
69          IF v_count = 0 THEN
70              RAISE_APPLICATION_ERROR(-20096, 'Presenza non trovata per assegnazione e data indicate.');
71          END IF;
72
73          SELECT tipo_giornata,
74                 ore_svolte,
75                 ore_da_recuperare
76          INTO v_tipo_giornata_finale,
77               v_ore_svolte_finali,
78               v_ore_recupero_finali
```



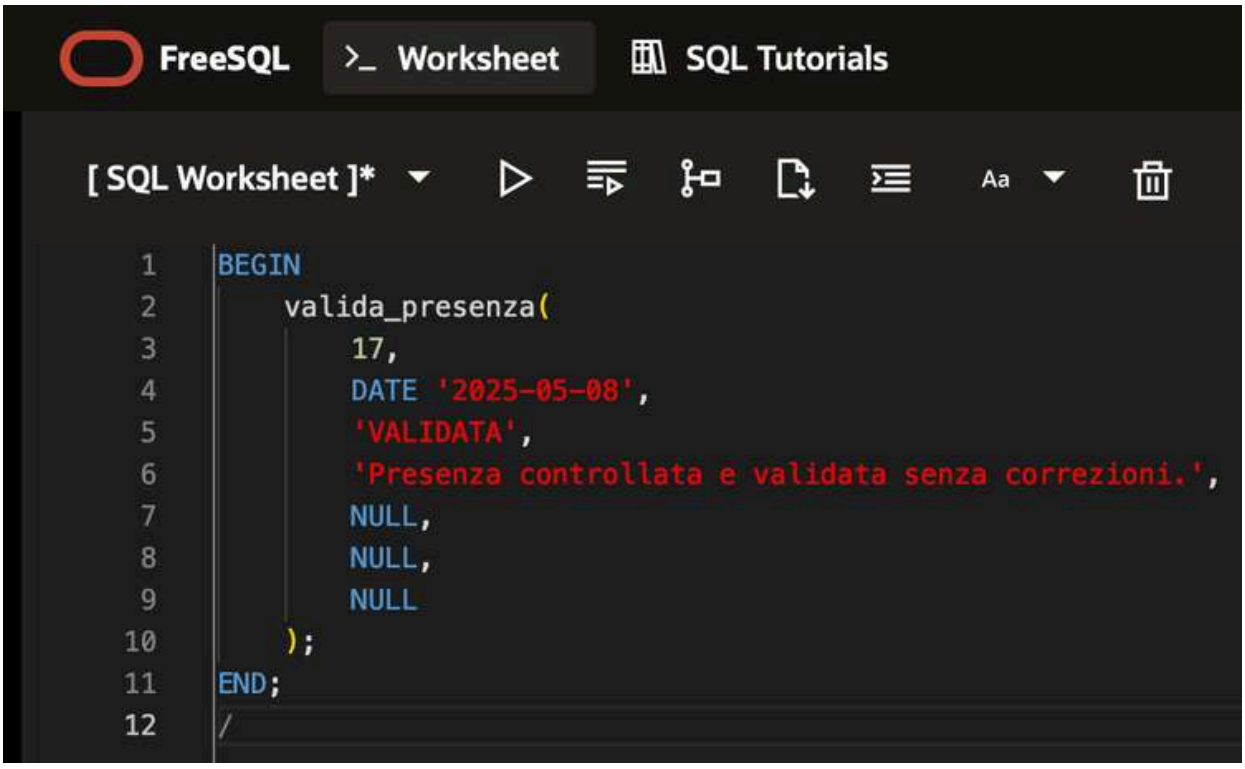
```
19      BEGIN
76          INTO v_tipo_giornata_finale,
79          FROM REGISTRO_SERVIZIO
80          WHERE id_assegnazione = p_id_assegnazione
81            AND data_servizio = p_data_servizio;
82
83          IF p_nuovo_tipo_giornata IS NOT NULL THEN
84              IF UPPER(TRIM(p_nuovo_tipo_giornata)) NOT IN (
85                  'SERVIZIO',
86                  'PERMESSO_ORDINARIO',
87                  'MALATTIA_RETRIBUITA',
88                  'MALATTIA_NON_RETRIBUITA',
89                  'PERMESSO_STRAORDINARIO'
90              ) THEN
91                  RAISE_APPLICATION_ERROR(-20097, 'Nuovo tipo di giornata non valido.');
92              END IF;
93
94              v_tipo_giornata_finale := UPPER(TRIM(p_nuovo_tipo_giornata));
95          END IF;
96
97          IF p_nuove_ore_svolte IS NOT NULL THEN
98              IF p_nuove_ore_svolte < 0 THEN
99                  RAISE_APPLICATION_ERROR(-20098, 'Le nuove ore svolte non possono essere negative.');
100             END IF;
101
102             v_ore_svolte_finali := p_nuove_ore_svolte;
103         END IF;
104
105         IF p_nuove_ore_da_recuperare IS NOT NULL THEN
106             IF p_nuove_ore_da_recuperare < 0 THEN
107                 RAISE_APPLICATION_ERROR(-20099, 'Le nuove ore da recuperare non possono essere negative.');
108             END IF;
109
110             v_ore_recupero_finali := p_nuove_ore_da_recuperare;
111         END IF;
112
113         IF v_tipo_giornata_finale = 'SERVIZIO' AND v_ore_svolte_finali <= 0 THEN
114             RAISE_APPLICATION_ERROR(-20100, 'Una giornata di servizio deve avere ore svolte maggiori di zero.');
115         END IF;
116
```

Implementazione

Procedura valida_presenza

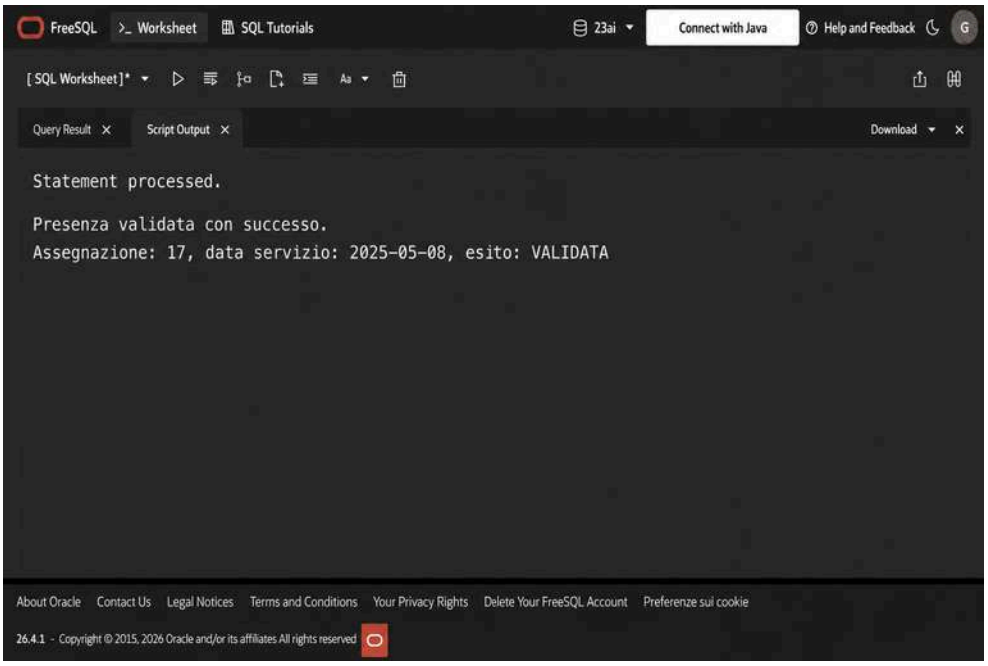


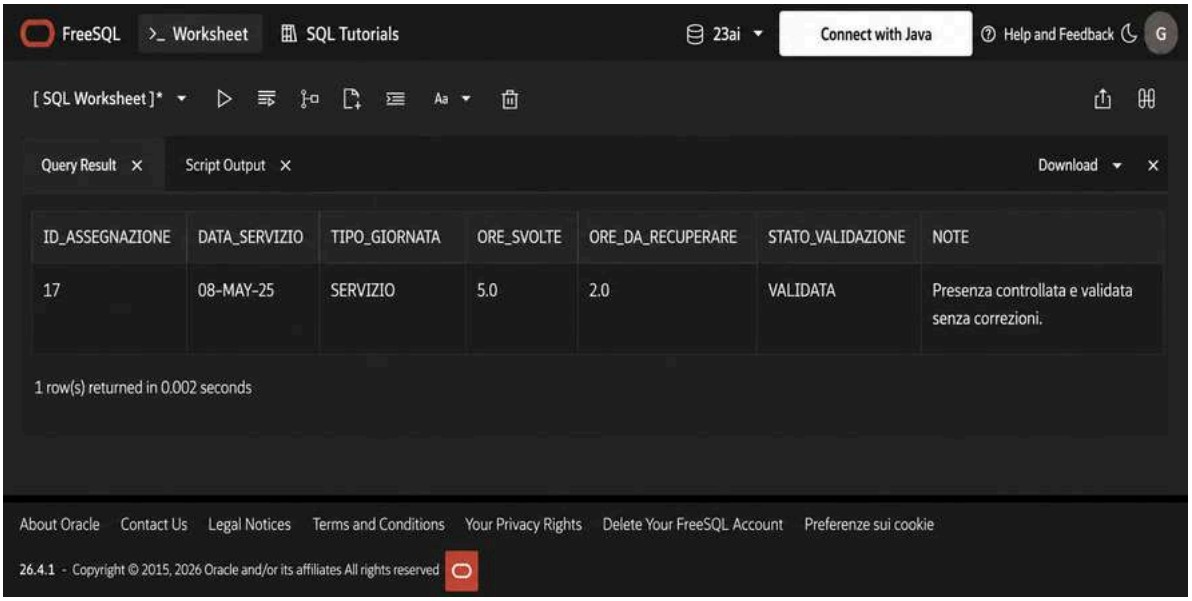
Procedura valida_presenza (caso corretto)



Implementazione

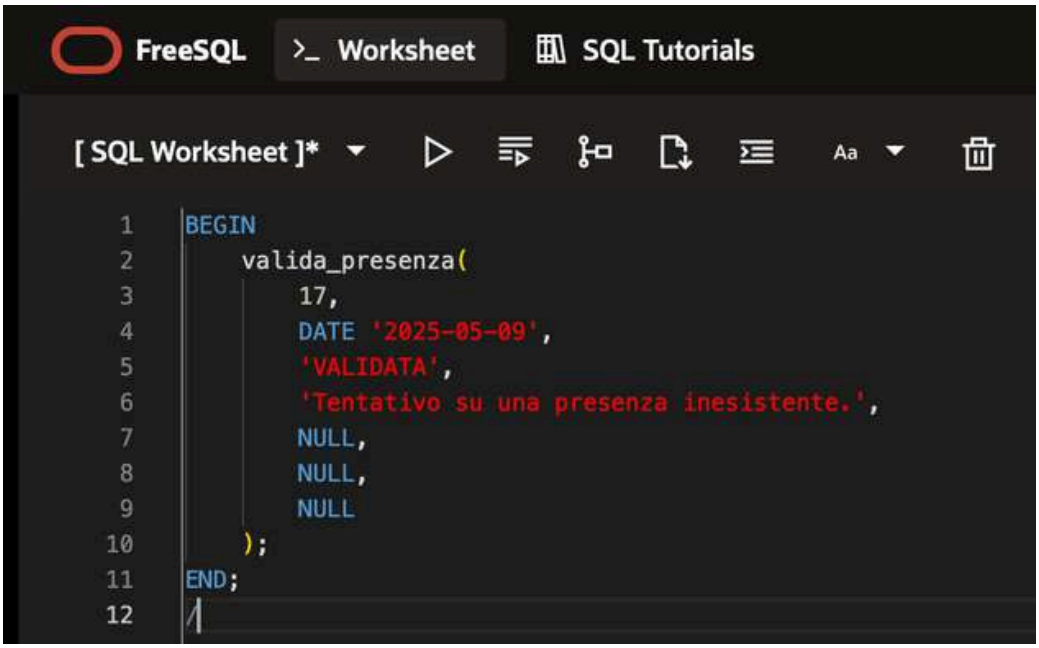
Procedura valida_presenza (caso corretto)

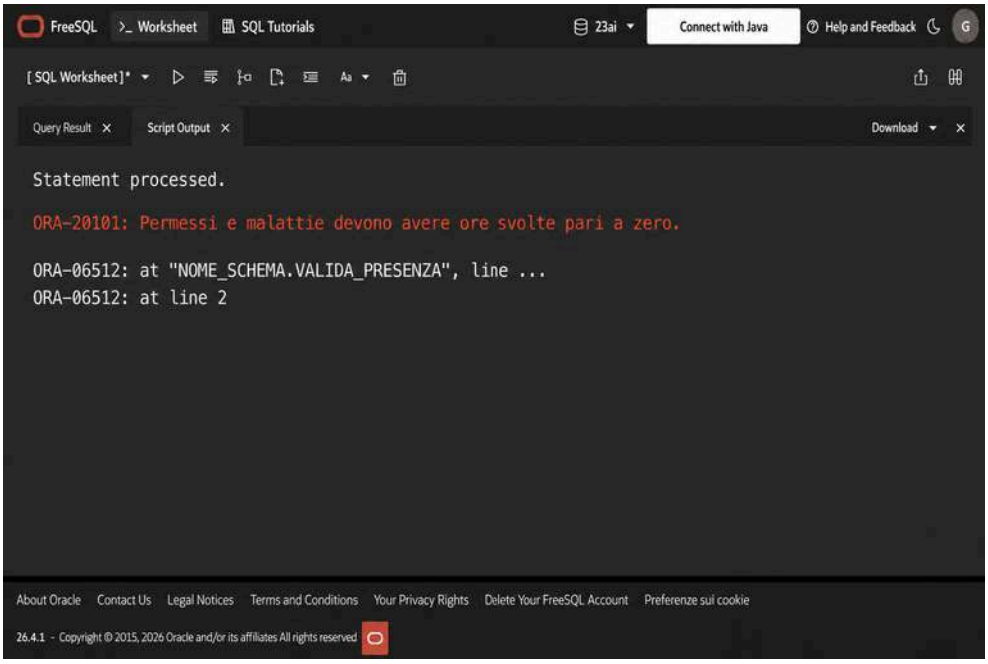




Implementazione

Procedura valida_presenza (caso errato)





Implementazione

Procedura registra_formazione

La procedura **registra_formazione** è progettata per registrare un nuovo evento formativo nel sistema, associandolo all'operatore responsabile e memorizzandone i dati descrittivi, temporali e organizzativi.
I passaggi eseguiti sono i seguenti:

1. **Controllo dei dati obbligatori:**

La procedura verifica che siano presenti:

- titolo
- tipologia
- descrizione
- data della formazione
- durata in ore
- operatore responsabile

2. **Controllo della tipologia:**

La tipologia deve rientrare tra i valori ammessi dal sistema.

3. **Controllo della durata:**

La durata deve essere strettamente positiva.

4. **Controllo della data:**

La data della formazione deve essere valorizzata correttamente. In questa implementazione, oltre a non essere nulla, non può essere futura rispetto alla data corrente. Questo rende il controllo più forte e difendibile.

5. **Controllo esistenza dell'operatore**

La procedura verifica che l'operatore responsabile esista.

6. **Inserimento della formazione**

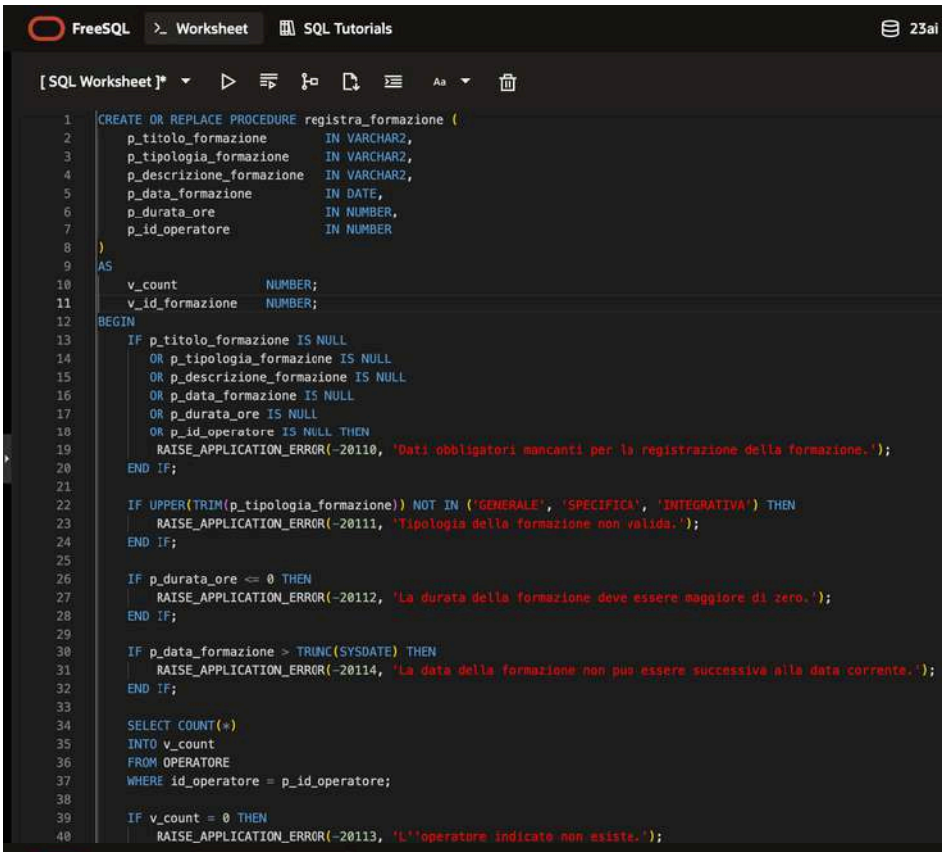
Se tutti i controlli hanno esito positivo, viene generato un nuovo identificativo e viene inserito il nuovo evento formativo.

7. **Gestione transazionale**

Se tutto va bene, viene eseguito **COMMIT;** in caso di errore, **ROLLBACK** e rilancio dell'eccezione.

Implementazione

Procedura registra_formazione




```sql
CREATE OR REPLACE PROCEDURE registra_formazione (
    p_titolo_formazione       IN VARCHAR2,
    p_tipologia_formazione    IN VARCHAR2,
    p_descrizione_formazione  IN VARCHAR2,
    p_data_formazione         IN DATE,
    p_durata_ore              IN NUMBER,
    p_id_operatore            IN NUMBER
)
AS
    v_count           NUMBER;
    v_id_formazione   NUMBER;
BEGIN
    IF p_titolo_formazione IS NULL
       OR p_tipologia_formazione IS NULL
       OR p_descrizione_formazione IS NULL
       OR p_data_formazione IS NULL
       OR p_durata_ore IS NULL
       OR p_id_operatore IS NULL THEN
        RAISE_APPLICATION_ERROR(-20110, 'Dati obbligatori mancanti per la registrazione della formazione.');
    END IF;

    IF UPPER(TRIM(p_tipologia_formazione)) NOT IN ('GENERALE', 'SPECIFICA', 'INTEGRATIVA') THEN
        RAISE_APPLICATION_ERROR(-20111, 'Tipologia della formazione non valida.');
    END IF;

    IF p_durata_ore <= 0 THEN
        RAISE_APPLICATION_ERROR(-20112, 'La durata della formazione deve essere maggiore di zero.');
    END IF;

    IF p_data_formazione > TRUNC(SYSDATE) THEN
        RAISE_APPLICATION_ERROR(-20114, 'La data della formazione non può essere successiva alla data corrente.');
    END IF;

    SELECT COUNT(*)
    INTO v_count
    FROM OPERATORE
    WHERE id_operatore = p_id_operatore;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20113, 'L''operatore indicato non esiste.');
    END IF;

    SELECT NVL(MAX(id_formazione), 0) + 1
    INTO v_id_formazione
    FROM FORMAZIONE;

    INSERT INTO FORMAZIONE (
        id_formazione,
        id_operatore,
        titolo,
        tipologia,
        data_formazione,
        durata_ore,
        descrizione
    )
    VALUES (
        v_id_formazione,
        p_id_operatore,
        p_titolo_formazione,
        UPPER(TRIM(p_tipologia_formazione)),
        p_data_formazione,
        p_durata_ore,
        p_descrizione_formazione
    );

    COMMIT;

    DBMS_OUTPUT.PUT_LINE(
        'Formazione registrata con successo. ID formazione: ' || v_id_formazione
    );

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
```

Implementazione

Procedura registra_formazione (caso corretto)

FreeSQL | Worksheet | SQL Tutorials

[SQL Worksheet]*

```
BEGIN
    registra_formazione(
        'Laboratorio cittadinanza digitale',
        'INTEGRATIVA',
        'Modulo dedicato all uso consapevole degli strumenti digitali nel servizio civile.',
        DATE '2025-06-10',
        3,
        2
    );
END;
/
```

FreeSQL | Worksheet | SQL Tutorials | 23ai | Connect with Java | Help and Feedback

[SQL Worksheet]*

Query Result | Script Output | Download

```
Statement processed.

Formazione registrata con successo.
ID formazione: 11
```

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account | Preferenze sui cookie

26.4.1 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

FreeSQL | Worksheet | SQL Tutorials | 23ai | Connect with Java | Help and Feedback

[SQL Worksheet]*

Query Result | Script Output | Download

ID_FORMAZIONE	ID_OPERATORE	TITOLO	TIPOLOGIA	DATA_FORMAZIONE	DURATA_ORE	DESCRIZIONE
11	2	Laboratorio cittadinanza digitale	INTEGRATIVA	10-JUN-25	3	Modulo dedicato all uso consapevole degli strumenti digitali nel servizio civile.

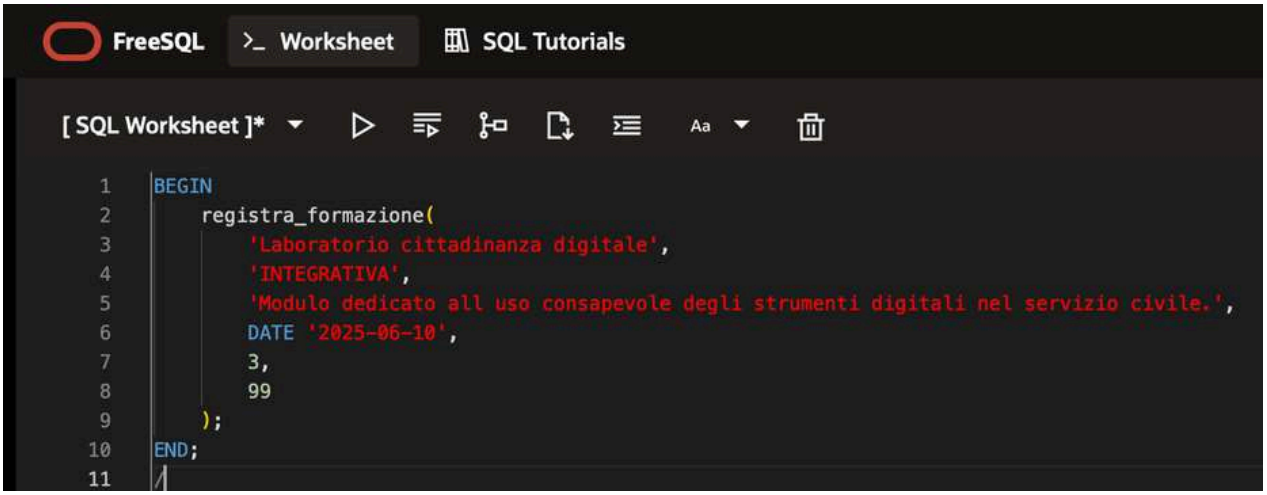
1 row(s) returned in 0.001 seconds

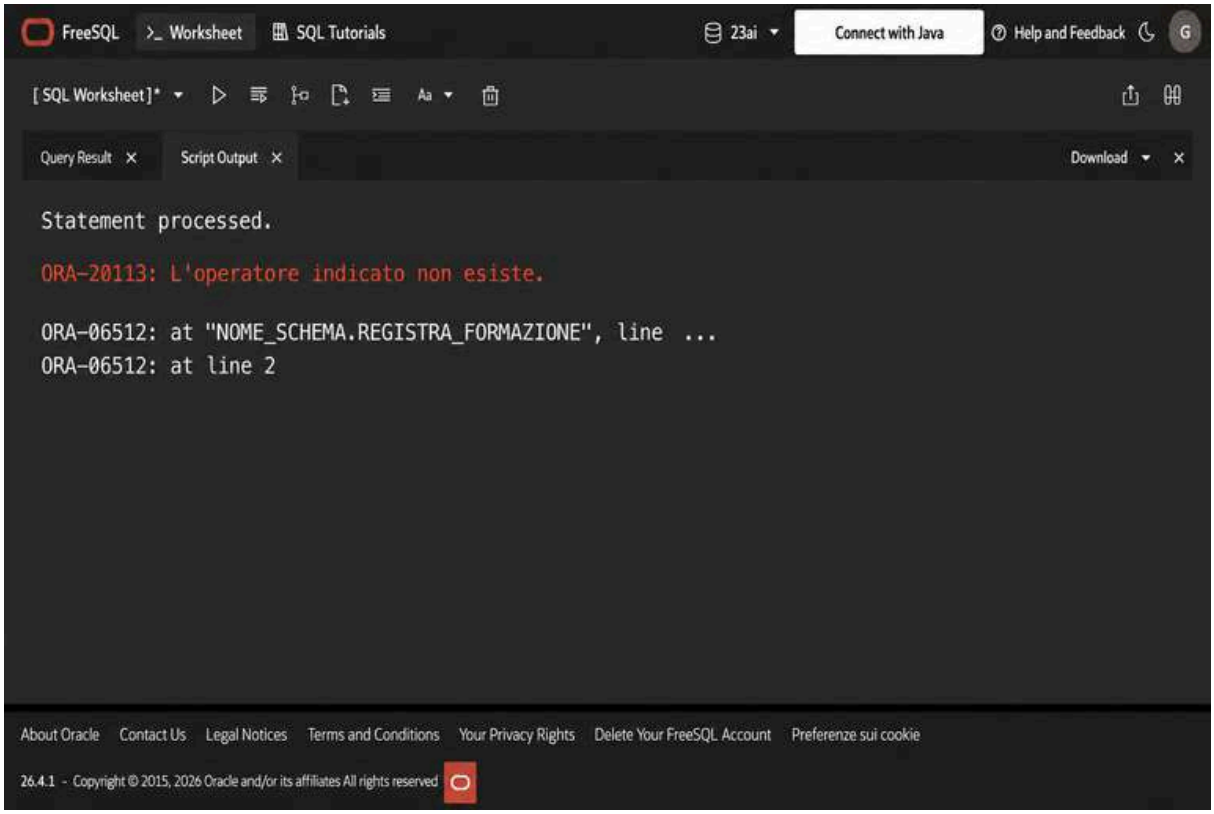
About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account | Preferenze sui cookie

26.4.1 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

Implementazione

Procedura registra_formazione (caso errato)





Implementazione

Procedura registra_partecipazione_formazione

La procedura **registra_partecipazione_formazione** è progettata per registrare la partecipazione di un volontario a un evento formativo nel contesto della propria assegnazione.
L'operazione viene svolta dall'operatore del sistema e consente di inserire una nuova riga nella tabella **ASSEGNAZIONE_FORMAZIONE**, collegando:

- una specifica assegnazione;
- una specifica formazione;
- il relativo stato di presenza;
- il numero di ore frequentate;
- eventuali note descrittive.

I controlli effettuati sono i seguenti:

1. **Controllo dei dati obbligatori:** La procedura verifica che siano presenti:
   - identificativo dell'assegnazione;
   - identificativo della formazione;
   - stato di presenza;
   - ore frequentate.
2. **Controllo esistenza dell'assegnazione:**

La procedura verifica che l'assegnazione indicata esista nella tabella **ASSEGNAZIONE,** poiché la partecipazione formativa deve essere riferita a un percorso amministrativo reale del volontario. La tabella **ASSEGNAZIONE** è infatti il fulcro del percorso amministrativo del volontario.

3. **Controllo esistenza della formazione:**

La procedura verifica che la formazione indicata esista nella tabella **FORMAZIONE,** poiché non è possibile registrare una partecipazione a un evento formativo inesistente. La tabella **FORMAZIONE** rappresenta proprio un evento formativo del sistema.

5. **Controllo validità del valore di presenza:**

Il valore di **p_presenza** deve appartenere ai valori ammessi dal sistema. Nel DDL reale della tabella associativa i valori previsti sono:

- PRESENTE
- ASSENTE
- PARZIALE.

6. C**ontrollo non negatività delle ore frequentate:**

La procedura verifica che p_ore_frequentate sia maggiore o uguale a zero, in coerenza con i vincoli statici della partecipazione alla formazione.

7. **Controllo di coerenza tra presenza e ore frequentate:**

La procedura applica il controllo logico previsto dal modello:

- se il volontario risulta ASSENTE, allora ore_frequentate deve essere uguale a zero;
- se il volontario risulta PRESENTE o PARZIALE, allora ore_frequentate deve essere maggiore di zero.

8. **Inserimento della partecipazione:**

Se tutti i controlli hanno esito positivo, la procedura inserisce il nuovo record nella tabella ASSEGNAZIONE_FORMAZIONE.

9. **Gestione transazionale:**

Se tutto va bene:

- viene eseguito il COMMIT;
- viene mostrato un messaggio di successo con DBMS_OUTPUT.PUT_LINE.

In caso di errore:

- viene eseguito il ROLLBACK;
- l'eccezione viene rilanciata.

Implementazione

Procedura registra_partecipazione_formazione

FreeSQL >_ Worksheet SQL Tutorials

[SQL Worksheet]*

```
CREATE OR REPLACE PROCEDURE registra_partecipazione_formazione (
    p_id_assegnazione    IN NUMBER,
    p_id_formazione      IN NUMBER,
    p_presenza           IN VARCHAR2,
    p_ore_frequentate    IN NUMBER,
    p_note               IN VARCHAR2 DEFAULT NULL
)
AS
    v_count NUMBER;
BEGIN
    IF p_id_assegnazione IS NULL
       OR p_id_formazione IS NULL
       OR p_presenza IS NULL
       OR p_ore_frequentate IS NULL THEN
        RAISE_APPLICATION_ERROR(
            -20120,
            'Dati obbligatori mancanti per la registrazione della partecipazione alla formazione.'
        );
    END IF;

    SELECT COUNT(*)
    INTO v_count
    FROM ASSEGNAZIONE
    WHERE id_assegnazione = p_id_assegnazione;

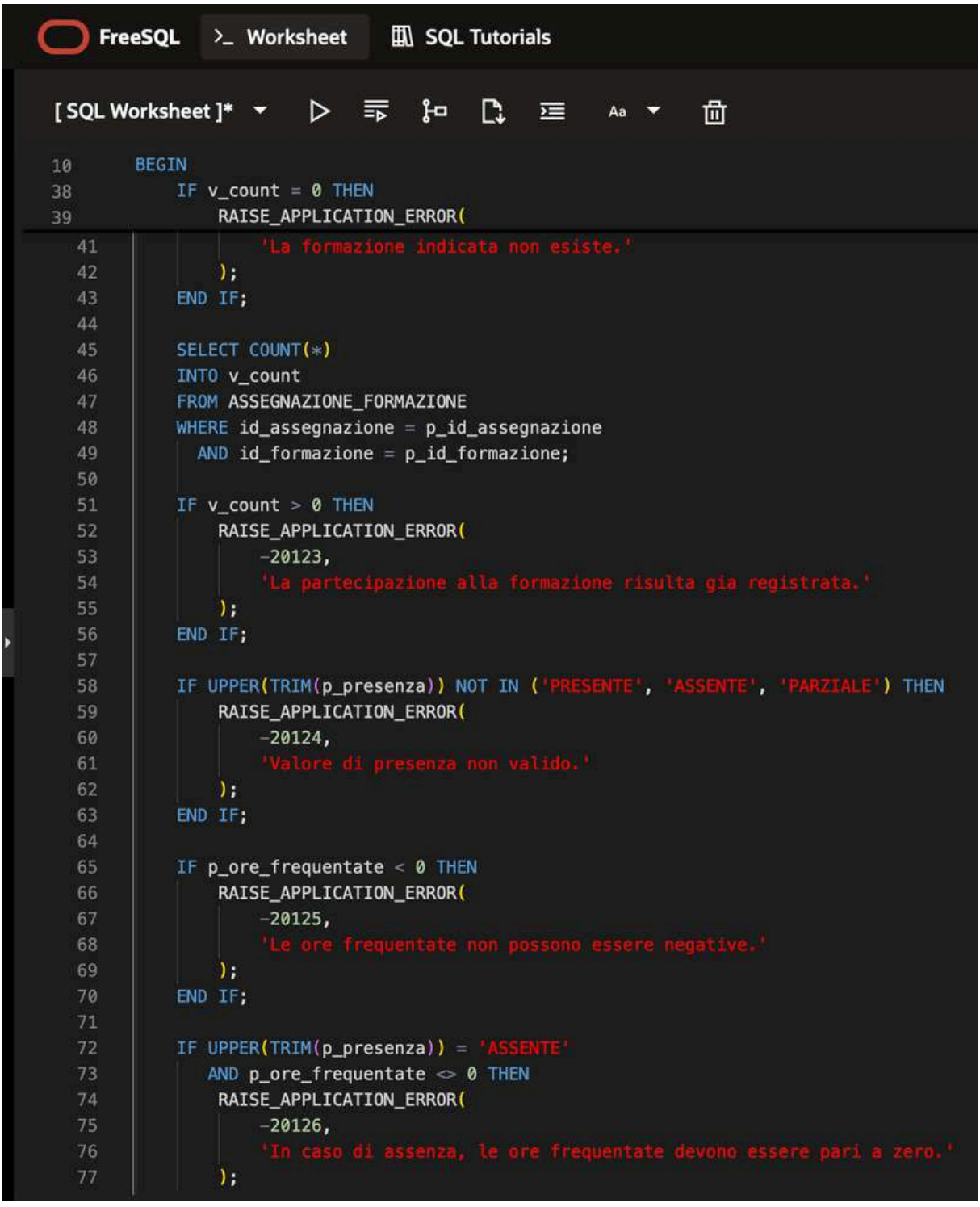
    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(
            -20121,
            'L''assegnazione indicata non esiste.'
        );
    END IF;

    SELECT COUNT(*)
    INTO v_count
    FROM FORMAZIONE
    WHERE id_formazione = p_id_formazione;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(
            -20122,
```

Implementazione

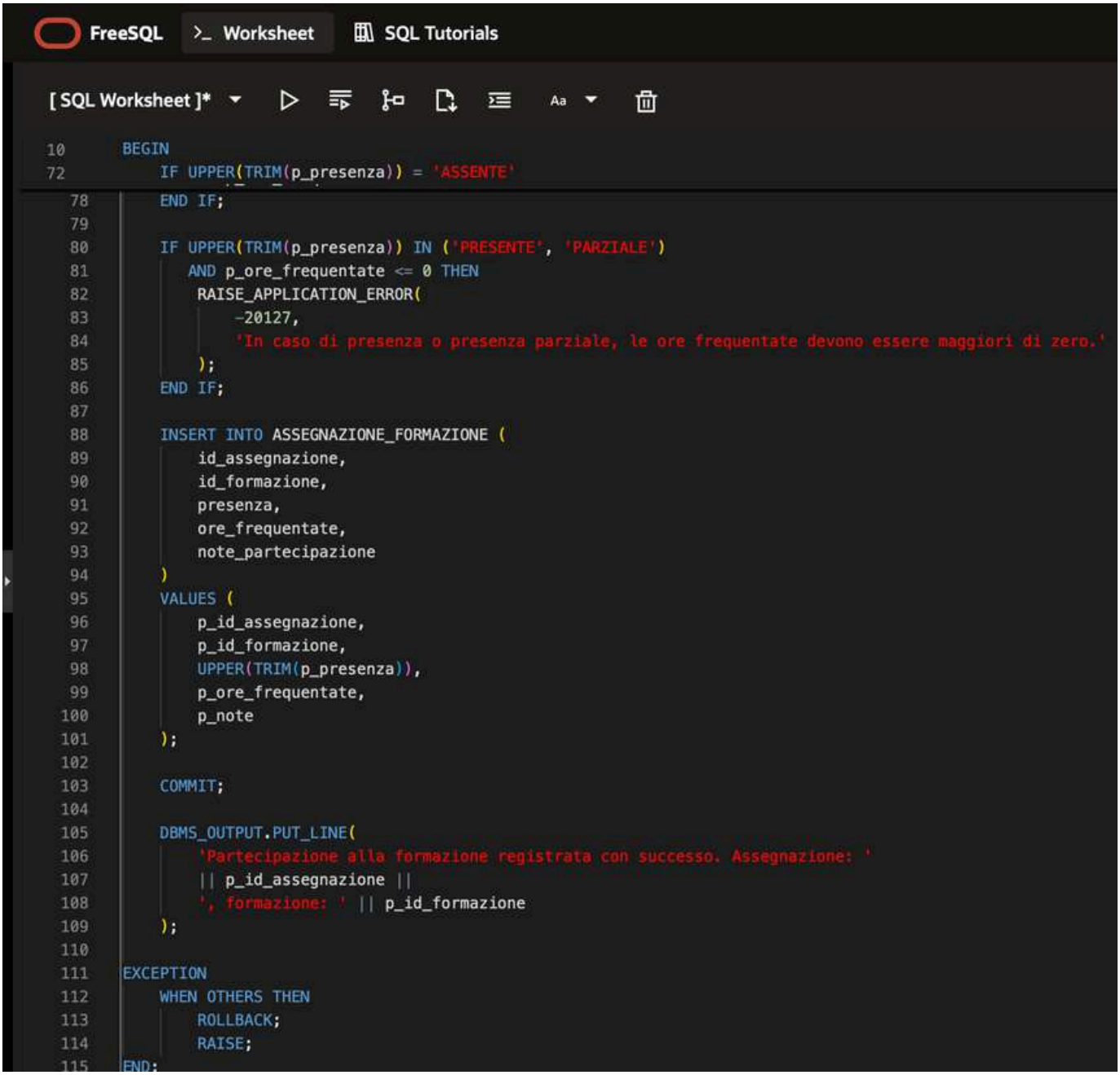
Procedura registra_partecipazione_formazione




```
10      BEGIN
38          IF v_count = 0 THEN
39              RAISE_APPLICATION_ERROR(
41                  'La formazione indicata non esiste.'
42              );
43      END IF;
44
45      SELECT COUNT(*)
46      INTO v_count
47      FROM ASSEGNAZIONE_FORMAZIONE
48      WHERE id_assegnazione = p_id_assegnazione
49        AND id_formazione = p_id_formazione;
50
51      IF v_count > 0 THEN
52          RAISE_APPLICATION_ERROR(
53              -20123,
54              'La partecipazione alla formazione risulta gia registrata.'
55          );
56      END IF;
57
58      IF UPPER(TRIM(p_presenza)) NOT IN ('PRESENTE', 'ASSENTE', 'PARZIALE') THEN
59          RAISE_APPLICATION_ERROR(
60              -20124,
61              'Valore di presenza non valido.'
62          );
63      END IF;
64
65      IF p_ore_frequentate < 0 THEN
66          RAISE_APPLICATION_ERROR(
67              -20125,
68              'Le ore frequentate non possono essere negative.'
69          );
70      END IF;
71
72      IF UPPER(TRIM(p_presenza)) = 'ASSENTE'
73         AND p_ore_frequentate <> 0 THEN
74          RAISE_APPLICATION_ERROR(
75              -20126,
76              'In caso di assenza, le ore frequentate devono essere pari a zero.'
77          );
```

Implementazione

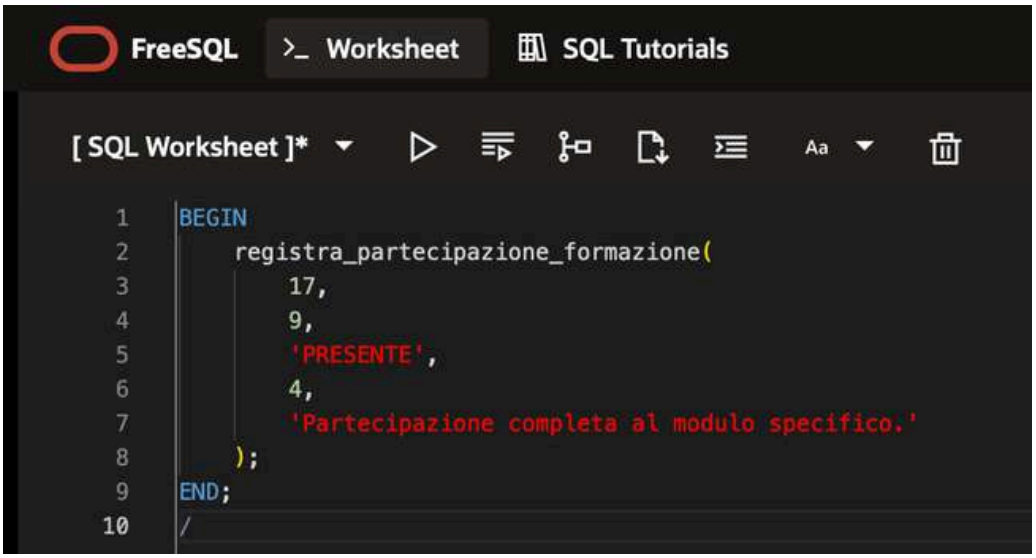
Procedura registra_partecipazione_formazione

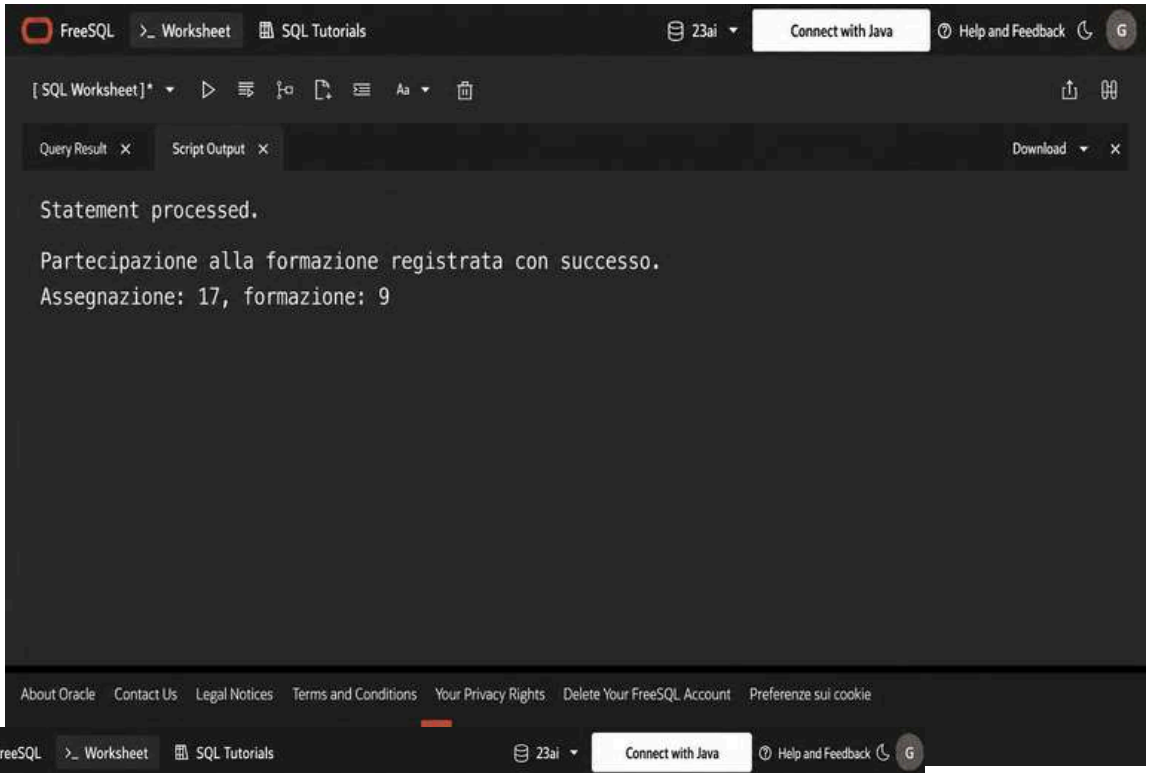


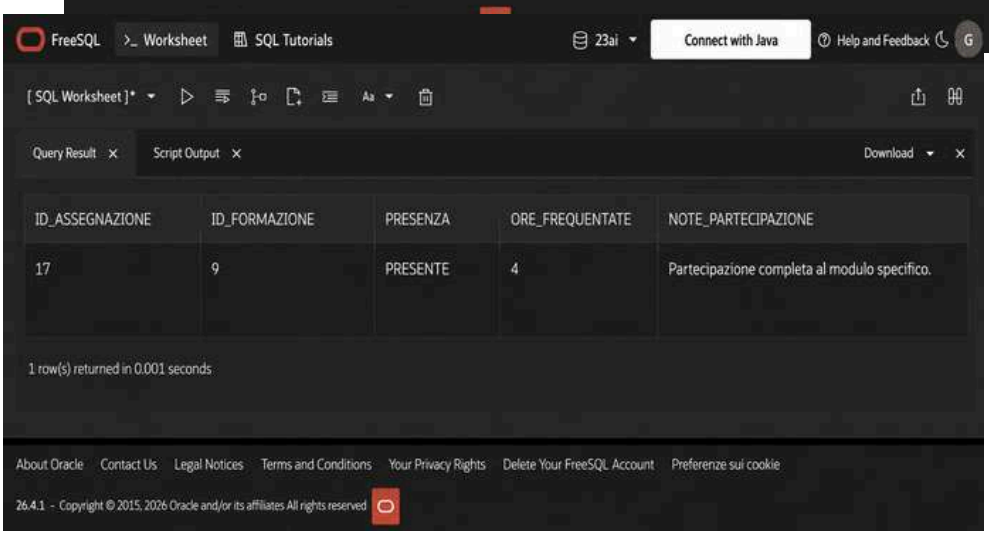

```
10      BEGIN
72          IF UPPER(TRIM(p_presenza)) = 'ASSENTE'
78          END IF;
79
80          IF UPPER(TRIM(p_presenza)) IN ('PRESENTE', 'PARZIALE')
81             AND p_ore_frequentate <= 0 THEN
82              RAISE_APPLICATION_ERROR(
83                  -20127,
84                  'In caso di presenza o presenza parziale, le ore frequentate devono essere maggiori di zero.'
85              );
86          END IF;
87
88          INSERT INTO ASSEGNAZIONE_FORMAZIONE (
89              id_assegnazione,
90              id_formazione,
91              presenza,
92              ore_frequentate,
93              note_partecipazione
94          )
95          VALUES (
96              p_id_assegnazione,
97              p_id_formazione,
98              UPPER(TRIM(p_presenza)),
99              p_ore_frequentate,
100             p_note
101         );
102
103         COMMIT;
104
105         DBMS_OUTPUT.PUT_LINE(
106             'Partecipazione alla formazione registrata con successo. Assegnazione: '
107             || p_id_assegnazione ||
108             ', formazione: ' || p_id_formazione
109         );
110
111     EXCEPTION
112         WHEN OTHERS THEN
113             ROLLBACK;
114             RAISE;
115     END;
```

Implementazione

Procedura registra_partecipazione_formazione (caso corretto)

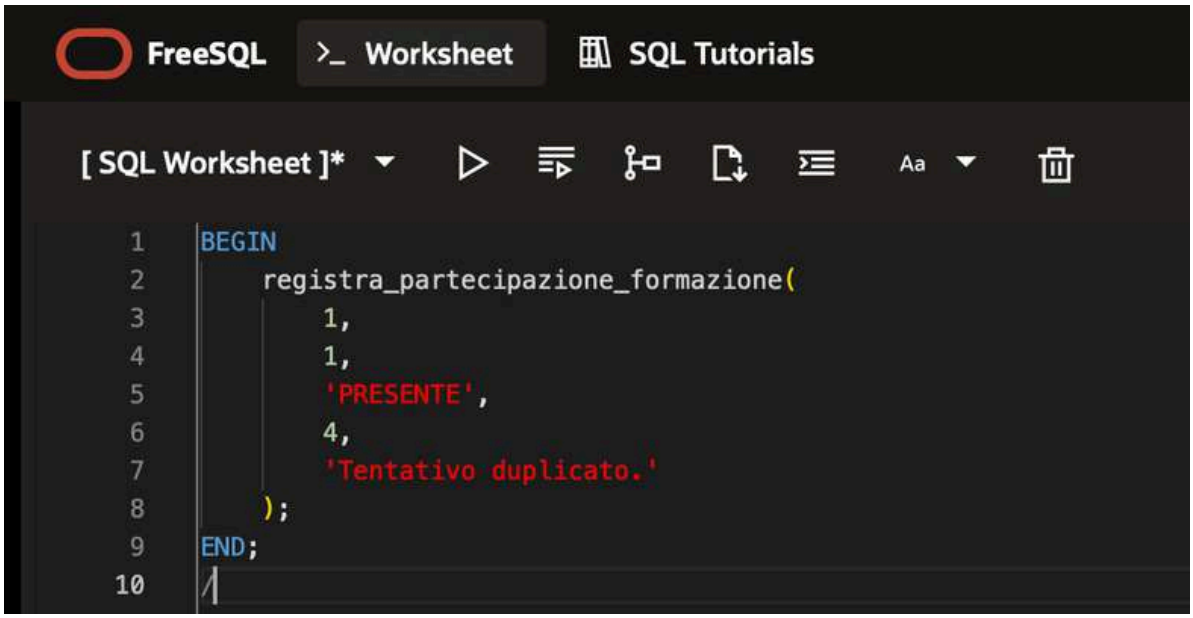


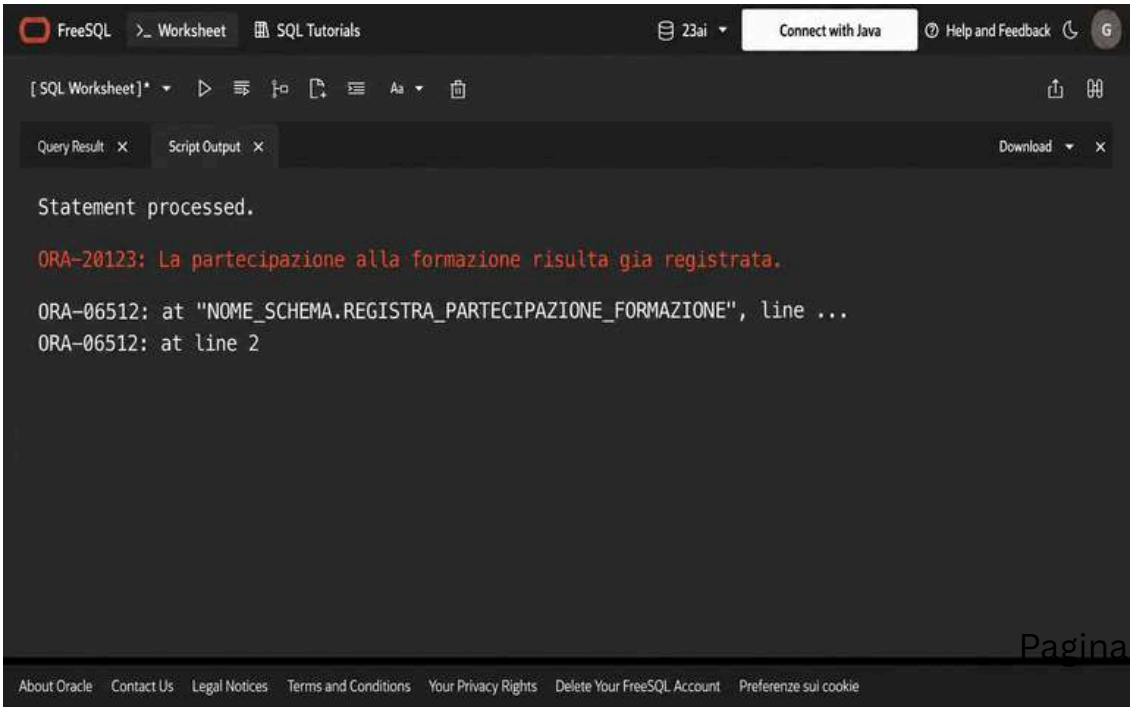




Implementazione

Procedura registra_partecipazione_formazione (caso errato)





Implementazione

Procedura registra_monitoraggio

La procedura **registra_monitoraggio** è progettata per registrare un monitoraggio periodico relativo al percorso del volontario, associandolo a una specifica assegnazione. Il monitoraggio rappresenta una sintesi periodica e valutativa del percorso del volontario, costruita a partire dai dati amministrativi disponibili e integrata con osservazioni qualitative.
I passaggi eseguiti dalla procedura sono i seguenti:

1. **Controllo dei dati obbligatori:**

La procedura verifica che siano presenti:

- identificativo dell'assegnazione
- data del monitoraggio
- tipologia del monitoraggio
- esito del monitoraggio

2. **Controllo esistenza dell'assegnazione:**

La procedura verifica che l'assegnazione indicata esista nella tabella **ASSEGNAZIONE,** perché il monitoraggio deve sempre riferirsi a un percorso reale del volontario.

2. **Controllo di coerenza temporale:**

La procedura verifica **che p_data_monitoraggio** rientri nel periodo amministrativo dell'assegnazione:

- non può essere precedente a data_inizio
- non può essere successiva a data_fine_effettiva, se presente
- altrimenti non può essere successiva a data_fine_prevista, se presente

Questo recepisce il vincolo dinamico sul monitoraggio coerente con il periodo del percorso.

4. **Controllo validità di tipo_monitoraggio:**

La procedura verifica che il tipo di monitoraggio appartenga ai valori ammessi dal sistema.

5. **Controllo validità di esito:**

La procedura verifica che l'esito del monitoraggio appartenga ai valori ammessi dal sistema.

Procedura registra_monitoraggio

7. **Inserimento del monitoraggio**

Se tutti i controlli precedenti hanno esito positivo:

- viene generato un nuovo identificativo
- viene inserita una nuova riga nella tabella MONITORAGGIO

8. **Gestione transazionale**

Se l'inserimento va a buon fine:

- viene eseguito il **COMMIT**
- viene mostrato un messaggio di successo con **DBMS_OUTPUT.PUT_LINE**

In caso di errore:

- viene eseguito il **ROLLBACK**
- l'eccezione viene rilanciata

Implementazione

Procedura registra_monitoraggio

FreeSQL >_ Worksheet SQL Tutorials

[SQL Worksheet]*

```
CREATE OR REPLACE PROCEDURE registra_monitoraggio (
    p_id_assegnazione        IN NUMBER,
    p_data_monitoraggio      IN DATE,
    p_tipo_monitoraggio      IN VARCHAR2,
    p_esito_monitoraggio     IN VARCHAR2,
    p_note                   IN VARCHAR2 DEFAULT NULL
)
AS
    v_count                NUMBER;
    v_id_monitoraggio      NUMBER;
    v_data_inizio          ASSEGNAZIONE.data_inizio%TYPE;
    v_data_fine_prevista   ASSEGNAZIONE.data_fine_prevista%TYPE;
    v_data_fine_effettiva  ASSEGNAZIONE.data_fine_effettiva%TYPE;
BEGIN
    IF p_id_assegnazione IS NULL
       OR p_data_monitoraggio IS NULL
       OR p_tipo_monitoraggio IS NULL
       OR p_esito_monitoraggio IS NULL THEN
        RAISE_APPLICATION_ERROR(
            -20130,
            'Dati obbligatori mancanti per la registrazione del monitoraggio.'
        );
    END IF;

    SELECT COUNT(*)
    INTO v_count
    FROM ASSEGNAZIONE
    WHERE id_assegnazione = p_id_assegnazione;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(
            -20131,
            'L''assegnazione indicata non esiste.'
        );
    END IF;

    SELECT data_inizio,
           data_fine_prevista,
           data_fine_effettiva
    INTO v_data_inizio,
```

Implementazione

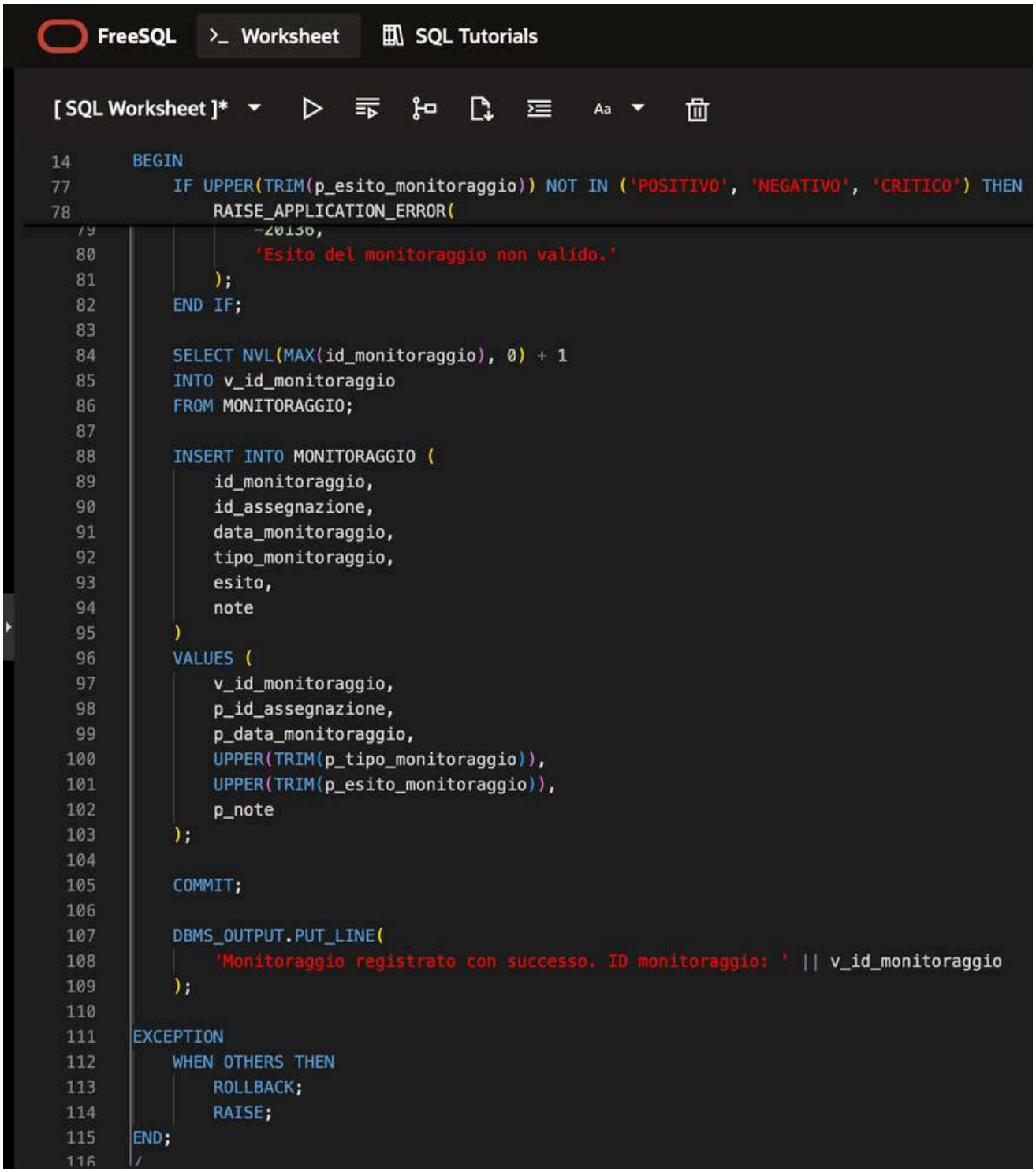
FreeSQL > Worksheet SQL Tutorials

[SQL Worksheet]*

```
14      BEGIN
40          INTO v_data_inizio,
41               v_data_fine_prevista,
42               v_data_fine_effettiva
43          FROM ASSEGNAZIONE
44          WHERE id_assegnazione = p_id_assegnazione;
45
46          IF p_data_monitoraggio < v_data_inizio THEN
47              RAISE_APPLICATION_ERROR(
48                  -20132,
49                  'La data del monitoraggio non puo essere precedente alla data di inizio dell''assegnazione.'
50              );
51          END IF;
52
53          IF v_data_fine_effettiva IS NOT NULL
54             AND p_data_monitoraggio > v_data_fine_effettiva THEN
55              RAISE_APPLICATION_ERROR(
56                  -20133,
57                  'La data del monitoraggio e successiva alla fine effettiva dell''assegnazione.'
58              );
59          END IF;
60
61          IF v_data_fine_effettiva IS NULL
62             AND v_data_fine_prevista IS NOT NULL
63             AND p_data_monitoraggio > v_data_fine_prevista THEN
64              RAISE_APPLICATION_ERROR(
65                  -20134,
66                  'La data del monitoraggio e successiva alla fine prevista dell''assegnazione.'
67              );
68          END IF;
69
70          IF UPPER(TRIM(p_tipo_monitoraggio)) NOT IN ('INIZIALE', 'INTERMEDIO', 'FINALE') THEN
71              RAISE_APPLICATION_ERROR(
72                  -20135,
73                  'Tipologia di monitoraggio non valida.'
74              );
75          END IF;
76
77          IF UPPER(TRIM(p_esito_monitoraggio)) NOT IN ('POSITIVO', 'NEGATIVO', 'CRITICO') THEN
78              RAISE_APPLICATION_ERROR(
```

Implementazione

Procedura registra_monitoraggio




```
14      BEGIN
77          IF UPPER(TRIM(p_esito_monitoraggio)) NOT IN ('POSITIVO', 'NEGATIVO', 'CRITICO') THEN
78              RAISE_APPLICATION_ERROR(
79                  -20136,
80                  'Esito del monitoraggio non valido.'
81              );
82          END IF;
83
84          SELECT NVL(MAX(id_monitoraggio), 0) + 1
85          INTO v_id_monitoraggio
86          FROM MONITORAGGIO;
87
88          INSERT INTO MONITORAGGIO (
89              id_monitoraggio,
90              id_assegnazione,
91              data_monitoraggio,
92              tipo_monitoraggio,
93              esito,
94              note
95          )
96          VALUES (
97              v_id_monitoraggio,
98              p_id_assegnazione,
99              p_data_monitoraggio,
100             UPPER(TRIM(p_tipo_monitoraggio)),
101             UPPER(TRIM(p_esito_monitoraggio)),
102             p_note
103         );
104
105         COMMIT;
106
107         DBMS_OUTPUT.PUT_LINE(
108             'Monitoraggio registrato con successo. ID monitoraggio: ' || v_id_monitoraggio
109         );
110
111     EXCEPTION
112         WHEN OTHERS THEN
113             ROLLBACK;
114             RAISE;
115     END;
116     /
```

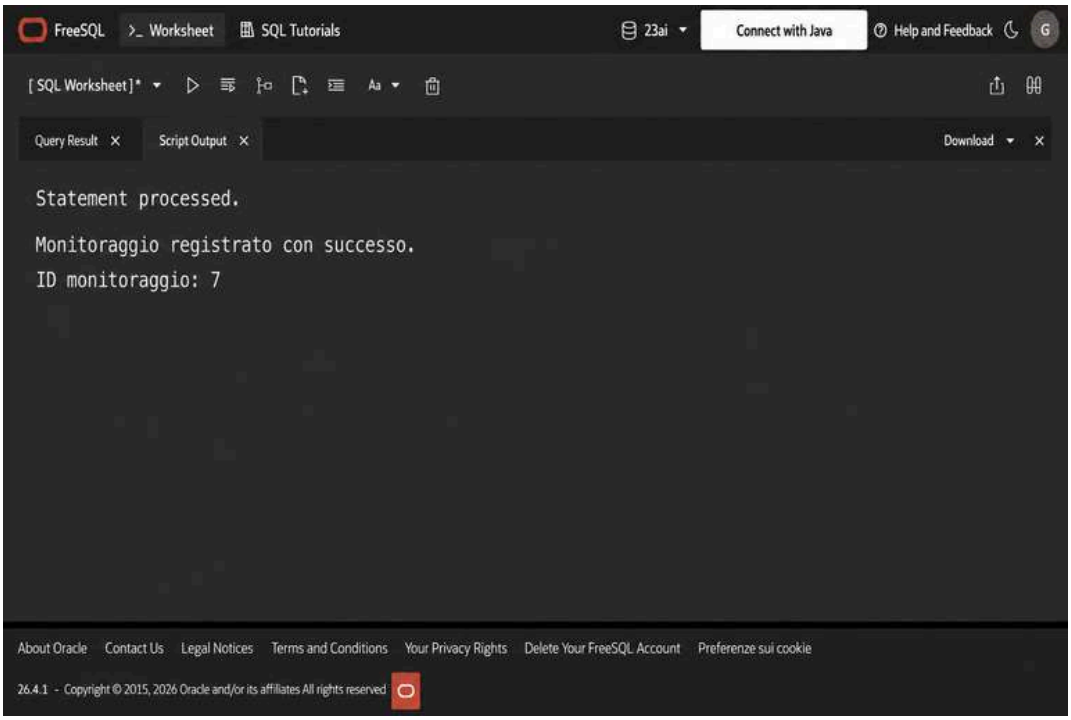
Implementazione

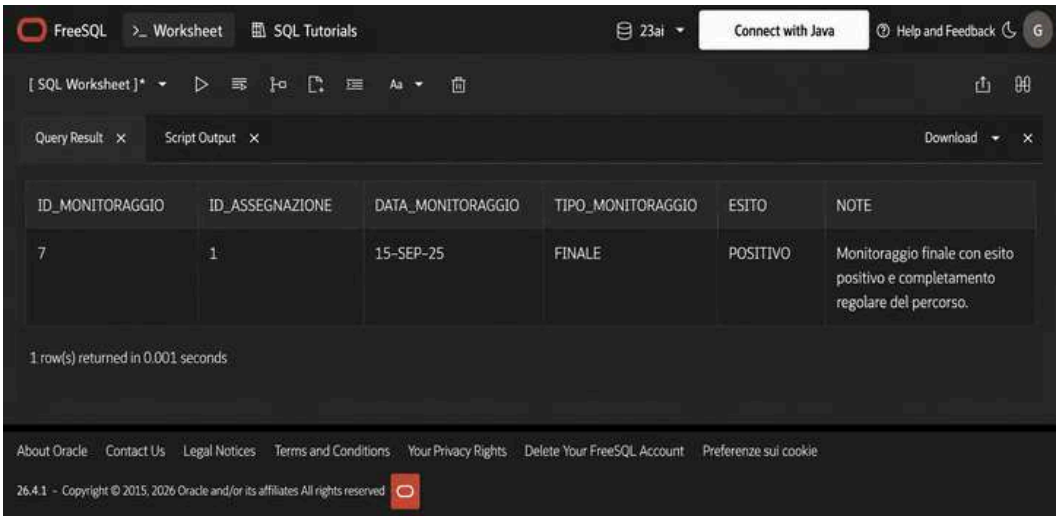
Procedura registra_monitoraggio (caso corretto)

FreeSQL | Worksheet | SQL Tutorials

[SQL Worksheet]*

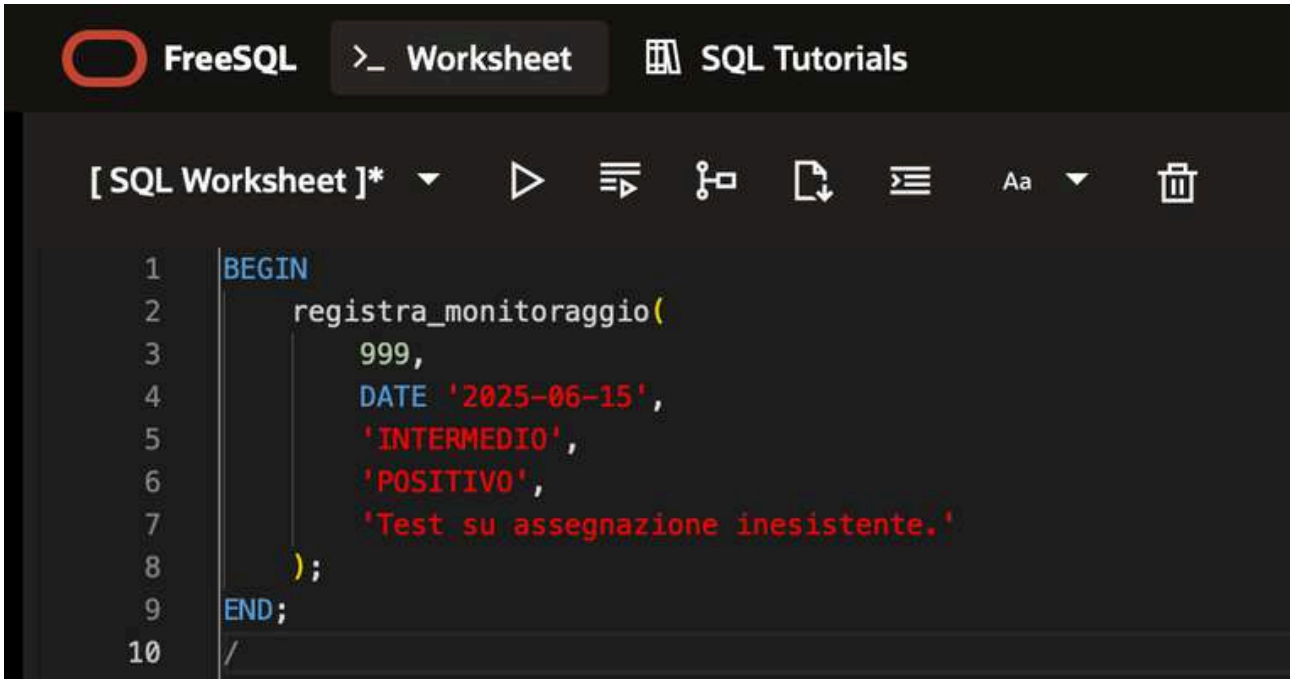
```
BEGIN
    registra_monitoraggio(
        1,
        DATE '2025-09-15',
        'FINALE',
        'POSITIVO',
        'Monitoraggio finale con esito positivo e completamento regolare del percorso.'
    );
END;
/
```





Implementazione

Procedura registra_monitoraggio (caso errato)




```
ORA-20131: L'assegnazione indicata non esiste.

ORA-06512: at "NOME_SCHEMA.REGISTRA_MONITORAGGIO", line ...
ORA-06512: at line 2

Statement processed.
```

Implementazione

Procedura inserisci_presenza

La procedura **inserisci_presenza** è progettata per consentire al volontario di registrare la propria presenza giornaliera nel sistema, in riferimento a una assegnazione attiva e valida nella data considerata.
L'operazione inserisce una nuova riga nella tabella **REGISTRO_SERVIZIO,** che sarà poi sottoposta alla successiva validazione amministrativa. Per questo motivo, lo stato iniziale della registrazione viene impostato automaticamente a **DA_VALIDARE.**
I passaggi eseguiti dalla procedura sono i seguenti:

1. **Controllo dei dati obbligatori:**
    - La procedura verifica che siano presenti:
    - identificativo del volontario
    - data del servizio
    - tipo di giornata
    - ore svolte
    - ore da recuperare
2. **Controllo esistenza del volontario:**

La procedura verifica che il volontario indicato esista nella tabella **VOLONTARIO.**

3. **Controllo validità del tipo di giornata:**

La procedura verifica che il tipo di giornata appartenga ai valori ammessi dal sistema.

4. **Controllo validità delle ore:**

La procedura verifica che:

- ore_svolte sia maggiore o uguale a zero
- ore_da_recuperare sia maggiore o uguale a zero

Implementazione

Procedura inserisci_presenza

5. **Controllo di coerenza minima tra tipo giornata e ore svolte:**

La procedura applica il controllo logico di base:

- se tipo_giornata = 'SERVIZIO', le ore svolte devono essere maggiori di zero
- se tipo_giornata rappresenta permesso o malattia, le ore svolte devono essere pari a zero

6. **Ricerca dell'assegnazione attiva nella data considerata:**

La procedura individua una assegnazione del volontario che sia valida nella data indicata:

- data_servizio deve essere successiva o uguale a data_inizio
- se è presente data_fine_effettiva, la data del servizio deve essere minore o uguale a quella
- altrimenti, se è presente data_fine_prevista, la data del servizio deve essere minore o uguale a quella
- se entrambe sono nulle, l'assegnazione viene trattata come ancora aperta
- lo stato dell'assegnazione deve essere **ATTIVA**

7. **Controllo di non duplicazione:**

La procedura verifica che non esista già una presenza registrata per la stessa assegnazione e la stessa data.

8. **Inserimento della presenza:**

Se tutti i controlli hanno esito positivo:

- viene generato un nuovo id_registro
- viene inserita una nuova riga in **REGISTRO_SERVIZIO**
- stato_validazione viene impostato a **DA_VALIDARE**

9. **Gestione transazionale:**

Se l'inserimento va a buon fine:

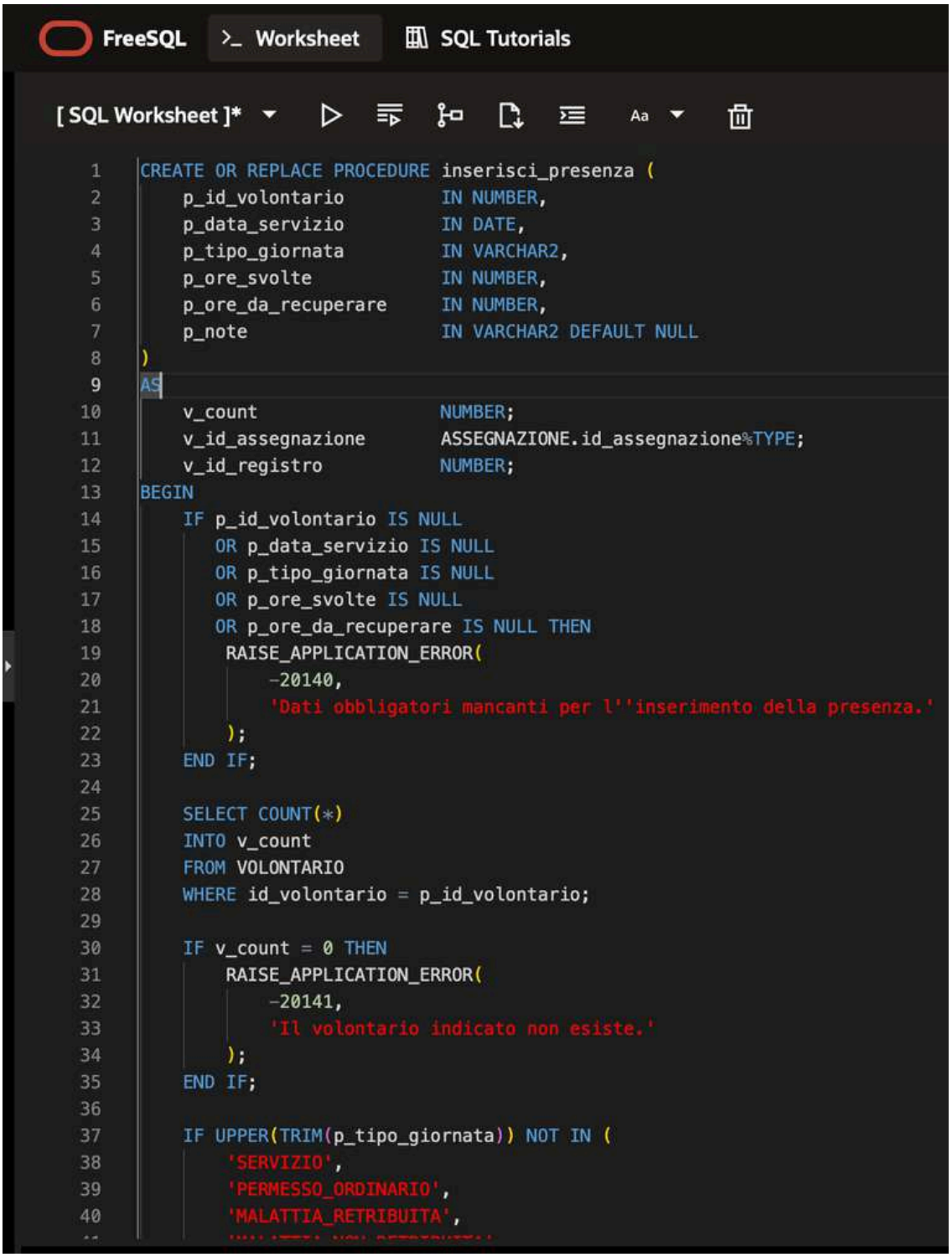
- viene eseguito il **COMMIT**
- viene mostrato un messaggio di successo

In caso di errore:

- viene eseguito il **ROLLBACK**
- l'eccezione viene rilanciata

Implementazione

Procedura inserisci_presenza




```sql
CREATE OR REPLACE PROCEDURE inserisci_presenza (
    p_id_volontario         IN NUMBER,
    p_data_servizio         IN DATE,
    p_tipo_giornata         IN VARCHAR2,
    p_ore_svolte            IN NUMBER,
    p_ore_da_recuperare     IN NUMBER,
    p_note                  IN VARCHAR2 DEFAULT NULL
)
AS
    v_count                 NUMBER;
    v_id_assegnazione       ASSEGNAZIONE.id_assegnazione%TYPE;
    v_id_registro           NUMBER;
BEGIN
    IF p_id_volontario IS NULL
       OR p_data_servizio IS NULL
       OR p_tipo_giornata IS NULL
       OR p_ore_svolte IS NULL
       OR p_ore_da_recuperare IS NULL THEN
        RAISE_APPLICATION_ERROR(
            -20140,
            'Dati obbligatori mancanti per l''inserimento della presenza.'
        );
    END IF;

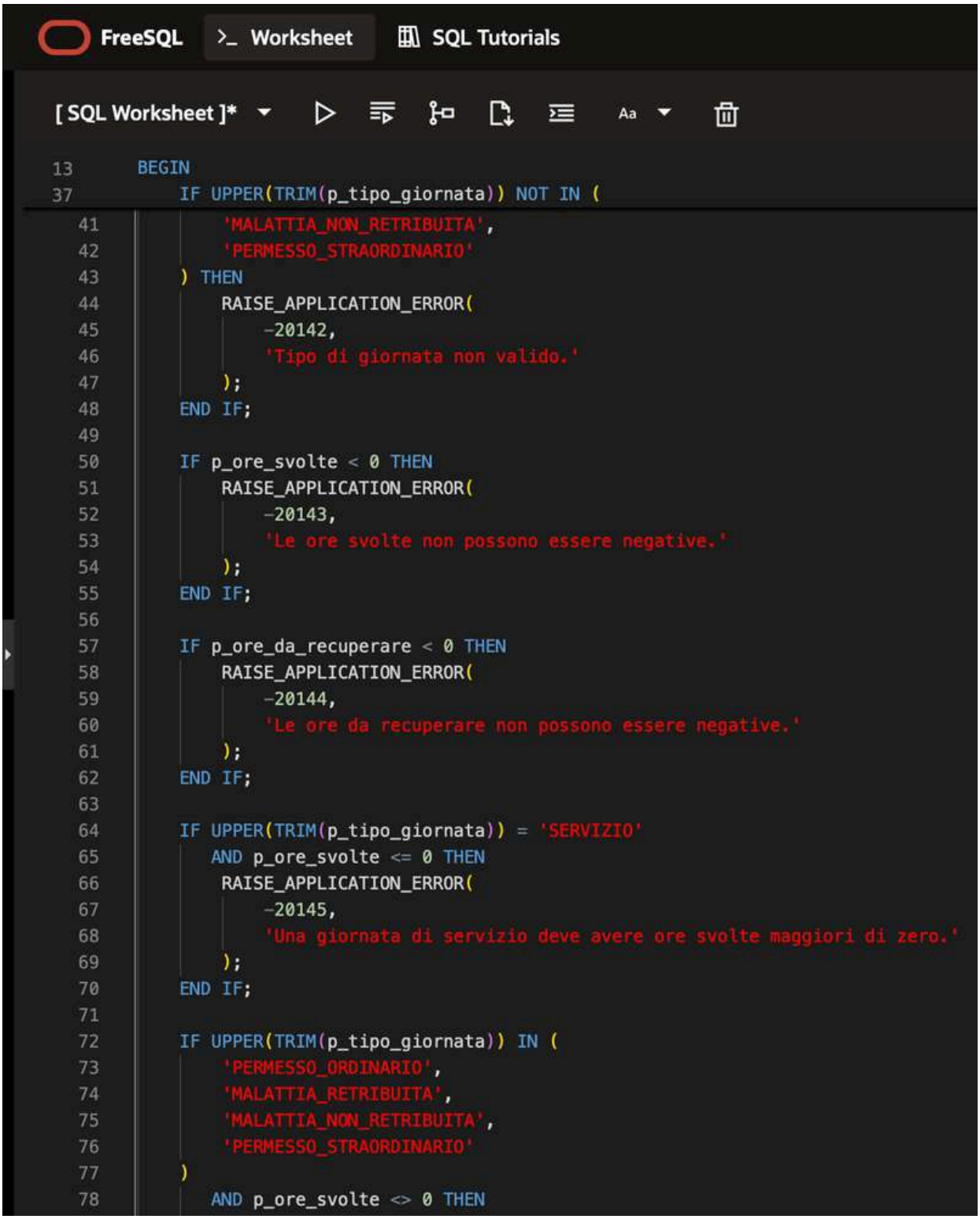
    SELECT COUNT(*)
    INTO v_count
    FROM VOLONTARIO
    WHERE id_volontario = p_id_volontario;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(
            -20141,
            'Il volontario indicato non esiste.'
        );
    END IF;

    IF UPPER(TRIM(p_tipo_giornata)) NOT IN (
        'SERVIZIO',
        'PERMESSO_ORDINARIO',
        'MALATTIA_RETRIBUITA',
```

Implementazione

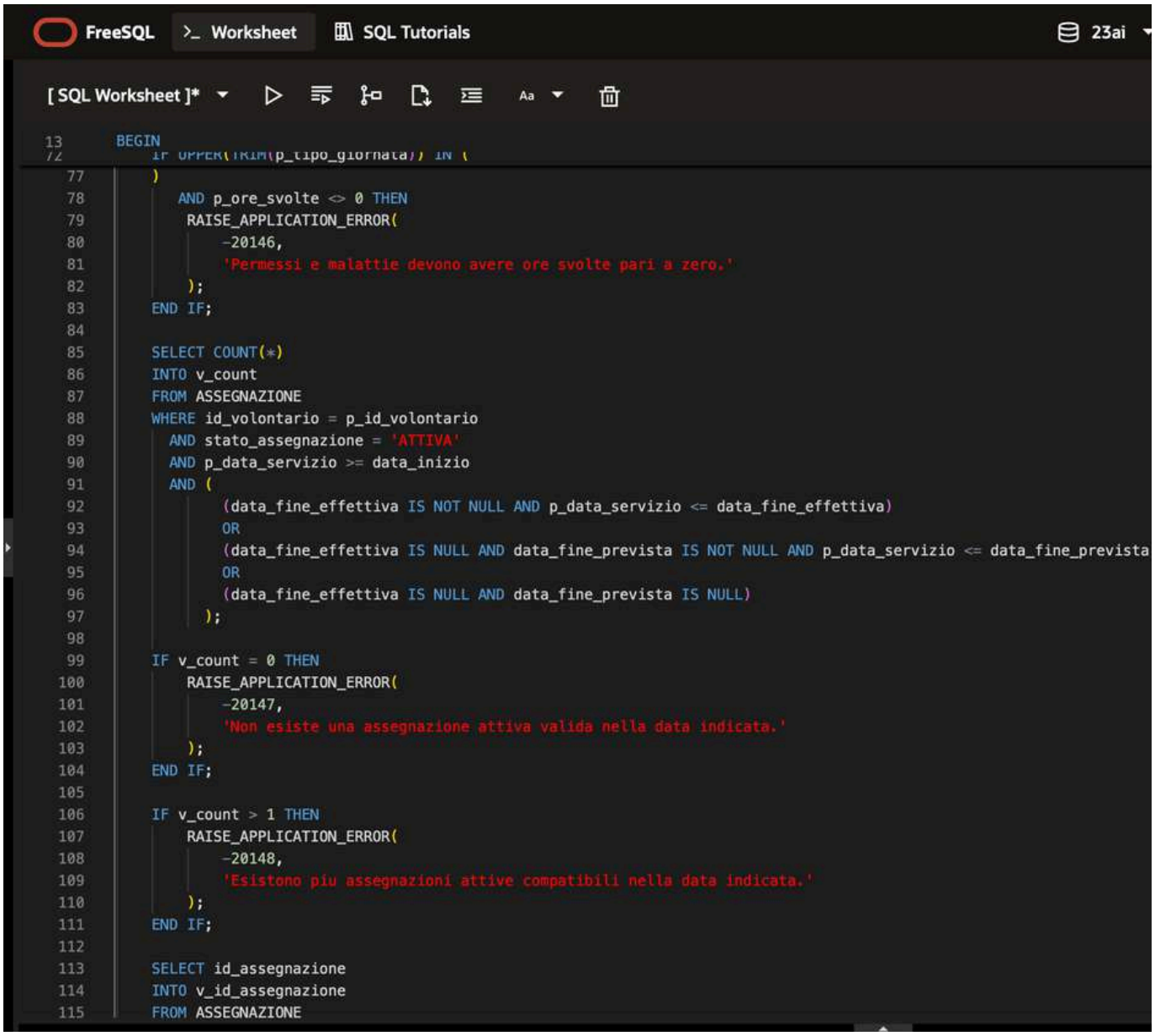
Procedura inserisci_presenza




```
13      BEGIN
37          IF UPPER(TRIM(p_tipo_giornata)) NOT IN (
41              'MALATTIA_NON_RETRIBUITA',
42              'PERMESSO_STRAORDINARIO'
43          ) THEN
44              RAISE_APPLICATION_ERROR(
45                  -20142,
46                  'Tipo di giornata non valido.'
47              );
48          END IF;
49
50          IF p_ore_svolte < 0 THEN
51              RAISE_APPLICATION_ERROR(
52                  -20143,
53                  'Le ore svolte non possono essere negative.'
54              );
55          END IF;
56
57          IF p_ore_da_recuperare < 0 THEN
58              RAISE_APPLICATION_ERROR(
59                  -20144,
60                  'Le ore da recuperare non possono essere negative.'
61              );
62          END IF;
63
64          IF UPPER(TRIM(p_tipo_giornata)) = 'SERVIZIO'
65             AND p_ore_svolte <= 0 THEN
66              RAISE_APPLICATION_ERROR(
67                  -20145,
68                  'Una giornata di servizio deve avere ore svolte maggiori di zero.'
69              );
70          END IF;
71
72          IF UPPER(TRIM(p_tipo_giornata)) IN (
73              'PERMESSO_ORDINARIO',
74              'MALATTIA_RETRIBUITA',
75              'MALATTIA_NON_RETRIBUITA',
76              'PERMESSO_STRAORDINARIO'
77          )
78             AND p_ore_svolte <> 0 THEN
```

Implementazione

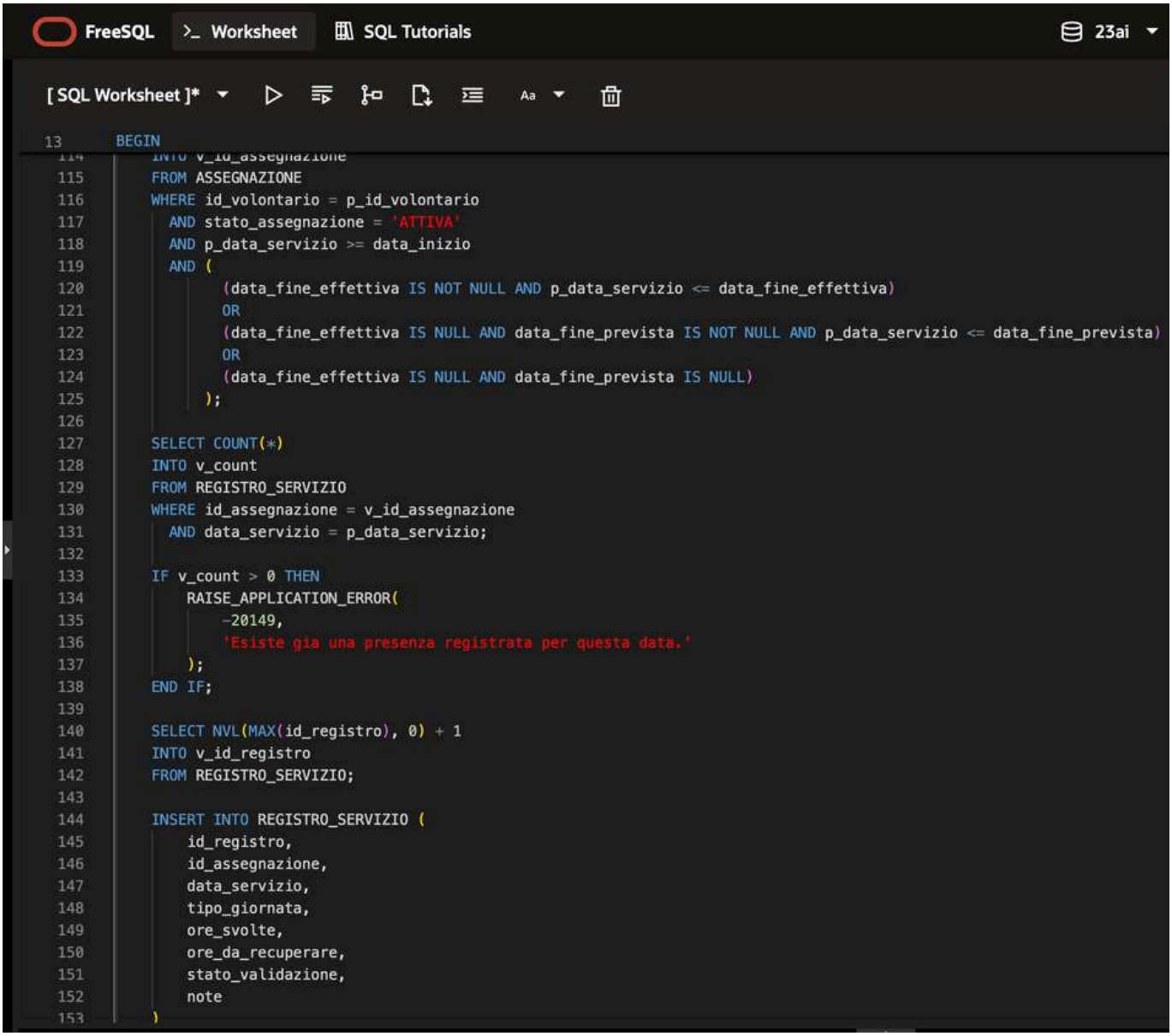
Procedura inserisci_presenza




```
13      BEGIN
72          IF UPPER(TRIM(p_tipo_giornata)) IN (
77          )
78             AND p_ore_svolte <> 0 THEN
79              RAISE_APPLICATION_ERROR(
80                  -20146,
81                  'Permessi e malattie devono avere ore svolte pari a zero.'
82              );
83          END IF;
84
85          SELECT COUNT(*)
86          INTO v_count
87          FROM ASSEGNAZIONE
88          WHERE id_volontario = p_id_volontario
89            AND stato_assegnazione = 'ATTIVA'
90            AND p_data_servizio >= data_inizio
91            AND (
92                  (data_fine_effettiva IS NOT NULL AND p_data_servizio <= data_fine_effettiva)
93                  OR
94                  (data_fine_effettiva IS NULL AND data_fine_prevista IS NOT NULL AND p_data_servizio <= data_fine_prevista
95                  OR
96                  (data_fine_effettiva IS NULL AND data_fine_prevista IS NULL)
97                );
98
99          IF v_count = 0 THEN
100             RAISE_APPLICATION_ERROR(
101                 -20147,
102                 'Non esiste una assegnazione attiva valida nella data indicata.'
103             );
104         END IF;
105
106         IF v_count > 1 THEN
107             RAISE_APPLICATION_ERROR(
108                 -20148,
109                 'Esistono piu assegnazioni attive compatibili nella data indicata.'
110             );
111         END IF;
112
113         SELECT id_assegnazione
114         INTO v_id_assegnazione
115         FROM ASSEGNAZIONE
```

Implementazione

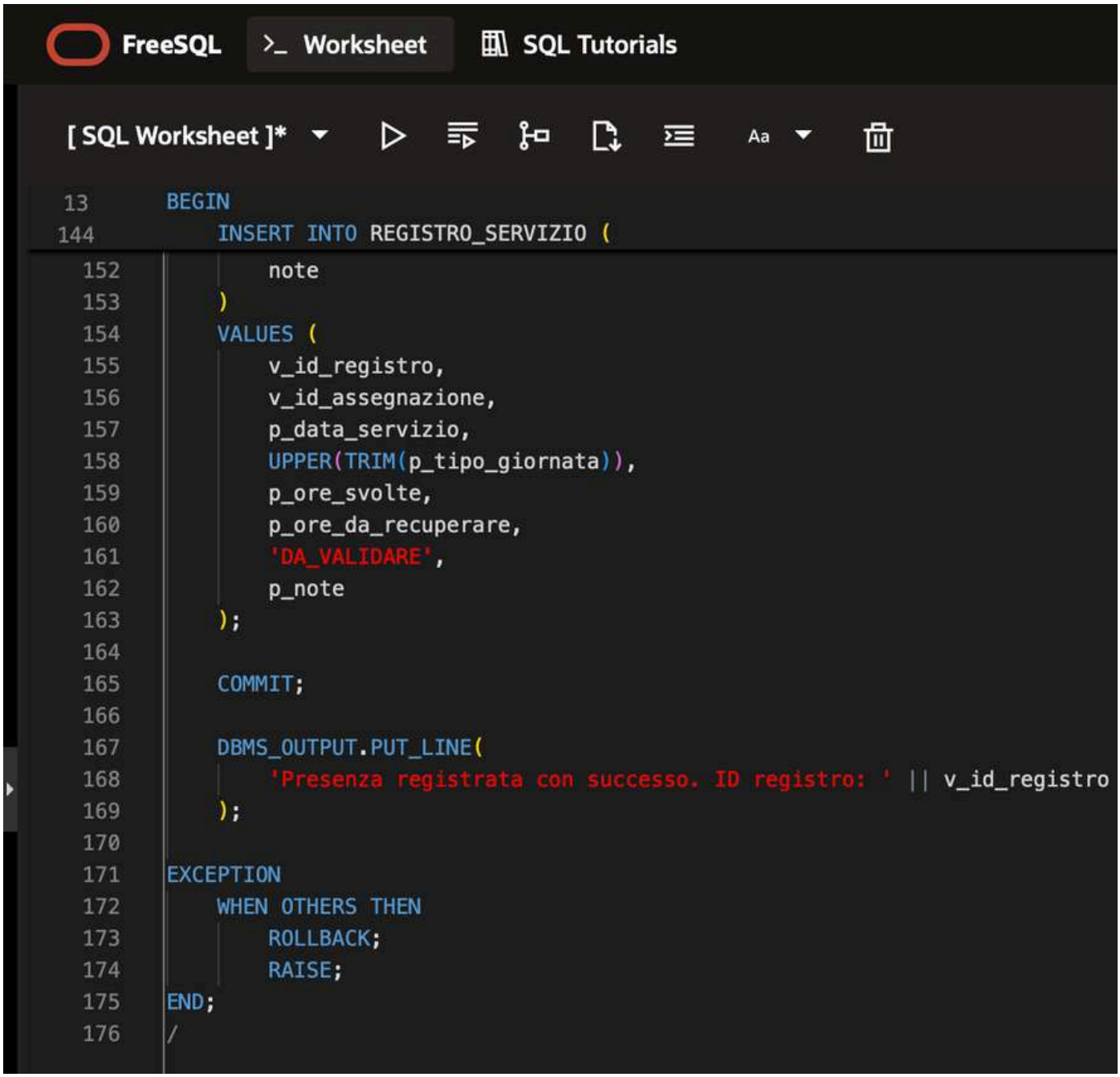
Procedura inserisci_presenza




```
13      BEGIN
115         FROM ASSEGNAZIONE
116         WHERE id_volontario = p_id_volontario
117           AND stato_assegnazione = 'ATTIVA'
118           AND p_data_servizio >= data_inizio
119           AND (
120                 (data_fine_effettiva IS NOT NULL AND p_data_servizio <= data_fine_effettiva)
121                 OR
122                 (data_fine_effettiva IS NULL AND data_fine_prevista IS NOT NULL AND p_data_servizio <= data_fine_prevista)
123                 OR
124                 (data_fine_effettiva IS NULL AND data_fine_prevista IS NULL)
125               );
126
127         SELECT COUNT(*)
128         INTO v_count
129         FROM REGISTRO_SERVIZIO
130         WHERE id_assegnazione = v_id_assegnazione
131           AND data_servizio = p_data_servizio;
132
133         IF v_count > 0 THEN
134             RAISE_APPLICATION_ERROR(
135                 -20149,
136                 'Esiste gia una presenza registrata per questa data.'
137             );
138         END IF;
139
140         SELECT NVL(MAX(id_registro), 0) + 1
141         INTO v_id_registro
142         FROM REGISTRO_SERVIZIO;
143
144         INSERT INTO REGISTRO_SERVIZIO (
145             id_registro,
146             id_assegnazione,
147             data_servizio,
148             tipo_giornata,
149             ore_svolte,
150             ore_da_recuperare,
151             stato_validazione,
152             note
153         )
```

Implementazione

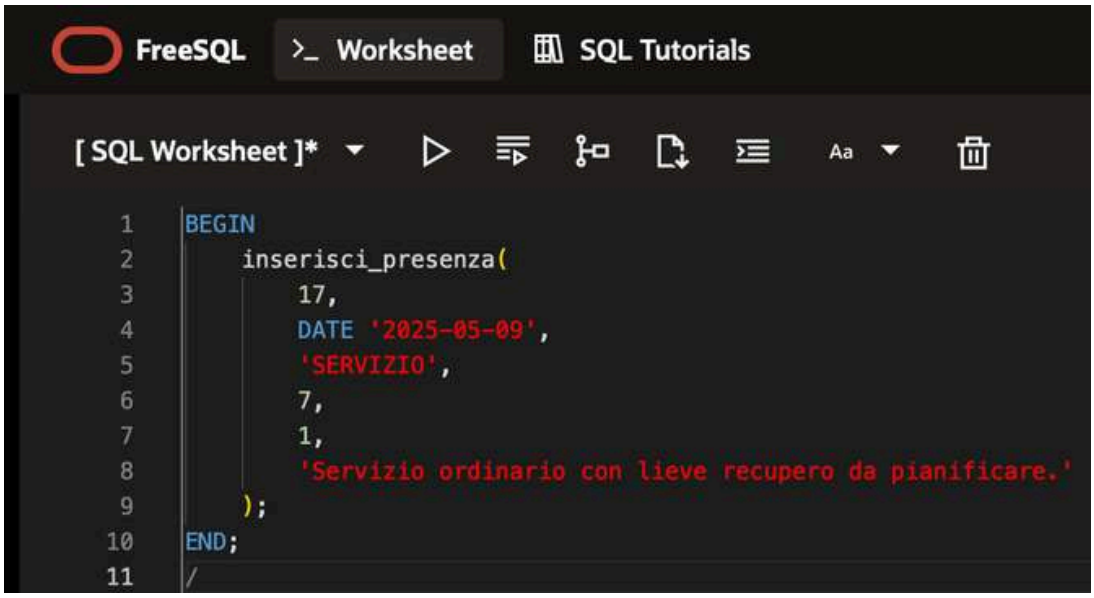
Procedura inserisci_presenza

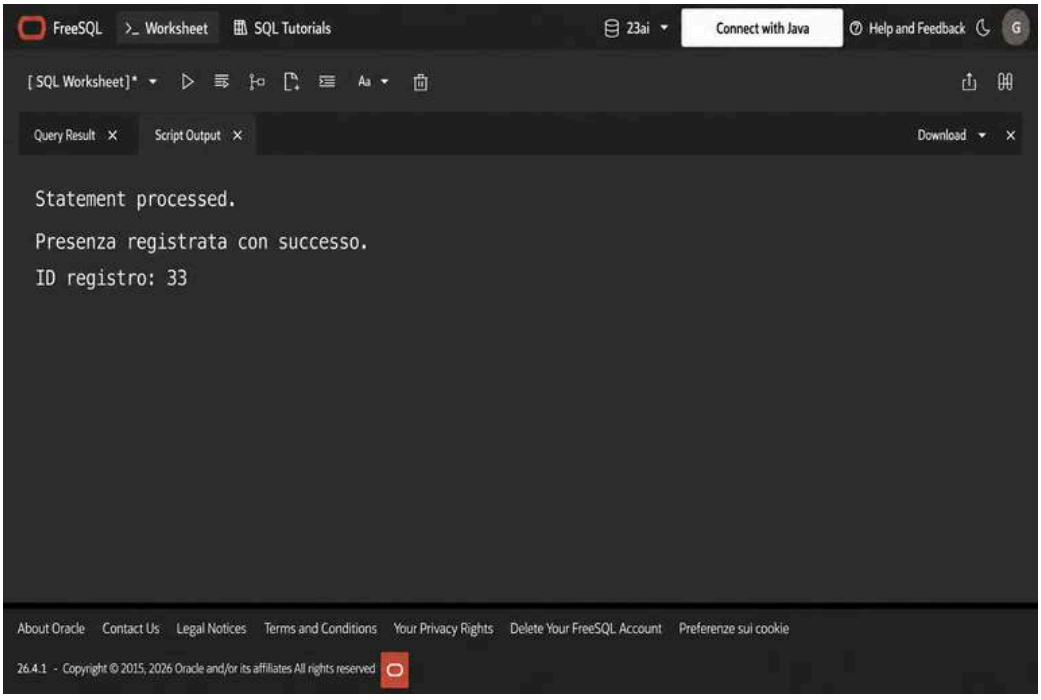


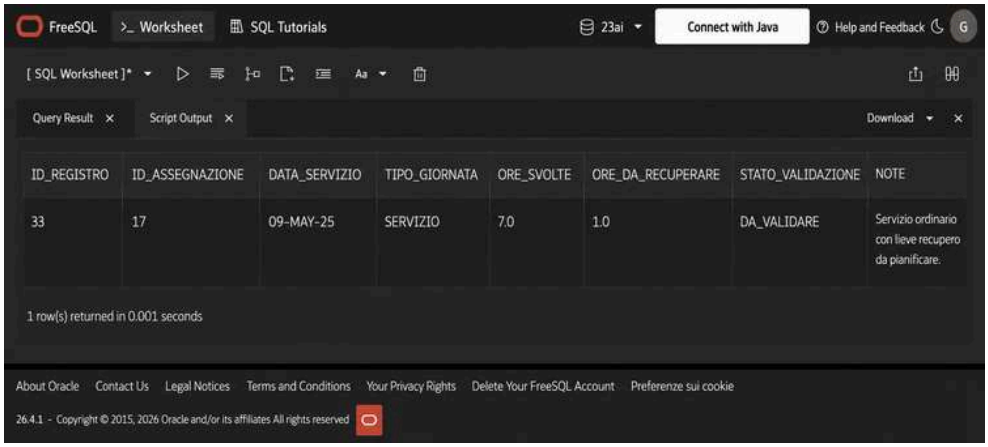

```
13      BEGIN
144         INSERT INTO REGISTRO_SERVIZIO (
152             note
153         )
154         VALUES (
155             v_id_registro,
156             v_id_assegnazione,
157             p_data_servizio,
158             UPPER(TRIM(p_tipo_giornata)),
159             p_ore_svolte,
160             p_ore_da_recuperare,
161             'DA_VALIDARE',
162             p_note
163         );
164
165         COMMIT;
166
167         DBMS_OUTPUT.PUT_LINE(
168             'Presenza registrata con successo. ID registro: ' || v_id_registro
169         );
170
171     EXCEPTION
172         WHEN OTHERS THEN
173             ROLLBACK;
174             RAISE;
175     END;
176     /
```

Implementazione

Procedura inserisci_presenza (caso corretto)







Implementazione

Procedura inserisci_presenza (caso errato)

FreeSQL | Worksheet | SQL Tutorials

[SQL Worksheet]*

```
1  BEGIN
2      inserisci_presenza(
3          999,
4          DATE '2025-05-09',
5          'SERVIZIO',
6          7,
7          1,
8          'Test su volontario inesistente.'
9      );
10 END;
11 /
```

```
ORA-20141: Il volontario indicato non esiste.

ORA-06512: at "NOME_SCHEMA.INSERISCI_PRESENZA", line ...
ORA-06512: at line 2

Statement processed.
```

Implementazione

VISTE

Le viste in SQL sono oggetti del database che rappresentano una query salvata. Si comportano come tabelle virtuali che permettono agli utenti di visualizzare e interagire con i dati da una o più tabelle senza modificarli direttamente. Le viste offrono una serie di vantaggi, tra cui la semplificazione delle query complesse, la sicurezza dei dati e la gestione p**iù efficiente delle autorizzazioni.**

Caratteristiche delle Viste

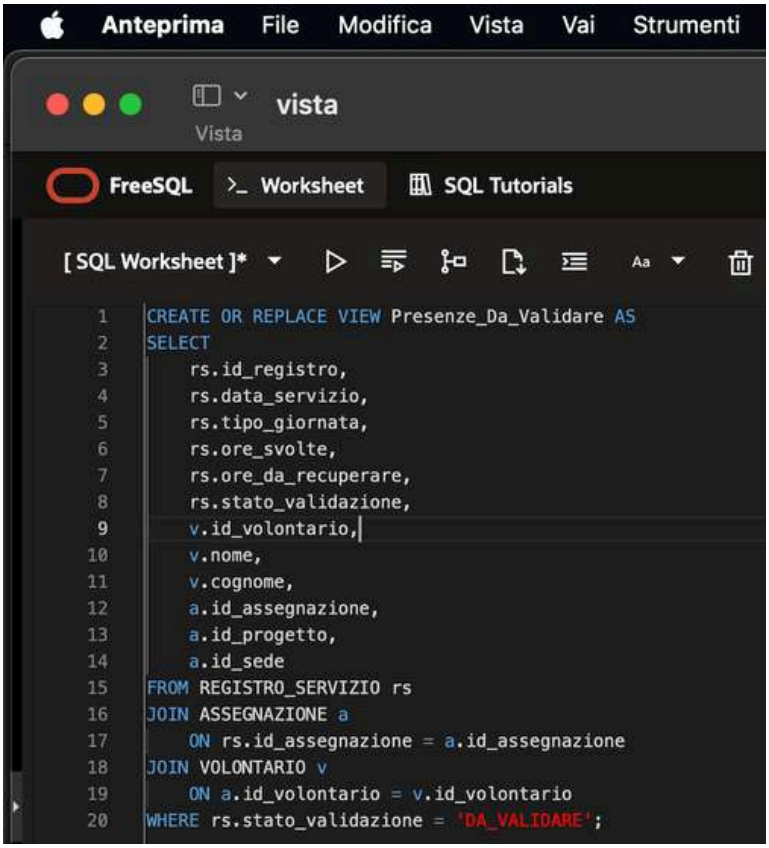
-Virtuali: Le viste non memorizzano fisicamente i dati. Quando una vista viene interrogata, SQL Server esegue la query definita dalla vista e restituisce il risultato.
-Dinamiche: Il contenuto di una vista è sempre aggiornato in base ai dati delle tabelle sottostanti. Qualsiasi modifica ai dati nelle tabelle base si riflette immediatamente nella vista.
-Semplicità: Le viste possono semplificare le query complesse combinando più tabelle e selezionando solo le colonne necessarie, presentando una struttura più semplice agli utenti finali.

Implementazione

VISTE

Nel contesto del presente progetto, il database del Servizio Civile Universale è stato progettato principalmente attorno alle tabelle operative fondamentali, in particolare **ASSEGNAZIONE, REGISTRO_SERVIZIO, ASSEGNAZIONE_FORMAZIONE, RICHIESTA_VARIAZIONE e MONITORAGGIO,** che descrivono il percorso amministrativo del volontario. Per questo motivo, le viste non costituiscono un elemento strutturale centrale del sistema, ma possono essere introdotte come supporto alla consultazione di dati ricorrenti o di sintesi. Una vista particolarmente utile nel contesto del progetto potrebbe essere, ad esempio, una vista sulle assegnazioni attive, utile per mostrare rapidamente i volontari che risultano attualmente inseriti in un progetto e in una sede, oppure una vista sulle presenze da validare, utile per l'attività dell'amministratore nella fase di controllo finale del **REGISTRO_SERVIZIO.** Questo tipo di soluzione è coerente con la logica del progetto, nel quale il volontario inserisce le proprie giornate di servizio e l'amministratore procede poi alla validazione amministrativa.

Vista presenze_da_validare



VISTE

Vista presenze_da_validare

Questa vista è coerente con la struttura del database, perché **REGISTRO_SERVIZIO** è collegata a **ASSEGNAZIONE, e ASSEGNAZIONE** è a sua volta collegata a **VOLONTARIO**. La sua utilità pratica consiste nel permettere all'amministratore di individuare rapidamente le presenze ancora da esaminare, senza dover ogni volta scrivere manualmente una query di join tra più tabelle. **REGISTRO_SERVIZIO** contiene infatti la registrazione giornaliera delle attività del volontario, mentre il volontario dispone del solo inserimento e aggiornamento delle proprie giornate, che vengono poi controllate dall'amministratore.

Note di revisione

Aggiornamenti successivi alla consegna. Per lo stato attuale fanno fede il codice e il CHANGELOG del repository.

- Utenti e ruoli: gli utenti con password in chiaro sono sostituiti da tre ruoli (ruolo_amministratore, ruolo_operatore, ruolo_volontario). Gli utenti vanno creati a parte.
- Trigger sui limiti di assenza: check_limite_permessi_ordinari, check_limite_malattia_retribuita e check_limite_malattia_non_retribuita sono unificati in check_limiti_assenze.
- Trigger check_limite_ore_giornaliere: rimosso. Le ore oltre 8 sono rifiutate dal vincolo CHECK invece di essere troncate.
- Trigger check_presenza_assegnazione_attiva: controlla solo il periodo dell'assegnazione, così lo storico delle assegnazioni concluse è ammesso.
- Procedure: gli ID derivano da sequenze (seq_formazione, seq_monitoraggio) invece di MAX+1; nessun COMMIT o ROLLBACK interno, la transazione è del chiamante; disattiva_progetto ha una sola definizione.
- Schema: aggiunti gli indici sulle chiavi esterne e il vincolo per cui un'assegnazione CONCLUSA ha la data di fine effettiva.
- Le procedure crea_progetto, modifica_progetto, crea_assegnazione, gestisci_richiesta_variazione, valida_presenza e inserisci_presenza descritte in questa relazione non sono ancora incluse negli script del repository. Il codice mostrato nelle pagine seguenti è quello della versione consegnata.